Specification Handbook

Kiro Specification Reference

Build date: 2026-11-11T22:37:15Z

Source revision: 1d38a32

Generated automatically from Kiro specifications. Do not edit by hand.

Contents

1 Document Information **3**

2 Revision History **4**

3 Architecture Overview **5**

4 Specification Index **6**

5 Requirements Index **7**

6 Tasks Index **9**

7 Diagram Index **12**

8 Steering Documents **13**
- 8.0.1 Agent workflow . . . 13
- 8.0.2 Architecture rules . . . 14
- 8.0.3 Licensing and distribution . . . 15
- 8.0.4 Progress tracking: the SVG mindmap . . . 17
- 8.0.5 Toolchain . . . 17

9 Specifications **20**
- 9.1 Phase 1: Application Foundation — Requirements — SPEC-PHASE-1-FOUNDATION . . . 21
  - 9.1.1 Requirements . . . 21
  - 9.1.2 Design . . . 26
  - 9.1.3 Tasks . . . 33
- Notes . . . 37
- 9.2 Phase 10: Road Trip Planning — Requirements — SPEC-PHASE-10-ROAD-TRIP-PLANNING 39
  - 9.2.1 Requirements . . . 39
  - 9.2.2 Design . . . 49
  - 9.2.3 Tasks . . . 63
- Notes . . . 69
- 9.3 Phase 11: Geotagged Photos from a Linked Folder — Requirements — SPEC-PHASE-11-GEOTAGGED-PHOTO
  - 9.3.1 Requirements . . . 71
  - 9.3.2 Design . . . 78
  - 9.3.3 Tasks . . . 96
- Notes . . . 100
- 9.4 Phase 2: Routes and Waypoints — Requirements — SPEC-PHASE-2-ROUTES-WAYPOINTS102
  - 9.4.1 Requirements . . . 102
  - 9.4.2 Design . . . 104
  - 9.4.3 Tasks . . . 107
- Notes . . . 110
- 9.5 Phase 3: Tracks and Import/Export — Requirements — SPEC-PHASE-3-TRACKS-IMPORT-EXPORT 112
  - 9.5.1 Requirements . . . 112
  - 9.5.2 Design . . . 114
  - 9.5.3 Tasks . . . 118
- Notes . . . 122
- 9.6 Phase 4: Advanced Mapping — Requirements — SPEC-PHASE-4-ADVANCED-MAPPING . 124
  - 9.6.1 Requirements . . . 124
  - 9.6.2 Design . . . 127
  - 9.6.3 Tasks . . . 132
- Notes . . . 137

9.7 Phase 5: Object and point management — Requirements — SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT13
9.7.1 Requirements . . . 139
9.7.2 Design . . . 143
9.7.3 Tasks . . . 147
Notes . . . 151
9.8 Phase 6: Route alternatives and shaping — Requirements — SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPI
9.8.1 Requirements . . . 153
9.8.2 Design . . . 159
9.8.3 Tasks . . . 166
Notes . . . 170
9.9 Phase 7: UK regions, road-following by default, and a fuller profile — Requirements — SPEC-PHASE-7-REGIONS-PROFILE . . . 172
9.9.1 Requirements . . . 172
9.9.2 Design . . . 177
9.9.3 Tasks . . . 183
Notes . . . 188
9.10 Phase 8: Intelligent Windy Roads Routing — Requirements — SPEC-PHASE-8-WINDY-ROADS-ROUTING190
9.10.1 Requirements . . . 190
Notes . . . 197
9.11 Phase 9: Garmin Compatibility — Requirements — SPEC-PHASE-9-GARMIN-COMPATIBILITY199
9.11.1 Requirements . . . 199
Notes . . . 201

10 Architecture Decisions 203

11 Traceability Matrix 204

12 Git Change History 211

13 Appendices 212
13.1 Appendix A — Reference Files . . . 212
13.2 Appendix B — Tokenizer Summary . . . 212
13.2.1 Specifications . . . 212
13.2.2 Steering Documents . . . 212

1 Document Information

Field	Value
Title	Specification Handbook
Organisation	—
Build date	2026-11-11T22:37:15Z
Source revision	1d38a32
Specifications	11
Requirements	106
Tasks	161
Diagrams	0
Source of truth	`.kiro/specs`

This handbook is generated automatically from the Kiro specifications. Do not edit it by hand — regenerate with `yarn handbook build`. The specs remain the single source of truth.

2 Revision History

Revision	Date	Notes
1d38a32	2026-11-11T22:37:15Z	Handbook build

3 Architecture Overview

No dedicated `architecture` spec was found in the source tree.

Status	Specs
unspecified	11

4 Specification Index

Spec ID	Title	Ver	Status	Reqs	Tasks	Figs
SPEC-PHASE-1-FOUNDATION	Phase 1: Application Foundation — Requirements	—	unspecified	14	18	0
SPEC-PHASE-10-ROAD-TRIP-PLANNING	Phase 10: Road Trip Planning — Requirements	—	unspecified	17	30	0
SPEC-PHASE-11-GEOTAGGED-PHOTOS	Phase 11: Geotagged Photos from a Linked Folder — Requirements	—	unspecified	11	13	0
SPEC-PHASE-2-ROUTES-WAYPOINTS	Phase 2: Routes and Waypoints — Requirements	—	unspecified	7	16	0
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT	Phase 3: Tracks and Import/Export — Requirements	—	unspecified	8	21	0
SPEC-PHASE-4-ADVANCED-MAPPING	Phase 4: Advanced Mapping — Requirements	—	unspecified	8	20	0
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT	Phase 5: Object and point management — Requirements	—	unspecified	6	12	0
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING	Phase 6: Route alternatives and shaping — Requirements	—	unspecified	11	15	0
SPEC-PHASE-7-REGIONS-PROFILE	Phase 7: UK regions, road-following by default, and a fuller profile — Requirements	—	unspecified	6	16	0
SPEC-PHASE-8-WINDY-ROADS-ROUTING	Phase 8: Intelligent Windy Roads Routing — Requirements	—	unspecified	13	0	0
SPEC-PHASE-9-GARMIN-COMPATIBILITY	Phase 9: Garmin Compatibility — Requirements	—	unspecified	5	0	0

5 Requirements Index

Requirement ID	Requirement
SPEC-PHASE-1-FOUNDATION:R1	Req 1: Application launches as a native macOS app
SPEC-PHASE-1-FOUNDATION:R2	Req 2: Electron security baseline
SPEC-PHASE-1-FOUNDATION:R3	Req 3: Three-panel application layout
SPEC-PHASE-1-FOUNDATION:R4	Req 4: Trip library tree
SPEC-PHASE-1-FOUNDATION:R5	Req 5: Trip and folder management
SPEC-PHASE-1-FOUNDATION:R6	Req 6: Local SQLite persistence with migrations
SPEC-PHASE-1-FOUNDATION:R7	Req 7: Persistence across restart
SPEC-PHASE-1-FOUNDATION:R8	Req 8: OpenStreetMap display behind a provider abstraction
SPEC-PHASE-1-FOUNDATION:R9	Req 9: Backup and restore
SPEC-PHASE-1-FOUNDATION:R10	Req 10: Application menus and keyboard shortcuts
SPEC-PHASE-1-FOUNDATION:R11	Req 11: Settings and theming
SPEC-PHASE-1-FOUNDATION:R12	Req 12: Build, packaging and distribution
SPEC-PHASE-1-FOUNDATION:R13	Req 13: Testing baseline
SPEC-PHASE-1-FOUNDATION:R14	Req 14: Licence compliance and attribution
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R1	Req 1: Trip schedule
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R2	Req 2: Stops and durations
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R3	Req 3: Legs and the timeline
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R4	Req 4: Calendar view
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R5	Req 5: POI data source
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R6	Req 6: Finding POIs along a route
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R7	Req 7: POI details and honesty
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R8	Req 8: Adding a POI to the trip
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R9	Req 9: Fuel range check
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R10	Req 10: Export
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R11	Req 11: Postcode and place search (offline)
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R12	Req 12: Security, privacy and licensing
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R13	Req 13: Testing
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R14	Req 14: The trip in the library summarises it
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R15	Req 15: Stops are joined by road automatically
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R16	Req 16: Panels resize and hide
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R17	Req 17: Icon buttons with hints
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R1	Req 1: Linking a folder
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R2	Req 2: Reading location and time
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R3	Req 3: Keeping in step with the folder
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R4	Req 4: Photos in the library
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R5	Req 5: Photos on the map
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R6	Req 6: The preview box
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R7	Req 7: Using a photo's location
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R8	Req 8: Read-only, privacy and security
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R9	Req 9: Scale and responsiveness
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R10	Req 10: Backups and moving to another Mac
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R11	Req 11: Testing
SPEC-PHASE-2-ROUTES-WAYPOINTS:R1	Req 1: Route creation
SPEC-PHASE-2-ROUTES-WAYPOINTS:R2	Req 2: Route editing
SPEC-PHASE-2-ROUTES-WAYPOINTS:R3	Req 3: Waypoint management
SPEC-PHASE-2-ROUTES-WAYPOINTS:R4	Req 4: Route and waypoint statistics
SPEC-PHASE-2-ROUTES-WAYPOINTS:R5	Req 5: Undo and redo
SPEC-PHASE-2-ROUTES-WAYPOINTS:R6	Req 6: Map selection and interaction
SPEC-PHASE-2-ROUTES-WAYPOINTS:R7	Req 7: Trip organisation completion
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R1	Req 1: GPX import
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R2	Req 2: Additional format support
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R3	Req 3: Import workflow
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R4	Req 4: Export
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R5	Req 5: Track display and statistics
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R6	Req 6: Track editing, non-destructive
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R7	Req 7: Library completion
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R8	Req 8: Test fixtures
SPEC-PHASE-4-ADVANCED-MAPPING:R1	Req 1: Multiple map providers
SPEC-PHASE-4-ADVANCED-MAPPING:R2	Req 2: Routing provider abstraction

Requirement ID	Requirement
SPEC-PHASE-4-ADVANCED-MAPPING:R3	Req 3: Snap to road
SPEC-PHASE-4-ADVANCED-MAPPING:R4	Req 4: Improved map controls
SPEC-PHASE-4-ADVANCED-MAPPING:R5	Req 5: Route and track profile
SPEC-PHASE-4-ADVANCED-MAPPING:R6	Req 6: Speed identification and distribution
SPEC-PHASE-4-ADVANCED-MAPPING:R7	Req 7: Waze hand-off
SPEC-PHASE-4-ADVANCED-MAPPING:R8	Req 8: Offline map groundwork
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R1	Req 1: Multi-selection in the library
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R2	Req 2: Batch move and delete
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R3	Req 3: Point table for a route
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R4	Req 4: Point table for a track
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R5	Req 5: Map highlighting of selected points
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R6	Req 6: Consistency with what exists
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R1	Req 1: Candidates while drawing
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R2	Req 2: Candidates for a leg of a saved route
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R3	Req 3: A chosen alternative survives re-routing
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R4	Req 4: Drag the line to shape it
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R5	Req 5: Drag control points
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R6	Req 6: Add, remove and convert points
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R7	Req 7: Via and shaping points are distinct
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R8	Req 8: Undo, integrity and limits
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R9	Req 9: Responsiveness and failure
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R10	Req 10: Provider contract
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R11	Req 11: Testing
SPEC-PHASE-7-REGIONS-PROFILE:R1	Req 1: Choosing regions on a UK county map
SPEC-PHASE-7-REGIONS-PROFILE:R2	Req 2: Following roads is the default
SPEC-PHASE-7-REGIONS-PROFILE:R3	Req 3: Altitude in the profile
SPEC-PHASE-7-REGIONS-PROFILE:R4	Req 4: A twisty score on every profile
SPEC-PHASE-7-REGIONS-PROFILE:R5	Req 5: UK default limits where none is posted
SPEC-PHASE-7-REGIONS-PROFILE:R6	Req 6: A plainer route panel (owner feedback)
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R1	Req 1: Routing engine capability assessment
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R2	Req 2: Routing modes and preferences
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R3	Req 3: Road curvature and twistiness scoring
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R4	Req 4: Road suitability and restrictions
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R5	Req 5: Custom routing cost model
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R6	Req 6: Detour management
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R7	Req 7: Snap-to-road integration and geometry integrity
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R8	Req 8: Geometry volume and undo
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R9	Req 9: User interface
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R10	Req 10: GPX compatibility
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R11	Req 11: Performance, caching and reliability
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R12	Req 12: Security, privacy and licensing
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R13	Req 13: Testing
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R1	Req 1: GDB support
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R2	Req 2: Garmin IMG investigation
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R3	Req 3: Device transfer investigation
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R4	Req 4: Expanded import and export
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R5	Req 5: Compatibility testing

6 Tasks Index

Task ID	Done	Task
SPEC-PHASE-1-FOUNDATION:T1	☑	Scaffold the project and prove the toolchain installs
SPEC-PHASE-1-FOUNDATION:T2	☑	Verify the native module against the Electron ABI
SPEC-PHASE-1-FOUNDATION:T3	☑	Minimal secure Electron shell
SPEC-PHASE-1-FOUNDATION:T4	☑	Shared contracts
SPEC-PHASE-1-FOUNDATION:T5	☑	Database layer
SPEC-PHASE-1-FOUNDATION:T6	☑	Repositories and services for folders and trips
SPEC-PHASE-1-FOUNDATION:T7	☑	IPC and preload bridge
SPEC-PHASE-1-FOUNDATION:T8	☑	Three-panel layout
SPEC-PHASE-1-FOUNDATION:T9	☑	Trip library panel
SPEC-PHASE-1-FOUNDATION:T10	☑	Map panel
SPEC-PHASE-1-FOUNDATION:T11	☑	Properties panel
SPEC-PHASE-1-FOUNDATION:T12	☑	Application menus
SPEC-PHASE-1-FOUNDATION:T13	☑	Settings surface
SPEC-PHASE-1-FOUNDATION:T14	☑	Backup and restore
SPEC-PHASE-1-FOUNDATION:T15	☑	End-to-end tests
SPEC-PHASE-1-FOUNDATION:T16	☑	Licence compliance surface
SPEC-PHASE-1-FOUNDATION:T17	☑	Build, packaging and ad-hoc signing
SPEC-PHASE-1-FOUNDATION:T18	☑	Phase 1 verification pass
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T1	☑	Spec, ADR 0019 and the out-of-order build
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T2	☑	Zoned time and offline time-zone lookup
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T3	☑	Itinerary types and the timeline
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T4	☑	Corridor geometry and the range check
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T5	☑	POI categories, attributes and opening hours
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T6	☑	Postcodes, OSGB and search text
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T7	☑	iCalendar writer
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T8	☑	Itinerary storage
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T9	☑	Itinerary service and commands
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T10	☑	Itinerary IPC, preload and tree detail
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T11	☑	Place data: extract and ingest
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T12	☑	Postcode data
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T13	☑	Search service
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T14	☑	POI search and range service
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T15	☑	From POI to waypoint and stop; planning legs
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T16	☑	Export
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T17	☑	Itinerary editor
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T18	☑	Calendar drawer
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T19	☑	Along-the-route panel and POI layer
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T20	☑	Map search box
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T21	☑	Settings, About and the register
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T22	☑	End-to-end tests and verification pass
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T23	☑	Spec for the feedback
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T24	☑	Leg modes in storage and the timeline
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T25	☑	Automatic planning
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T26	☑	Leg modes in the editor
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T27	☑	The trip in the library summarises it
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T28	☑	Resizable, hideable panels
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T29	☑	Icon buttons with hints
SPEC-PHASE-10-ROAD-TRIP-PLANNING:T30	☑	End-to-end tests and verification
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T1	☑	Spec, ADR 0020 and the out-of-order build
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T2	☑	Synthetic photo fixtures
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T3	☑	TIFF parsing and EXIF values (pure)
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T4	☑	Containers and the metadata reader
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T5	☑	Storage: linked folders and the file index
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T6	☑	Linking, refusals, undo and the write guard
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T7	☑	Scanner, fingerprints and watcher
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T8	☑	Path resolution, thumbnails, IPC and preload
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T9	☑	Library: tree, Photos view and the photo list
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T10	☑	Photo map layer, one trip and all trips
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T11	☑	The preview box
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T12	☑	Photo to waypoint, export and backups
SPEC-PHASE-11-GEOTAGGED-PHOTOS:T13	☑	End-to-end and verification pass
SPEC-PHASE-2-ROUTES-WAYPOINTS:T1	☑	Shared contracts for routes and waypoints
SPEC-PHASE-2-ROUTES-WAYPOINTS:T2	☑	Route and waypoint repositories
SPEC-PHASE-2-ROUTES-WAYPOINTS:T3	☑	Route service with geometry and derived statistics
SPEC-PHASE-2-ROUTES-WAYPOINTS:T4	☑	Waypoint service
SPEC-PHASE-2-ROUTES-WAYPOINTS:T5	☑	Command registry and undo service
SPEC-PHASE-2-ROUTES-WAYPOINTS:T6	☑	IPC and preload wiring

Task ID	Done	Task
SPEC-PHASE-2-ROUTES-WAYPOINTS:T7	☒	Waypoint map layer
SPEC-PHASE-2-ROUTES-WAYPOINTS:T8	☒	Route map layer
SPEC-PHASE-2-ROUTES-WAYPOINTS:T9	☒	Route drawing mode
SPEC-PHASE-2-ROUTES-WAYPOINTS:T10	☒	Route vertex editing
SPEC-PHASE-2-ROUTES-WAYPOINTS:T11	☒	Route and waypoint properties panels
SPEC-PHASE-2-ROUTES-WAYPOINTS:T12	☒	Library tree completion
SPEC-PHASE-2-ROUTES-WAYPOINTS:T13	☒	Undo and redo in the UI
SPEC-PHASE-2-ROUTES-WAYPOINTS:T14	☒	Context menus and map controls
SPEC-PHASE-2-ROUTES-WAYPOINTS:T15	☒	End-to-end tests
SPEC-PHASE-2-ROUTES-WAYPOINTS:T16	☒	Phase 2 verification pass
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T1	☒	Shared IO contracts and geometry helpers
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T2	☒	Synthetic test fixtures
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T3	☒	Track repository and service
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T4	☒	Format detection
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T5	☒	GPX read
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T6	☒	GPX write and round-trip
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T7	☒	KML and KMZ
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T8	☒	TCX read
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T9	☒	FIT read
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T10	☒	CSV waypoints
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T11	☒	JSON native backup
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T12	☒	Import service
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T13	☒	Export service
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T14	☒	IPC and preload wiring
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T15	☒	Track map layer
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T16	☒	Track properties and editing
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T17	☒	Import dialog
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T18	☒	Export dialog
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T19	☒	Library completion
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T20	☒	End-to-end tests
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:T21	☒	Phase 3 verification pass
SPEC-PHASE-4-ADVANCED-MAPPING:T1	☒	Shared contracts
SPEC-PHASE-4-ADVANCED-MAPPING:T2	☒	Pure geometry: bearing, profile, speed bands
SPEC-PHASE-4-ADVANCED-MAPPING:T3	☒	Map provider registry and dynamic CSP
SPEC-PHASE-4-ADVANCED-MAPPING:T4	☒	Migration 0002 and repositories
SPEC-PHASE-4-ADVANCED-MAPPING:T5	☒	Routing client and Valhalla adapter
SPEC-PHASE-4-ADVANCED-MAPPING:T6	☒	Routing service: snap a route
SPEC-PHASE-4-ADVANCED-MAPPING:T7	☒	Road attributes service
SPEC-PHASE-4-ADVANCED-MAPPING:T8	☒	Settings and consent
SPEC-PHASE-4-ADVANCED-MAPPING:T9	☒	IPC and preload wiring
SPEC-PHASE-4-ADVANCED-MAPPING:T10	☒	Map controls
SPEC-PHASE-4-ADVANCED-MAPPING:T11	☒	Measure tool
SPEC-PHASE-4-ADVANCED-MAPPING:T12	☒	Snap-to-road UI
SPEC-PHASE-4-ADVANCED-MAPPING:T13	☒	Profile panel
SPEC-PHASE-4-ADVANCED-MAPPING:T14	☒	Speed distribution UI
SPEC-PHASE-4-ADVANCED-MAPPING:T15	☒	Waze hand-off
SPEC-PHASE-4-ADVANCED-MAPPING:T16	☒	Phase 4 verification pass
SPEC-PHASE-4-ADVANCED-MAPPING:T17	☒	Interactive snap routing tool
SPEC-PHASE-4-ADVANCED-MAPPING:T18	☒	Embed the routing engine
SPEC-PHASE-4-ADVANCED-MAPPING:T19	☒	Wire the UI to the embedded engine
SPEC-PHASE-4-ADVANCED-MAPPING:T20	☒	Region download from the visible map area
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T1	☒	Repair the track undo gap
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T2	☒	Composite batch command
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T3	☒	Batch service
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T4	☒	Non-contiguous track point deletion
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T5	☒	Track point editing and paged reads
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T6	☒	IPC and preload wiring
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T7	☒	Multi-selection in the selection store
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T8	☒	Library tree: selection and keyboard model
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T9	☒	Batch actions in the UI

Task ID	Done	Task
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T10	☒	Point table
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T11	☒	Map highlighting
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:T12	☒	Phase 5 verification pass
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T1	☒	Shared contracts
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T2	☒	Pure geometry: divergence and reversal
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T3	☒	Leg boundaries and splicing
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T4	☒	Edit planner
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T5	☒	Migration 0003 and repositories
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T6	☒	Valhalla adapter: alternatives and sections
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T7	☒	Routing service, IPC and undo
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T8	☒	Making a chosen alternative stick
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T9	☒	Shaping a saved route on the map
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T10	☒	Candidates on the map
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T11	☒	The snap routing tool
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T12	☒	Extend a saved route
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T13	☒	TrailRT JSON carries control points
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T14	☒	Live tests and the latency measurement
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:T15	☒	End-to-end and verification pass
SPEC-PHASE-7-REGIONS-PROFILE:T1	☒	Spec, ADR 0018 and renumbering
SPEC-PHASE-7-REGIONS-PROFILE:T2	☒	Twistiness scoring
SPEC-PHASE-7-REGIONS-PROFILE:T3	☒	UK boundary and default limits
SPEC-PHASE-7-REGIONS-PROFILE:T4	☒	Road data carries one-way, and assumed limits reach the services
SPEC-PHASE-7-REGIONS-PROFILE:T5	☒	Terrain: HGT sampling and the tile store
SPEC-PHASE-7-REGIONS-PROFILE:T6	☒	Profile: terrain elevation and twistiness
SPEC-PHASE-7-REGIONS-PROFILE:T7	☒	Region map data and multi-region build
SPEC-PHASE-7-REGIONS-PROFILE:T8	☒	Region picker: UK map and Elsewhere
SPEC-PHASE-7-REGIONS-PROFILE:T9	☒	Following roads by default
SPEC-PHASE-7-REGIONS-PROFILE:T10	☒	Profile and distribution display
SPEC-PHASE-7-REGIONS-PROFILE:T11	☒	Attribution and licence register
SPEC-PHASE-7-REGIONS-PROFILE:T12	☒	Live tests
SPEC-PHASE-7-REGIONS-PROFILE:T13	☒	End-to-end and verification pass
SPEC-PHASE-7-REGIONS-PROFILE:T14	☒	Fix: Enter and double-click did not finish a new route
SPEC-PHASE-7-REGIONS-PROFILE:T15	☒	Fix: the map went blank after switching base map while zoomed in
SPEC-PHASE-7-REGIONS-PROFILE:T16	☒	Owner feedback: a plainer route panel

7 Diagram Index

Figure	Diagram ID	Caption	Spec

8 Steering Documents

Project-wide rules and guidance from `.kiro/steering`, reproduced in full.

8.0.1 Agent workflow

This repository is worked on by both Kiro and Claude Code. They share one set of rules and one set of specs, and neither tool keeps private instructions.

Single source of truth

What	Where	Read by
Always-applied rules	`.kiro/steering/*.md`	Kiro directly; Claude via `@` imports in `CLAUDE.md`
Phase specs	`.kiro/specs/<phase>/`	Both
Decisions	`docs/adr/`	Both
Progress	`docs/progress-mindmap.svg`	Both

To add or change a rule, edit or add a file in `.kiro/steering/`. Do not put project rules in `CLAUDE.md`, because Kiro never reads it. If you add a new steering file, also add an `@` import for it in `CLAUDE.md`, or Claude will not see it.

Spec-driven development Each phase lives in `.kiro/specs/phase-N-<slug>/` and moves through three documents, in order:

1. `requirements.md` — numbered requirements (`1.1`, `1.2`, …) with acceptance criteria.
2. `design.md` — how the requirements are met, written against the code as it actually is.
3. `tasks.md` — an ordered checklist that implements the design.

Do not write a phase's design or tasks until the previous phase is built and tested (see `.kiro/specs/README.md`). Phases with requirements only are deliberately unplanned.

Task format Keep `tasks.md` in the format Kiro parses, so either tool can pick up where the other left off:

```
- [ ] 3. Short task title
  - Concrete sub-step naming the file or module
  - Unit tests: what they prove
  - _Requirements: 1.2, 4.1_
```

- `- [ ]` not started, `- [x]` done. Number tasks sequentially and do not renumber completed ones.
- Every task ends with a `_Requirements: …_` line that traces back to `requirements.md`.
- Tick a task only once it is verified, meaning `npm run verify` passes or the behaviour is confirmed. Written code alone is not verified. In the same change, update `docs/progress-mindmap.svg` (see `progress-tracking.md`).
- Work on one task at a time and in order, unless the tasks file says otherwise.

Verification

```
npm run lint && npm run typecheck && npm test    # fast loop
npm run verify                                   # full gate incl. build + e2e
```

8.0.2 Architecture rules

Module boundaries Keep map rendering, routing, trip management, file conversion and database access in separate modules. These boundaries are load-bearing, not cosmetic: Garmin map support and alternative routing providers are planned, and they must drop in without touching trip management.

```
src/
  main/        Electron main process. Owns filesystem, database, dialogs, menus.
    db/        drizzle client, migrations, repositories
    services/  use-case layer called by IPC handlers
    ipc/       one module per namespace; validates then delegates
    io/        import/export (Phase 3+), io/garmin/ (Phase 9)
    security/  CSP, navigation and permission guards
    menu/      application menu definition
  preload/     contextBridge surface only. No logic.
  shared/      contracts imported by BOTH processes
    domain/    entity types
    ipc/       channel names + zod request/response schemas
    geo/       pure geometry maths (distance, bbox, simplify)
  renderer/    React. No filesystem, no database, no node builtins.
    features/  library/ map/ properties/ settings/
    stores/    zustand
    components/
```

Rules that follow from this:

- `shared/` must not import `electron`, `node:*`, `react`, or any driver. It is plain TypeScript so both sides and unit tests can use it freely.
- `renderer/` reaches the system only through the preload bridge. Never add `nodeIntegration`, and never pass a path to the renderer expecting it to read it.
- Geometry maths lives in `shared/geo/` as pure functions. Distance, ascent and bbox calculations are used by import, statistics and the map alike, and they must be unit-testable without Electron.
- A provider interface (`MapProvider`, `RoutingProvider`, `GarminMapProvider`) is defined in `shared/` and implemented elsewhere. Trip management code must never reference a concrete provider.

Electron security (non-negotiable)

- `contextIsolation: true`, `nodeIntegration: false`, `sandbox: true`.
- Preload exposes a narrow, explicitly enumerated API. No generic `invoke(channel, args)` passthrough, which would defeat the whole exercise.
- Every IPC handler validates its payload with a zod schema before use, and returns a discriminated `{ ok: true, data } | { ok: false, error }` result rather than throwing across the boundary.
- A strict CSP is set on responses. Tile hosts are allowlisted in `img-src` only; `script-src` stays `'self'`.
- `setWindowOpenHandler` denies all window opens; external links go through `shell.openExternal` after the URL is validated against an allowlist of schemes and hosts.
- Imported file content is untrusted. Never render imported HTML or SVG, and treat GPX/KML description fields as plain text.

Data integrity

- Imported geometry is preserved byte-faithfully in its original form. Never re-sample, simplify or round an import on the way in. Simplification is an explicit, user-initiated, non-destructive action.
- Missing data stays missing. Absent elevation is `null`, never `0`, and elevation gain over a route with no elevation data is `null`, never `0`. Do not synthesise values to make a UI field look populated.
- Derived values (distance, ascent, simplified render geometry) are caches. Mark them as such, recompute from the source of truth, and never let a cache become the only copy of something.

Honesty about limitations If a feature cannot be implemented reliably, say so in the spec and in the UI. Do not ship a control that looks functional but does nothing, and do not claim support for a proprietary format that has not been tested against real sample files. A documented gap is worth more than a misleading placeholder.

Testing Unit tests sit alongside the code they cover (`*.test.ts`). Pure logic — geometry, parsers, statistics, validation — is tested directly with no Electron in the loop. Playwright drives the packaged app for launch, persistence and restart flows. Test fixtures live in `test-data/` and are synthetic; never use real user GPS data in automated tests.

One test geography: the Buxton square Anywhere the geography is load-bearing — snapping, map-region selection, road attributes, the end-to-end fixtures — import it from `@fixtures/places` rather than inventing coordinates. It is a loop round four real market towns:

```
Macclesfield     A537 "Cat and Fiddle"     Buxton

   A523                                        A515

   Leek       A523 / A52           Ashbourne
```

Real roads are what make these tests able to fail usefully. A snapped route must stay near the control points it was given; posted limits must come back as whole mph values; and the loop spans three counties, so no Geofabrik county extract covers it — the case the region picker exists for. Invented coordinates in empty country can only establish that something came back.

Two rules follow:

- **Do not guess a coordinate.** Resolve it against OpenStreetMap. Three of the via points were out by up to 3.4 km when first written, and only the live corridor assertion caught it.
- **A local graph must cover the geography.** Use `A515_TRACE`, which stays inside Derbyshire, so a 44 MB county extract is enough. The Cat and Fiddle crosses into Cheshire and belongs to the live tests.

Format fixtures are the exception and stay where they are. Parsing and round-trip tests use arbitrary coordinates by design, and moving them would churn exact-value assertions for no coverage. The one gap that left — every longitude positive, so a dropped minus sign round-trips unseen — is closed by `generated/peak-district-loop.gpx`.

8.0.3 Licensing and distribution

The application is **GPL-3.0-or-later**. It is free to download and use, with no subscriptions, no mandatory accounts, no advertising, no paid feature tiers and no cloud dependency for core functionality. See ADR 0007.

These are policy, not preferences. Do not introduce a paywall, an account requirement, a feature gate or ad delivery, and do not make core functionality depend on a network service.

Before adding any dependency Check the licence first. Adding it and auditing later is how incompatible code ends up shipped.

```
npm view <package> license
```

Licence	Verdict
MIT, ISC, BSD-2/3-Clause, 0BSD, Apache-2.0, Unlicense, CC0, Blue Oak, MPL-2.0	Fine
GPL-2.0-**or-later**, GPL-3.0, LGPL-2.1+, LGPL-3.0, AGPL-3.0	Fine, now permitted
GPL-2.0-**only**	**Incompatible** with GPL-3.0. Reject.
Hippocratic, ethical-source, or any licence with field-of-use restrictions	**Reject.** Additional restrictions GPL-3.0 §7 forbids, and not open source.
`SEE LICENSE IN …`, `UNLICENSED`, bilateral EULAs	**Reject** unless the text is read and confirmed compatible. Assume not.
No declared licence	**Reject.**

The distinction that catches people out: GPL-2.0-only cannot be combined with GPL-3.0 code. "Or later" is what makes a GPL-2 era dependency usable here.

A build-only dependency that never reaches users does not bind the combined work, so the bar is lower. It still has to permit contribution, and it still goes in the register.

The register `docs/licences/third-party-register.md` must be regenerated whenever dependencies change, and the change committed alongside. A register that lags the lockfile is a compliance failure, not an inconvenience.

Obligations on every release

- Publish the complete corresponding source for the whole combined work, including any bundled GPL components.
- Reproduce third-party notices in the About window, including Electron's and Chromium's.
- Keep `LICENSE` at the repository root unmodified.
- Carry per-file copyright and licence headers on source files.

Distribution channels GitHub releases plus a DMG for direct download. The DMG is also committed under `downloads/` through Git LFS, in the same commit as the source it was built from, so the corresponding source for every committed binary is the commit that carries it. Notarisation is compatible with GPL-3.0 — it attests to the binary rather than restricting the user — but there is no Developer ID yet, so builds are **ad-hoc signed** and notarisation is configured but disabled. See ADR 0009, and document the Gatekeeper consequence honestly rather than hiding it.

Not the Mac App Store. Its terms impose usage and copy restrictions and a DRM layer that conflict with GPL-3.0 §7 and §3. Do not propose it, and do not add App Store targets to the build configuration.

Garmin formats Prioritise functionality and Garmin compatibility over licensing caution, within the law. GPL tooling such as GPSBabel (GPL-2.0-or-later) is now permitted.

Two limits remain absolute: no circumvention of encryption, DRM or map licensing, and no proprietary SDK whose EULA conflicts with GPL-3.0 — which includes Garmin's own FIT SDK. Use the MIT-licensed FIT parser instead.

8.0.4 Progress tracking: the SVG mindmap

`docs/progress-mindmap.svg` is the canonical at-a-glance view of implementation progress across all phases. Keep it current.

When to update it Update the mindmap in the same change as the work it describes:

- a task moves to in-progress or done
- a task is added, removed or re-scoped
- a phase completes
- a risk is retired or a new blocker appears

Never batch mindmap updates up for later. A stale map is worse than none, because it is believed.

How to update it Edit the SVG by hand. It is authored as plain, readable SVG for exactly this reason: no diagram tool, no build step, no generated blob.

Each leaf node carries a `data-status` attribute and takes its fill from the status palette. Change both together, then update the counters in the header band so the summary matches the nodes.

Status	`data-status`	Colour
Done	`done`	`#16a34a`
In progress	`in-progress`	`#f59e0b`
Not started	`todo`	`#94a3b8`
Investigation only	`investigation`	`#8b5cf6`
Blocked / at risk	`blocked`	`#dc2626`

Mark a task `done` only when it is verified — tests passing or behaviour confirmed — not when the code is merely written. `investigation` is for work whose deliverable is a documented finding rather than a feature, which is most of Phase 9, and part of Phase 8.

After editing, render it to confirm it still displays correctly, and surface it in chat as an artifact so the user sees the current state without opening a file.

Scope The mindmap tracks implementation status only. Requirements live in `.kiro/specs/`, decisions in `docs/adr/`. Do not duplicate their content into the diagram; keep leaf labels short and let the specs carry the detail.

8.0.5 Toolchain

Policy: best in class, verified Prefer the current stable major of every dependency. “Current” means the published `latest` tag, not what happens to be in anyone’s memory.

Two hard rules qualify that preference:

1. **Newest is not best if it breaks the toolchain.** Before adopting a major, check `peerDependencies` of everything that consumes it. A version that forces `--legacy-peer-deps`, strands the linter, or has no working native build is not best in class.
2. **Verify, do not assume.** Resolve the real version with `npm view <pkg> version` and prove the set composes with `npm install --dry-run` before writing it into `package.json`.

When a newer major has to be skipped, record why in `docs/adr/` and revisit it rather than leaving it undocumented.

Node.js This project targets **Node.js 26 (LTS "Lithium")**, pinned in `.nvmrc` and in `package.json` `engines`.

- Run `nvm use` in the repo root before any npm/npx command. This shell auto-reverts to a Node 20 default on directory change, and npm 10 has a resolution bug (`Cannot read properties of null (reading 'edgesOut')`) that will bite on this dependency graph. If a command behaves oddly, check `node -v` first.
- Do not downgrade the pin to satisfy a package. If a package only supports an older line, say so and propose an alternative instead of moving the pin.
- CI and local development use the same major.

Host Node only drives build tooling and tests. The Electron main and renderer processes run the Node bundled with Electron (24.22.2, modules ABI 150 for Electron 45), so a native module compiled against the host ABI will not necessarily load. `postinstall` runs `electron-builder install-app-deps` for exactly this reason.

One correction to the usual advice: `better-sqlite3` 13 is **N-API** based, and N-API is ABI-stable across Node and Electron, so it needs no Electron-specific rebuild. Do not assume that of the next native dependency. Verify instead — `npm run verify:native` runs a real SQLite round trip inside Electron, which is the only check that can actually prove it. See ADR 0010.

Electron 45 ships no postinstall script and downloads its binary lazily on first `require('electron')`. `postinstall` therefore runs `install-electron` explicitly, so a clean install is deterministic rather than deferring a ~130 MB download to an arbitrary later command.

Package manager npm, as shipped with Node 26 (npm 11). Commit `package-lock.json`.

npm 11 blocks lifecycle scripts by default. `electron` (binary download) and `better-sqlite3` (native build) both need them. Approve explicitly and record the approved set in `package.json`; never disable the gate wholesale.

Verified version matrix Resolved and dry-run installed together on Node 26.13.0 / npm 11.20.0 with no peer conflicts. Treat this as the baseline; re-verify before changing any line.

Role	Package	Version
Runtime	node	26.13.0 (LTS Lithium)
Desktop shell	electron	45.2.3
Electron build	electron-vite	6.0.0
Packaging	electron-builder	26.18.0
Native rebuild	@electron/rebuild	4.2.1
Bundler	vite	8.3.6
React plugin	@vitejs/plugin-react	6.2.1
UI	react / react-dom	19.3.1
Language	typescript	7.0.2
Styling	tailwindcss + @tailwindcss/vite	4.3.4
Mapping	leaflet	1.9.4
Icons	lucide-react (ISC)	1.72.0
XML parsing	fast-xml-parser	5.12.0
KMZ (zip)	jszip	3.10.2
FIT	fit-file-parser	6.2.7
Database driver	better-sqlite3	13.0.3
ORM	drizzle-orm	0.45.4
Migrations	drizzle-kit	0.31.11
State	zustand	5.0.16

Role	Package	Version
Validation	zod	4.6.7
Unit tests	vitest + @vitest/coverage-v8	5.1.0
E2E tests	playwright	1.65.0
Lint	oxlint	1.91.0

Linting: oxlint, not typescript-eslint TypeScript 7 is the native-port compiler and is the current stable major. `typescript-eslint` 8.72.1 still declares `typescript: >=4.8.4 <6.1.0`, so type-aware ESLint cannot run on TS 7 at all.

Resolution: keep TypeScript 7 and lint with **oxlint**. Type safety comes from `tsc --noEmit` in CI, which is the stronger check anyway. Do not add `typescript-eslint` while this conflict stands, and do not pin back to TypeScript 5.9.3 to accommodate it. See `docs/adr/0002-typescript-7-and-oxlint.md`.

No React binding for Leaflet `react-leaflet` is deliberately absent. It is licensed Hippocratic-2.1, which is incompatible with the project's GPL-3.0 licence and is not OSI-approved open source. Leaflet itself (BSD-2-Clause) is used directly through in-house hooks. Do not reintroduce `react-leaflet` or add another binding without running the licence check first. See ADR 0008.

Tailwind 4 Tailwind 4 is configured through the `@tailwindcss/vite` plugin and CSS-first `@theme` directives. There is no `tailwind.config.js` and no PostCSS pipeline.

No GeoJSON intermediate for KML `@tmcw/togeojson` was considered for KML and rejected. It converts to GeoJSON, which flattens names, descriptions and `ExtendedData` that the import-fidelity rule requires us to preserve or report. GPX, KML and TCX are all parsed directly with `fast-xml-parser`, which keeps one parsing approach and full control over what survives. It also avoids pulling in an XML DOM implementation, since togeojson needs one under Node.

SQLite driver `better-sqlite3` + `drizzle-orm`, not Node's built-in `node:sqlite`. Drizzle has a first-class better-sqlite3 driver and none for `node:sqlite`, and the Electron bundled Node is not the host Node, so the built-in module's availability is not ours to assume. Revisit when Drizzle ships a supported `node:sqlite` driver.

9 Specifications

9.1 Phase 1: Application Foundation — Requirements — SPEC-PHASE-1-FOUNDATION

Version: — · **Status:** unspecified · **Slug:** `phase-1-foundation`

9.1.1 Requirements

Introduction Phase 1 delivers a running, secure, persistent macOS desktop shell: the three-panel BaseCamp-style layout, a trip library backed by SQLite, an OpenStreetMap view behind a provider abstraction, native menus, settings, and backup. No routes, tracks, waypoints, or import/export — those are Phases 2–3.

The goal is a foundation that later phases extend without restructuring. Where Phase 1 deliberately stops short of the source specification, that is recorded in `.kiro/specs/spec-review.md` rather than left implicit.

Phase 1 is complete when all acceptance criteria below are verified and the four source-spec acceptance criteria hold: the app launches on macOS, the map renders, trips can be created, renamed and deleted, and trips survive a restart.

Requirements

Requirement 1: Application launches as a native macOS app **User story:** As a user, I want to launch TrailRT like any other Mac application, so that it feels like a desktop tool rather than a web page.

Acceptance criteria

1. WHEN the application is launched THEN the system SHALL display a single main window with a native macOS title bar and standard window controls.
2. WHEN the window is resized or moved THEN the system SHALL persist the geometry and SHALL restore it on next launch.
3. WHEN the window is closed THEN the system SHALL quit the application, as is conventional for single-window document-less Mac apps.
4. WHEN the application starts THEN the window SHALL become visible only after the first render is ready, with no white flash.
5. IF a second instance is launched THEN the system SHALL focus the existing window instead of opening a second one, because two processes writing one SQLite file is a corruption risk.

Requirement 2: Electron security baseline **User story:** As a user, I want the application to be hardened against malicious file content, so that opening someone else's GPS file cannot compromise my machine.

Acceptance criteria

1. THE renderer SHALL run with `contextIsolation: true`, `nodeIntegration: false` and `sandbox: true`.
2. THE preload script SHALL expose only an explicitly enumerated API surface, and SHALL NOT expose a generic channel-passthrough function.
3. WHEN any IPC message is received THEN the main process SHALL validate the payload against a zod schema before acting on it, and SHALL reject invalid payloads with a structured error.
4. WHEN an IPC handler fails THEN it SHALL return a `{ ok: false, error }` result rather than propagating an exception across the process boundary.
5. THE main process SHALL set a Content-Security-Policy on renderer responses with `script-src 'self'`, and SHALL allowlist tile hosts in `img-src` only.

6. WHEN the renderer attempts to navigate to a non-local URL, or to open a new window THEN the system SHALL deny it.
7. WHEN an external link is activated THEN the system SHALL validate its scheme and host against an allowlist before passing it to `shell.openExternal`.
8. THE renderer SHALL have no direct filesystem or database access by any path.

Requirement 3: Three-panel application layout **User story:** As a BaseCamp user, I want the familiar library / map / properties arrangement, so that I can work without relearning the interface.

Acceptance criteria

1. THE main window SHALL present a left library panel, a centre map panel and a right properties panel.
2. WHEN the user drags a panel divider THEN the system SHALL resize the adjacent panels, SHALL enforce minimum widths, and SHALL persist the sizes across restarts.
3. WHEN the user toggles the library or properties panel THEN the system SHALL hide or show it and the map SHALL reflow to fill the space.
4. WHEN the window is resized THEN the map SHALL resize correctly with no grey tile gutters and no stale tile layout.
5. THE layout SHALL remain usable down to a window width of 1024 px.

Requirement 4: Trip library tree **User story:** As a user, I want a hierarchical library of my trips and folders, so that I can organise my trip planning.

Acceptance criteria

1. THE library SHALL display a tree rooted at “My Collection” containing a Trips branch and a Folders hierarchy.
2. THE tree SHALL include placeholder branches for Routes, Tracks, Waypoints, Imported Data and Recently Used, rendered as cross-trip views per ADR 0004. IF the underlying feature is not yet implemented THEN the branch SHALL state that it arrives in a later phase rather than appearing broken or empty.
3. WHEN the user expands or collapses a node THEN the system SHALL persist that state across restarts.
4. WHEN the user selects a node THEN the properties panel SHALL show that object’s details.
5. THE tree SHALL support sorting trips by name and by date.
6. Sort by distance SHALL NOT appear in Phase 1, because no distance-bearing objects exist until Phase 3.

Requirement 5: Trip and folder management **User story:** As a user, I want to create, rename, move and delete trips and folders, so that I can keep my collection organised.

Acceptance criteria

1. WHEN the user creates a trip THEN the system SHALL persist it with a generated ID, a name, creation and modification timestamps, and SHALL select it in the tree.
2. WHEN the user renames a trip or folder THEN the system SHALL persist the new name and SHALL update the modification timestamp.
3. WHEN the user duplicates a trip THEN the system SHALL create an independent copy including its child objects, named distinctly from the original.
4. WHEN the user deletes a trip or a non-empty folder THEN the system SHALL require confirmation, and the confirmation SHALL state what will be deleted including child counts.
5. WHEN the user moves a trip to a folder, by menu or by drag-and-drop THEN the system SHALL persist the new parent.

6. WHEN the user drags a folder into its own descendant THEN the system SHALL reject the move rather than creating a cycle.
7. WHEN the user archives a trip THEN the system SHALL set its status to archived and SHALL hide it from the default tree while keeping it reachable through a filter.
8. WHEN the user types in the library search field THEN the system SHALL filter the tree to matching trips and folders.
9. THE system SHALL enforce uniqueness of nothing by name; duplicate names are permitted, as in BaseCamp.

Requirement 6: Local SQLite persistence with migrations **User story:** As a user, I want my data stored locally with no account, so that my trips are mine and work offline.

Acceptance criteria

1. THE system SHALL store all user data in a SQLite database in the macOS application support directory.
2. THE system SHALL NOT require any network service or account to start or to operate on existing data.
3. WHEN the application starts THEN it SHALL apply any pending migrations before serving queries, and SHALL fail loudly with an actionable message if migration fails.
4. THE initial migration SHALL create the full schema from section 12 of the source specification: folders, trips, routes, route points, tracks, track points, waypoints, attachments, settings and import history.
5. THE database SHALL use `journal_mode = WAL` and SHALL enforce `foreign_keys = ON`.
6. WHEN a multi-statement mutation is performed THEN it SHALL run in a single transaction.
7. THE schema SHALL store coordinates and elevations in SI units and timestamps as UTC epoch milliseconds.
8. THE schema SHALL make elevation and time columns nullable, and absent values SHALL be stored as NULL rather than zero.

Requirement 7: Persistence across restart **User story:** As a user, I want everything exactly as I left it when I reopen the app.

Acceptance criteria

1. WHEN the user creates, renames, moves or deletes a trip and then restarts the application THEN the change SHALL be present.
2. WHEN the application restarts THEN it SHALL restore window geometry, panel sizes, panel visibility, tree expansion state, theme, selected object and last map position and zoom.
3. IF the application is terminated uncheckedly mid-write THEN the system SHALL recover to the last committed transaction on next start without data loss beyond that transaction.

Requirement 8: OpenStreetMap display behind a provider abstraction **User story:** As a user, I want an interactive map, and as a maintainer I want to add map providers later without touching trip management.

Acceptance criteria

1. THE map SHALL render OpenStreetMap tiles from `https://tile.openstreetmap.org/{z}/{x}/{y}.png` via Leaflet.
2. THE map SHALL display OpenStreetMap attribution visibly at all times.
3. THE tile source SHALL be supplied by a `MapProvider` implementation resolved from configuration, and the tile URL SHALL NOT be hardcoded in a component.
4. THE system SHALL send a descriptive User-Agent identifying the application and version, as the OSM tile usage policy requires.

5. THE system SHALL NOT prefetch, bulk download, or offer any area-download of tiles. Caching SHALL be limited to tiles fetched for user-initiated viewing, honouring HTTP cache headers.
6. THE map SHALL support zoom and pan by mouse, trackpad and keyboard.
7. THE map SHALL provide zoom in, zoom out and a layer selector control.
8. WHEN tiles cannot be fetched THEN the map SHALL remain interactive over cached or blank tiles and SHALL show a non-blocking offline indicator.
9. THE map module SHALL NOT import trip-management or database code.

Requirement 9: Backup and restore **User story:** As a user, I want my trip collection protected against corruption and mistakes.

Acceptance criteria

1. THE system SHALL create an automatic backup of the database on a rolling schedule, retaining a bounded number of generations.
2. WHEN the user requests a manual backup THEN the system SHALL write a consistent snapshot to a user-chosen location using SQLite's backup API, not a naive file copy.
3. WHEN the user restores a backup THEN the system SHALL require confirmation, SHALL back up the current database first, and SHALL reload the application state afterwards.
4. IF a backup file is not a valid TrailRT database THEN the restore SHALL be refused with a clear message and the current database SHALL be untouched.

Requirement 10: Application menus and keyboard shortcuts **User story:** As a Mac user, I want native menus and standard shortcuts.

Acceptance criteria

1. THE system SHALL install a native menu bar with the application menu plus File, Edit, View, Trip and Help, matching section 16 of the source specification.
2. Menu items whose feature arrives in a later phase SHALL be present but disabled, so the menu structure is honest about what is not yet available.
3. THE File menu SHALL expose New Trip, Save a Copy…, Backup, Restore and Quit in Phase 1. Open, Import and Export SHALL be disabled until Phase 3. Per ADR 0001 there is no bare "Save".
4. THE View menu SHALL expose Show/Hide Library, Show/Hide Properties, Map Layers, Zoom In and Zoom Out, all functional in Phase 1.
5. Standard shortcuts SHALL follow macOS conventions, including ⌘N for New Trip, ⌘W, ⌘Q and ⌘, for Settings.
6. Edit menu Undo and Redo SHALL be present and disabled in Phase 1, with the history stack arriving in Phase 2.

Requirement 11: Settings and theming **User story:** As a user, I want to configure the app and match my system appearance.

Acceptance criteria

1. THE system SHALL provide a settings surface reachable from the application menu, covering theme, map provider, units and backup frequency.
2. THE system SHALL support light and dark themes plus a follow-system option, and SHALL apply changes without a restart.
3. WHEN a setting is changed THEN it SHALL be persisted to the settings table immediately.
4. THE units setting SHALL affect display only; stored values SHALL remain SI.

Requirement 12: Build, packaging and distribution **User story:** As a maintainer, I want a reproducible dev loop and a distributable Mac build.

Acceptance criteria

1. THE project SHALL provide a development command with hot module replacement in the renderer and automatic restart of the main process.
2. THE project SHALL build a macOS application bundle and a DMG installer via electron-builder.
3. THE bundle SHALL carry an application icon, a version number and a bundle identifier.
4. THE build SHALL use ad-hoc signing (`mac.identity: "-"`) with hardened runtime retained, since no Developer ID is available initially. The entitlements file SHALL include `com.apple.security.cs.disable-library-validation` and `com.apple.security.cs.allow-jit`, without which an ad-hoc signed Electron app fails to launch. See ADR 0009.
5. THE build configuration SHALL be shaped so that obtaining a Developer ID later requires only supplying a certificate and enabling notarisation, with no change to entitlements, hardened runtime or targets.
6. THE ad-hoc signed bundle SHALL be verified to launch on Apple Silicon and to pass `codesign --verify --deep --strict`.
7. THE README and release notes SHALL document the Gatekeeper block that an unnotarised download produces, including the System Settings → Privacy & Security → Open Anyway sequence. They SHALL NOT instruct users to disable Gatekeeper system-wide.
8. Native modules SHALL be rebuilt against the Electron ABI as part of install.
9. THE build SHALL target both Apple Silicon and Intel.
10. Platform-specific code SHALL be isolated so that a future Windows target does not require restructuring.

Requirement 13: Testing baseline **User story:** As a maintainer, I want the foundation covered by tests so that later phases can change it safely.

Acceptance criteria

1. THE project SHALL run unit tests with Vitest and end-to-end tests with Playwright's Electron support.
2. Unit tests SHALL cover trip and folder creation, renaming, duplication, moving, deletion, cycle rejection, migration application, IPC schema validation and settings persistence.
3. End-to-end tests SHALL cover application launch, map render, the trip create-rename-delete cycle, and persistence across a restart.
4. Tests SHALL run against a temporary database, SHALL NOT touch the real application support directory, and SHALL NOT use real user GPS data.
5. THE type check `tsc --noEmit` and `oxlint` SHALL pass with no errors.

Requirement 14: Licence compliance and attribution **User story:** As a user of free software, I want to know what the application is made of and how to get its source; as a maintainer, I need the GPL-3.0 obligations met from the first release rather than retrofitted.

Acceptance criteria

1. THE repository SHALL contain the full GPL-3.0 licence text at `LICENSE`, unmodified.
2. THE Help menu About window SHALL state the application licence, and SHALL list third-party components with their versions and licences, including Electron's and Chromium's notices.
3. THE About window SHALL present the written offer of source, with a link to the public repository opened via the validated external-link path.
4. Source files SHALL carry a copyright and licence header.
5. THE third-party register at `docs/licences/third-party-register.md` SHALL be regenerated whenever dependencies change, and the regeneration SHALL be part of the same change.
6. THE build SHALL NOT include a Mac App Store target, which is incompatible with GPL-3.0.

7. THE application SHALL NOT require an account, display advertising, gate any feature behind payment, or depend on a network service for core functionality.

9.1.2 Design

Overview Three processes, one contract. The Electron **main** process owns everything privileged: the SQLite file, the filesystem, dialogs and menus. The **renderer** is a React app with no system access. **Preload** is a thin, enumerated bridge between them. A fourth directory, `shared/`, holds the types and zod schemas both sides compile against, so the IPC contract cannot drift.

The foundational constraint is that Phase 1 must not have to be restructured by Phases 2–5. Three decisions carry that weight: the full database schema is created in migration 0001 rather than grown per phase; every mutation goes through a service method with an explicit inverse so the Phase 2 undo stack has something to record; and the map talks to a `MapProvider` interface from day one so Garmin and offline providers are additive.

Product identity Fixed now because these values propagate into the bundle, the DMG, the window title, the app support directory path and the tile User-Agent, and changing them later means migrating a user's database location.

Field	Value
Display name	TrailRT
Bundle identifier	`com.trailrt.app`
npm package name	`trailrt`
App support directory	`~/Library/Application Support/TrailRT/`
Database file	`trailrt.db` (WAL sidecars alongside)
Backups directory	`~/Library/Application Support/TrailRT/backups/`
Attachments directory	`~/Library/Application Support/TrailRT/attachments/`
Tile User-Agent	`TrailRT/<version> (+https://github.com/…)`

`productName` and `appId` are set explicitly in `electron-builder.yml`, so the workspace directory name has no effect on the build.

Signing posture No Developer ID initially, so the build is **ad-hoc signed** with hardened runtime retained (ADR 0009). The detail that matters: an ad-hoc signature has no Apple Team ID, which mismatches the Apple-signed Electron frameworks and causes a launch failure unless the entitlements grant `com.apple.security.cs.disable-library-validation`. Electron additionally needs `com.apple.security.cs.allow-jit`. Both go in `build/entitlements.mac.plist` from the start, which is also what the eventually-signed build wants, so acquiring a certificate later changes only `mac.identity` and the notarisation switch — no restructuring.

The user-visible cost is a Gatekeeper block on a downloaded DMG, and macOS no longer offers the Control-click bypass. That is a documentation obligation, not something to engineer around.

Project structure

```
trailrt/
  electron.vite.config.ts        three builds: main, preload, renderer
  electron-builder.yml
  drizzle.config.ts
  vitest.config.ts
  playwright.config.ts
  .nvmrc                         26
```

```
build/                          icon.icns, entitlements
test-data/                      synthetic fixtures
docs/
    adr/
    progress-mindmap.svg
src/
    main/
        index.ts                app lifecycle, single-instance lock
        window.ts               BrowserWindow + geometry persistence
        security/
            csp.ts              CSP header, tile host allowlist
            navigation.ts       window-open and navigation guards
        db/
            client.ts           better-sqlite3 + drizzle, pragmas
            schema.ts           drizzle table definitions
            migrate.ts          migration runner
            migrations/
            repositories/       trips, folders, settings, importHistory
        services/
            tripService.ts
            folderService.ts
            settingsService.ts
            backupService.ts
        ipc/
            register.ts         wires all namespaces
            handler.ts          validate → delegate → Result wrapper
            trips.ts
            folders.ts
            settings.ts
            backup.ts
        menu/
            appMenu.ts
            actions.ts
    preload/
        index.ts                contextBridge surface, no logic
    shared/
        domain/                 Trip, Folder, Settings, TreeNode types
        ipc/
            channels.ts         channel name constants
            schemas.ts          zod request/response pairs
            result.ts           Result<T> discriminated union
        geo/                    distance, bbox, formatting (pure)
        map/
            MapProvider.ts      interface + OSM descriptor
    renderer/
        main.tsx
        index.css               Tailwind 4 @theme tokens
        app/
            App.tsx
            Layout.tsx          three-panel split, persisted sizes
        features/
            library/            tree, search, context menus, DnD
            map/                MapView, providers, controls
```

```
    properties/        inspector panel
    settings/
  stores/              zustand: library, selection, ui, settings
  components/          Button, Dialog, Tree, Split primitives
  hooks/
```

Boundary rules are enforced by convention plus an oxlint `no-restricted-imports` rule: `shared/` imports nothing privileged, `renderer/` never imports from `main/`, and `features/map/` never imports trip-management or database code.

Database schema SQLite via better-sqlite3, described with Drizzle, migrated with drizzle-kit. Migration 0001 creates the whole schema. Phase 1 only wires repositories for folders, trips, settings and import history, but the tables exist so later phases add behaviour rather than structure.

Conventions throughout: text UUID primary keys from `crypto.randomUUID()`; timestamps as `INTEGER` UTC epoch milliseconds; distances in metres and elevations in metres as `REAL`; `NULL` for genuinely absent data, never a sentinel zero.

```
folders
  id              TEXT PK
  parent_id       TEXT NULL → folders(id) ON DELETE CASCADE
  name            TEXT NOT NULL
  sort_order      INTEGER NOT NULL DEFAULT 0
  created_at      INTEGER NOT NULL
  updated_at      INTEGER NOT NULL
  INDEX (parent_id)

trips
  id              TEXT PK
  folder_id       TEXT NULL → folders(id) ON DELETE SET NULL
  name            TEXT NOT NULL
  description     TEXT NULL
  notes           TEXT NULL
  status          TEXT NOT NULL DEFAULT 'active'   CHECK IN ('active','archived')
  metadata        TEXT NULL                        -- JSON
  created_at      INTEGER NOT NULL
  updated_at      INTEGER NOT NULL
  INDEX (folder_id), INDEX (status), INDEX (updated_at)

routes                                             -- Phase 2
  id              TEXT PK
  trip_id         TEXT NOT NULL → trips(id) ON DELETE CASCADE
  name            TEXT NOT NULL
  colour          TEXT NULL
  source          TEXT NOT NULL DEFAULT 'user'     -- 'user' | 'import'
  point_count     INTEGER NOT NULL DEFAULT 0       -- cache
  distance_m      REAL NULL                        -- cache
  elev_gain_m     REAL NULL                        -- cache, NULL if no ele data
  elev_loss_m     REAL NULL                        -- cache
  created_at      INTEGER NOT NULL
  updated_at      INTEGER NOT NULL
  INDEX (trip_id)

route_points                                       -- Phase 2
```

```
  id              TEXT PK
  route_id        TEXT NOT NULL → routes(id) ON DELETE CASCADE
  seq             INTEGER NOT NULL
  lat             REAL NOT NULL
  lon             REAL NOT NULL
  ele_m           REAL NULL
  name            TEXT NULL
  symbol          TEXT NULL
  notes           TEXT NULL
  UNIQUE (route_id, seq)

tracks                                          -- Phase 3
  id              TEXT PK
  trip_id         TEXT NOT NULL → trips(id) ON DELETE CASCADE
  name            TEXT NOT NULL
  colour          TEXT NULL
  width           INTEGER NULL
  source          TEXT NOT NULL DEFAULT 'import'
  point_count     INTEGER NOT NULL DEFAULT 0      -- cache
  distance_m      REAL NULL                       -- cache
  elev_gain_m     REAL NULL                       -- cache
  elev_loss_m     REAL NULL                       -- cache
  started_at      INTEGER NULL
  ended_at        INTEGER NULL
  render_geometry BLOB NULL                       -- cache, see ADR 0003
  render_tolerance_m REAL NULL
  created_at      INTEGER NOT NULL
  updated_at      INTEGER NOT NULL
  INDEX (trip_id)

track_points                                    -- Phase 3, source of truth
  id              TEXT PK
  track_id        TEXT NOT NULL → tracks(id) ON DELETE CASCADE
  segment_index   INTEGER NOT NULL DEFAULT 0
  seq             INTEGER NOT NULL
  lat             REAL NOT NULL
  lon             REAL NOT NULL
  ele_m           REAL NULL
  recorded_at     INTEGER NULL
  UNIQUE (track_id, segment_index, seq)

track_sources                                   -- Phase 3, ADR 0003
  id              TEXT PK
  track_id        TEXT NOT NULL → tracks(id) ON DELETE CASCADE
  format          TEXT NOT NULL
  content_hash    TEXT NOT NULL
  original_bytes  BLOB NOT NULL
  created_at      INTEGER NOT NULL

waypoint_groups                                 -- Phase 2
  id              TEXT PK
  trip_id         TEXT NOT NULL → trips(id) ON DELETE CASCADE
  name            TEXT NOT NULL
```

```
  created_at      INTEGER NOT NULL

waypoints                                          -- Phase 2
  id              TEXT PK
  trip_id         TEXT NOT NULL → trips(id) ON DELETE CASCADE
  group_id        TEXT NULL → waypoint_groups(id) ON DELETE SET NULL
  name            TEXT NOT NULL
  lat             REAL NOT NULL
  lon             REAL NOT NULL
  ele_m           REAL NULL
  description     TEXT NULL
  symbol          TEXT NULL
  notes           TEXT NULL
  source          TEXT NOT NULL DEFAULT 'user'
  created_at      INTEGER NOT NULL
  updated_at      INTEGER NOT NULL
  INDEX (trip_id), INDEX (lat, lon)

attachments
  id              TEXT PK
  owner_type      TEXT NOT NULL   CHECK IN ('trip','route','track','waypoint')
  owner_id        TEXT NOT NULL
  kind            TEXT NOT NULL   -- 'photo' | 'file'
  relative_path   TEXT NOT NULL   -- under appSupport/attachments
  content_hash    TEXT NOT NULL
  byte_size       INTEGER NOT NULL
  mime_type       TEXT NULL
  created_at      INTEGER NOT NULL
  INDEX (owner_type, owner_id)

settings
  key             TEXT PK
  value           TEXT NOT NULL   -- JSON
  updated_at      INTEGER NOT NULL

import_history
  id              TEXT PK
  source_name     TEXT NOT NULL
  source_format   TEXT NOT NULL
  destination_trip_id TEXT NULL → trips(id) ON DELETE SET NULL
  status          TEXT NOT NULL   CHECK IN ('success','partial','failed')
  report          TEXT NOT NULL   -- JSON: counts, warnings, skipped
  started_at      INTEGER NOT NULL
  finished_at     INTEGER NULL
```

Attachments store bytes on disk and metadata in the database, keeping the database small enough that rolling backups stay cheap. `track_points` as source of truth with `render_geometry` as an explicit, rebuildable cache is ADR 0003. Strict `NOT NULL` + `CASCADE` ownership of child objects is ADR 0004.

IPC contract One pattern, applied uniformly. Channels are namespaced constants; each has a zod request schema and a response type; handlers validate then delegate to a service and never contain business logic.

```ts
// shared/ipc/result.ts
export type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: { code: ErrorCode; message: string; detail?: string } };

// main/ipc/handler.ts - the only place ipcMain.handle is called
export function handle<Req, Res>(
  channel: string,
  schema: ZodType<Req>,
  fn: (req: Req) => Promise<Res> | Res,
): void {
  ipcMain.handle(channel, async (_event, raw): Promise<Result<Res>> => {
    const parsed = schema.safeParse(raw);
    if (!parsed.success) {
      return { ok: false, error: { code: "INVALID_REQUEST", message: "…" } };
    }
    try {
      return { ok: true, data: await fn(parsed.data) };
    } catch (err) {
      return { ok: false, error: toAppError(err) };
    }
  });
}
```

Phase 1 channels: `trips:list|create|rename|update|duplicate|move|archive|delete`, `folders:list|create|rename|move|delete`, `library:tree`, `settings:get|set|all`, `backup:create|restore` `window:state`, `shell:openExternal`.

Preload exposes these as named methods grouped by namespace — `window.trailrt.trips.create(...)` — and nothing else. No generic `invoke`. The renderer never sees a channel string.

Renderer state Four zustand stores with distinct jobs, which keeps re-renders local:

- `libraryStore` — tree data, expansion state, sort, search query
- `selectionStore` — currently selected object reference
- `uiStore` — panel sizes, panel visibility, theme
- `settingsStore` — mirror of persisted settings

Stores hold no business logic. Mutations call the bridge, then apply the returned canonical object. The main process is the single source of truth; stores are a cache of it. No optimistic writes in Phase 1 — correctness over perceived speed while the foundation settles.

Mutation shape for future undo Per ADR 0001 there is no save step, so undo must be a command history. Phase 1 does not build the stack, but every service mutation is written as a method that returns enough information to invert it, and components never write to the database through any other route. Phase 2 adds the stack above this without touching call sites.

Map module

```ts
// shared/map/MapProvider.ts
export interface MapProvider {
  readonly id: string;
  readonly name: string;
  readonly kind: "raster-tiles" | "vector-tiles" | "offline" | "garmin";
  readonly attribution: string;
```

```
  readonly minZoom: number;
  readonly maxZoom: number;
  readonly tileUrlTemplate: string;
  readonly subdomains?: readonly string[];
  readonly requiresApiKey: boolean;
}
```

Phase 1 registers one provider, OpenStreetMap, from a registry module. The component reads whichever provider the settings store names; no tile URL appears in JSX. Adding OpenTopoMap in Phase 4 is a registry entry plus a CSP allowlist entry, with no component change.

`RoutingProvider` and `GarminMapProvider` are deliberately *not* stubbed in Phase 1. An empty interface with no implementation and no caller is a placeholder, and the steering rules forbid those. They are designed in Phases 4 and 5, which the module layout accommodates.

There is no `react-leaflet`. It is Hippocratic-2.1 licensed, which is incompatible with the project's GPL-3.0 licence, so Leaflet (BSD-2-Clause) is driven directly through in-house hooks — see ADR 0008. In practice this costs little, because the lifecycle below is the imperative escape hatch rather than the declarative API that a binding would provide. `features/map/` owns a `useLeafletMap` hook for instance creation, teardown and resize, plus one effect per layer type.

Leaflet lifecycle details that matter: the map instance is created once in a ref and never recreated on re-render; a `ResizeObserver` on the container calls `invalidateSize()` to satisfy Requirement 3.4; tile loading uses Leaflet defaults with no prefetch, and `crossOrigin` is left unset so the browser cache handles tiles normally per Requirement 8.5.

Security implementation CSP is applied via `onHeadersReceived`, assembled from the tile hosts of the registered providers:

```
default-src 'self';
script-src 'self';
style-src 'self' 'unsafe-inline';
img-src 'self' data: blob: https://tile.openstreetmap.org;
connect-src 'self';
font-src 'self';
object-src 'none';
frame-src 'none';
base-uri 'none';
form-action 'none';
```

`style-src 'unsafe-inline'` is required by Leaflet's inline transforms. It is the one relaxation, and it is scoped to styles only.

`setWindowOpenHandler` returns `{ action: 'deny' }` unconditionally. `will-navigate` is cancelled for anything that is not the renderer entry point. External links go through a validator that requires `https:` and a host on the allowlist before reaching `shell.openExternal`.

A single-instance lock guards the database, per Requirement 1.5.

Backup strategy `backupService` uses better-sqlite3's `db.backup()` rather than copying the file, because WAL means the main file alone is not a consistent snapshot. Automatic backups run on a timer and at clean shutdown, writing to `appSupport/backups/` with a bounded generation count. Restore validates the candidate by opening it read-only and checking `user_version` and the presence of expected tables before swapping, and snapshots the current database first.

Technical risks

Risk	Impact	Mitigation
`better-sqlite3` native build against Electron 45 ABI	Blocks everything	`@electron/rebuild` in a postinstall step; verified early as task 2, before any feature work
npm 11 blocks lifecycle scripts; `electron` and `better-sqlite3` need them	Install appears to succeed but produces a broken tree	Explicit approved-scripts list in `package.json`, verified in task 1
Leaflet in React 19 strict mode double-invokes effects	Duplicate map instances, grey tiles	Single imperative instance in a ref, guarded init, `invalidateSize()` on resize
TypeScript 7 ecosystem gaps	Lint or tooling breakage	oxlint instead of typescript-eslint, ADR 0002; `tsc --noEmit` is the type gate
OSM tile policy breach	Access blocked for all users	Descriptive User-Agent, no prefetch, no area download, documented in Requirement 8
Tree drag-and-drop cycles	Corrupt hierarchy	Server-side ancestry check rejects the move, Requirement 5.6; unit tested
Playwright Electron support is less mature than its browser support	Flaky E2E	Keep E2E to the four acceptance flows; put detailed logic in Vitest
Panel and map resize interaction	Visual breakage, Requirement 3.4	`ResizeObserver`, not window resize events
In-house Leaflet integration replaces a community binding	Map bugs we own rather than inherit fixes for	Small surface, covered by Phase 1 map tests; forced by licence, ADR 0008
A dependency added without a licence check	GPL-3.0 violation in a shipped build	Check before adding per `.kiro/steering/licensing.md`; register regenerated in same change

Out of scope for Phase 1 Stated explicitly so their absence is not read as an oversight: routes, tracks, waypoints, import, export, elevation profiles, routing, Garmin anything, undo history, sort by distance, attachments UI, and offline map sources. Menu items for these are present and disabled per Requirement 10.2.

9.1.3 Tasks

Execute in order. Tasks 1–3 de-risk the toolchain before any feature work, by design: a broken native build or a blocked lifecycle script discovered at task 12 would invalidate everything above it.

Update `docs/progress-mindmap.svg` as each task changes status.

☒ 1. Scaffold the project and prove the toolchain installs
- Initialise `package.json` with the verified version matrix from `.kiro/steering/toolchain.md`, `engines.node >= 26`, and `"type": "module"`
- Approve lifecycle scripts for `electron` and `better-sqlite3` explicitly; confirm `node_modules/electron/dist` and a compiled `better_sqlite3.node` both exist after a clean install
- Add `electron.vite.config.ts` with main, preload and renderer builds
- Add `tsconfig.json` with `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, and path aliases for `@main`, `@shared`, `@renderer`
- Add `.oxlintrc.json` including `no-restricted-imports` rules enforcing the module boundaries from `.kiro/steering/architecture.md`
- Add `vitest.config.ts`, `.gitignore`, `.nvmrc`

  - *Requirements: 12.1, 12.8, 13.5*
- ☒ 2. Verify the native module against the Electron ABI
  - Wire `@electron/rebuild` into a postinstall script
  - Open a SQLite database from the main process and read `sqlite_version()`, confirming the module loads under Electron and not merely under host Node
  - *Requirements: 6.1, 12.8*
- ☒ 3. Minimal secure Electron shell
  - `src/main/index.ts`: app lifecycle, single-instance lock, macOS activate handling
  - `src/main/window.ts`: `BrowserWindow` with `contextIsolation: true`, `nodeIntegration: false`, `sandbox: true`; show on `ready-to-show`
  - `src/main/security/csp.ts`: CSP via `onHeadersReceived`, tile hosts derived from the provider registry
  - `src/main/security/navigation.ts`: deny all window opens, cancel external navigation, allowlist-validated `shell.openExternal`
  - Render a placeholder React root to confirm the shell boots
  - *Requirements: 1.1, 1.3, 1.4, 1.5, 2.1, 2.5, 2.6, 2.7*
- ☒ 4. Shared contracts
  - `shared/domain/`: `Trip`, `Folder`, `Settings`, `TreeNode`, `ObjectRef`
  - `shared/ipc/result.ts`: `Result<T>` union and `ErrorCode`
  - `shared/ipc/channels.ts`: namespaced channel constants
  - `shared/ipc/schemas.ts`: zod request schemas for every Phase 1 channel, including coordinate and name constraints
  - `shared/geo/`: haversine distance, bbox, unit formatting — pure, no imports
  - Unit tests for geo maths and schema rejection of malformed payloads
  - *Requirements: 2.3, 6.7, 11.4*
- ☒ 5. Database layer
  - `main/db/client.ts`: better-sqlite3 + drizzle, `journal_mode = WAL`, `foreign_keys = ON`, `synchronous = NORMAL`, database in app support dir
  - `main/db/schema.ts`: all tables from the design document
  - `drizzle.config.ts` and generated migration `0001_initial`
  - `main/db/migrate.ts`: apply pending migrations at startup, fail loudly with an actionable message
  - Unit tests: migration applies to an empty temp database, is idempotent on second run, foreign keys are enforced, nullable elevation stays NULL
  - *Requirements: 6.1, 6.3, 6.4, 6.5, 6.8*
- ☒ 6. Repositories and services for folders and trips
  - `main/db/repositories/`: folders, trips, settings, importHistory
  - `main/services/folderService.ts`: create, rename, move with ancestry cycle rejection, delete with child counting
  - `main/services/tripService.ts`: create, rename, update, duplicate (deep copy), move, archive, delete; transactional; each mutation returns the canonical object and the information needed to invert it
  - `main/services/settingsService.ts`: typed get/set with zod-validated values
  - Unit tests for every service method including cycle rejection and cascade behaviour on delete
  - *Requirements: 5.1–5.9, 6.6, 7.1, 11.3*
- ☒ 7. IPC and preload bridge
  - `main/ipc/handler.ts`: the single `ipcMain.handle` wrapper, validate → delegate → `Result`
  - Namespace modules for trips, folders, library tree, settings, backup, window state, shell
  - `preload/index.ts`: `contextBridge` surface with named methods per namespace and no generic passthrough
  - Typed renderer client wrapping the bridge and unwrapping `Result`
  - Unit tests: invalid payloads are rejected, thrown service errors become `{ ok: false }`

  - *Requirements: 2.2, 2.3, 2.4, 2.8*
- ☒ 8. Three-panel layout
  - Tailwind 4 setup via `@tailwindcss/vite` with `@theme` tokens for light and dark
  - `Layout.tsx`: resizable split panes with minimum widths
  - `uiStore`: panel sizes, visibility, theme; persisted through settings
  - Theme application including follow-system via `nativeTheme`
  - *Requirements: 3.1, 3.2, 3.3, 3.5, 7.2, 11.2*
- ☒ 9. Trip library panel
  - Tree component: My Collection root, Folders hierarchy, Trips branch, and the Routes / Tracks / Waypoints / Imported Data / Recently Used view branches showing a "arrives in a later phase" state
  - Expansion state persistence, name and date sorting, search filtering
  - Context menus: new trip, new folder, rename, duplicate, archive, delete
  - Inline rename, drag-and-drop move with cycle rejection surfaced to the user
  - Destructive-action confirmation dialog stating child counts
  - *Requirements: 4.1–4.6, 5.4, 5.5, 5.6, 5.8, 7.2*
- ☒ 10. Map panel
  - `shared/map/MapProvider.ts` interface and the OSM provider registry entry
  - `MapView`: single Leaflet instance held in a ref, strict-mode safe, `ResizeObserver` driving `invalidateSize()`
  - Attribution control, always visible
  - Descriptive User-Agent on tile requests; no prefetch, no area download
  - Controls: zoom in, zoom out, layer selector
  - Map position and zoom persisted and restored
  - Non-blocking offline indicator when tiles fail
  - *Requirements: 8.1–8.9, 3.4, 7.2*
- ☒ 11. Properties panel
  - Inspector driven by `selectionStore`
  - Trip and folder property editing: name, description, notes; writes through the service layer
  - Empty and multi-selection states
  - *Requirements: 3.1, 4.4, 5.2*
- ☒ 12. Application menus
  - Native menu bar: app, File, Edit, View, Trip, Help per source spec §16
  - Phase 1 functional items: New Trip, Save a Copy…, Backup, Restore, Quit, panel toggles, Map Layers, Zoom In/Out, Settings
  - Later-phase items present and disabled: Open, Import, Export, Undo, Redo, New Route, New Track, New Waypoint, Fit Trip to Screen
  - macOS standard shortcuts: ⌘N, ⌘W, ⌘Q, ⌘,
  - Menu-to-renderer action dispatch
  - *Requirements: 10.1–10.6*
- ☒ 13. Settings surface
  - Settings panel or window: theme, map provider, units, backup frequency
  - Immediate persistence, live application, no restart
  - *Requirements: 11.1–11.4*
- ☒ 14. Backup and restore
  - `backupService` using better-sqlite3 `db.backup()`, not file copy
  - Rolling automatic backups on a timer and at clean shutdown, bounded generations
  - Manual backup to a chosen location; restore with validation, pre-restore snapshot, confirmation, and state reload
  - Unit tests: snapshot restores to an equivalent database; a non-TrailRT file is refused and leaves the current database untouched
  - *Requirements: 9.1–9.4, 7.3*

☒ 15. End-to-end tests
- Playwright Electron harness pointed at a temporary app support directory
- Launch and window visible
- Map renders with tiles and visible attribution
- Create, rename, delete a trip
- Restart and confirm trip plus UI state persistence
- *Requirements: 13.1, 13.3, 13.4, 1.2, 7.1, 7.2*

☒ 16. Licence compliance surface
- About window: application licence, written offer of source, repository link through the validated external-link path
- Third-party component list with versions and licences, including Electron and Chromium notices, generated from the register rather than hand-maintained
- Copyright and licence headers on source files
- A script that regenerates `docs/licences/third-party-register.md`, so it stays in step with the lockfile
- *Requirements: 14.1–14.5*

☒ 17. Build, packaging and ad-hoc signing
- `electron-builder.yml`: macOS target, DMG, arm64 and x64; no Mac App Store target, which GPL-3.0 precludes
- Application icon, version, bundle identifier `com.trailrt.app`
- `mac.identity: "-"` for ad-hoc signing with `hardenedRuntime: true`
- `build/entitlements.mac.plist` with `com.apple.security.cs.disable-library-validation` and `com.apple.security.cs.allow-jit`; without these an ad-hoc signed Electron app will not launch
- Notarisation configured but disabled, with the switch documented, so a Developer ID can be dropped in later without restructuring
- Verify by doing: the bundle launches on Apple Silicon, `codesign --verify --deep --strict` passes, the DMG mounts, and a deliberately quarantined copy reproduces the expected Gatekeeper block so the documented workaround is tested rather than assumed
- Document the System Settings Privacy & Security Open Anyway sequence in the README; do not tell users to disable Gatekeeper
- *Requirements: 12.2–12.7, 12.9, 12.10, 14.6*

☒ 18. Phase 1 verification pass
- `tsc --noEmit`, `oxlint`, Vitest and Playwright all green
- Walk the four source-spec acceptance criteria against the packaged build: launches on macOS, map renders, trip create/rename/delete works, trips persist across restart
- Update the mindmap to show Phase 1 complete
- Record any discovered limitation in `docs/adr/` rather than leaving a placeholder in the UI
- *Requirements: all*

Notes

9.2 Phase 10: Road Trip Planning — Requirements — SPEC-PHASE-10-ROAD-TRIP-PLANNING

Version: — · **Status:** unspecified · **Slug:** phase-10-road-trip-planning

9.2.1 Requirements

Status: first build complete (10–11 November 2026), built ahead of Phases 8 and 9 at the owner's request; see ADR 0019 §1. Feedback Requirements 14–17 built and verified (11 November 2026).

Depends on Phase 4 (routing engine, leg travel-time estimates) and Phase 7 (UK county regions), both shipped. It does **not** depend on Phase 8 or Phase 9, so it can be moved ahead of either without rework. Its position after them is a default, not a dependency.

Introduction A trip today is a container: a name, a description, notes, a status, and an unordered set of routes, tracks and waypoints. It has no dates, no order, and no idea how long anything takes. This phase adds three things the owner asked for:

1. **A calendar-based way to plan a trip.** The trip gets a start date and an ordered itinerary of stops. Each stop has a duration in hours or days. The app works out arrival and departure times from the driving time between stops, and shows the result as a calendar as well as on the map.
2. **Finding points of interest along a route.** Hotels, petrol, food, attractions and the like, found within a chosen distance of the route, listed in the order you reach them, and addable to the trip as stops.
3. **Postcode and place search, offline.** Type a postcode, a town or a street and go there, then drop a waypoint or start a route from it. The owner asked for it to stay offline (10 November 2026), so no online geocoder is offered at all. See Requirement 11.

What the codebase already establishes These findings come from the code as it stands after Phase 7. They constrain the design and are recorded here so the design does not have to rediscover them.

- `Trip` **has no temporal fields** (`src/shared/domain/types.ts`). Routes in a trip carry no sort order (`routes` in `src/main/db/schema.ts` is indexed by `trip_id` only). An itinerary therefore needs new storage. It cannot be inferred from what exists.
- **The region pipeline throws its map data away.** `regionCatalogue.ts` downloads Geofabrik `.osm.pbf` extracts, merges them with `osmium`, builds the routing graph, then deletes every extract. The graph keeps roads, not hotels or fuel stations. Offline POI search therefore has to capture POIs while the extract still exists. `osmium` is already bundled for the merge step, so filtering by tag at build time adds no new tool.
- `connect-src 'self'` **blocks every outbound request** except those the routing provider declares. Nothing in the app can query an online POI service today, and that is the correct default.
- **The waypoint symbol set already has** `lodging`, `fuel`, `food`, `parking`, `viewpoint` **and** `campground`, each paired with a Garmin `<sym>` name (`src/shared/domain/symbols.ts`). POI categories should map onto this set rather than invent a parallel one.
- **Valhalla can resolve time zones** when `timezones.sqlite` sits beside the tiles (`embedded.ts`). Today it is optional and usually absent.
- `planningSpeedKmh` **defaults to 4.5 km/h**, which is walking pace. A road trip timeline built on it would be absurd. Phase 8 already raises this as an unresolved question. This phase forces an answer. Since this was written, Phase 7 Requirement 6.2 added per-mode averages for the properties panel's estimate (`src/shared/routing/travelTime.ts`), with walking still at the planning speed. The timeline should reuse that module, not add a second set of speeds.
- **There is no geocoder anywhere.** Phase 6 left place search out of scope because a geocoder would be a network dependency for a core feature, which ADR 0007 rules out "without an

offline source". Requirement 11 is that offline source. The same region build that has to capture POIs (Requirement 5.1) can capture place and street names in the same pass.

Three distinctions this phase must keep

1. **A plan is not a record.** Arrival and departure times are planned values derived from estimates. They are never presented as measurements, the same rule ADR 0011 §8 applies to route duration.
2. **No result is not proof of absence.** OpenStreetMap coverage of hotels and fuel stations is good in the UK but not complete. "No fuel found in the next 60 km" means none is *mapped*. The UI must say so.
3. **Durations in days are calendar days, not multiples of 24 hours.** "Stay two days" means leave on the morning two dates later, not 48 hours after arriving at 17:40. See Terms.

Terms

Term	Meaning
Itinerary	The ordered list of stops for a trip, with a start date and time. Optional. A trip without one behaves exactly as it does today.
Stop	A place in the itinerary: start, end, or somewhere in between. It references a location and has a duration.
Duration	How long is spent at a stop. Given in **hours** (e.g. 1.5 h for lunch) or **days** (e.g. 2 days at a hotel).
Day-stay	A stop whose duration is in days. You leave on the date N days after arriving, at the stop's departure time.
Leg	The driving between two consecutive stops. It has a distance and an estimated driving time.
Timeline	The derived arrival and departure time of every stop. A cache, always recomputable from the itinerary and the legs.
Corridor	The area within a chosen distance of a route, used to find POIs along it.
POI	A point of interest from OpenStreetMap, such as a hotel, fuel station or café. It is not part of the library until the user adds it.
POI snapshot	The set of POIs captured from a region's extract, with the date of that extract.
Gazetteer	The local, searchable index of postcodes, place names and street names that Requirement 11 searches. Built on this machine; never queried remotely.
Postcode centre	The single point a postcode dataset gives for a postcode. It stands for the postcode's area, which may cover several streets, and is not an address.

User stories

1. As someone planning a five-day road trip, I want to set a start date and see each day laid out on a calendar, so I can check the plan fits the week I have off.
2. As the same traveller, I want to say “two nights in Buxton” or “an hour at the Cat and Fiddle” and have every later time move when I change it, so I am not doing the arithmetic by hand.
3. As a rider on a motorcycle with a 200 km range, I want to see where the fuel stations are along tomorrow’s route, and be warned about any stretch with none mapped, so I do not run dry on a moor.
4. As a traveller looking for somewhere to stay, I want to see the hotels near where a day’s driving ends, and add one as the night’s stop.
5. As someone who has to be somewhere at a given time, I want to see when I will arrive at each stop, and be told when a day’s driving runs long.
6. As a privacy-conscious user, I want POI search to work from data already on my machine, and to be asked before my route is sent anywhere.
7. As a user who shares plans, I want to export the itinerary to my calendar app, and the route and stops to my GPS.
8. As an existing user who never plans by date, I want trips to work exactly as they do now.
9. As someone planning from a hotel confirmation, I want to type its postcode and go there on the map, without the search leaving my machine.

Requirements

Requirement 1: Trip schedule Acceptance criteria

1. A trip SHALL optionally carry an itinerary with a start date and start time. A trip without one SHALL behave exactly as it does today: same tree, same properties panel, same export. This is a regression requirement and SHALL be covered by tests that would fail if it broke.
2. THE user SHALL be able to add an itinerary to an existing trip, and remove one, without changing the trip’s routes, tracks or waypoints. Removing an itinerary SHALL be undoable.
3. THE trip’s end date SHALL be derived from the timeline, not entered separately, so the two cannot disagree.
4. THE library tree SHALL show a scheduled trip’s date range as secondary text, using the existing `TreeNode.detail` field.
5. Archiving a scheduled trip SHALL keep its itinerary intact.

Requirement 2: Stops and durations Acceptance criteria

1. An itinerary SHALL be an ordered list of stops, with a start stop and an end stop. The start and end MAY be the same place, so a round trip is a loop.
2. A stop SHALL reference a location in the trip, either an existing waypoint or a new one created for it. The itinerary SHALL NOT keep a second copy of the coordinates.
3. Each intermediate stop SHALL have a duration in **hours** (with minute precision) or in **days** (a whole number of days).
4. A day-stay SHALL also have a departure time of day, defaulting to a user-set setting (e.g. 09:00). The user SHALL be able to change it per stop.
5. A stop SHALL accept an optional fixed time, such as “arrive by 15:00” or “depart at 10:30”. WHERE a fixed time conflicts with the timeline THE system SHALL show the conflict and SHALL NOT silently move the fixed time.
6. THE user SHALL be able to add, remove and reorder stops. Each change SHALL be undoable, using the existing command and batch-undo machinery (ADR 0014).
7. Stop notes SHALL be plain text, like every other notes field.

Requirement 3: Legs and the timeline Acceptance criteria

1. Each leg SHALL be linked to a route in the trip. A leg with no route SHALL show as unplanned. Its driving time SHALL be unknown, not zero, and every later time SHALL show as unknown too.
2. Leg driving time SHALL come from the routing engine's estimate for that route when one exists. Otherwise it SHALL come from distance and a planning speed. Either way it SHALL be labelled an estimate, per ADR 0011 §8. *Amended at design:* no saved route stores an engine estimate (ADR 0011 §8), so every leg is timed from distance and the per-mode average in `shared/routing/travelTime.ts`, and the UI names the speed used.
3. THE planning speed used for road trips SHALL suit the travel mode. The current 4.5 km/h default SHALL NOT be used to time a driving leg.
4. Arrival at a stop SHALL be the previous departure plus the leg's driving time. Departure SHALL be arrival plus the duration, for an hours stop, or the departure time on the date N days later, for a day-stay.
5. THE timeline SHALL be derived, SHALL be recomputed whenever a stop, a duration or a linked route's geometry changes, and SHALL NOT be the only copy of anything, per the derived-values rule in `.kiro/steering/architecture.md`.
6. THE user SHALL be able to set a maximum driving time per day. Days over it SHALL be flagged, and the system SHALL NOT re-plan the trip to fix them.
7. WHERE a day-stay's departure would come before its arrival (e.g. arriving at 11:00 on the last day, with a 09:00 departure time) THE system SHALL flag it rather than wrap to the next day. *Amended at design:* a day-stay is a whole number of days, at least one, counted from the arrival date, so it always leaves on a later date than it arrives and cannot depart first. The case a user can hit is a fixed departure on an hours stop that is earlier than the arrival. That is flagged, and the time is not wrapped. See design §14.
8. Times SHALL be shown in the local time of the stop they belong to. WHERE a leg crosses a time-zone boundary, or a stay spans a daylight-saving change, THE timeline SHALL stay correct and SHALL show the change. Hours durations SHALL be elapsed time; day-stays SHALL be wall-clock dates.
9. Time-zone lookup SHALL work offline.

Requirement 4: Calendar view Acceptance criteria

1. A scheduled trip SHALL be viewable as a calendar with at least **day**, **week** and **month** views.
2. THE calendar SHALL show driving legs and stops as distinct blocks, with driving time, stop name and duration on each.
3. Each day SHALL show a summary: driving distance, driving time, and where the night is spent.
4. Selecting a block in the calendar SHALL select the matching route or waypoint on the map and in the properties panel, using the existing selection store. Selecting a stop on the map SHALL show it in the calendar.
5. THE user SHALL be able to change a stop's duration from the calendar, for example by dragging the end of a block. It SHALL go through the same undoable command as editing it in the properties panel.
6. Times that depend on an unknown leg SHALL be shown as unknown in the calendar, not placed at a guessed position.
7. THE calendar SHALL be built in-house or from a dependency that passes the licence check in `.kiro/steering/licensing.md`. It SHALL be usable at both desktop and narrow window widths.
8. THE calendar SHALL honour the app's light and dark themes and the user's unit setting.

Requirement 5: POI data source Acceptance criteria

1. **Offline first.** WHEN a region is downloaded or imported THE system SHALL extract POIs in the supported categories from the same `.osm.pbf` before it is deleted, and SHALL store them

locally as a POI snapshot. The design SHALL measure the snapshot size for the three-county Buxton set and SHALL state it.
2. POIs mapped as areas (many hotels, fuel stations and supermarkets are building outlines, not points) SHALL be included as a representative point. They SHALL NOT be dropped because they are not nodes.
3. THE snapshot SHALL record the date of the extract it came from, and the UI SHALL show that date wherever POIs are listed.
4. Rebuilding or replacing the routing regions SHALL rebuild the snapshot in the same pass. Regions loaded before this phase SHALL be flagged as having no POI data, with a one-press rebuild. They SHALL NOT be shown as having no POIs.
5. **Online search is optional and off by default.** IF an online source is offered (e.g. an Overpass API instance) THEN it SHALL need explicit opt-in. The opt-in SHALL name the service and say what is sent: the route corridor, which reveals where the user plans to go. It SHALL respect that service's usage policy and rate limits, and SHALL be added to the CSP deliberately, not by widening `connect-src`.
6. POI search SHALL work fully with no network connection, using the snapshot.
7. THE mapping from OSM tags to POI categories SHALL be defined in one place in `src/shared/`, with the reasoning for each tag.

Requirement 6: Finding POIs along a route Acceptance criteria

1. THE user SHALL be able to search for POIs along a route, or along the whole itinerary, by category. At minimum: **accommodation** (hotel, motel, guest house, hostel, campsite), **fuel**, **EV charging**, **food and drink**, **attractions and viewpoints**, **parking**, **toilets**, **supermarkets**, and **vehicle services**.
2. THE corridor width SHALL be user-selectable, with at least 0.5, 1, 2, 5 and 10 km options.
3. Results SHALL be listed in the order they are reached along the route. Each SHALL show its distance from the start and its straight-line distance off the route. For a scheduled trip it SHALL also show the day and the estimated time it is reached.
4. THE user SHALL be able to limit a search to a section of the route: between two distances, within one itinerary day, or near a chosen point (for example "hotels within 10 km of where day 2 ends").
5. WHERE the user asks for it THE system SHALL compute the real road detour to a POI with the routing engine, and SHALL label it an estimate. Straight-line off-route distance SHALL NOT be presented as a detour.
6. Results SHALL be shown on the map as a separate layer, with a category symbol, clustered at low zoom, and toggled like the existing layers.
7. Corridor geometry SHALL be computed by pure functions in `src/shared/geo/`, unit-testable without Electron.
8. Search over a 500 km route SHALL NOT block the UI, and SHALL be cancellable.

Requirement 7: POI details and honesty Acceptance criteria

1. A POI SHALL show its name, category and any of these OSM attributes that are present: address, phone, website, opening hours, brand, star rating, and fuel types or charging connectors.
2. Every attribute SHALL be treated as untrusted plain text. A website SHALL open only through `shell.openExternal` after the existing scheme and host validation.
3. WHERE opening hours can be parsed THE system SHALL say whether the POI is expected to be open at the planned arrival time. Opening hours that cannot be parsed SHALL be shown as raw text and SHALL NOT be guessed at.
4. A missing attribute SHALL be shown as absent, not defaulted. A fuel station with no opening hours is not "open 24 hours".

5. An empty result SHALL say “no mapped in OpenStreetMap within ”, along with the snapshot date. It SHALL NOT say there are none.
6. THE system SHALL NOT show prices, room availability, ratings from other sites or booking links. See Out of scope.
7. OpenStreetMap attribution SHALL be shown wherever POI data is displayed.

Requirement 8: Adding a POI to the trip Acceptance criteria

1. THE user SHALL be able to add a POI to the trip as a waypoint, or as an itinerary stop (which creates the waypoint as well).
2. THE waypoint SHALL take the POI’s name and coordinates, a symbol mapped from its category through `src/shared/domain/symbols.ts`, and a plain-text description built from its attributes.
3. THE waypoint SHALL record where it came from (the OSM element type and id, and the snapshot date) so it can be traced, and so a later version can offer to refresh it. Its `source` SHALL be distinct from a hand-placed waypoint.
4. Adding a POI as a stop between two others SHALL offer to re-route the affected leg through it, using the existing snapping and shaping-point model (Phase 6). It SHALL NOT edit the route silently.
5. Adding, and re-routing through, a POI SHALL each be undoable.

Requirement 9: Fuel range check Acceptance criteria

1. THE user SHALL be able to set a vehicle range in the trip or in settings, and SHALL be able to leave it unset. Unset means no check runs. It does not mean unlimited range.
2. WITH a range set THE system SHALL flag every stretch of the itinerary longer than that range with no mapped fuel station within the corridor. The corridor width SHALL be shown alongside the warning.
3. THE check SHALL allow for a user-set reserve (default e.g. 20%).
4. THE warning SHALL carry the honesty caveat from Requirement 7.5. It SHALL NOT claim the stretch has no fuel.
5. THE check SHALL handle EV charging the same way, using the charging category, when the trip’s vehicle is set to electric.

Requirement 10: Export Acceptance criteria

1. A scheduled trip SHALL export to **iCalendar** (`.ics`, RFC 5545): one event per stop and, optionally, per driving leg, with correct time zones. Times derived from unknown legs SHALL be left out, and the export SHALL say so.
2. THE GPX export of a scheduled trip SHALL write stops as waypoints in itinerary order, and the routes in leg order, through the existing GPX writer with no new extensions.
3. Planned times SHALL NOT be written as GPX `<time>` elements. In GPX those mean recorded time, and a device or app would read a plan as a recording.
4. Itinerary fields GPX cannot carry (duration, fixed times, day-stays) SHALL be reported as unmapped on export, following the Phase 3 precedent.
5. Importing iCalendar is out of scope.

Requirement 11: Postcode and place search (offline) The owner asked for address and postcode lookup, and for it to stay offline. Every search is answered from data on this machine. There is no online fallback, so the CSP does not change for search.

Acceptance criteria

1. THE map SHALL have a search box that accepts a UK postcode, a place name (city, town, village, hamlet or named locality), a street name with or without a town, or the name of a POI

in the snapshot. Results SHALL appear as the user types.

2. Choosing a result SHALL centre the map on it and mark it. It SHALL then offer: add as a waypoint, start a New Route here, and, for a scheduled trip, add as a stop. None of these SHALL happen without that choice.
3. **Offline only.** No search text and no result SHALL leave the machine. THE system SHALL NOT offer an online geocoder, and SHALL NOT widen `connect-src` for search. Search SHALL work with no network connection.
4. **Postcodes** SHALL come from Ordnance Survey **Code-Point Open**, under the Open Government Licence v3.0. Entry SHALL ignore case and spacing: `de451aa`, `DE45 1AA` and `de45  1aa` are the same postcode.
5. A full postcode SHALL resolve to its postcode centre. The result SHALL be labelled as the postcode's centre, not as an address, because one postcode can cover several streets.
6. A partial postcode (an outcode such as `DE45`, or a sector such as `DE45 1`) SHALL resolve to the area it covers, labelled approximate. It SHALL NOT be shown as a single address.
7. A postcode that is well formed but not in the data SHALL say "not found in the postcode data dated ". It SHALL NOT be replaced with a nearby postcode. A malformed entry SHALL be treated as a place-name search, not rejected.
8. **Coverage SHALL be stated.** Code-Point Open covers Great Britain only. A Northern Ireland (`BT`), Isle of Man (`IM`), Jersey (`JE`) or Guernsey (`GY`) postcode SHALL produce a message saying that area's postcodes are not included. It SHALL NOT say "not found".
9. THE postcode data SHALL be obtained with one press from Settings, with its size shown before it is fetched, the same pattern as map regions. It SHALL then be stored locally, with its release date shown in Settings and in search results. Re-fetching a newer release SHALL replace it in one step. Fetching it is the only network action in this requirement, and it is taken by the user.
10. **Places and streets** SHALL be captured from the OpenStreetMap extract during the region build, in the same pass as the POI snapshot (Requirement 5.1), before the extract is deleted. That covers `place=*` features, named roads, and `addr:*` tags where they are present. A road mapped as many ways SHALL appear once per street per settlement, not once per way.
11. Place and street search SHALL cover the loaded map regions only. WHEN it finds nothing THE system SHALL name the loaded regions, so "no match" reads as "not in your regions" and not as "does not exist". Postcode search covers all of Great Britain regardless of the loaded regions.
12. **House numbers.** WHERE OpenStreetMap has a building with that `addr:housenumber` and `addr:street`, a full address SHALL resolve to that building. Otherwise it SHALL fall back to the street, and the result SHALL say that only the street was matched. THE system SHALL NOT interpolate a house number along a street, because a position invented between two numbers looks exact and is not.
13. Results SHALL be ranked with an exact postcode first, then places by kind (city, town, village, hamlet) and distance from the map centre. Names that occur more than once (there are many places called Newtown) SHALL show their county, so the user can tell them apart.
14. Matching SHALL ignore case and punctuation, SHALL match from the start of a word as the user types, and SHALL treat common abbreviations as their full form (`St` / `Street`, `Rd` / `Road`).
15. Data from Ordnance Survey SHALL carry the attribution its licence requires, including the Royal Mail and National Statistics statements that Code-Point Open specifies. OpenStreetMap-derived names SHALL carry OpenStreetMap attribution. Both SHALL appear in the About window and beside search results, and both datasets SHALL be added to the third-party register.
16. Names and addresses SHALL be rendered as plain text.
17. Postcode parsing and normalisation SHALL be pure functions in `src/shared/`. Search SHALL sit behind an interface in `src/shared/`, like `MapProvider` and `RoutingProvider`, with the index in `src/main/`. Trip management SHALL NOT reference a concrete search source.

Requirement 12: Security, privacy and licensing Acceptance criteria

1. No route, corridor or location data SHALL leave the machine unless the user has opted in to an online POI source, per Requirement 5.5.
2. POI data derived from OpenStreetMap SHALL meet ODbL attribution requirements. The design SHALL state whether the local snapshot or exported waypoints form a derivative database, and what follows from that.
3. Any new dependency (calendar, iCalendar writer, opening-hours parser, POI icons) SHALL pass the licence check and be added to the third-party register in the same change.
4. Imported POI text SHALL be rendered as text, never as HTML.
5. THE application SHALL remain fully functional for all existing features with no network connection.

Requirement 13: Testing Acceptance criteria

1. Unit tests SHALL cover timeline arithmetic: hours stops, day-stays, fixed times and their conflicts, unknown legs propagating forward, the daily driving cap, and a stay across the UK clock change (BST ends 25 October 2026).
2. Unit tests SHALL cover corridor search: distance along route, off-route distance, section limits, ordering, and an empty result.
3. Unit tests SHALL cover the fuel range check: a stretch flagged, a stretch cleared by a station, unset range running no check, and the reserve.
4. Tests SHALL cover POI extraction from an extract, including a POI mapped as an area.
5. Geography-dependent fixtures SHALL use the Buxton square from `@fixtures/places`. POI fixtures SHALL be resolved against OpenStreetMap, not invented, per the "do not guess a coordinate" rule.
6. Tests SHALL assert that a trip with no itinerary is unchanged in the tree, the properties panel, and GPX export.
7. iCalendar output SHALL be checked against RFC 5545 structure, including time-zone components.
8. End-to-end tests SHALL cover building an itinerary, viewing it as a calendar, finding fuel and a hotel along a route, adding the hotel as a stop, and restarting the app with everything intact.
9. Unit tests SHALL cover postcode normalisation and classification: spacing and case, full postcodes, outcodes and sectors, malformed entries falling through to place search, and the `BT`, `IM`, `JE` and `GY` coverage messages.
10. Unit tests SHALL cover place search: prefix matching, abbreviations, ranking, a name that occurs in two counties, a house number that resolves to a building, and one that falls back to its street.
11. Gazetteer fixtures SHALL use the Buxton square. Postcode fixtures SHALL be a small excerpt of the real Code-Point Open release, committed with its attribution, and SHALL NOT be invented. Place and street fixtures SHALL be resolved against OpenStreetMap.
12. An end-to-end test SHALL search for a postcode and for a place with all network access blocked, centre the map on each, and add one as a waypoint.

Feedback after the first build (11 November 2026) The owner used the built phase and asked for four changes. They are added as Requirements 14–17 rather than folded into the earlier ones, so the record shows what was built first and what changed after.

Requirement 14: The trip in the library summarises it The library tree is where a trip is found and recognised. For a dated trip it should say what the plan is, not only list what the trip contains.

Acceptance criteria

1. A scheduled trip's row SHALL show its dates, its number of days and its total planned distance,

e.g. “23–25 Oct 2026 · 3 days · 98 km”. A distance that is partly unknown SHALL be marked as such, not shown as a total.
2. A scheduled trip’s children SHALL follow the plan: one group per day, each holding that day’s stops and drives in the order they happen, with their local times. Selecting a stop SHALL select its waypoint; selecting a drive SHALL select its route.
3. Routes, tracks and waypoints of a scheduled trip that are not in the plan SHALL still be listed, after the days, under a group that says they are not in the plan. Nothing in the trip SHALL disappear from the tree.
4. A stop or drive whose time is unknown SHALL be listed under an “Unknown times” group in plan order, not placed on a guessed day.
5. A trip with no itinerary SHALL be shown exactly as before (Requirement 1.1).

Requirement 15: Stops are joined by road automatically Acceptance criteria

1. Each leg SHALL have a mode: **by road** (the default), **a chosen route**, or **other transport** (“don’t link”).
2. WHEN stops are added, removed or reordered, or a stop’s place moves, THE system SHALL plan every by-road leg that has no route, or whose route no longer joins its two stops, with the routing provider, save the route and link it. The structural change and the planning SHALL be one undoable step.
3. A route the system planned this way SHALL be recorded as planned for the itinerary. WHEN it no longer joins its stops THE system SHALL replace it, and the replacement SHALL be undoable. A route the user drew, imported or chose SHALL never be modified or deleted by this.
4. WHERE no routing provider is available THE system SHALL leave the leg unplanned and SHALL say why (for example, “No map regions cover this drive”), rather than drawing a straight line that looks planned.
5. An **other transport** leg SHALL take a kind (ferry, train, flight, or other, with a free-text label) and an optional travel time. With a time it SHALL be timed by that; without one its arrival and every later time SHALL be unknown. It SHALL NOT be searched for POIs or counted as driving, and the range check SHALL treat it as a stretch it cannot check.
6. Choosing a route by hand SHALL switch that leg to **a chosen route**; choosing **by road** again SHALL return it to automatic planning.

Requirement 16: Panels resize and hide Acceptance criteria

1. The panel below the map (profile, calendar, places along the route) SHALL be resizable in height by dragging its top edge, and SHALL remember its height between sessions.
2. The library (left), properties (right) and the panel below the map (bottom) SHALL each have a visible control to hide and show it, and their state SHALL persist.
3. Hiding a panel SHALL give its space to the map. Resizing SHALL keep a minimum size for the map, so it can never be squeezed to nothing.
4. The controls SHALL be usable from the keyboard and named for screen readers.

Requirement 17: Icon buttons with hints Acceptance criteria

1. Action buttons in the trip, route, itinerary, calendar, along-the-route and search panels SHALL be compact icon buttons.
2. Each SHALL show a hint naming its action when hovered or focused, and SHALL carry that name as its accessible label. An icon with no name is not acceptable.
3. A one-off call to action whose meaning an icon cannot carry (“Plan by date”, “Download”) MAY stay as text.
4. Icons SHALL come from an open-source set whose licence passes the check in `.kiro/steering/licensing.md`, recorded in the register.

Non-functional requirements

1. **No regression.** Every existing test still passes, and `npm run verify` stays green.
2. **Search is instant.** Results for each typed character SHALL appear within 100 ms on the three-county Buxton region set plus all of Great Britain's postcodes. The design SHALL measure this, and the size of the gazetteer, and state both.
3. **Module boundaries hold.** Timeline maths and corridor geometry are pure and live in `shared/`. POI extraction and storage live in `main/`. The renderer reaches them only through the preload bridge. Trip management does not reference a concrete POI source. A `PoiProvider` interface sits in `shared/`, like `MapProvider` and `RoutingProvider`.
4. **Errors cross IPC as results, never exceptions.**
5. **No new dependency without a licence check** recorded in the register.
6. **Missing stays missing.** Unknown driving times, absent attributes and unchecked fuel ranges are shown as unknown, never as zero, open, or fine.

Out of scope

- **Hotel booking, prices and room availability.** Every source needs a commercial API with an account, an API key and terms that restrict use and redistribution. That breaks the no-account, no-cloud-dependency policy in ADR 0007. The app can show the hotel's own website from OSM, and nothing more.
- **Live fuel prices.** The UK has been moving towards a statutory open fuel-price data scheme. Whether a usable, licence-compatible feed exists that the app can read offline is a question for the design stage, not a requirement of this phase.
- **Traffic-aware timing.** Driving times are the engine's free-flow estimates.
- **Automatic itinerary generation**, such as "plan me five days around the Peak District". The user builds the plan; the app does the arithmetic and the searching.
- **Sharing or syncing an itinerary between people.** Export covers sharing.
- **Online geocoding.** The owner chose offline only. Nominatim, Photon and postcodes.io are all left out, including as an opt-in.
- **Complete house-number coverage.** The one complete UK address list, OS AddressBase, is paid and licensed per use, which ADR 0007 rules out. Search matches house numbers only where OpenStreetMap maps them (Requirement 11.12).
- **Reverse lookup**: clicking the map to get an address. The gazetteer could support it later; nobody has asked for it yet.

Unresolved questions for the design stage

1. **Legs and routes.** Is each leg its own route, or is the itinerary one multi-stop route whose stops are shaping points (ADR 0017)? One route per leg fits a per-day calendar and a per-leg re-route. One route fits how a device navigates a whole day. The answer decides the storage.
2. **Where do POI snapshots live?** In the main SQLite database, keyed to the loaded regions, or in a separate file beside the routing tiles that is replaced when they are? The second keeps the library database small, and ties POIs to the graph they were built with.
3. **Online POI source: include it at all?** Offline from the snapshot covers the UK regions. An Overpass option adds a network path, a consent flow and a usage-policy obligation for the cases a snapshot cannot cover, such as a route outside the loaded regions. It could be left out until it is asked for.
4. **Time-zone data.** Ship `timezones.sqlite` with the routing tiles, which Valhalla already supports, or bundle a small boundary dataset in `shared/`. For a UK-only region set, every stop is Europe/London, but the design should not hard-code that.
5. **Planning speed.** Replace the single `planningSpeedKmh` with per-mode defaults (shared with the Phase 8 question), or rely on the engine's leg times and use the planning speed only for legs without a route.

6. **Should the calendar be its own pane, or a mode of the map area?** This is a layout decision that touches `PanelLayout`.
7. **OS Open Names for places outside the loaded regions?** It is a free Ordnance Survey gazetteer of places, roads and postcodes for all of Great Britain, under the same licence as Code-Point Open. It would make place search work before any region is downloaded, at the cost of a second, much larger download. Requirement 11.11 currently limits place search to the loaded regions instead.
8. **Bundle the postcode data, or fetch it?** Requirement 11.9 fetches it on request. Bundling would make search work on first launch, but would put the dataset in every installer and need an app release for each refresh. The design should measure it before deciding.
9. **Northern Ireland postcodes.** Is there a source whose licence is compatible with GPL-3.0 redistribution? If not, Requirement 11.8 stands and `BT` postcodes stay unsupported, with that said in the UI.

9.2.2 Design

Written against the code after Phase 7, with the uncommitted Phase 7 follow-up work (`shared/routing/travelTime.ts`, the profile drawer target) in the tree. ADR 0019 records the decisions that are more than implementation.

Built out of order. The owner asked for this phase to be built on 10 November 2026, ahead of Phases 8 and 9. It depends only on Phases 4 and 7, which are complete, so nothing it builds on is unfinished. The README's rule against designing ahead exists to stop guessing at unsettled interfaces; the interfaces this phase uses are all shipped.

0. Measurements Taken on 10 November 2026 against real data. They settle the storage questions before any code is written.

Region data. Derbyshire, Cheshire and Staffordshire from Geofabrik (44, 39 and 31 MB), merged with the bundled `osmium` 1.19.1:

Step	Time	Output
`osmium merge`	1.6 s	113 MB `.osm.pbf`
`osmium tags-filter` (POI, place, highway, `addr:housenumber`)	1.6 s	47 MB
`osmium export -f geojsonseq`	4.1 s	1,219,532 features, 459 MB (streamed, never kept)
Ingest into SQLite (prototype)	3.7 s	see below

Kept after ingest: 35,567 POIs, 2,588 places, 64,428 street rows (39,303 street names, one row per name per settlement), 574,150 addressed buildings. On disk: POIs 10.0 MB, addresses 23.9 MB, streets 3.1 MB, search tokens and index 6.2 MB — **about 43 MB for the three counties**, against a 141 MB routing graph for the same set. The design trims POI tags to the attributes in Requirement 7.1, so production should come in below that.

Measured again once built (task 11). The production extract and ingest on the same three counties: `osmium` filter and export 5.4 s, ingest 3.2 s, peak memory about 300 MB, **37.9 MB** on disk. It keeps 26,388 POIs (private and unnamed kerbside parking excluded), 2,587 places, 62,151 streets and 300,963 addressed buildings. The prototype's 574,150 addresses had counted every building twice, once as a line and once as an area, which is the duplication in fact 1 below. Addresses are written as they stream past rather than held, so memory does not grow with them.

Postcodes. Code-Point Open, release 2026-11 (copyright date 19 October 2026): `codepo_gb.zip`, 14.5 MB, no API key, MD5 published in the OS downloads listing. 120 CSV files, 1,749,385 postcodes. Ingest 1.9 s. Stored as postcode lat/lon: **32.6 MB**.

Search latency. Prefix lookups on an indexed token table (`tok >= ? AND tok < ? || '\uffff'`) plus a postcode prefix query: 0.04–0.29 ms per keystroke on the combined set. The 100 ms budget (non-functional requirement 2) is spent on IPC and rendering, not the query.

Measured again once built (task 13), with `npm run verify:search`: the production ingest of all 1,749,385 postcodes takes 3.5 s and stores 51.2 MB. Postcodes are kept in their spaced form, which costs more than the prototype's compact keys and buys exact outcode prefixes. Across every keystroke of seven typed phrases (postcodes, streets, house numbers and places) against that plus the three-county place data, the **worst was 12.4 ms** in the search service.

Two facts about the data that the design has to handle:

1. `osmium export` emits every closed way **twice**: once as a LineString and once as a (Multi)Polygon. Buxton's Palace Hotel (`way/117882931`) appears in both forms. Ingest deduplicates by `type/id` and keeps the area.
2. 888 Code-Point postcodes have positional quality **90** and coordinates `0, 0`. They are real postcodes with no location. They are stored and reported as "has no location in the postcode data". Plotting them at 0, 0 would put a Scottish postcode off the coast of Ghana.

1. Invariants this phase collides with, and how each is resolved

Invariant	Collision	Resolution
Derived values are caches (architecture.md)	A timeline looks like stored data	Only inputs are stored: start date/time, stops, durations, fixed times, leg links. The timeline is computed on every read by a pure function (§3) and never written
Missing stays missing	Unlinked legs, unknown zones, unparsed hours	`null` throughout. An unknown leg makes every later time `null`, a zone not in the table is reported as unknown, and unparsed opening hours are shown raw
Engine durations stay out of stored data (ADR 0011 §8)	Requirement 3.2 asks for the engine's leg time	No engine time is stored on a saved route, deliberately. Legs are timed by `travelTime.travelSpeedKmh` for the route's mode, the module Phase 7 added for exactly this. Requirement 3.2's fallback branch is therefore the only branch, and the UI says which speed was used

Invariant	Collision	Resolution
No cloud dependency for core functionality (ADR 0007)	POIs and search need data	Both are built on this machine from data the user fetches once: the region extract already being downloaded, and Code-Point Open. Nothing is queried remotely. No online POI source is offered (§6.4)
Imported data is untrusted	OSM tags become UI text	Rendered as text nodes only; `website` goes through the existing `openExternal` allowlist
No generic IPC passthrough	Many new calls	One namespace per area (`itinerary`, `places`, `search`), each method named, each with a zod schema
Every mutation is undoable or says why not	Itinerary edits, POI-to-stop, re-routing	Every itinerary command returns an inverse. Compound actions (add POI as stop, re-route through a stop) are `batch` commands, so one ⌘Z unwinds them (ADR 0014)
`connect-src 'self'`	Postcode download	The renderer never fetches. The main process downloads from `api.os.uk`, following exactly one named cross-origin redirect (§7.2). The CSP does not change
Module boundaries	Trip management must not know a concrete POI or search source	`PoiProvider` and `PlaceSearchProvider` interfaces in `shared/`; `itineraryService` imports neither concrete implementation

2. The itinerary model

2.1 Legs are routes, one per leg Unresolved question 1 is settled for **one route per leg**:

- The calendar is per day, and a day is one or more legs. A leg that is a section of a longer route would need a "from control point *i* to *j*" reference, and Phase 6 has shown that control-point indices move under shaping edits.
- Re-routing one leg through a new stop becomes "create two routes", never "edit a route" (§9.3). That satisfies Requirement 8.4's "SHALL NOT edit the route silently" by construction.
- A device that navigates a whole day still gets the day: GPX export writes the routes in leg order (§10.2).

2.2 Storage — migration `0004_itinerary.sql`

```
CREATE TABLE itineraries (
  trip_id            TEXT PRIMARY KEY REFERENCES trips(id) ON DELETE CASCADE,
  start_date         TEXT NOT NULL,      -- 'YYYY-MM-DD', wall clock
```

```
  start_time          TEXT NOT NULL,       -- 'HH:mm', wall clock at the first stop
  default_depart      TEXT NOT NULL DEFAULT '09:00',
  max_drive_minutes   INTEGER,             -- null: no daily cap
  range_km            REAL,                -- null: no range check (Req 9.1)
  reserve_pct         INTEGER NOT NULL DEFAULT 20,
  energy              TEXT NOT NULL DEFAULT 'fuel',    -- 'fuel' | 'electric'
  created_at, updated_at
);

CREATE TABLE itinerary_stops (
  id                  TEXT PRIMARY KEY,
  trip_id             TEXT NOT NULL REFERENCES itineraries(trip_id) ON DELETE CASCADE,
  seq                 INTEGER NOT NULL,
  waypoint_id         TEXT NOT NULL,       -- no FK, see below
  leg_route_id        TEXT,                -- route of the leg ARRIVING here; no FK
  duration_kind       TEXT NOT NULL,       -- 'hours' | 'days'
  duration_minutes    INTEGER,             -- hours stops
  duration_days       INTEGER,             -- day-stays
  depart_time         TEXT,                -- day-stays: 'HH:mm'; null uses default
  arrive_by           TEXT,                -- 'HH:mm' check
  depart_at           TEXT,                -- 'HH:mm' fixed departure, hours stops
  notes               TEXT,
  created_at, updated_at
);
```

Why the stop does not foreign-key its waypoint or route. A cascade would delete the stop when its waypoint is deleted, and undoing the waypoint deletion (`waypoint:restore`, which reuses the id) would not bring the stop back. A `SET NULL` would keep the stop but lose the link for good. With a plain id, a deleted waypoint shows as "location deleted", and undo heals the link automatically because the id comes back. The same applies to `leg_route_id` and `route:restore`. This follows the "no orphan surprises" reading of ADR 0001.

The first stop has no arriving leg and no duration that matters; its `duration_kind` is stored as `hours` with `0` minutes. The last stop's duration is ignored for the same reason. The start and end may be the same waypoint, so a loop needs no special case.

Removing an itinerary (Requirement 1.2) deletes the `itineraries` row and cascades the stops. Its inverse, `itinerary:restore`, carries the full snapshot: one row and at most a few dozen stops. It is bounded (§2.4), so removal is undoable. Waypoints and routes are untouched.

2.3 Fixed times Both are wall-clock times on a specific date, so they cannot drift:

- `arrive_by` is a check. A conflict is raised when the computed arrival is later than `arrive_by` on the arrival date. Nothing moves.
- `depart_at`, on an hours stop, fixes departure to that time on the arrival date. You wait if early. A conflict is raised if arrival plus the stop's duration is later; the departure stays at `depart_at`, as Requirement 2.5 requires.
- A **day-stay** already has a departure time, so `depart_at` is not offered for one.

2.4 Bounds At most **100 stops** per itinerary, enforced by the schema. That bounds the restore payload and keeps the calendar readable. It is far beyond the length of any road trip a person plans by hand.

3. Time

3.1 `shared/time/zoned.ts` (pure) There is no date library in the project and none is needed. `Intl` with full ICU is present in both Node 26 and Electron 45:

- `zoneOffsetMs(utcMs, zone)` from `Intl.DateTimeFormat(…).formatToParts`
- `wallClockToUtc({ date, time }, zone)` solves for the instant. In a **spring-forward gap**, a time that does not exist moves forward by the gap. In an **autumn overlap**, the earlier instant is chosen. Both are deterministic and tested.
- `utcToWallClock(utcMs, zone)` → `{ date, time, offsetMinutes }`
- `addDays(date, n)` on `YYYY-MM-DD` strings, with no time involved, so daylight saving cannot touch it

An hours duration is elapsed time: `departure = arrival + minutes`. A day-stay is wall-clock: `departure = wallClockToUtc(addDays(arrivalLocalDate, n), departTime)`. A stay across 25 October 2026, when BST ends, therefore leaves at 09:00 GMT on the right morning. A 48-hour block would leave at 08:00.

3.2 `shared/time/timeZones.ts` (pure) — offline zone lookup `TimeZoneResolver = (point) => string | null`. The default resolver tests a small table of polygons:

Polygon	Zone	Source
`UK_BOUNDARY` (existing, 150 points)	`Europe/London`	Geofabrik `united-kingdom`, already in `shared/geo/ukBoundary.ts`

Every region the picker offers is in the UK, apart from “Ireland and Northern Ireland”, whose Northern Ireland part is inside `UK_BOUNDARY`. A point outside every polygon resolves to `null`. The timeline then shows that stop’s times in the computer’s own zone, labelled “time zone unknown here — shown in ”. Times are not silently assumed to be London’s (unresolved question 4). Adding a country is a new row in the table, not a code change. Valhalla’s `timezones.sqlite` was rejected: building it downloads the timezone-boundary-builder release (about 60 MB), which is a network dependency for a 2-line answer in the UK.

The **calendar zone** is the first stop’s zone (or the computer’s, if unknown). Days are laid out on that zone’s dates. A stop in another zone shows its own local time and offset beside it (Requirement 3.8).

3.3 `shared/itinerary/timeline.ts` (pure)

```
computeTimeline(input: TimelineInput, resolveZone: TimeZoneResolver): Timeline
```

`TimelineInput` holds the itinerary row, the stops in order, each with its resolved waypoint position (or `null` if deleted) and its leg: `{ routeId, distanceM, mode } | null`. The output gives each stop `arrival`, `departure` (UTC ms or `null`), its zone, its local wall clock, and a list of flags:

Flag	When
`unplanned-leg`	no linked route, or the route has no distance
`unknown-time`	an earlier leg is unplanned
`location-deleted`	waypoint id does not resolve
`arrive-by-missed`	§2.3
`depart-at-too-early`	§2.3
`zone-unknown`	§3.2

Requirement 3.7's "departure before arrival" flag is covered by `depart-at-too-early`; see §14 for why a day-stay cannot produce it.

Leg driving time is `distanceM / travelSpeedKmh(mode, planningSpeedKmh)`. The mode is the route's recorded `transport_mode`, or the settings mode for a hand-drawn route. Walking legs therefore still use the planning speed, and a driving leg never does (Requirement 3.3).

Days. `days[]` groups everything by local date in the calendar zone. Each day has its driving distance, driving time (a leg counts on its departure date), the stop where the night is spent (the stop you are at at local midnight), and `over-daily-cap` when driving time exceeds `max_drive_minutes`. Nothing is re-planned (Requirement 3.6).

The end date of the trip is the last stop's arrival date. It is derived, never stored (Requirement 1.3), and is `null` when the last arrival is unknown.

4. Itinerary service, commands and IPC `main/services/itineraryService.ts`, following `waypointService`:

Operation	Inverse
`create(tripId, start)`	`itinerary:remove`
`remove(tripId)`	`itinerary:restore` (snapshot)
`update(tripId, patch)` (start, default depart, cap, range, reserve, energy)	`itinerary:update` (previous values)
`addStop(tripId, { waypointId, at? , duration… })`	`itinerary:removeStop`
`removeStop(stopId)`	`itinerary:restoreStop` (row + position)
`moveStop(stopId, toIndex)`	`itinerary:moveStop` (previous index)
`updateStop(stopId, patch)`	`itinerary:updateStop` (previous values)
`linkLeg(stopId, routeId | null)`	`itinerary:linkLeg` (previous)

All are registered in `commands.ts` with labels ("itinerary edit", "stop move", …). `get(tripId)` returns `ItineraryView`: the stored rows, the resolved waypoints and legs, and `computeTimeline`'s output. It is computed in the main process, which has the routes' distances and modes, and is sent whole.

`addStop` validates that the waypoint belongs to the trip. `linkLeg` validates that the route does. Moving a waypoint or route to another trip does not cascade; the stop then shows `location-deleted` or `unplanned-leg`, because from this trip's point of view that is what happened.

IPC namespace `itinerary`: `get`, `create`, `remove`, `update`, `addStop`, `removeStop`, `moveStop`, `updateStop`, `linkLeg`, `planLeg` (§9.2), `addPoiStop` (§9.1), `exportIcs` (§10.1). Each has a zod schema in `shared/ipc/schemas.ts`. `HH:mm` and `YYYY-MM-DD` are validated as real times and dates, not only by pattern.

Library tree. `libraryService` gives a scheduled trip `detail` = its date range ("12–16 May 2027"), or "from 12 May 2027" when the end is unknown. A trip with no itinerary gets no `detail`, exactly as now. That is a regression test (Requirement 1.1).

5. Places: one build, three uses

5.1 Pipeline `main/places/` is new and owns everything about place data:

```
region extract(s)    [merge]    extract.osm.pbf    valhalla_build_tiles  (as now)
                                                       osmium tags-filter
                                                          osmium export geojsonseq
                                                          ingest   places.sqlite
```

- `extract.ts`: two child processes, the same pattern and kill-on-abort handling as `mergeExtracts.ts`. The filter expression is generated from the category table (§6.1), so the table stays the one place a tag is named.
- `ingest.ts`: streams the GeoJSON lines; never holds the file. Writes `places.sqlite.tmp`, then renames over `places.sqlite` only once the graph build also succeeded. A failed or cancelled build leaves the previous snapshot in place alongside the previous graph.
- `downloadAndBuild` and the `.osm.pbf` import both run it, between the merge and the deletion in the existing `finally`. A new `BuildSteps.places` step keeps the orchestration testable offline, as the other steps are.
- Progress stage "Reading places" goes through the existing region progress events.

5.2 `places.sqlite` Beside the tiles in `userData/valhalla/`. It is replaced when they are, so the POIs always describe the same data as the graph (unresolved question 2). It is a cache of the extract: rebuildable, not backed up, and not the only copy of anything (the extract on Geofabrik is).

Table	Columns	Notes
`meta`	`key`, `value`	`extract_date` (from the PBF header `osmosis_replication_timestamp`, else the file's download date, labelled so), `regions` (names), `coverage` (region rings as JSON), `schema`
`poi`	`id`, `osm` (`node/123`), `category`, `name`, `lat`, `lon`, `attrs` (JSON of the Requirement 7.1 fields only)	index on `(lat, lon)`
`place`	`id`, `kind`, `name`, `county`, `lat`, `lon`	`place=*` nodes
`street`	`id`, `name`, `settlement`, `county`, `lat`, `lon`	one row per name per settlement
`address`	`id`, `street_key`, `number`, `name`, `lat`, `lon`	`addr:housenumber` + `addr:street`
`token`	`token`, `kind`, `ref`	index on `token`; kind 1 place, 2 street, 3 POI

Area POIs (Requirement 5.2) take the centroid of their outer ring. A centroid can fall outside an L-shaped building, but for "where is this hotel" on a map that is metres. It is better than the first vertex, which is a corner.

One street per settlement (Requirement 11.10): each named way is assigned to its nearest `city|town|village|hamlet|suburb` node by equirectangular distance. Ways with the same name and settlement collapse to one row, with the vertex nearest their combined centre as the point.

County (Requirement 11.13): the Geofabrik region whose polygon contains the feature, from the regions recorded in `meta`. A user-imported `.osm.pbf` has no regions, so its features have no county, and duplicates are told apart by settlement instead.

5.3 Regions built before this phase `placesState()` reports `missing` when the graph exists but `places.sqlite` does not. The POI panel and the search box then say "Place data was not built with your regions" and offer **Rebuild regions**. That reruns `downloadAndBuild` with `region.json`'s recorded `regionIds` (Requirement 5.4). It never shows "no POIs". An imported-file graph records no region ids, so it offers re-import instead.

6. Points of interest

6.1 `shared/poi/categories.ts` The single tag table (Requirement 5.7). Each row: category id, label, the OSM tags that select it, the waypoint symbol from `symbols.ts`, and a one-line reason.

Category	OSM tags	Symbol
`accommodation`	`tourism=hotel,motel,guest_house,hostel,camp_site,caravan_site,cha`	`lodging` (camp/caravan sites)
`fuel`	`amenity=fuel`	`fuel`
`charging`	`amenity=charging_station`	`fuel` (no charging symbol exists; adding one would need a Garmin name, which has none for it)
`food`	`amenity=restaurant,cafe,pub,fast_food,bar`	`food`
`attractions`	`tourism=attraction,museum,viewpoint,gallery,zoo,theme_park,picnic historic=castle,ruins,monument`	`scenic`
`parking`	`amenity=parking,motorcycle_parking`	`parking`
`toilets`	`amenity=toilets`	`pin`
`supermarket`	`shop=supermarket,convenience`	`pin`
`vehicle`	`shop=car_repair,motorcycle,tyres`	`pin`

`parking` excludes `access=private` and `parking=underground`-only rows with no name, which are 15,653 of the 35,567 in the measurement and would bury every other result. `attrs` keeps only `addr:*`, `phone`/`contact:phone`, `website`/`contact:website`, `opening_hours`, `brand`, `stars`, `fuel:*=yes` and `socket:*`.

6.2 `shared/poi/openingHours.ts` An in-house parser for the common subset: `24/7`; rules of weekday ranges and lists (`Mo-Fr`, `Sa,Su`, `PH` ignored with a note); time ranges, including past midnight (`18:00-02:00`); `off`; rules separated by `;`, later rules overriding earlier ones for the days they name. Anything else returns `unparsed`, and the raw text is shown (Requirement 7.3). The full `opening_hours` grammar has seasons, sunrise and holidays; the library that implements it was not adopted, because an unparsed answer shown raw is honest, and a 30 kB dependency for the long tail is not needed. `openAt(parsed, wallClock)` → `open | closed | unknown`.

6.3 Corridor search — `shared/geo/corridor.ts` (pure)

- `buildLineIndex(points)`: cumulative distance per vertex, plus a grid of segments in ~1 km cells (equirectangular metres around the line's mean latitude).
- `project(index, point, maxM)` → `{ alongM, offM }` or `null`: the nearest segment within `maxM`, using only the cells around the point.
- `corridorBox(points, widthM)` → the bounding box to query candidates with.
- `filterSection(results, { fromM, toM } | { near: point, radiusM })`.

The main process queries `poi` by the corridor box, projects each candidate, keeps those with `offM` `width`, and sorts by `alongM`. For an itinerary, the leg lines are concatenated in order, and each result also gets its leg, day and estimated pass time from the timeline (`departure + alongWithinLeg / speed`). A 500 km route at ~20,000 vertices projects ~30,000 candidates in tens of milliseconds. To keep the main process responsive regardless, the loop yields every 2,000 candidates (`setImmediate`) and checks an `AbortSignal` (Requirement 6.8). The renderer cancels by request id. No worker thread is introduced: chunked yielding is enough at these sizes, and Phase 8 records that a worker would be its own decision.

Road detour (Requirement 6.5), on request for one result: two `routeLeg` calls through the existing provider (route point before → POI, POI → route point after). The detour is their sum minus the route's distance between those two points, labelled "estimated detour by road". It needs a provider; without one, the button says why it is unavailable.

6.4 No online POI source Unresolved question 3: **left out.** The owner chose offline only for search, and the same reasoning applies here. Requirement 5.5 is conditional ("IF an online source is offered"), so not offering one satisfies it. A route outside the loaded regions gets "outside your map regions — POIs not checked there" for that stretch (§6.5), not an online lookup.

6.5 Coverage `meta.coverage` holds the loaded regions' rings. A stretch of route outside them is reported as **not covered**. A POI search there never says "none found", and the range check (§8) marks the stretch "not checked".

7. Search

7.1 `shared/search/` (pure)

- `postcode.ts`: `normalisePostcode` (case, all whitespace); `classify(text)` → `full` | `sector` | `outcode` | `uncovered` (`BT`, `IM`, `JE`, `GY`) | `none`. It follows the published UK postcode format (`A9 9AA`, `A99 9AA`, `A9A 9AA`, `AA9 9AA`, `AA99 9AA`, `AA9A 9AA`), plus `GIR 0AA`.
- `text.ts`: case and punctuation folding, diacritics stripped, tokens, and the abbreviation table (`st`→`street`, `rd`→`road`, `ave`→`avenue`, `ln`→`lane`, `dr`→`drive`, `cl`→`close`, `cres`→`crescent`, `pl`→`place`, `sq`→`square`, `ter`→`terrace`, `gdns`→`gardens`, `pk`→`park`). `St` before a name, as in `St Ann's Well`, matches both `saint` and `street`, and ranking sorts it out.
- `rank.ts`: exact postcode, then places by kind (city > town > village > hamlet > suburb > locality), then streets, then POIs, each by distance from the map centre. Duplicate display names get `, <county>` (or settlement).
- `PlaceSearchProvider` in `shared/search/PlaceSearchProvider.ts`; `PoiProvider` in `shared/poi/PoiProvider.ts`.

7.2 Postcode data — `main/places/postcodes.ts`

- **Listing**: `GET https://api.os.uk/downloads/v1/products/CodePointOpen` (version, e.g. `2026-11`) and `…/downloads` (size and MD5 of the GB CSV). Settings shows the size before anything is fetched (Requirement 11.9).
- **Download**: the `…downloads?area=GB&format=CSV&redirect` URL answers 307 to `omseprd1stdstordownlo` The Geofabrik client refuses any cross-origin redirect. This client allows **exactly** that one origin pair, and verifies the MD5 from the listing before using the file. The policy stays narrow: one named host, one hop.
- **Ingest**: `jszip` (already a dependency for KMZ) reads the zip in memory (14.5 MB). Each `Data/CSV/*.csv` row → normalised postcode, OSGB36 eastings/northings → WGS84 by `shared/geo/osgb.ts`, quality code. Quality 90 is stored with null coordinates. `Doc/metadata.txt` gives the copyright date, which is shown as the release date. Written to `userData/gazetteer/postcodes.sqlite.tmp`, then renamed (Requirement 11.9, "replace it in one step").

7.3 `shared/geo/osgb.ts` Inverse Transverse Mercator on the Airy 1830 ellipsoid (OS National Grid parameters), then the OSGB36 → WGS84 Helmert transform (OS published parameters, accuracy about 5 m). A postcode centre is already an approximation of an area, so that is ample. OSTN15, the centimetre-grade transform, is a 15 MB grid and is not needed. Tested against the worked example in OS's *A guide to coordinate systems in Great Britain* for the projection step, and against Buxton-area postcode centres for the whole chain, within 10 m.

7.4 `main/places/searchService.ts` Opens both databases read-only with `better-sqlite3`. A query:

1. If `classify` says `uncovered`, return the coverage message (Requirement 11.8). If it says `full`, do an exact lookup: found, found with no location, or "not found in the postcode data dated …" (Requirement 11.7). For `sector`/`outcode`, return the mean of its postcodes' points plus their bounding box, labelled approximate (Requirement 11.6).
2. Otherwise tokenise. Look up the last token as a prefix and the others as whole or abbreviated tokens. Intersect by ref, rank, and take 20.
3. A house number followed by a street ("12 Spring Gardens") looks up `address`. Found → that building. Not found → the street, flagged "only the street was matched" (Requirement 11.12). Nothing is interpolated.
4. No result → "no match in " (Requirement 11.11).

Answers carry the dataset dates for the attribution line beside results.

8. Range check — `shared/itinerary/rangeCheck.ts` (pure) Input: the concatenated itinerary line's length, the `alongM` of every fuel (or `charging`, when `energy = 'electric'`) POI within the corridor, the uncovered stretches, `range_km` and `reserve_pct`. Usable range = `range × (1 − reserve)`. It **assumes a full tank at the start**, and the warning says so. Every gap between consecutive refuelling points (start, POIs, end) that is longer than the usable range is a warning, with the corridor width, and the caveat "no fuel station mapped in OpenStreetMap" (Requirement 9.4). A gap that touches an uncovered stretch is "not checked", not a warning and not a pass. An unset range returns `{ checked: false }`, which the UI shows as "no range set", not as "fine" (Requirement 9.1).

9. From POI to stop

9.1 Adding `itinerary.addPoiStop` and `places.addPoiWaypoint` take a POI id. The main process looks the POI up again, so the renderer never supplies coordinates or tags it could have altered. A waypoint is created with:

- `name` = POI name (or the category label when unnamed)
- `symbol` from §6.1
- `description` = plain-text lines from `attrs` ("Address: …", "Phone: …")
- `source = 'poi'`, a new waypoint source distinct from `user` and `import` (Requirement 8.3). `GeometrySource` stays `user | import` for routes and tracks; waypoints get `WaypointSource = GeometrySource | 'poi'`. The waypoints table has no CHECK on `source`, so this needs no migration.
- `metadata.osm = { type, id, snapshotDate }`, using the existing metadata column.

Waypoints from postcode and place search are `source = 'user'`: the user chose to put a point there, and the dataset only helped find it. They go through the existing `waypoints.create` channel, which takes no metadata, so no lookup provenance is recorded for them. That is a deliberate simplification recorded here, not an omission.

Found while building (task 15). Undoing a waypoint deletion dropped the waypoint's `metadata`, because the deletion's inverse never carried it. That was already true for imported waypoints' preserved GPX extras. With POIs it would also have turned `poi` back into `user` and lost the OSM provenance. The inverse now carries both, and `waypoint:restore` writes them back.

`addPoiStop` is one `batch` command: create the waypoint, then add the stop at the chosen position. One undo removes both.

9.2 Planning a leg `itinerary.planLeg(stopId)`: `routeLeg` from the previous stop's waypoint to this stop's, in the settings transport mode, then `createSnappedRoute` with two via points and the previewed leg, named " □ ", then `linkLeg`. One batch. It needs a routing provider, and the button says so when there is none. Without one, the user can still link an existing route, or draw one by hand and link it.

9.3 Re-routing through a new stop (Requirement 8.4) When a stop is inserted between A and C and the leg into C had a route R, the UI offers **Re-route through** . Accepting plans A□new and new□C (§9.2) and links them. R is **not modified and not deleted**: it stays in the trip, unlinked, and the confirmation says so. One batch; one undo.

10. Export

10.1 iCalendar — `main/io/ics/write.ts` Hand-written RFC 5545, with no dependency:

- `VCALENDAR` with `PRODID:-//TrailRT//Road trip//EN`, `VERSION:2.0`, `CALSCALE:GREGORIAN`.
- One `VTIMEZONE` per zone used. Its `STANDARD`/`DAYLIGHT` sub-components are generated from the actual transitions in the trip's date span, found by scanning `zoneOffsetMs` day by day and bisecting to the minute, each with explicit `DTSTART`, `TZOFFSETFROM` and `TZOFFSETTO`. No RRULE is needed for a bounded span. Stops with an unknown zone are written in UTC (…`Z`).
- One `VEVENT` per stop (`DTSTART;TZID=` arrival, `DTEND` departure, `SUMMARY` stop name, `LOCATION`, `GEO:lat;lon`, `DESCRIPTION` duration and notes), and optionally one per leg ("Drive: A □ B, 84 km, est. 1 h 32 min"). `UID` = `<stop id>@trailrt`, `DTSTAMP` = export time.
- Text escaped (`\\`, `\;`, `\,`, `\n`), lines folded at 75 octets on UTF-8 boundaries, CRLF line endings.
- Stops whose times are unknown are left out, and the export result lists them (Requirement 10.1).

10.2 GPX for a scheduled trip `exportService.gather` for a single scheduled trip orders waypoints with stop waypoints first, in itinerary order, then the rest by name. It orders routes with linked leg routes first, in leg order, then the rest. **No `<time>` is written for a plan** (Requirement 10.3). The writer only writes time for track points and for waypoints with a recorded time, which stops do not have. That is asserted in a test, so it stays true. The plan's warnings gain one line: "Itinerary details (start, durations, fixed times) are not part of GPX and are not written" (Requirement 10.4). A trip with no itinerary exports byte-for-byte as before, which is a regression test.

11. Renderer

11.1 Itinerary in the trip properties `features/itinerary/ItinerarySection.tsx` inside the trip's properties: "Plan by date" creates an itinerary. It contains: start date and time; the stop list (drag handle or □/□ buttons; the drag is not the only way); per stop a duration control (hours and minutes, or days), departure time for day-stays, arrive-by and depart-at, notes; per leg the linked route (a select of the trip's routes), "Plan this leg" and the leg's distance and estimated time; then the trip-level settings (daily driving cap, range, reserve, fuel or electric). Flags render as inline messages on the stop or leg they belong to. "Add stop" offers the trip's waypoints. "Remove itinerary" asks first, then reports "Itinerary removed — Undo".

11.2 Calendar drawer `features/itinerary/CalendarPanel.tsx` opens in the drawer below the map, the slot the profile uses (`uiStore.drawer`). That answers unresolved question 6 without touching `PanelLayout`. The map stays visible while the calendar is open, which Requirement 4.4's "select in the calendar, see it on the map" needs. In-house, with no calendar dependency:

- **Day**: an hour grid (00–24) for one date, blocks positioned by local time.
- **Week**: seven such columns.

- **Month**: a date grid with each day's summary (distance, driving time, overnight stop) and its blocks as chips.
- Leg blocks and stop blocks are visibly distinct (filled vs outlined) and labelled ("Drive · 1 h 32 min est.", "Buxton · 2 days").
- Selecting a block sets the selection store (route for a leg, waypoint for a stop). The map's existing selection highlighting does the rest. Selecting a waypoint that is a stop scrolls the calendar to it.
- **Dragging the bottom edge** of a stop block changes its duration in 15-minute steps (hours stops) or whole days (day-stays), committed on release through `itinerary.updateStop`. The keyboard path is the duration field.
- A block whose times are unknown is not placed on the grid. It is listed in an "Unknown times" strip above the grid (Requirement 4.6).
- Colours come from the existing theme tokens, so light and dark work (Requirement 4.8). Below 640 px, week view becomes a stacked day list.

11.3 Along-the-route panel `features/poi/PoiPanel.tsx`, opened from a route's or the itinerary's properties ("Find along route"). It has: category checkboxes; a corridor width select (0.5, 1, 2, 5, 10 km); a section select (whole route, a day, between distances, near a stop with a radius); and a result list (name, category, km from start, km off route, day and est. time when scheduled, opening status at that time). Each row has **Show**, **Add as waypoint** and **Add as stop**, and **Road detour** on request. The empty state reads "No fuel mapped in OpenStreetMap within 2 km · data from 3 Nov 2026". Range warnings sit at the top. OSM attribution sits at the bottom.

`features/map/usePoiLayer.ts`: results as category-symbol markers on a dedicated pane, clustered by `shared/geo/cluster.ts` (pure grid clustering by screen cell, recomputed on zoom). Toggled with the other layers via a new `layerVisibility.pois` key, default on, so existing settings stay valid.

11.4 Search box `features/search/SearchBox.tsx`, top-left of the map. Debounced at 60 ms and keyed by request sequence, so a slow answer never replaces a newer one. Keyboard navigable. Choosing a result centres the map, drops a temporary marker, and shows the actions: Add as waypoint; Start New Route here; Add as stop (scheduled trip only). Nothing happens until one is chosen (Requirement 11.2). Approximate results draw their area's bounding box. Attribution and dataset dates sit under the list.

11.5 Settings and About Settings gains a **Postcodes** section: dataset status (none, or "Code-Point Open, 19 Oct 2026, 1,749,385 postcodes"), download size, Download / Update with progress, and the licence statements. The **Map regions** section gains the places status and Rebuild. About gains Ordnance Survey and Royal Mail / National Statistics statements, and OSM's ODbL statement for place data.

12. Licensing

- **Code-Point Open**: Open Government Licence v3.0. It allows copying, adaptation and commercial use, and requires the attribution statements. It is compatible with redistribution inside a GPL-3.0 application. Nothing is bundled: the user's machine fetches it, and fixtures carry a small excerpt with the statements.
- **OpenStreetMap data**: ODbL 1.0. `places.sqlite` is a Derivative Database. It is built and used on the user's own machine and is not publicly distributed, so ODbL's share-alike (§4.4) is not triggered. Attribution is shown wherever it is displayed (§4.3). A waypoint exported from a POI is an insubstantial extract and carries no share-alike obligation, but the description notes "© OpenStreetMap contributors". The committed fixture excerpt is a

Derivative Database that *is* distributed, so it is published under ODbL with its source and date in `test-data/places-excerpt/README.md`.

- No new npm dependency is added. The register gains the two datasets, under a “Data” heading.

13. Testing

Area	Where	Proves
Zoned time	`shared/time/zoned.test.ts`	gap and overlap; BST end 25 Oct 2026 and start 29 Mar 2026; `addDays` across month and year ends
Timeline	`shared/itinerary/timeline.test.ts`	Requirement 13.1 in full
Corridor	`shared/geo/corridor.test.ts`	Requirement 13.2, on `A515_TRACE`
Range	`shared/itinerary/rangeCheck.test.ts`	Requirement 13.3, plus “not checked”
Opening hours	`shared/poi/openingHours.test.ts`	the subset, past midnight, override order, unparsed
Postcodes, text, rank	`shared/search/*.test.ts`	Requirements 13.9, 13.10
OSGB	`shared/geo/osgb.test.ts`	the OS worked example; SK17 centres
Ingest	`main/places/ingest.test.ts`	the committed Buxton excerpt: Palace Hotel as an area; duplicate LineString/Polygon kept once; streets per settlement; addresses
osmium	`main/places/extract.test.ts`	runs the real tools on a tiny committed `.osm` XML when `resources/valhalla/bin/osmium` exists, skipped otherwise, like the live tests
Postcode ingest	`main/places/postcodes.test.ts`	the committed Code-Point excerpt: quality 90 ▯ no location; redirect allowlist refuses any other host; MD5 mismatch refused
Itinerary service	`main/services/itinerary.test.ts`	undo of every command; trip without itinerary unchanged; archive keeps; deleted waypoint heals on undo
iCalendar	`main/io/ics/write.test.ts`	Requirement 13.7: structure, VTIMEZONE, folding, escaping, unknown stops left out
GPX	`main/io/exportService` tests	order; no `<time>`; unscheduled trip byte-identical
E2E	`tests/e2e/roadtrip.spec.ts`	Requirements 13.8, 13.12 with seeded `places.sqlite` and `postcodes.sqlite` built from the fixtures, all network blocked

Fixtures (Requirements 13.5, 13.11):

- `test-data/places-excerpt/buxton.geojsonseq`: real OSM features from a box around Buxton town centre, trimmed to a few hundred, from the `osmium export` of the 2026-11-10 Derbyshire extract. ODbL, attributed.
- `test-data/places-excerpt/tiny.osm`: a handful of those features as OSM XML, for the osmium integration test.
- `test-data/codepoint/sk.csv`: the real `SK17 6` rows from the 2026-11 release, plus `licence.txt` and `metadata.txt` copied verbatim, and two quality-90 rows from the same release.

14. Requirement amendments Two places where the requirements assumed something the code does not have:

- **3.2**: "the routing engine's estimate for that route when one exists". No saved route has one, by ADR 0011 §8, so every leg is timed by the per-mode average. The requirement is amended to say so.
- **3.7**: a day-stay must be at least one day (validated), so "departure before arrival" can only happen when arrival on the arrival day is later than the departure time, and N = 1 still leaves the next morning. The flag is kept for the case the user can actually hit: `depart-at` **on an hours stop earlier than arrival**, which is `depart-at-too-early`. The requirement's example ("arriving at 11:00 on the last day") cannot occur with whole-day stays measured from the arrival date, and the design records that rather than building a flag nothing can raise.

15. Feedback after the first build (Requirements 14–17)

15.1 Leg modes — migration `0005_leg_modes.sql` `itinerary_stops` gains three columns, each describing the leg **arriving** at the stop:

Column	Values	Meaning
`leg_mode`	`road` (default), `route`, `other`	planned automatically, chosen by hand, or not by road
`leg_transport`	`ferry`, `train`, `flight`, `other`, null	for `other` only
`leg_minutes`	integer, null	travel time for `other`; null means not known

`leg_label` holds a free-text name for `other` ("Holyhead–Dublin ferry"). Existing rows migrate as `route` when they have a `leg_route_id` (the user chose it) and `road` otherwise. Nothing is planned during the migration, so an upgrade never makes routes on its own.

15.2 Automatic planning `main/services/autoPlanService.ts`, `autoPlan(db, provider | null, mode, tripId)`, async. For every `road` leg after the first stop:

1. If its route exists, is in the trip, and joins its two stops (design §4's 1 km test), nothing happens.
2. If its route was **planned automatically** and no longer fits, that route is deleted (`route:restore` is its inverse).
3. A new leg is routed (`routeLeg`), saved with `createSnappedRoute` and linked. The route's metadata records `{ plannedBy: 'itinerary' }`, which is the only thing that lets step 2 delete it. A route without that marker is the user's, and is never touched (Requirement 15.3).

It returns one `batch` mutation. The IPC handlers for `addStop`, `removeStop`, `moveStop`, `addPoiStop`, `setLegMode`, and `waypoints.move` / `waypoints.update` (when the waypoint is a stop) apply their

own change, run `autoPlan`, and record **one** batch of both, so a single ⌘Z undoes the stop and the drives planned for it. Undo order is the batch's reverse: the planned routes go first, then the structural change restores its own links.

With no provider, or when the engine finds no road, the leg is left without a route. `ItineraryView` gains `planIssue` per stop, from the provider status ("No map regions are loaded", "Outside your map regions") or the engine's own answer, so the editor says why (Requirement 15.4).

15.3 Timeline `TimelineLeg` gains `kind: 'road' | 'other'` and `minutes`. An `other` leg with minutes takes exactly that long. Without minutes it is flagged `no-travel-time`, and everything after it is unknown, as an unplanned road leg is. Day totals count road legs as driving. Other transport is carried separately as `otherSeconds`, so the daily driving cap is about driving. Along-the-route search and the range check skip `other` legs. The range check reports them as stretches it cannot check, never as covered.

15.4 The library tree for a dated trip `libraryService` builds a scheduled trip's children from its `ItineraryView`. A new `TreeNodeKind`, `group`, is a heading row: not selectable, no `ref`, expandable. Days are expanded by default. Children:

```
Peak weekend                 23–25 Oct 2026 · 3 days · 98 km
  Fri 23 Oct                 1 h 32 min driving
    Buxton                   leave 16:00            (waypoint)
    Buxton → Leek            21 min                 (route)
    Leek                     17:21 – Sun 09:00      (waypoint)
  Sun 25 Oct                 …
  Unknown times              (only when some are)
  Not in the plan            (only when some are)
```

Node ids are positions (`trip:…/day:2026-10-23/stop:…`), so a waypoint that is both start and end appears twice without a key clash. An undated trip's node is built by the old code path, unchanged (Requirement 14.5). A total distance with any road leg unknown is shown as "at least 98 km".

15.5 Panels `PanelLayout` gains `drawerHeight` (default 300) and `drawerVisible` (default true), both optional so settings saved before this load unchanged. `SplitPane` gains a `top` side: the drawer sits in a container whose height is dragged by its top edge and clamped so the map keeps at least 160 px. Three toggles (library, properties, bottom panel) sit in a small group at the map's bottom-left. They are keyboard-reachable and named, and they mirror the existing View menu items. Opening a drawer (calendar, profile, places) makes the bottom panel visible again.

15.6 Icon buttons `lucide-react` 1.72.0 (ISC, verified with `npm view` and a dry-run install; added to the register). `IconButton` takes a required `hint`, which is both its `aria-label` and a tooltip shown on hover after 300 ms and immediately on keyboard focus. The tooltip is rendered in a portal, positioned from the button's rectangle, so overflow containers never clip it. The trip, route, itinerary, calendar, along-the-route and search panels use it for their actions. "Plan by date" and "Download" stay as text (Requirement 17.3).

9.2.3 Tasks

Execute in order. Pure modules come first, each proven offline, then storage and the main process, then the UI, then the end-to-end and verification pass.

Update `docs/progress-mindmap.svg` as each task changes status.

- ☒ 1. Spec, ADR 0019 and the out-of-order build

  - `design.md`, `tasks.md`, ADR 0019; README row and decision index; the note that this phase is built ahead of Phases 8 and 9 at the owner's request
  - Requirements 3.2 and 3.7 amended per design §14
  - Mindmap: the phase's leaves become these tasks
  - *Requirements:* *12.2*
- ☒ 2. Zoned time and offline time-zone lookup
  - `shared/time/zoned.ts`: `zoneOffsetMs`, `wallClockToUtc`, `utcToWallClock`, `addDays`, `isValidDate`, `isValidTime`
  - `shared/time/timeZones.ts`: `TimeZoneResolver`, polygon table with `UK_BOUNDARY` → `Europe/London`
  - Unit tests: 01:30 on 29 Mar 2026 moves forward; 01:30 on 25 Oct 2026 takes the earlier instant; 09:00 the day after BST ends is 09:00 GMT; `addDays` across 28/29 Feb and 31 Dec; Buxton → London; Dublin and Douglas → null
  - *Requirements:* *3.8*, *3.9*, *13.1*
- ☒ 3. Itinerary types and the timeline
  - `shared/domain/itinerary.ts`: stored and view types
  - `shared/itinerary/timeline.ts`: `computeTimeline`, days, flags, end date
  - Unit tests: hours stop; 2-day stay across the clock change leaves 09:00 GMT; arrive-by missed; depart-at waits when early and flags when late; an unplanned leg makes every later time null; deleted location; daily cap flagged and nothing moved; a walking leg uses the planning speed and a driving leg does not; the night is spent at the right stop
  - *Requirements:* *1.3*, *2.3*, *2.4*, *2.5*, *3.1*, *3.2*, *3.3*, *3.4*, *3.6*, *3.7*, *3.8*, *13.1*
- ☒ 4. Corridor geometry and the range check
  - `shared/geo/corridor.ts`: line index, `project`, `corridorBox`, sections
  - `shared/itinerary/rangeCheck.ts`
  - Unit tests on `A515_TRACE`: along and off distances; points beyond the width rejected; a point near both ends of a loop projects to the nearer segment; section by distance and by "near a point"; ordering; empty. Range: a gap flagged; a station clears it; unset range runs no check; the reserve shrinks the usable range; an uncovered stretch is "not checked"
  - *Requirements:* *6.2*, *6.3*, *6.4*, *6.7*, *9.1*, *9.2*, *9.3*, *9.4*, *9.5*, *13.2*, *13.3*
- ☒ 5. POI categories, attributes and opening hours
  - `shared/poi/categories.ts` (the tag table, reasons, symbols, osmium filter expressions), `shared/poi/attributes.ts`, `shared/poi/openingHours.ts`, `shared/poi/PoiProvider.ts`
  - Unit tests: every category maps to a real symbol id; private unnamed parking excluded; attributes keep only the Requirement 7.1 fields; `24/7`; `Mo-Fr 08:00-18:00; Sa 09:00-12:00`; past midnight; later rule overrides; `off`; an unparsed expression stays unparsed; a missing value is absent, not open
  - *Requirements:* *5.7*, *6.1*, *7.1*, *7.3*, *7.4*
- ☒ 6. Postcodes, OSGB and search text
  - `shared/search/postcode.ts`, `shared/search/text.ts`, `shared/search/rank.ts`, `shared/search/PlaceSearchProvider.ts`, `shared/geo/osgb.ts`
  - Unit tests: `de451aa`, `DE45 1AA`, `de45  1aa` equal; full, sector, outcode; `BT1 1AA`, `IM1 1AA`, `JE2 3AB`, `GY1 1AA` uncovered; malformed falls through; abbreviations both ways; prefix tokens; ranking order; duplicate names get their county; OS worked example; SK17 centres within 10 m
  - *Requirements:* *11.4*, *11.5*, *11.6*, *11.7*, *11.8*, *11.13*, *11.14*, *11.17*, *13.9*, *13.10*
- ☒ 7. iCalendar writer
  - `main/io/ics/write.ts`: events, VTIMEZONE from real transitions, escaping, folding, CRLF
  - Unit tests: structure; a trip across 25 Oct 2026 has both offsets; a 75-octet fold never splits a UTF-8 sequence; `;` `,` `\` and newlines escaped; unknown-time stops left out and reported; unknown zone written as UTC
  - *Requirements:* *10.1*, *13.7*

- ☒ 8. Itinerary storage
  - Migration `0004_itinerary.sql`, `schema.ts`, `db/repositories/itineraries.ts`
  - Unit tests: migration applies on a Phase 7 database; deleting a trip cascades; deleting a waypoint leaves the stop; `seq` ordering
  - *Requirements: 1.1, 1.5, 2.1, 2.2*
- ☒ 9. Itinerary service and commands
  - `main/services/itineraryService.ts`; commands registered with labels
  - Unit tests: every operation's undo and redo; remove-then-undo restores every stop; a deleted waypoint shows `location-deleted` and heals on undo; foreign waypoint or route refused; 100-stop bound; archived trip keeps its itinerary
  - *Requirements: 1.2, 1.5, 2.1, 2.2, 2.6, 2.7, 3.5*
- ☒ 10. Itinerary IPC, preload and tree detail
  - Channels, schemas, handlers, preload methods; `libraryService` date range
  - Unit tests: schemas reject `25:00`, `2026-02-30`, 101 stops; tree detail for a scheduled trip; **no detail and an identical node for a trip with no itinerary**
  - *Requirements: 1.1, 1.4, 12.5*
- ☒ 11. Place data: extract and ingest
  - `main/places/extract.ts`, `main/places/ingest.ts`, `main/places/store.ts` (`placesState`); `BuildSteps.places` wired into `downloadAndBuild` and `.osm.pbf` import; "Reading places" progress
  - Fixtures: `test-data/places-excerpt/` from the real extract, with ODbL attribution
  - Unit tests: Palace Hotel kept once, as an area, at its centroid; streets collapse per settlement; addresses; county from region rings; failed build keeps the previous snapshot; osmium integration test (skipped when the tool is absent)
  - *Requirements: 5.1, 5.2, 5.3, 5.4, 11.10, 13.4, 13.5*
- ☒ 12. Postcode data
  - `main/places/postcodes.ts`: listing, size, one-hop allowlisted download, MD5 check, jszip ingest, atomic replace
  - Fixtures: `test-data/codepoint/` real excerpt with licence and metadata
  - Unit tests: quality 90 stored without a location; any other redirect host refused; MD5 mismatch refused and the old data kept; release date from `metadata.txt`
  - *Requirements: 11.4, 11.9, 11.15, 13.11*
- ☒ 13. Search service
  - `main/places/searchService.ts` implementing `PlaceSearchProvider`; IPC `search.query`, `search.status`, `postcodes.*`
  - Unit tests against the fixtures: postcode exact, sector, not found with date, uncovered message, no location; place prefix; street with house number to the building; house number falling back to its street; empty result names the loaded regions; latency measured on the full data in the live test
  - *Requirements: 11.1, 11.3, 11.5, 11.6, 11.7, 11.8, 11.11, 11.12, 11.13, 11.16, NFR 2*
- ☒ 14. POI search and range service
  - `main/places/poiService.ts` implementing `PoiProvider`: corridor query for a route or an itinerary, sections, chunked and cancellable; detour by road; range check; IPC `places.*`
  - Unit tests against the fixture: ordered results; itinerary results carry day and time; cancel stops work; uncovered stretch reported; detour needs a provider
  - No online POI source is offered (design §6.4), which is how 5.5 is met
  - *Requirements: 5.5, 5.6, 6.1, 6.3, 6.4, 6.5, 6.8, 9.2, 9.5, 12.1*
- ☒ 15. From POI to waypoint and stop; planning legs
  - `WaypointSource` gains `poi`; provenance in metadata; `addPoiWaypoint`, `addPoiStop`, `planLeg`, re-route through a stop, each a batch
  - Unit tests: the waypoint's symbol, plain-text description and provenance; one undo removes waypoint and stop; re-route creates two routes and leaves the old one unchanged

and in the trip
  - Found while testing: undoing a waypoint deletion lost its `metadata` (also true of imported waypoints before this phase). Fixed: the inverse carries metadata and source, read before the row is deleted
  - *Requirements:* *8.1*, *8.2*, *8.3*, *8.4*, *8.5*
- ☒ 16. Export
  - GPX order for a scheduled trip and the unmapped warning; `itinerary.exportIcs` with a save dialog
  - Unit tests: stop waypoints in order; no `<time>` on stops; a trip with no itinerary exports byte-identically to before
  - iCalendar import stays out (10.5): no reader is added
  - *Requirements:* *10.1*, *10.2*, *10.3*, *10.4*, *10.5*, *13.6*
- ☒ 17. Itinerary editor
  - `features/itinerary/ItinerarySection.tsx` in trip properties; store
  - Unit tests for the view-model helpers (duration text, flag messages)
  - *Requirements:* *1.2*, *2.1*, *2.3*, *2.4*, *2.5*, *2.6*, *2.7*, *3.1*, *3.6*, *9.1*, *9.3*, *9.5*
- ☒ 18. Calendar drawer
  - `features/itinerary/CalendarPanel.tsx`, day, week and month; pure layout helpers in `calendarLayout.ts`; drag to change duration; selection sync
  - Unit tests: block placement and overlaps; unknown times listed, not placed; drag snaps to 15 minutes or whole days
  - *Requirements:* *4.1*, *4.2*, *4.3*, *4.4*, *4.5*, *4.6*, *4.7*, *4.8*
- ☒ 19. Along-the-route panel and POI layer
  - `features/poi/PoiPanel.tsx`, `features/map/usePoiLayer.ts`, `shared/geo/cluster.ts`, `layerVisibility.pois`
  - Unit tests: clustering per zoom; empty-state wording carries the date
  - *Requirements:* *6.1*, *6.2*, *6.3*, *6.4*, *6.5*, *6.6*, *7.1*, *7.2*, *7.5*, *7.6*, *7.7*, *8.1*, *9.2*, *9.4*
- ☒ 20. Map search box
  - `features/search/SearchBox.tsx`; result actions; approximate areas
  - Unit tests: stale responses are dropped; keyboard selection
  - *Requirements:* *11.1*, *11.2*, *11.6*, *11.15*, *11.16*
- ☒ 21. Settings, About and the register
  - Postcode section; places status and rebuild; About statements; third-party register “Data” section
  - *Requirements:* *5.4*, *11.9*, *11.15*, *12.2*, *12.3*
- ☒ 22. End-to-end tests and verification pass
  - `tests/e2e/roadtrip.spec.ts` with seeded `places.sqlite` and `postcodes.sqlite`, all network blocked: build an itinerary, calendar, fuel and a hotel along a route, hotel as a stop, postcode and place search, restart with everything intact
  - `npm run verify` green; README row marked complete
  - Found while testing: a library change (any edit, or an undo) reloaded the itinerary and briefly unmounted the “Plan by date” form, losing what was being typed. The form now stays mounted while the same trip reloads.
  - Found while reviewing screenshots: short calendar blocks overprinted each other, and the clock-change note was clipped. Blocks are now drawn at least 45 minutes tall and allocated lanes on that drawn length
  - Result: 1,533 unit tests and 138 end-to-end tests pass, 8 of them this phase’s, with every network request refused
  - *Requirements:* *13.8*, *13.12*, *12.4*, *NFR 1*

Feedback after the first build (11 November 2026)

- ☒ 23. Spec for the feedback

  - Requirements 14–17, design §15, ADR 0019 §8, these tasks, mindmap leaves
  - *Requirements: 14.5, 15.1, 16.1, 17.4*
- ☒ 24. Leg modes in storage and the timeline
  - Migration `0005_leg_modes.sql`; `ItineraryStop` leg fields; service `setLegMode`; timeline `other` legs, `no-travel-time`, `otherSeconds`; range check and POI search skip `other` legs
  - Unit tests: migration marks linked legs `route`; a ferry with 2 h is timed 2 h and not counted as driving; a ferry with no time makes the rest unknown; choosing a route switches the mode; undo of each
  - *Requirements: 15.1, 15.5, 15.6*
- ☒ 25. Automatic planning
  - `autoPlanService.autoPlan`; `plannedBy` marker; replanning of stale planned routes only; one batch with the structural change; `planIssue`
  - Unit tests with a fake provider: adding a stop plans both legs; moving a stop replaces only planned routes and leaves a drawn route untouched; one undo removes stop and routes; no provider leaves the leg unplanned with a reason; an `other` leg is never planned
  - *Requirements: 15.2, 15.3, 15.4*
- ☒ 26. Leg modes in the editor
  - Mode control per leg (by road, choose a route, other transport with kind, label and time); reason when a road leg could not be planned
  - Unit tests: a plan issue is given for a by-road leg with no route, and never for a chosen route or other transport
  - Built with task 29's icon buttons, which were done first so these controls never had text buttons; "Route through this stop" was removed, since inserting a stop now plans both sides itself
  - *Requirements: 15.1, 15.4, 15.5, 15.6*
- ☒ 27. The trip in the library summarises it
  - `group` tree nodes; day groups, unknown times, not in the plan; trip detail with days and distance; days expanded by default
  - Unit tests: a dated trip's tree in plan order with times; a stop used twice gets two distinct rows; nothing in the trip is lost; an undated trip's node is unchanged
  - *Requirements: 14.1, 14.2, 14.3, 14.4, 14.5*
- ☒ 28. Resizable, hideable panels
  - `PanelLayout.drawerHeight` and `drawerVisible`; top-edge resize with a map minimum; panel toggles at the map's bottom-left
  - Unit tests: height clamping keeps the map minimum; old settings without the new keys load with defaults
  - *Requirements: 16.1, 16.2, 16.3, 16.4*
- ☒ 29. Icon buttons with hints
  - `lucide-react` added and registered; `IconButton` with `hint` and a portal tooltip; the panels converted
  - Unit tests: tooltip placement stays inside the window
  - Registering it showed the About list's hand-kept set was already missing three runtime dependencies (`fast-xml-parser`, `fit-file-parser`, `jszip`); all four are now listed
  - *Requirements: 17.1, 17.2, 17.3, 17.4*
- ☒ 30. End-to-end tests and verification
  - Road trip spec updated for automatic legs, the tree summary, panel toggles and icon buttons; `npm run verify` green
  - Without map regions the e2e app has no routing, so the automatic leg is proven by its stated reason, not by a route; the unit tests drive a fake provider
  - The full run caught what the panel conversion missed: the track and waypoint action rows were still text, and the trip's import icon shared the accessible name "Import files" with the library's. Both fixed; the trip's is now "Import GPX, KML or other files"
  - Screenshots caught two layout faults, fixed: a width class appended to `INPUT_CLASS` lost

to its `w-full` (new `inputAt()` swaps it instead), and the calendar opened at midnight rather than at the first thing planned
- *Requirements: 14.2, 15.2, 16.2, 17.2*

Notes

9.3 Phase 11: Geotagged Photos from a Linked Folder — Requirements — SPEC-PHASE-11-GEOTAGGED-PHOTOS

Version: — · **Status:** unspecified · **Slug:** `phase-11-geotagged-photos`

9.3.1 Requirements

Status: designed (11 November 2026), see `design.md` and `tasks.md`. Built ahead of Phases 8 and 9 at the owner's request. Revised on 12 November 2026 with the owner's answers: folders are linked per trip, with a view of every trip's photos together (question 2); the linked folder is the trip's own folder on disk, which Phase 12 extends to GPX files; and the scale to design for is a 40,000-photo library built up over many years. Open questions 3–5 are answered by the design (§14). Question 1 moves to Phase 12, where the tracks it needs arrive from the same folder.

Depends on Phases 1–3 (library, waypoints, tracks) and Phase 10 (marker clustering, icon buttons with hints, resizable panels), all shipped. It does **not** depend on Phase 8 or Phase 9. It sits after them by default, not by dependency, so the owner can move it ahead of either without rework.

Introduction The owner asked for a way to link an external folder of images, so that photos appear on the map at the place they were taken, using the location recorded in each photo. Selecting a photo shows it in a preview box in the bottom-left corner of the map.

Linked is the key word. The photos stay where the user keeps them. The app reads them and records where they are, and never copies, moves, renames or rewrites them. That is a different model from the one the codebase already has, described below, and the difference is deliberate.

The trip's folder on disk The owner's feedback on the design (12 November 2026) widened the idea. The linked folder is not only a photo folder. It is **the trip's folder on disk**: the planned routes, the tracks recorded when the trip was actually done, and the photos, all kept together where the user can see them. For someone who travels widely that is a better record than anything locked inside one app, and keeping GPX files outside the app lets people feel in control of their data. Later, sharing a trip as a flythrough video will make keeping images and video organised matter even more.

The owner's own library is the scale to design for: about **40,000 photos and hundreds of GPX files, built up over many years**. The app must not reread thousands of files each time it starts. It keeps an index of each folder, and knows from that index what has changed.

That work is split into three phases, so each can be built and tested before the next is designed:

Phase	What the linked folder gains
11 (this one)	Linking, the file index, and photos read from it. Read-only.
12	GPX kept in step with the folder in both directions, with a hash store. TrailRT writes only files it created.
13	Video, flythroughs, and sharing, with what they produce kept in the folder.

This phase builds the parts the other two depend on: the folder link, the index, the scanner and the watcher. It therefore records every file of a kind a later phase will read (GPX and other track files, video), so those phases start from an index that already exists rather than adding a second walk.

What the codebase already establishes These findings come from the code after Phase 10. They constrain the design and are recorded here so the design does not have to rediscover them.

- **An `attachments` table already exists and is unused.** Phase 1 created it, with `attachmentsDir` in `src/main/paths.ts`, to settle gap 12 of the spec review. That decision was to **copy** files into the app support directory and keep metadata, a relative path and a content hash in the database. Linked photos are the opposite: nothing is copied. Copying a 40 GB photo folder would double the disk use and leave two copies to drift apart. The design should keep linked photos separate from `attachments`, and record in an ADR why both models exist. The copy model fits a single picture pinned to a waypoint; the link model fits a photo library.
- **The CSP allows images only from `'self'`, `data:`, `blob:` and tile hosts** (`src/main/security/csp.ts`). `csp.test.ts` asserts that `file:` is not in `img-src`, and it must stay that way. Thumbnails therefore have to reach the renderer as bytes produced by the main process, or through a narrowly scoped custom protocol. The renderer is never handed a path to load, as the architecture rules require.
- **The bottom-left corner of the map is already occupied.** The scale bar and cursor readout (`MapReadout.tsx`) sit at `bottom-3 left-3`. The panel toggles (`PanelToggles.tsx`, Phase 10 Requirement 16) sit above them at `bottom-12`. The preview box has to share that corner with both, not cover them.
- **Marker clustering already exists** in `src/shared/geo/cluster.ts`, built for points of interest in Phase 10. A photo folder can hold tens of thousands of images, so the photo layer needs it from the start.
- **Track points carry a recorded time** (`track_points.recorded_at`). That is what would let a photo with no location be placed by matching its time against a recorded track. This is the usual way to geotag photos from a camera with no GPS. It is left as an open question below, not a requirement, because the owner asked for photos placed by their own location data.
- **The waypoint symbol set already has `photo`** (`src/shared/domain/symbols.ts`), paired with a Garmin symbol name. A waypoint made from a photo uses it.
- **Library top-level nodes are views, not containers** (ADR 0004). A cross-trip "Photos" view fits beside Routes, Tracks and Waypoints in `libraryService.ts` `VIEWS`.
- **The library already has folders.** `folderService.ts` organises trips into folders in the library tree. A "trip folder" in the app would be confused with those, so the app calls a folder on disk a **linked folder**, and these documents use "the trip's folder on disk" only for the idea.
- **Backups are a SQLite snapshot of the database only** (`backupService.ts`). A linked photo is not in the database, so it is not in a backup. The UI has to say so (Requirement 10).

Three distinctions this phase must keep

1. **The photo is the source of truth; the app holds a cache.** Location, time and thumbnails read from a file are derived values. They are recomputed when the file changes and never become the only copy of anything. This is the architecture's rule for derived data.
2. **No location is not location zero.** A photo without GPS data has no position. It is listed as such and never placed at 0°, 0°, at the folder's "average" position, or anywhere else.
3. **Camera time is not clock time.** Many photos record the local time with no time zone. A time without a zone is shown as the camera recorded it, and labelled as such. It is not silently converted as if it were UTC or the computer's current zone.

Terms

Term	Meaning
Linked folder	A folder on disk the user has linked to a trip: the trip's folder on disk. In this phase the app reads it and never writes to it.

Term	Meaning
File index	The app's record of the files in a linked folder: each one's path within the folder, kind, size, modified time and fingerprint. It is how the app knows a file is unchanged without opening it.
Fingerprint	A hash of a file's size and the bytes the app already reads for its metadata. It recognises a photo that has been moved or renamed within the folder. It is not a hash of the whole file (see the design, §0).
Photo	An image file inside a linked folder that the app recognises as a photograph format.
Geotag	Location data recorded in the photo's metadata (EXIF GPS), usually by the phone or camera when the picture was taken.
Located photo	A photo with a valid geotag. It can be placed on the map.
Unlocated photo	A photo with no geotag, or one the app rejects as invalid (Requirement 2.5). It is listed but not placed.
Taken time	When the photo was taken, from its metadata. It may or may not carry a time zone.
Photo cache	What the app stores about a photo: its path within the folder, size, modified time, geotag, taken time and thumbnail. Rebuildable from the file.
Preview box	The panel in the bottom-left corner of the map that shows the selected photo.

User stories

1. As someone back from a trip, I want to point the app at the folder my phone photos were copied into and see each picture on the map where I took it.
2. As the same person, I want to click a photo marker and see the picture in a small preview without leaving the map, then step through the others from that day.
3. As a planner, I want to turn a photo of a good viewpoint into a waypoint I can route to next time.
4. As someone with a large photo library, I want linking a folder to leave my files exactly as they were, with nothing moved, renamed, copied or edited.
5. As a user whose camera has no GPS, I want to be told which photos have no location, rather than have them disappear or land in the sea off Africa.
6. As someone who keeps photos on an external drive, I want the app to cope when the drive is unplugged, and pick up where it left off when it is plugged back in.
7. As someone who has travelled for years, with tens of thousands of photos across many trips, I want to see each trip's photos with that trip, and also every photo from every trip on one map.

Requirements

Requirement 1: Linking a folder Acceptance criteria

1. THE user SHALL be able to link a folder from a trip, through an icon button with a hint in the trip's actions and through a File menu item. Either way opens the system folder chooser in

the main process. The wording is about linking a folder to a trip, not about photos, because Phase 12 adds GPX to the same link.
2. A linked folder SHALL belong to one trip. Its photos SHALL appear under that trip in the library and with that trip on the map. A trip MAY have several linked folders. The same folder MAY be linked to more than one trip (confirmed by the owner, 12 November 2026).
3. THE user SHALL choose whether subfolders are included, defaulting to included.
4. Hidden files and folders, and macOS package directories, SHALL be skipped.
5. IF the chosen folder is an Apple Photos library (`.photoslibrary`) THEN the app SHALL refuse it with an explanation. That library is a database whose layout Apple does not document. Reading it would be fragile, and the honest answer is to export the photos to a folder first.
6. THE same folder SHALL NOT be linked to the same trip twice. Linking a folder that is inside, or contains, a folder already linked to that trip SHALL be refused with an explanation, so no photo appears twice.
7. IF macOS refuses access to the folder (for example Desktop, Documents, Downloads or a removable volume without permission) THEN the app SHALL say so and point to the setting that grants it. It SHALL NOT report the folder as empty.
8. THE user SHALL be able to unlink a folder after a confirmation that says the files on disk are not touched. Unlinking removes the folder's photo cache and thumbnails, and nothing else. It SHALL be undoable.

Requirement 2: Reading location and time Acceptance criteria

1. THE app SHALL read a photo's geotag from its EXIF GPS data: latitude, longitude and, where present, altitude, horizontal accuracy and the direction the camera faced.
2. JPEG and HEIC (the iPhone default) SHALL be supported. Any other format (TIFF, PNG, DNG and other raw formats) SHALL be listed as supported only after it has been tested against real sample files, in line with the architecture's honesty rule. Until then it is not offered.
3. Hemisphere references SHALL be honoured: south latitudes and west longitudes are negative. The whole test geography is west of Greenwich, so a dropped sign moves a photo about 250 km east and must fail a test.
4. Altitude below sea level SHALL be honoured. Absent altitude SHALL stay absent, never 0.
5. A geotag SHALL be rejected, and the photo treated as unlocated with the reason recorded, when it is exactly 0°, 0°, outside the valid range, or malformed. Cameras without a GPS fix commonly write 0°, 0°.
6. THE taken time SHALL come from the original capture time in the metadata, with its time-zone offset when the photo records one. WHERE it records none THE time SHALL be stored and shown as camera time, without a zone. The GPS timestamp, which is UTC, MAY be used to establish the offset, and the design decides whether to.
7. Text fields in the metadata (title, description, comments) SHALL be treated as untrusted plain text, as the architecture requires of imported content.
8. A file that cannot be read, is truncated, or is not the image its extension claims SHALL be listed with the reason. It SHALL NOT stop the rest of the folder being read.

Requirement 3: Keeping in step with the folder Acceptance criteria

1. THE app SHALL rescan linked folders at launch and on request. While it is running it SHALL notice files that are added, changed or removed.
2. A file SHALL be re-read only when its size or modified time has changed. An unchanged folder SHALL rescan without reopening its photos. The app SHALL keep this state in a file index per linked folder.
3. A photo removed from disk SHALL disappear from the map and the library on the next scan. Anything the user made from it, such as a waypoint (Requirement 7), SHALL stay.
4. IF a linked folder is missing, for example because the drive is unplugged or the folder was moved or renamed, THEN its photos SHALL be shown as unavailable with their last known

locations. They SHALL NOT be deleted. When the folder returns they SHALL become available again without relinking.
5. THE user SHALL be able to relink a moved folder to its new location. Its photos SHALL be matched by their paths within the folder, so anything made from them keeps its link.
6. Files in iCloud Drive that are not downloaded to this Mac SHALL be listed as not downloaded. The app SHALL NOT force a download of a whole folder to read its metadata.
7. A photo moved or renamed within its linked folder SHALL be recognised as the same photo, by its fingerprint, so anything made from it keeps its link. It SHALL NOT be treated as one photo removed and another added.
8. THE file index SHALL record every file of a kind a later phase reads: GPX, KML, KMZ, TCX and FIT files (Phase 12), and video files (Phase 13). This phase does not open them. The folder's properties SHALL count them and say they are not read yet, so their absence from the map is explained.

Requirement 4: Photos in the library Acceptance criteria

1. A trip with linked folders SHALL show a Photos group in the library tree. It holds one node per linked folder, with the folder name, its photo count and how many have no location.
2. A top-level **Photos** view SHALL list every linked folder across all trips, in keeping with ADR 0004. It is a view, not a container.
3. THE photos in a folder SHALL be listable in taken-time order, each showing whether it is located, unlocated (with the reason), unavailable or not downloaded.
4. Selecting a photo in the library SHALL select it on the map and open the preview box. Selecting a located photo off screen SHALL bring it into view.
5. FOR a dated trip (Phase 10), the day groups in the tree SHOULD show the number of photos taken that day. The design decides how photo times without a zone are assigned to days, and the UI says when that assignment is a guess.
6. THE Photos view SHALL be filterable by year taken and by trip, so a library built up over many years stays usable.

Requirement 5: Photos on the map Acceptance criteria

1. Located photos SHALL be drawn as a separate map layer, which can be shown and hidden from the layers control like routes, tracks, waypoints and points of interest.
2. Photos SHALL be clustered, with a count, until zoomed in far enough to separate them. The layer SHALL stay responsive with 50,000 located photos in view of the clustering, which covers the owner's 40,000 with headroom. The design fixes the measured target.
3. Photos at the same spot, such as a burst of shots, that no zoom level can separate SHALL open the preview box at the first of them, with a "1 of N" step through the rest.
4. WHERE a photo records the direction the camera faced, the selected photo's marker SHOULD show that direction.
5. Unavailable photos SHALL be drawn distinctly from available ones, or hidden, at the user's choice. They SHALL never be indistinguishable.
6. WHEN the Photos view is selected, THE map SHALL show the located photos of every trip together, subject to the view's filters (Requirement 4.6). Otherwise it shows the photos of the trip on the map. A folder linked to several trips SHALL have its photos shown once in the all-trips map, not once per trip.

Requirement 6: The preview box The owner asked for it in the bottom-left corner of the map.

Acceptance criteria

1. Selecting a photo SHALL open the preview box in the bottom-left corner of the map area. It SHALL stack above the panel toggles and the scale bar, which stay visible and usable. It

SHALL NOT cover the search box (top left) or the zoom and layer controls (top right).
2. THE preview box SHALL stay inside the map area when the panels around the map are resized or hidden (Phase 10 Requirement 16), including when the panel below the map is open. On a narrow map it SHALL shrink, keeping the picture's aspect ratio, rather than overflow.
3. THE preview box SHALL show:
   - the picture, the right way up according to its orientation metadata;
   - the file name, with its path inside the linked folder as a hint;
   - the taken time, with its zone when known, otherwise labelled as camera time;
   - the location in the user's coordinate format, with altitude and accuracy when present and nothing in their place when absent; for an unlocated photo, "No location" and the reason.
4. ITS actions SHALL be icon buttons with hints (Phase 10 Requirement 17): previous and next photo in taken-time order, zoom to the photo, add it as a waypoint (Requirement 7), open it in the default app, show it in Finder, and close.
5. THE keyboard SHALL work: left and right arrows step through the photos and Escape closes the box, while focus is in the box.
6. Only a thumbnail sized for the box SHALL be loaded, never the full-size original. Opening the original goes to the default app.
7. WHEN the photo is unavailable or not downloaded THE box SHALL say so and show the cached thumbnail if one exists.

Requirement 7: Using a photo's location Acceptance criteria

1. THE user SHALL be able to add a located photo as a waypoint in its trip, at the photo's location and altitude, with the `photo` symbol and the file name as the default name. It SHALL be a single undoable step.
2. A waypoint made from a photo SHALL record which photo it came from, so the waypoint's properties can open that photo's preview while the photo is available. The waypoint SHALL be an ordinary waypoint in every other respect, and SHALL export and survive the photo's removal like any other.
3. THE user SHALL be able to add several selected located photos as waypoints in one step, recorded as one undo step (ADR 0014).

Requirement 8: Read-only, privacy and security Acceptance criteria

1. THE app SHALL NOT write to, rename, move, delete or change the metadata of any file in a linked folder. No code path that writes SHALL be able to address a linked folder. The design SHALL show how that is enforced, not only intended. Phase 12 relaxes this for GPX files TrailRT itself creates, and only for those. Photos stay read-only in every phase.
2. THE renderer SHALL request photos and thumbnails by photo id. The main process SHALL resolve the id to a path and serve only files inside a currently linked folder. A symbolic link that resolves outside the folder SHALL NOT be followed.
3. `file:` SHALL stay out of the CSP. Thumbnails SHALL reach the renderer as bytes or through a custom protocol that serves only cached thumbnails by id.
4. "Open in default app" and "Show in Finder" SHALL act only on a path the main process resolved from a photo id. They SHALL NOT accept a path from the renderer.
5. Reading a photo SHALL involve no network request. There SHALL be no online reverse geocoding of photo locations.
6. Photo locations SHALL NOT leave the machine except through an export the user starts, and photo-derived waypoints export only as waypoints.

Requirement 9: Scale and responsiveness Acceptance criteria

1. Scanning SHALL run without blocking the main process's handling of other requests. It SHALL show progress and SHALL be cancellable.

2. THE app SHALL read only as much of each file as it needs for the metadata, not the whole image.
3. Thumbnails SHALL be made when first needed and cached in the app support directory. The cache SHALL be bounded in size and rebuildable at any time.
4. THE design SHALL measure, on the target hardware, the first scan of a folder of 40,000 mixed JPEG and HEIC photos, an unchanged rescan of the same photos spread across 100 linked folders, and the time to show the first preview. It SHALL record these as targets the tests hold to.
5. Starting the app SHALL NOT wait for linked folders to be scanned. Folders are checked after the window has shown, and the map shows the indexed photos straight away.

Requirement 10: Backups and moving to another Mac Acceptance criteria

1. A backup SHALL include the linked-folder records and the photo cache metadata, and SHALL NOT include the photos. The backup UI SHALL say that linked photos are not backed up by the app.
2. Restoring a backup on another Mac SHALL show its linked folders as unavailable until they are relinked (Requirement 3.5). Nothing SHALL be lost in that state.
3. Thumbnails SHALL be left out of the backup, as a cache that rebuilds.

Requirement 11: Testing Acceptance criteria

1. Unit tests SHALL cover EXIF location and time reading with no Electron in the loop: hemisphere signs, altitude below sea level, absent altitude, 0°, 0° rejection, out-of-range and malformed values, time with and without an offset, and every orientation value.
2. Fixture photos SHALL be synthetic and SHALL NOT contain real user GPS data, in line with the testing rules. Fixtures made from real device files (to prove HEIC and other formats against real encoders) SHALL have their metadata replaced with locations from `@fixtures/places`, the Buxton square, before they are committed.
3. Fixtures SHALL include: located JPEG and HEIC; a photo with no geotag; one at 0°, 0°; a truncated file; a non-image with an image extension; a nested folder; a hidden folder; a symbolic link pointing outside the folder; and a `.photoslibrary` directory.
4. A test SHALL prove the read-only rule: after a link, scan, thumbnail build, add-as-waypoint, unlink and rescan, every fixture file is byte-identical and has its original modified time.
5. End-to-end tests SHALL link a fixture folder (with the folder chooser stubbed, as the import tests stub the file chooser), and cover:
   - located photos on the map and the unlocated ones listed;
   - the preview box opening in the bottom-left corner, clear of the panel toggles and the scale bar;
   - stepping through photos;
   - adding a photo as a waypoint and undoing it;
   - the folder going missing and coming back;
   - persistence across a restart.

Constraints carried from the rest of the project

1. **Module boundaries.** Folder scanning, metadata reading and thumbnails live in the main process (`src/main/linkedFolders/` or similar). Pure parsing of metadata values (coordinates from degrees, minutes and seconds, offsets, orientation) lives in `shared/` so it can be unit-tested directly. Trip management does not reference a concrete metadata reader.
2. **Errors cross IPC as results, never exceptions**, and every handler validates its payload with zod.
3. **No new dependency without a licence check** recorded in the register.

4. **Missing stays missing.** Absent location, altitude, accuracy, direction and zone are shown as absent, never as zero or a guess.

Out of scope

- **Writing geotags into photos.** It contradicts Requirement 8.1. If it is ever wanted, it needs its own requirement, an explicit per-action confirmation, and a backup of each original.
- **Editing, rotating, cropping or deleting photos.**
- **Reading GPX and other track files from the folder.** That is Phase 12. This phase records them in the index (Requirement 3.8) and does nothing else with them.
- **Video.** Phone videos carry their location in a different container. Video is part of Phase 13, with flythroughs and sharing. This phase records video files in the index (Requirement 3.8) and does not read or place them.
- **The Apple Photos library and iCloud Photos** (Requirement 1.5). Photos exported from them to a folder work like any others.
- **Online reverse geocoding** ("taken in Buxton"). Phase 10's offline gazetteer could support it later, offline, if asked for.
- **Face, object or scene recognition.**

Unresolved questions for the owner and the design stage

1. **Match unlocated photos to a track by time?** Track points record their time, so a photo taken at 14:32 can be placed where the track was at 14:32. It is the standard answer for cameras with no GPS, and BaseCamp did it. It needs a camera-clock offset control and a clear "placed from the track, not the photo" label. Nothing would be written to the photo. **Moved to Phase 12** (its open question 6), because that phase brings the trip's recorded tracks in from the same folder.
2. **Folder per trip, or folder for the library? Answered by the owner on 12 November 2026:** per trip, with the ability to see all. Folders are linked per trip (Requirement 1.2). The Photos view shows every trip's photos together (Requirements 4.6 and 5.6).
3. **Which metadata reader?** Candidates checked on 11 November 2026:
   - `exifr` 7.1.3: MIT, no dependencies, reads JPEG, HEIC, TIFF and PNG and reads only the first part of the file. Its last release was May 2022, which is a maintenance risk the design must weigh.
   - `exiftool-vendored` 39.2.0: MIT wrapper around ExifTool, which is under the Perl Artistic licence or GPL. It reads almost everything, but runs a separate Perl process that would have to be bundled, signed and size-checked.
   - A small in-house reader for the GPS and time tags of JPEG and HEIC only.

   The design decides after testing all three against the fixtures.
4. **Thumbnails.** Electron's `nativeImage.createThumbnailFromPath` uses macOS Quick Look, which handles HEIC and many raw formats with no new dependency. It needs measuring for speed and for whether it applies the orientation tag.
5. **Watching.** Is the recursive file watcher Node offers on macOS reliable enough for folders on external and network drives? Or should those be rescanned on demand only?

9.3.2 Design

Written against the code after Phase 10, with the uncommitted fix for the packaged tile builder (`build/sign-mac.cjs`, `src/main/routing/toolExit.ts`) in the tree. ADR 0020 records the decisions that are more than implementation.

Revised on 12 November 2026 after the owner's feedback (requirements, "The trip's folder on disk"). The linked folder is the trip's folder on disk, which Phase 12 extends to GPX and Phase 13 to video. This phase therefore keeps a general **file index** of each linked folder rather than a photo table alone

(§2). It adds fingerprints so a moved photo keeps its links (§5.3), and a map of every trip's photos (§9.1). It is sized for a 40,000-photo library (§0). In the code and the UI the folder is a **linked folder**, because the library's existing folders hold trips (`folderService.ts`).

Built out of order. The owner asked for this phase on 11 November 2026, ahead of Phases 8 and 9. It depends only on Phases 1–3 and 10, which are complete. As with Phase 10, the README's rule against designing ahead exists to stop guessing at unsettled interfaces, and every interface used here has shipped.

0. Measurements Taken on 11 November 2026 on the target hardware: Apple M4, 24 GB, internal SSD, macOS 26.6.2, Node 26.13.0, Electron 45.2.3. The scripts are reproduced by task 2's fixture generator and task 13's measurement test.

Fixtures. JPEGs made by Electron's `nativeImage`, with an EXIF block from a small in-house writer: GPS at Buxton and Leek, an altitude below sea level, an orientation-6 photo, a time with and without an offset, 0°, 0°, and no GPS. HEICs converted from those by `sips` (Apple's ImageIO encoder, the one the iPhone uses). `sips` kept GPS, time, offset and orientation, and rewrote the TIFF block big-endian (`MM`), so the reader must handle both byte orders.

Where the metadata sits.

File	Size	EXIF block
1200×800 JPEG	16 KB	APP1, byte 2
4032×3024 HEIC from `sips`	175 KB	start of `mdat`, byte 2,838

Apple's encoder puts the EXIF item first in `mdat`, but HEIF does not require it. The `iloc` box says where it is, so the general reader follows `iloc` and costs one extra ranged read when the item is further in.

Metadata readers.

Candidate	Result
`exifr` 7.1.3 (MIT, last release May 2022)	Its file reader **fails on Node 26**: it calls the file APIs with arguments they no longer accept, and an unclosed `FileHandle` then aborts the process. Given buffers, its parser reads every fixture correctly. Error reporting is opaque: a truncated HEIC returns `undefined`, a truncated JPEG returns `{"errors":[{}]}`.
`exiftool-vendored` 39.2.0 (MIT) + ExifTool 13.63	Runs ExifTool on the **system Perl**, which Apple has deprecated, as an extra process. 22 MB unpacked. The same class of hidden host dependency that broke the 0.1.0 tile build (ADR 0013 amendment). Not tested further.
In-house	Chosen. See §3 and ADR 0020.

Scan cost, 10,000 files (5,000 JPEG, 5,000 HEIC, 20 folders, warm cache):

Step	Time
Walk and `stat` every file	31 ms
Read the first 64 KB of each and parse it (`exifr` on buffers)	223 ms, 22 µs per file

Parsing is negligible. The cost of a first scan is I/O, which is asynchronous, so the scan does not need a worker thread (§5.5).

Hashing, measured 12 November 2026 (Node 26 `crypto`, SHA-256):

Input	Time
64 KB, the bytes already read for a photo's metadata	0.023 ms
A whole 3 MB photo, already in memory	0.95 ms
A 2 MB GPX file	0.62 ms

So a hash of the whole of every photo is affordable for the CPU but not for the disk. 40,000 photos of 3 MB is 120 GB to read: about 20 minutes from a USB hard drive at 100 MB/s, on every first scan. A photo's **fingerprint** is therefore a hash of its size and the first 64 KB, which the metadata read has already fetched, so it costs nothing extra (§5.3). Whole-file hashes are kept for GPX and other track files, which are small. Phase 12 needs them to tell whether a file is in step.

Clustering, 50,000 points in 100 trip-sized clumps around the world, `clusterPoints` over the whole set:

Zoom	Time	Clusters
2	4.0 ms	72
6	4.7 ms	754
10	8.6 ms	37,477
14	9.1 ms	49,937

Clustering the whole all-trips set is cheap at every zoom. The cost to bound is the number of markers drawn at high zoom, which the viewport limit handles (§9.2).

Thumbnails with `nativeImage.createThumbnailFromPath` (Quick Look):

Source	640 px, first	640 px, again
4032×3024 JPEG	38 ms	5 ms
4032×3024 HEIC	41 ms	5 ms

Quick Look **applies the orientation tag** and keeps the aspect ratio: the orientation-6 fixture came back portrait, with its corner marker rotated 90° clockwise. HEIC needs no new dependency.

Targets the tests hold to (Requirement 9.4), set with headroom over the measurements because synthetic images compress better than real ones and the cache was warm:

Measure	Target
First scan, 40,000 mixed JPEG and HEIC of 3 MB each, internal SSD	≤ 20 s
Unchanged rescan, the same 40,000 across 100 linked folders	≤ 2 s
First preview, thumbnail not yet cached	≤ 250 ms
Preview with a cached thumbnail	≤ 50 ms
Loading the all-trips map points, 50,000 located photos	≤ 300 ms

Measure	Target
Re-cluster and redraw on `moveend`, 50,000 located photos	≤ 50 ms, ≤ one marker per 56 px cell in view
Window shown at launch	Before any folder is scanned (Requirement 9.5)

The rescan target comes from the measured walk: 31 ms to `stat` 10,000 files, so about 125 ms for 40,000, leaving the rest for comparing against the index.

A cold-cache first scan cannot be measured without `sudo purge`. Task 13 records the cold figure if the owner runs it, and the target above stands until then.

1. Invariants this phase collides with, and how each is resolved

Invariant	Collision	Resolution
Phase 1 `attachments` copies files into app storage	Copying a photo library doubles disk use and drifts	Separate tables, `linked_folders`, `folder_files` and `photo_metadata`. Nothing is copied. ADR 0020 §1 says why both models exist.
The library already has folders, which hold trips (`folderService.ts`)	"Trip folder" would mean two things	The code and the UI say **linked folder** (`linked_folders`, `main/linkedFolders/`, "Link Folder to Trip…").
Phases 12 and 13 read the same folders	A photo-only table would leave them a second walk and a second index	One `folder_files` index per linked folder, with a `kind`. This phase records GPX, other track files and video, and reads only photos (§2, §5.2).
Waypoints made from a photo point back at it	Photos get moved and renamed within a folder	A fingerprint identifies a photo whose path changed (§5.3). The waypoint keeps the fingerprint, so the link survives without the scanner rewriting waypoints (§11.3).
CSP: `img-src 'self' data: blob:` and tile hosts; no `file:`	The renderer must show photos	Thumbnails cross IPC as bytes and become `blob:` URLs, which the policy already allows. **No CSP change and no custom protocol.** (§6.3)
The renderer never receives a path to read	Previews, Open, Show in Finder	Everything is by photo id. The main process resolves the path and checks it is inside a linked folder (§7.2).
The library tree is rebuilt whole on every change	A folder can hold tens of thousands of photos	Photos are **not tree nodes**. The tree has one node per linked folder (Requirement 4.1); the photos are listed in a virtualised drawer (§8.3).

Invariant	Collision	Resolution
Undo carries bounded inverses (ADR 0014)	Unlinking drops thousands of cache rows	The inverse restores the folder row with the same id, and a rescan rebuilds the cache. Anything made from a photo carries `(folderId, relativePath, fingerprint)`, which survives (§4.4).
Backups are a SQLite snapshot of the whole database	Thumbnails must stay out (Requirement 10.3)	Thumbnails live on disk under the app support directory; the cache metadata is in the database and is backed up (§12).
Missing stays missing	EXIF fields are often absent; cameras write 0°, 0°	Every optional value is a nullable column. 0°, 0° is rejected with a reason (§3.3).
Collection export writes `metadata` verbatim	Photo provenance would leave the machine (Requirement 8.6)	The collection writer strips the `photo` key (§11.3). GPX, KML and CSV never write `metadata`.
The POI layer clusters every point, globally	50,000 photos at high zoom become 50,000 DOM markers	The photo layer clusters only the points in view (§9.2).
Bottom-left: scale bar at `bottom-3`, panel toggles at `bottom-12`	The preview box goes in the same corner	The three move into one flex column, so the preview box stacks above the others and none of them is positioned by guesswork (§10.1).
Trip management references no concrete provider	Photo metadata reader	`PhotoMetadataReader` is an interface in `shared/photos/`. The scanner is handed an implementation (§3.5).
The main process must stay responsive	Scanning 40,000 files at launch	Asynchronous I/O and cooperative yields, measured in §0, after the window shows. No worker thread (§5.5).

2. Storage — migration `0006_linked_folders.sql` Three new tables. Nothing existing changes, except that `waypoints.source` gains the value `'photo'`. That column has no `CHECK`, so only the TypeScript type and `isWaypointSource` change.

```
CREATE TABLE linked_folders (
  id                  TEXT PRIMARY KEY,
  trip_id             TEXT NOT NULL REFERENCES trips(id) ON DELETE CASCADE,
  path                TEXT NOT NULL,          -- absolute, as chosen; never sent to be read
  real_path           TEXT NOT NULL,          -- fs.realpath, NFC; for containment and de-dup
  name                TEXT NOT NULL,          -- basename at link time, for display
  include_subfolders  INTEGER NOT NULL DEFAULT 1,
  last_scan_at        INTEGER,                -- null until the first scan finishes
  last_seen_at        INTEGER,                -- last time the folder was reachable
  created_at          INTEGER NOT NULL,
  updated_at          INTEGER NOT NULL,
  UNIQUE (trip_id, real_path)
```

```
);

-- The file index. One row per recognised file, of any kind.
CREATE TABLE folder_files (
  id             TEXT PRIMARY KEY,
  folder_id      TEXT NOT NULL REFERENCES linked_folders(id) ON DELETE CASCADE,
  relative_path  TEXT NOT NULL,              -- POSIX separators, NFC
  kind           TEXT NOT NULL,              -- 'photo' | 'geodata' | 'video'
  format         TEXT NOT NULL,              -- 'jpeg' | 'heic' | 'gpx' | 'kml' | 'kmz' | 't
  size_bytes     INTEGER NOT NULL,
  mtime_ms       INTEGER NOT NULL,
  fingerprint    TEXT,                       -- §5.3; null for a file not read (every non-ph
  UNIQUE (folder_id, relative_path),
  CHECK (kind IN ('photo','geodata','video'))
);
CREATE INDEX idx_folder_files_kind ON folder_files(folder_id, kind);
CREATE INDEX idx_folder_files_fingerprint ON folder_files(folder_id, fingerprint);

-- What was read from a photo. A cache, rebuilt from the file.
CREATE TABLE photo_metadata (
  file_id          TEXT PRIMARY KEY REFERENCES folder_files(id) ON DELETE CASCADE,
  state            TEXT NOT NULL,            -- 'located' | 'unlocated' | 'unreadable' | 'no
  reason           TEXT,                     -- why unlocated or unreadable; null when locat
  lat              REAL,  lon REAL,          -- both or neither
  altitude_m       REAL,
  accuracy_m       REAL,
  direction_deg    REAL,
  direction_ref    TEXT,                     -- 'T' | 'M'; null with no direction
  taken_local      TEXT,                     -- 'YYYY-MM-DDTHH:MM:SS', the wall clock as rec
  taken_offset_min INTEGER,                  -- null = camera time, no zone
  orientation      INTEGER,                  -- 1..8, null when absent
  CHECK (state IN ('located','unlocated','unreadable','not-downloaded')),
  CHECK ((lat IS NULL) = (lon IS NULL)),
  CHECK (state <> 'located' OR lat IS NOT NULL),
  CHECK ((direction_deg IS NULL) = (direction_ref IS NULL))
);
CREATE INDEX idx_photo_metadata_taken ON photo_metadata(taken_local);
CREATE INDEX idx_photo_metadata_state ON photo_metadata(state);
```

- **One index for every kind.** `folder_files` is what Phases 12 and 13 build on. Phase 12 adds its sync record beside it, keyed by `file_id`, and fills `fingerprint` for track files with a whole-file SHA-256. Phase 13 adds video metadata the same way. Neither needs a second walk or a second watcher. Non-photo rows are counted in this phase and never opened (Requirement 3.8).
- **A folder's availability is not stored.** Whether it is reachable, missing or refused by macOS is worked out at runtime and held by the scanner. A stored "missing" flag would be wrong the moment the drive came back. `last_seen_at` is kept so the UI can say "last seen 3 days ago".
- **Files removed from disk are deleted from `folder_files`** (Requirement 3.3), and their metadata goes with them by cascade. Files in a missing folder are not, because absence of the folder says nothing about the files (Requirement 3.4).
- **A photo's identity is its path within the folder, then its fingerprint.** Row `id`s are stable while the row exists, including across a move the scanner recognises (§5.3), but nothing outside the cache refers to them.

- `real_path` is resolved at link and relink time. It drives the containment refusals (§4.2) and lets the all-trips map show a folder linked to two trips once (§9.1).
- Drizzle definitions are added to `schema.ts`, and the SQL is generated with `npm run db:generate` and checked in as migration version 7.

Repositories: `db/repositories/linkedFolders.ts`, `db/repositories/folderFiles.ts` and `db/repositories/photoMetadata.ts`. They provide upsert by `(folder_id, relative_path)`, a batched `deleteMissing(folderId, keepPaths)`, `movePath`, counts by kind and state, taken-time order, and compact map points.

3. Reading location and time

3.1 Formats `.jpg`, `.jpeg` and `.heic`, matched case-insensitively, then confirmed by the file's signature: `FF D8` for JPEG, an `ftyp` box with brand `heic`, `heix` or `mif1` for HEIC. A file whose signature does not match its extension is `unreadable`, with the reason "Not a JPEG image" or "Not a HEIC image" (Requirement 2.8).

PNG, TIFF, DNG and other raw formats are **not offered** (Requirement 2.2). They are counted, not listed: a folder's properties say "41 files in formats not supported yet (PNG, DNG)" so their absence is explained, not hidden.

Files for later phases are recognised by extension only and never opened (Requirement 3.8): `.gpx`, `.kml`, `.kmz`, `.tcx` and `.fit` as `geodata`, and `.mov`, `.mp4` and `.m4v` as `video`. The folder's properties say "14 GPX and track files and 210 videos — not read yet". The extension list sits in `main/linkedFolders/classify.ts`, the one place Phases 12 and 13 extend.

3.2 Containers — `shared/photos/containers.ts` (pure) Works over a `RangeReader` interface, `read(offset, length): Promise<Uint8Array>`, so it is pure and unit-testable against in-memory buffers.

- **JPEG:** walk the markers from `SOI` until `SOS`. Return the first `APP1` segment that starts `Exif\0\0`. One read of the first 64 KB covers it; APP1 is capped at 64 KB by the format.
- **HEIC:** read the first 64 KB and walk the top-level boxes to `meta`. In `meta`, find the `Exif` item in `iinf` and its extent in `iloc` (versions 0–2, construction method 0, offset and length sizes 0, 4 or 8). If the extent lies past the first read, make one more read of exactly that range. The item starts with a 4-byte offset to the TIFF header, which is skipped.
- Any inconsistency (a box running past the end, an extent past the file, an `iloc` version we do not handle) returns a typed failure with a reason, not an exception.

3.3 Values — `shared/photos/tiff.ts` and `shared/photos/exifValues.ts` (pure) `tiff.ts` parses a TIFF block in either byte order and reads only the tags below, following the `ExifIFD` (`0x8769`) and `GPSInfo` (`0x8825`) pointers. Every offset is bounds-checked, and cycles in the IFD chain are refused.

Tag	Use
`0x0112` Orientation	1–8, else null
`0x9003` DateTimeOriginal	taken time
`0x9011` OffsetTimeOriginal	its zone, `±HH:MM`
GPS `0x01`/`0x02`, `0x03`/`0x04`	latitude, longitude with their references
GPS `0x05`/`0x06`	altitude reference (1 = below sea level) and altitude
GPS `0x1F`	horizontal positioning error ▯ accuracy in metres
GPS `0x10`/`0x11`	direction reference and direction

`exifValues.ts` turns raw values into domain values:

- Degrees, minutes and seconds to decimal degrees. `S` and `W` are negative. Any other reference (absent, or not `N`/`S`/`E`/`W`) makes the geotag malformed rather than guessing a hemisphere.
- **Rejected geotags** (Requirement 2.5), each with its reason: exactly 0°, 0° ("Location recorded as 0°, 0° — usually a camera with no GPS fix"); latitude outside ±90 or longitude outside ±180; a zero denominator; a missing reference or component. The photo is `unlocated` and keeps the reason.
- Altitude: `ref = 1` negates. Absent altitude is null (Requirement 2.4).
- Accuracy and direction: null when absent; direction is normalised to [0, 360). Only `T` (true north) and `M` (magnetic) are accepted, and the reference is stored with the value so the UI can say which.
- **Text fields are not read.** Title, description and comments are not needed by any requirement. Not reading them is the simplest way to treat them as untrusted (Requirement 2.7).

3.4 Time (Requirement 2.6) `DateTimeOriginal` is stored as `taken_local`, exactly as recorded, after validating it as a real date and time. `OffsetTimeOriginal`, when present and valid, becomes `taken_offset_min`. Otherwise the offset is null and the UI says "camera time".

The GPS timestamp is not used to infer a zone. It is the time of the last fix, which can be minutes or hours stale on a camera that caches one. An offset worked out from a stale fix would be confidently wrong, which is worse than an honest "camera time". ADR 0020 §4.

Ordering ("taken-time order") sorts by `taken_local` as text, which is correct within one camera's clock. Photos with no taken time sort last, by file name.

3.5 The reader interface

```ts
// shared/photos/PhotoMetadataReader.ts
export interface PhotoMetadataReader {
  read(file: RangeReader, format: PhotoFormat): Promise<PhotoReadResult>;
}
export type PhotoReadResult =
  | { ok: true; location: Geotag | null; locationRejected: string | null;
      taken: TakenTime | null; orientation: Orientation | null }
  | { ok: false; reason: string };
```

`shared/photos/exifReader.ts` implements it from §3.2 and §3.3. The scanner is given a reader, so a second implementation (for raw formats, later) does not touch the scanner, the services or trip management.

4. Linking, unlinking and relinking

4.1 Choosing a folder Two IPC steps, like the import pipeline's staging:

1. `linkedFolders:chooseFolder({ tripId })` opens the native folder chooser in the main process. The choice is checked (§4.2) and held under a short-lived token. The renderer gets `{ token, name, displayPath, warnings }`. The path is for display only.
2. The renderer shows "Link *name* to *trip*", with **Include subfolders** ticked (Requirement 1.3), then sends `linkedFolders:link({ token, includeSubfolders })`.

Entry points (Requirement 1.1): an icon button with a hint in the trip's properties actions (`TripDetails`, `data-testid="trip-actions"`), the trip's context menu, and **File › Link Folder to Trip…**, which acts on the selected trip and is disabled without one. The wording names the folder and the trip, not photos, so it stays right when Phase 12 adds GPX to the same link.

4.2 Refusals, with their explanations

Case	Message
A `.photoslibrary` package, or a folder inside one (Requirement 1.5)	"This is an Apple Photos library. Its layout is private to Photos and changes between versions, so TrailRT does not read it. Export the photos to a folder (Photos › File › Export) and link that."
Already linked to this trip	"This folder is already linked to *trip*."
Inside, or containing, a folder linked to this trip (Requirement 1.6)	"This folder is inside *Name*, which is already linked to *trip*, so its photos are already shown." (or "contains")
Contains, or is inside, TrailRT's own data folder	"This folder contains TrailRT's own data, so it cannot be linked." This also keeps the thumbnail cache out of every linked folder (§7.3).
macOS refuses to list it, `EPERM` or `EACCES` (Requirement 1.7)	"macOS has not given TrailRT access to this folder. Allow it in System Settings › Privacy & Security › Files and Folders (or Full Disk Access), then rescan." It is never reported as empty.

Containment compares real paths (`fs.realpath`), with NFC normalisation, after resolving each side, so a symlinked or differently-cased spelling of the same folder is still caught.

The same folder may be linked to **different** trips. Requirement 1.2 ties a link to a trip, not the folder itself.

TCC and ad-hoc signing. macOS records folder permission against the app's code signature, and an ad-hoc signature changes with every build, so each new DMG can ask again. The message above covers that case. There is nothing to fix until a Developer ID exists (ADR 0009).

4.3 Services — `main/services/linkedFolderService.ts` `chooseFolder`, `link`, `unlink`, `restore`, `relink`, `setIncludeSubfolders`, `list`. Each mutation returns `{ result, inverse }` like every other service.

4.4 Undo

Action	Inverse	Notes
Link	`linkedFolder:unlink`	
Unlink	`linkedFolder:restore` with the folder row	The cache rows are **not** carried. Restore re-inserts the row with the same id, and the scanner rescans it. Waypoints made from its photos still carry `(folderId, relativePath, fingerprint)`, so their links come back.
Relink to a new path	`linkedFolder:relink` with the old path	

Action	Inverse	Notes
Include subfolders on or off	`linkedFolder:setIncludeSubfolders` with the old value	Switching off deletes cache rows for nested photos on the next scan; switching back rescans them.

`applyCommand` is synchronous. After any undo or redo, the IPC layer calls `folderScanner.reconcile()`, which starts a scan for any folder row with no live scan state. That is cheap when there is nothing to do.

Unlink asks for confirmation first: "Unlink *Name*? The files stay where they are on disk. TrailRT forgets their locations and thumbnails." (Requirement 1.8). It deletes the folder's thumbnail directory. Undo regenerates thumbnails on demand.

Trip deletion cascades to the trip's folder rows. It is already non-undoable (ADR 0001), and orphaned thumbnail directories are removed by a sweep at launch. **Trip duplication** copies the link rows, index rows and metadata rows with `INSERT … SELECT` under new ids. No files are touched. A duplicated waypoint made from a photo still names the original trip's folder id, so `photos:forWaypoint` finds its photo in the copy by fingerprint, through the trip-wide lookup (§11.3).

4.5 Relinking (Requirement 3.5) `linkedFolders:relink({ folderId })` opens the chooser and applies the same refusals. It updates `path` and `real_path`, keeps every row, and rescans. Files are matched by `relative_path`, so a moved folder comes back whole.

5. Scanning and watching — `main/linkedFolders/`

5.1 The walk

- `fs.promises.opendir`, iterating entries asynchronously.
- Skipped: names starting with `.`; macOS packages (directories ending `.app`, `.photoslibrary`, `.bundle`, `.pkg`, or carrying the package bit, which Node cannot read, so the suffix list is the check); symbolic links of any kind, from `Dirent.isSymbolicLink()`, so a link out of the folder is never followed (Requirement 8.2); and subfolders when `include_subfolders` is off.
- Each file is classified by `classify.ts` (§3.1): `photo`, `geodata`, `video`, or not recognised. Unrecognised files are not indexed.
- Skipped symbolic links and unsupported formats are counted, so the folder's properties can say what was left out.

5.2 Incremental reading (Requirements 3.2 and 9.2) For each file found, `lstat` gives size and `mtimeMs`. A file whose index row has the same size and modified time is **not opened**. A new or changed photo is opened read-only (`O_RDONLY | O_NOFOLLOW`), and the reader makes its one or two ranged reads (§3.2). The descriptor is closed in a `finally`. A new or changed `geodata` or `video` file only has its row's size and modified time updated, and is not opened in this phase. Rows for paths not seen in a complete walk are deleted at the end, in one transaction, after move detection (§5.3).

A walk interrupted by an error, a cancellation or the folder vanishing deletes nothing. Only a walk that finished can say a file is gone.

5.3 Fingerprints and moves (Requirement 3.7) A photo's fingerprint is `SHA-256(size as 8 bytes || first 64 KB)`, hex. Those bytes are already in memory from the metadata read, so it costs 23 µs (§0). The first 64 KB of a photo holds its EXIF block, with the capture time to the second, and the start of its compressed image, so two different photos sharing a fingerprint is not a practical concern. The pairing rule below does not rely on that: an ambiguous match is simply not paired.

At the end of a complete walk the scanner has two sets: rows whose path was not seen, and files read for the first time in this walk. A vanished row and a new file are **paired** when they have the same size and fingerprint and no other row or file in either set shares them. A pair is a move. The row's `relative_path` is updated in place (`movePath`), so its id, metadata and thumbnail survive. Anything unpaired is a removal or an addition, as before.

This covers moves and renames **within one linked folder**. A photo moved from one linked folder to another is a removal and an addition. A waypoint made from it still finds it through its fingerprint, because `photos:forWaypoint` looks in every folder of the trip (§11.3).

Not a whole-file hash. It would cost a read of every byte of every photo (§0), and nothing in this phase needs one. Phase 12 needs whole-file hashes for GPX files, and they are small.

5.4 Not downloaded (Requirement 3.6) Reading the first bytes of an iCloud Drive "dataless" file makes macOS download the whole file, so the check has to happen **before** opening it. A dataless file has the `SF_DATALESS` flag (`0x40000000`) and occupies no blocks. Node's `Stats` exposes `blocks` but not `st_flags`, so:

1. `size > 0 && blocks === 0` marks a candidate, which costs nothing.
2. Candidates are confirmed in batches of up to 200 paths with `/usr/bin/stat -f '%f'`, run with `execFile` and an argument array, never a shell. Those with the flag are stored as `not-downloaded` and never opened.
3. Legacy `.Name.ext.icloud` placeholders are listed as `not-downloaded` under their real name.

This is the same detection Finder's cloud badge relies on. It is **verified on a real iCloud Drive folder only in task 13**, and the requirement stays partially verified until then.

5.5 Responsiveness (Requirements 9.1 and 9.5) The scanner is asynchronous end to end. It awaits every directory read, `lstat` and file read, and yields with `setImmediate` after every 200 files whether or not anything was awaited, so IPC handlers interleave. One scan runs per folder, and a request to rescan a folder that is already scanning sets a "run again" flag rather than starting a second walk. At most two folders scan at once, so a library of 100 linked folders is worked through in turn rather than all at once.

The map and library read the index, which is already in the database, so a trip's photos are on the map at launch before any scan has started. Scans begin after the window has shown (Requirement 9.5).

Progress is sent as `linkedFolders:scanProgress { folderId, phase, seen, read, total? }` at most every 250 ms. A scan is cancelled through an `AbortSignal` from `linkedFolders:cancelScan`. A cancelled scan keeps every row it had already updated.

No worker thread. §0 shows parsing at 22 µs per file. The rest is I/O, which is already off the main thread. A worker would mean a second bundle entry and a second database connection, for no measured gain.

5.6 When scans run (Requirement 3.1)

- At launch, every folder, after the window has shown.
- On request: **Rescan** in the folder's properties and context menu.
- When the watcher reports a change (§5.7), debounced by 2 s.
- When the app regains focus and a folder is missing or was last scanned more than 10 minutes ago.
- Every 30 s **while any folder is missing**, a single `stat` of each missing folder. When one reappears, it is scanned (Requirement 3.4).

5.7 Watching `fs.watch(path, { recursive: true })`. On macOS this is FSEvents, which covers internal and externally attached drives. It reports changes made on this Mac only, so a **network volume** misses changes made by other machines. Its events are treated as hints that trigger the incremental rescan, never as the record of what changed. That makes a missed or coalesced event harmless, because the rescan works the truth out from `stat`. For network volumes the UI says "Changes made on other computers appear when you rescan." A watcher error (the volume unmounting) marks the folder missing and starts the 30 s check.

There is one watcher per linked folder that is currently reachable. Each is an FSEvents stream, which macOS handles cheaply. Task 7 measures 100 at once. If that proves costly, folders on the same volume can share one watcher on their common parent, with events filtered by path, without changing the scanner.

5.8 Folder states

State	Meaning	Photos
`available`	Reachable and scanned	Normal
`scanning`	A walk is in progress	Normal, with progress
`missing`	The path does not exist (Requirement 3.4)	Kept, drawn as unavailable with last known locations
`denied`	`EPERM`/`EACCES` (Requirement 1.7)	Kept, drawn as unavailable, with the System Settings message

6. Thumbnails — `main/photoCache/`

6.1 Making them `nativeImage.createThumbnailFromPath(path, { width: 640, height: 640 })`, then `toJPEG(80)`. Quick Look applies orientation and handles HEIC (§0).

640 px covers the preview box at 2× density. It is made **only when the preview asks for it** (Requirement 9.3). The map's markers do not use thumbnails.

Before calling Quick Look, the path is resolved and checked by the same guard as every other file access (§7.2). A `not-downloaded` photo is never passed to Quick Look, because that would download it.

6.2 The cache `<userData>/photo-thumbnails/<folderId>/<photoId>-<size>-<mtimeMs>.jpg`. The size and modified time in the name mean a changed photo gets a new thumbnail, and the stale one is swept.

Bounded at 512 MB (about 9,500 thumbnails at the measured size). A served thumbnail's file `mtime` is set to now. When the total passes the bound, the least recently served files are deleted down to 90%. Deleting the whole directory is always safe, which is what "rebuildable at any time" means. A launch sweep removes directories with no folder row.

6.3 Serving them (Requirement 8.3) `photos:thumbnail({ photoId })` returns `{ ok: true, data: { bytes: Uint8Array, width, height } }`. The renderer wraps the bytes in a `Blob`, makes a `blob:` URL, and revokes it when the preview changes. `blob:` is already in `img-src`. `file:` stays out of the CSP, and there is no custom protocol to secure.

7. IPC, privacy and security

7.1 Channels — `shared/ipc/` namespaces `linkedFolders` and `photos` Folder management is its own namespace, because Phase 12 adds GPX sync to the same folders and must not need channels named after photos.

Channel	Request	Notes
`linkedFolders:chooseFolder`	`{ tripId }`	Opens the chooser; returns a token
`linkedFolders:link`	`{ token, includeSubfolders }`	
`linkedFolders:unlink`	`{ folderId }`	Undoable
`linkedFolders:relink`	`{ folderId }`	Opens the chooser
`linkedFolders:setIncludeSubfolders`	`{ folderId, include }`	Undoable
`linkedFolders:rescan` / `linkedFolders:cancelScan`	`{ folderId }`	
`linkedFolders:list`	`{ tripId? }`	Folder rows with counts by kind and state, and runtime state
`photos:list`	`{ folderId }`	Rows for the drawer, taken-time order
`photos:mapPoints`	`{ tripId } | { all: true, year?, tripId? }`	Compact located points (§9.1)
`photos:years`	`{}`	Years with photos, and a count with no date, for the Photos view filter
`photos:get`	`{ photoId }`	One photo's details for the preview
`photos:thumbnail`	`{ photoId }`	Bytes
`photos:open` / `photos:reveal`	`{ photoId }`	`shell.openPath` / `shell.showItemInFolder`
`photos:addWaypoints`	`{ photoIds, tripId }`	One undo step
`photos:forWaypoint`	`{ waypointId }`	The photo a waypoint came from, if available

A photo id is its `folder_files.id`. Every request is validated with zod and every handler returns a result. Events: `linkedFolders:scanProgress` and `linkedFolders:changed`, which tells the renderer to reload folder and photo data. The preload exposes each by name; there is no generic passthrough.

7.2 Path resolution — `main/linkedFolders/resolve.ts` `resolveFilePath(db, fileId)`:

1. Looks up the row and its folder.
2. Joins `folder.path` and `relative_path`, normalised. A relative path that escapes with `..` is refused. They are written only by the scanner, but this is checked anyway.
3. `realpath`s both, and refuses unless the file's real path is inside the folder's real path. A symbolic link that has appeared since the scan, and points out, is refused here (Requirement 8.2).
4. Refuses unless the folder is currently linked and `available`.

`photos:thumbnail`, `photos:open`, `photos:reveal` and the scanner's own reads all go through it, so "Open" and "Show in Finder" act only on paths the main process resolved (Requirement 8.4).

7.3 Read-only, enforced (Requirement 8.1) Three layers, each tested:

1. **The access module cannot write.** `main/linkedFolders/` opens files only with `O_RDONLY | O_NOFOLLOW` through one helper. A unit test reads every source file in `src/main/linkedFolders/` and fails if any calls a write API (`writeFile`, `appendFile`, `createWriteStream`, `rename`, `rm`, `unlink`, `utimes`, `chmod`, `chown`, `copyFile`, `mkdir`, `truncate`, `symlink`, `link`, or the `w/a/+` open flags). Thumbnails live in `main/photoCache/`, which writes only below its own directory, and that directory cannot be inside a linked folder (§4.2).
2. **Every other writer is guarded.** The app writes outside its data directory in exactly two places: an export to a user-chosen file, and **Save a Copy…** of the database. Both call `assertNotInLinkedFolder(path)` first. It refuses a target inside any linked folder: "That folder is linked for photos, and TrailRT never writes into a linked folder. Choose somewhere else." Linked folders are read-only from TrailRT's point of view, including for new files.

   Phase 12 adds the one exception: GPX files TrailRT creates, written only by a separate sync module with its own rules (ADR 0020 §5). That module will live outside `main/linkedFolders/`, so the source scan above keeps holding for everything that reads photos.
3. **Proven by behaviour.** Requirement 11.4's test hashes every fixture file and records its modified time, links, scans, makes thumbnails, adds waypoints, unlinks, relinks and rescans, then checks every file is byte-identical with the same modified time.

7.4 No network (Requirement 8.5) Nothing in the photo path makes a network request, and there is no reverse geocoding. The CSP's `connect-src` is unchanged.

8. Photos in the library

8.1 Tree (Requirement 4.1) A trip with linked folders gets a **Linked folders** `group` row, after its routes, tracks and waypoints (or after the plan's days for a dated trip). It has one `linkedFolder` node per linked folder. `TreeNodeKind` and `ObjectKind` gain `'linkedFolder'`, so a folder is selectable, and its properties panel shows path, state, counts by kind, Include subfolders, and Rescan, Relink and Unlink.

Node detail: "312 photos · 18 without location", with "· unavailable" or "· no access" when the folder is not available. Files of kinds this phase does not read appear in the properties panel, not the node detail, so the tree stays short.

8.2 The Photos view (Requirement 4.2) `ViewId` gains `'photos'`, added to `VIEWS` after Waypoints: every linked folder across visible trips, annotated with its trip's name, as the Routes view does. It is a view, not a container (ADR 0004).

Selecting the Photos view itself puts the map into all-trips mode (Requirement 5.6): every trip's located photos, through `photos:mapPoints({ all: true })`. Selecting a folder within the view loads its trip on the map, as selecting a route in the Routes view does.

The view's header has two filters (Requirement 4.6), from `photos:years`: **Year**, "All years" by default, with a "No date" entry when any photo lacks a taken time; and **Trip**, "All trips" by default. Both narrow the folder list and the all-trips map together. The year is the year part of `taken_local`, the same wall-clock rule as §8.4.

8.3 The photo list drawer (Requirements 4.3 and 4.4) Selecting a `linkedFolder` node opens a new drawer kind, `{ kind: 'photos', folderId }`, beside `calendar` and `poi`. Rows are in taken-time order: file name, taken time (with zone, or "camera time"), and state ("No location — recorded as 0°, 0°", "Not downloaded", "Unreadable — not a HEIC image", "Unavailable"). It is virtualised with fixed row heights, as `PointTable` is, so 40,000 rows cost the same as 30.

Selecting a row selects the photo: it opens the preview box, and a located photo off screen is panned into view. Multi-select (shift and command, as in the library) enables **Add as waypoints**.

8.4 Photos per day (Requirement 4.5) For a dated trip, each day row's detail gains "· 23 photos". A photo is counted on the day whose date equals the **date part of** `taken_local`, the wall clock where and when it was taken. With a recorded offset that is exactly right. Without one it assumes the camera's clock was set to local time, which is the usual case and the only reasonable one. When any of a day's photos have no zone, the detail reads "· 23 photos (camera time)", and its hint says the assignment assumes the camera clock was on local time.

9. Photos on the map

9.1 Data `photoStore` (zustand) has two modes. In **trip mode** it loads `photos:mapPoints({ tripId })` for the trip `geometryStore` is showing. In **all-trips mode**, while the Photos view is selected, it loads `photos:mapPoints({ all: true, year?, tripId? })`. It reloads on `linkedFolders:changed`.

Points are sent **column-wise**: `ids: string[]`, `lat` and `lon` as `Float64Array`, `folder` as an index into a short `folders` array, and `dir` as `(number | null)[]`. That keeps 50,000 points to about 2.5 MB, once per change. Missing direction stays `null`, never a sentinel number.

In all-trips mode, a folder linked to more than one trip would put its photos on the map twice. The query takes one row per `(linked_folders.real_path, relative_path)`, the earliest-linked, so each photo appears once (Requirement 5.6).

9.2 Layer — `features/map/usePhotoLayer.ts`

- A separate `L.layerGroup`, shown and hidden by a new **Photos** checkbox in the layers menu (`layerVisibility.photos`, default on) (Requirement 5.1).
- **Viewport clustering.** On `moveend` and `zoomend`, points inside the map bounds padded by half a screen are clustered with `clusterPoints` at the current zoom, and only those clusters are drawn. A cell is 56 px, so the marker count is bounded by the screen, not the library (Requirement 5.2, target in §0). A cluster shows its count. Clicking it zooms in to fit its members, or opens the preview at the first member when they cannot be separated (§9.3).
- Markers use a camera glyph in a dedicated colour, distinct from waypoints and POIs.

9.3 Bursts (Requirement 5.3) A cluster is **inseparable** when its members' bounding box spans less than one metre, or the map is at its maximum zoom. Clicking it opens the preview at the earliest member, with "1 of N" stepping through that burst in taken-time order.

9.4 Direction (Requirement 5.4) The selected photo's marker, when it has a direction, shows a small wedge rotated to the bearing, with "true" or "magnetic" in its hint. Other markers do not, which keeps 50,000 markers plain.

9.5 Unavailable photos (Requirement 5.5) Photos in a `missing` or `denied` folder are drawn with a dashed outline at reduced opacity, and their hint says "Unavailable — *Folder* is not connected". The layers menu has **Show unavailable photos** (`photoUnavailable: 'show' | 'hide'`, default `show`). They are never drawn the same as available ones.

10. The preview box

10.1 Placement (Requirements 6.1 and 6.2) `MapReadout` and `PanelToggles` are each absolutely positioned today. They are moved into one `MapCornerStack`: `absolute bottom-3 left-3`, a `flex-col-reverse` column with a gap. The scale bar and readout come first, the toggles second and the preview box third, so the box always sits **above** the other two, and none of them needs a hand-picked `bottom-*` offset. The stack's `max-height` is the map's height minus the top controls' band, so it cannot reach the search box (top left) or the zoom and layer controls (top right).

The box is inside the map's container, which already resizes with the panels and the drawer. Its width is `min(320px, map width − 24px)`. The image keeps its aspect ratio inside a box whose height is limited by the space left in the stack, so on a short or narrow map it shrinks rather than overflowing.

10.2 Contents (Requirement 6.3)

- The thumbnail, already oriented by Quick Look.
- The file name, with its path inside the linked folder as a hint.
- Taken time: `14 Jun 2026, 10:32 (UTC+01:00)`, or `14 Jun 2026, 10:32 · camera time` with a hint that explains it.
- Location through `formatCoordinates` (`shared/geo/units.ts`). The app has one coordinate format and no user setting for it yet, so "the user's coordinate format" in Requirement 6.3 is that format. A setting, if one is added later, reaches the preview through the same function. Then altitude (`formatElevation`, in the user's units) and "±8 m" accuracy when present. Nothing when absent, never a dash or a zero. For an unlocated photo, "No location" and the reason.

10.3 Actions (Requirements 6.4 and 6.5) Icon buttons with hints, from lucide-react: previous and next in taken-time order (within the burst when opened from one, otherwise within the folder), zoom to the photo, add as a waypoint, open in the default app, show in Finder, and close. Location-dependent actions are disabled for an unlocated photo, with a hint saying why.

The box is a `region` with a label. While focus is inside it, the left and right arrow keys step and Escape closes.

10.4 Unavailable and not downloaded (Requirement 6.7) The box says "Unavailable — *Folder* is not connected", or "Not downloaded from iCloud — open it in Finder to download", and shows the cached thumbnail when one exists. The cache can serve a thumbnail without resolving the source path, so this works while the drive is unplugged. Open is disabled; Show in Finder stays enabled when only the file is missing but the folder is there.

10.5 What is loaded (Requirement 6.6) Only the 640 px thumbnail. The original is never read into the renderer; **Open** hands it to the default app.

11. Using a photo's location

11.1 One photo (Requirement 7.1) `photoWaypointService.addFromPhotos(db, { photoIds, tripId })` creates, for each located photo, an ordinary waypoint: the photo's latitude, longitude and altitude (null stays null), symbol `photo`, the file name without its extension as the name, `source: 'photo'`, and `metadata.photo = { folderId, relativePath, fingerprint }`. One photo gives one `waypoint:delete` inverse.

11.2 Several (Requirement 7.3) Several photos are wrapped in one `batch` (ADR 0014), so one ⌧Z removes them all. Unlocated photos in the selection are skipped and reported ("12 added; 3 have no location").

11.3 Provenance (Requirement 7.2)

- The waypoint's properties panel calls `photos:forWaypoint`. It looks the photo up by `(folderId, relativePath)`, then by fingerprint in that folder, then by fingerprint in the trip's other linked folders, so a photo moved or renamed since keeps its link (Requirement 3.7). Nothing rewrites the waypoint. If a photo is found and its folder is available, it shows **Show photo**, which selects it and opens the preview.
- The waypoint is otherwise ordinary. It is edited, moved, exported and deleted like any other, and survives the photo's removal and the folder's unlinking.
- `waypoint:delete` and `waypoint:restore` already carry `metadata`, so undoing a deletion keeps the link.
- **Export:** the TrailRT collection writer drops the `photo` key from waypoint metadata, so a relative path never leaves the machine (Requirement 8.6). GPX, KML, CSV and the itinerary exports never write `metadata` at all.

12. Backups and another Mac (Requirement 10)

- `linked_folders`, `folder_files` and `photo_metadata` are ordinary tables, so every backup includes the links, the index and the cache metadata (10.1). Thumbnails are files under `photo-thumbnails/`, outside the database, so they are never in a backup (10.3).
- The backup section of Settings and the restore dialog say: "Linked folders are remembered, but the files in them are not backed up by TrailRT. Back them up with Time Machine or your usual tool."
- After a restore on another Mac, each folder is `missing` until relinked (§4.5). Nothing is deleted while it is missing (10.2).

13. Testing

13.1 Fixtures — `scripts/generate-fixtures.mjs` gains a photos section Generated into `test-data/generated/photos/` (gitignored, like the other generated fixtures), by an Electron step that makes base JPEGs with `nativeImage`, writes EXIF with `scripts/lib/exif-write.mjs`, and converts to HEIC with `sips`. Every location comes from `@fixtures/places`, the Buxton square (Requirement 11.2).

Fixture	Proves
`buxton.jpg`, `buxton.heic`	Located; altitude, accuracy, direction (`T`), offset `+01:00`
`leek-o6.jpg`, `leek-o6.heic`	Orientation 6; no offset (camera time)
`ashbourne-below-sea.jpg`	Altitude reference 1 (negative)
`macclesfield-be.jpg`	Big-endian TIFF block in a JPEG
`no-gps.jpg`	Unlocated, no geotag
`zero-zero.jpg`	Rejected: 0°, 0°
`out-of-range.jpg`, `no-ref.jpg`, `zero-denominator.jpg`	Rejected: malformed
`orientation-1..8.jpg`	Every orientation value
`truncated.jpg`, `truncated.heic`	Unreadable, with a reason; the scan continues
`not-an-image.jpg`, `not-an-image.heic`	Unreadable: signature does not match
`nested/deeper/ashbourne.jpg`	Subfolders on and off
`.hidden/buxton.jpg`	Skipped
`link-out` → outside the folder	Not followed
`Library.photoslibrary/`	Refused when chosen; skipped when nested
`notes.png`	Counted as an unsupported format
`burst/1..5.jpg`	Same location: one inseparable cluster

Fixture	Proves
`tracks/day-1.gpx`, `tracks/day-2.fit`	Indexed as `geodata`, counted, never opened (Requirement 3.8)
`clip.mov`	Indexed as `video`, counted, never opened

Moves are made by the scanner tests at run time, by renaming a file in a temp copy of the tree, so the committed fixtures need no “before and after” pair.

For the §0 targets, task 13 generates a separate measurement tree: 40,000 photos of about 3 MB across 100 folders. It is generated, not committed, and deleted after the run.

The whole longitude range of the square is west of Greenwich, so a dropped `W` moves a photo about 250 km east and fails the location assertions (Requirement 2.3).

13.2 Unit tests (no Electron) `tiff.test.ts`, `exifValues.test.ts`, `containers.test.ts` and `exifReader.test.ts` cover Requirement 11.1 against both in-memory buffers and the fixture files. Also: the scanner against a temp folder (incremental rescan, deletion only after a complete walk, symlink and hidden skipping, the `.icloud` placeholder, the dataless heuristic with a stubbed `stat` batch), `resolve.test.ts`, fingerprint pairing (a rename is a move; two candidates with the same fingerprint are not paired), the refusals in §4.2, migration and repositories, undo of link, unlink, relink and subfolder toggling, add-as-waypoint (single and batch), the collection export dropping `photo`, the write guard, the read-only source scan (§7.3), viewport clustering, all-trips points with a folder linked to two trips appearing once, the year filter, and day counts.

13.3 End-to-end (Playwright, Requirement 11.5) The folder chooser is stubbed in the main process, as `import.spec.ts` stubs the file chooser. `photos.spec.ts` covers the six scenarios in 11.5. “Clear of the panel toggles and the scale bar” is asserted by comparing bounding boxes. “Missing and back” renames the fixture folder copy and renames it back. Persistence restarts the app with the same data directory.

14. Answers to the requirements’ open questions

1. **Placing unlocated photos from a track by time:** not in this phase. **Moved to Phase 12** (its open question 6), which brings the trip’s recorded tracks in from the same folder. The model leaves room for it: a future `placement` column, `photo` or `track`, in `photo_metadata`.
2. **Folder per trip or per library: answered by the owner on 12 November 2026**: per trip, with the ability to see all. Per trip as Requirement 1.2 says, with the same folder linkable to several trips; the Photos view shows every trip’s photos together (§8.2, §9.1).
3. **Metadata reader:** in-house (§0, §3; ADR 0020 §2).
4. **Thumbnails:** Quick Look through `nativeImage`. It applies orientation and reads HEIC, measured in §0.
5. **Watching:** `fs.watch` recursive, as a hint for an incremental rescan. Network volumes are rescanned on request and when the app regains focus, and the UI says so (§5.7).

15. Gaps stated honestly

- **iCloud “not downloaded”** detection is by `SF_DATALESS`, confirmed only against stubbed `stat` output until task 13 checks a real iCloud Drive folder.
- **Network volumes** do not report changes made by other computers. They are rescanned on request and on focus.
- **Formats** beyond JPEG and HEIC are counted and named, not read.
- **Cold-cache scan time** is not measured (§0).
- **Folder permission prompts** can recur with each ad-hoc-signed build (§4.2).

- **Moves between linked folders** are a removal and an addition in the index. Waypoint links survive them through the fingerprint (§11.3), but the thumbnail is made again.
- **The fingerprint is not a content hash.** A change that kept the size and the first 64 KB the same would not be seen as a change, but the scanner does not use it to detect changes. The size and modified time do that.
- **100 watchers at once** is unmeasured until task 7 (§5.7).

9.3.3 Tasks

Execute in order. Pure modules come first, each proven offline, then storage and the main process, then the UI, then the end-to-end and verification pass.

Update `docs/progress-mindmap.svg` as each task changes status.

☒ 1. Spec, ADR 0020 and the out-of-order build
- `design.md`, `tasks.md`, ADR 0020; README row and decision index; the note that this phase is built ahead of Phases 8 and 9 at the owner's request
- Measurements in design §0: metadata readers compared, scan cost, Quick Look thumbnails and orientation, targets
- Mindmap: the phase's leaves become these tasks
- Revised 12 November 2026 with the owner's answers: the linked folder is the trip's folder on disk, so storage is a general file index (design §2); fingerprints for moves (§5.3); an all-trips map (§9.1); 40,000-photo targets (§0), with hashing and clustering measured
- *Requirements: 2.2, 9.4*

☐ 2. Synthetic photo fixtures
- `scripts/lib/exif-write.mjs`: EXIF writer, both byte orders, every tag in design §3.3, with deliberate malformations (no reference, zero denominator, out of range)
- `scripts/generate-fixtures.mjs`: an Electron step for base JPEGs (`nativeImage`), HEIC through `sips`, and the folder tree in design §13.1 (nested, hidden, symlink out, `.photoslibrary`, `notes.png`, burst, truncated and fake files, plus `tracks/day-1.gpx`, `tracks/day-2.fit` and `clip.mov` for the index) under `test-data/generated/photos/`
- Every location from `@fixtures/places`; none invented
- Unit tests: the writer round-trips through the task 3 parser once that exists; until then, byte-level checks of the TIFF header and IFD counts
- *Requirements: 11.2, 11.3*

☐ 3. TIFF parsing and EXIF values (pure)
- `shared/photos/tiff.ts`: both byte orders, `ExifIFD` and `GPSInfo` pointers, bounds-checked offsets, IFD cycles refused
- `shared/photos/exifValues.ts`: DMS to degrees with hemisphere signs; altitude reference; accuracy; direction with `T/M`; 0°, 0°, out-of-range, zero-denominator and missing-reference rejection with reasons; `DateTimeOriginal` validation; `OffsetTimeOriginal` parsing; orientation 1–8
- No text fields read
- Unit tests: south and west negative (a Buxton fixture with `W` dropped lands ~250 km east and fails); altitude −12.25 m; absent altitude is null; each rejection with its reason; time with `+01:00`, `-05:30` and none; invalid dates refused; every orientation value; big- and little-endian blocks give identical results
- *Requirements: 2.1, 2.3, 2.4, 2.5, 2.6, 2.7, 11.1*

☐ 4. Containers and the metadata reader
- `shared/photos/containers.ts`: `RangeReader`; JPEG marker walk to the Exif `APP1`; HEIC `ftyp` brands, `meta` ▯ `iinf` ▯ `iloc` (versions 0–2), one extra ranged read when the item lies past the first 64 KB; typed failures
- `shared/photos/PhotoMetadataReader.ts` interface and `shared/photos/exifReader.ts` implementation

  - `main/linkedFolders/fileReader.ts`: the one place that opens a file in a linked folder, `O_RDONLY | O_NOFOLLOW`, closed in `finally`; signature checks against the extension
  - Unit tests: every fixture from task 2; an `iloc` extent past 64 KB read with exactly one extra read; truncated and fake files give their reasons and do not throw; a reader handed a 5 MB file reads at most 64 KB plus the item's length
  - *Requirements:* *2.2*, *2.8*, *9.2*, *11.1*
- ☐ 5. Storage: linked folders and the file index
  - `schema.ts`: `linked_folders`, `folder_files`, `photo_metadata` per design §2; migration `0006_linked_folders.sql` (version 7) generated and registered
  - `db/repositories/linkedFolders.ts`, `folderFiles.ts`, `photoMetadata.ts`: upsert by `(folder_id, relative_path)`, batched `deleteMissing`, `movePath`, counts by kind and state, taken-time order, column-wise map points for one trip or all trips, de-duplicated by `(real_path, relative_path)`, with year and trip filters
  - `WaypointSource` gains `'photo'`; `isWaypointSource` updated
  - Unit tests: migration applies on a Phase 10 database and is idempotent; the `CHECK`s refuse half a coordinate, a located row without one, and a direction without its reference; trip deletion cascades through all three tables; ordering puts photos with no taken time last; a folder linked to two trips yields its photos once in all-trips points
  - *Requirements:* *3.2*, *3.3*, *3.8*, *5.6*, *7.2*, *10.1*
- ☐ 6. Linking, refusals, undo and the write guard
  - `main/services/linkedFolderService.ts`: `chooseFolder` (token, design §4.1), `link`, `unlink`, `restore`, `relink`, `setIncludeSubfolders`, `list`; `real_path` resolved at link and relink
  - Refusals from design §4.2 with their messages: `.photoslibrary`, same folder twice, nested either way, TrailRT's data folder, `EPERM`/`EACCES`; real-path and NFC comparison
  - Commands `linkedFolder:unlink`, `linkedFolder:restore`, `linkedFolder:relink`, `linkedFolder:setIncludeSubfolders`; `reconcile()` after undo and redo
  - Trip duplication copies link, index and metadata rows
  - `main/linkedFolders/writeGuard.ts` `assertNotInLinkedFolder`, called by export and Save a Copy
  - Unit tests: each refusal; a symlinked spelling of a linked folder is caught; link ▯ unlink ▯ undo restores the same folder id; relink keeps rows matched by relative path; export into a linked folder is refused
  - *Requirements:* *1.2*, *1.3*, *1.5*, *1.6*, *1.7*, *1.8*, *3.5*, *8.1*
- ☐ 7. Scanner, fingerprints and watcher
  - `main/linkedFolders/classify.ts`: photo, geodata and video extensions (design §3.1)
  - `main/linkedFolders/scanner.ts`: async walk (design §5.1) skipping hidden entries, packages, symlinks and, when off, subfolders; counts of what was skipped; geodata and video indexed by size and `mtimeMs`, never opened
  - Incremental: unchanged size and `mtimeMs` are not opened; deletion only after a complete walk, in one transaction
  - Fingerprints and move pairing (design §5.3)
  - Not downloaded: `blocks === 0` candidates confirmed by batched `/usr/bin/stat -f '%f'` (`execFile`, no shell); `.icloud` placeholders
  - Folder states `available` / `scanning` / `missing` / `denied`; launch scan; focus rescan; 30 s check while missing; `fs.watch` recursive as a debounced hint; scans start after the window has shown
  - Progress events at most every 250 ms; `AbortSignal` cancel; yields every 200 files; one walk per folder, at most two folders at once
  - Unit tests against a temp copy of the fixtures: first scan counts by state; a second scan opens no file (spy on the reader); a deleted file's row goes; an interrupted walk deletes nothing; renaming the folder makes it `missing` with rows kept, and renaming back restores it without relinking; the dataless path never opens a candidate; a symlink

out is never followed; a renamed photo keeps its row id and metadata; two candidates sharing a fingerprint are not paired; a GPX file is indexed and never opened (spy on the reader)
  - Measured: 100 watchers at once, recorded in design §5.7
  - *Requirements:* 1.4, 3.1, 3.2, 3.3, 3.4, 3.6, 3.7, 3.8, 8.2, 9.1, 9.2, 9.5
- ☐ 8. Path resolution, thumbnails, IPC and preload
  - `main/linkedFolders/resolve.ts` per design §7.2
  - `main/photoCache/thumbnails.ts`: Quick Look at 640 px, JPEG 80, cache layout and 512 MB bound from design §6.2, launch sweep; never for a `not-downloaded` photo
  - The `linkedFolders` and `photos` IPC namespaces and events from design §7.1, zod schemas, results, preload entries by name; `photos:open` and `photos:reveal` by id
  - File › Link Folder to Trip… menu item
  - Read-only source scan test over `src/main/linkedFolders/` (design §7.3)
  - Unit tests: `..` and symlink-out refused by `resolve`; a changed photo gets a new thumbnail name; eviction keeps the bound; every handler rejects a malformed payload; the source scan fails when a write API is added (proven with a temporary file in the test)
  - *Requirements:* 1.1, 6.6, 8.2, 8.3, 8.4, 8.5, 9.3
- ☐ 9. Library: tree, Photos view and the photo list
  - Tree: Linked folders group and `linkedFolder` nodes (design §8.1); `ObjectKind`, `TreeNodeKind` and `ViewId` additions; folder properties with counts by kind, Rescan, Relink, Unlink (with its confirmation) and Include subfolders
  - Photos view: Year and Trip filters from `photos:years` (design §8.2)
  - Link dialog with Include subfolders; icon button in trip actions; trip context menu item
  - Drawer kind `photos`: virtualised list in taken-time order with states and reasons; selection opens the preview and pans to an off-screen photo; multi-select
  - Day counts for dated trips, with "(camera time)" when guessed
  - Unit tests: tree with and without photos; the Photos view across trips; the year filter, with "No date"; day assignment by wall-clock date, with the camera-time flag; list rows for every state; properties counting files not read yet
  - *Requirements:* 1.1, 1.3, 1.8, 3.8, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6
- ☐ 10. Photo map layer, one trip and all trips
  - `photoStore` with trip and all-trips modes (design §9.1); `features/map/usePhotoLayer.ts` with viewport clustering (design §9.2); inseparable bursts; direction wedge on the selected photo; unavailable styling
  - Layers menu: Photos and Show unavailable photos; `layerVisibility.photos` and `photoUnavailable` settings with schema and defaults
  - Unit tests: viewport filtering bounds the marker count by screen cells; the burst rule (< 1 m, or maximum zoom); 50,000 synthetic points re-cluster within the §0 target on the target hardware; selecting the Photos view switches to all-trips mode and leaving it switches back
  - *Requirements:* 5.1, 5.2, 5.3, 5.4, 5.5, 5.6
- ☐ 11. The preview box
  - `MapCornerStack` holding the readout, the panel toggles and the preview box (design §10.1)
  - `features/photos/PhotoPreview.tsx`: thumbnail via `blob:` URL (revoked on change); name with relative-path hint; taken time with zone or "camera time"; location, altitude and accuracy only when present; unlocated reason
  - Icon-button actions with hints; arrow keys and Escape while focused; unavailable and not-downloaded states with the cached thumbnail
  - Unit tests: the view model's text for every state; nothing rendered for an absent altitude or accuracy; "1 of N" stepping within a burst
  - *Requirements:* 6.1, 6.2, 6.3, 6.4, 6.5, 6.6, 6.7
- ☐ 12. Photo to waypoint, export and backups

  - `main/services/photoWaypointService.ts`: one photo → one waypoint; several → one batch; unlocated skipped and reported (design §11)
  - Waypoint properties: Show photo via `photos:forWaypoint`
  - Collection writer drops `metadata.photo`
  - Backup and restore UI text (design §12)
  - `photos:forWaypoint` lookup by path, then fingerprint in the folder, then fingerprint across the trip's folders (design §11.3)
  - Unit tests: altitude null stays null; symbol `photo`; one undo for a batch of 20; the waypoint survives unlinking and the photo's deletion; undoing a deletion keeps the link; the link survives a rename and a move to another of the trip's folders; a collection export contains no `photo` key or relative path
  - *Requirements: 7.1, 7.2, 7.3, 8.6, 10.1, 10.2, 10.3*
- ☐ 13. End-to-end and verification pass
  - `tests/e2e/photos.spec.ts` with the chooser stubbed: located on the map and unlocated listed; preview box in the bottom-left, clear of the panel toggles and scale bar by bounding box; stepping; add as waypoint and undo; folder missing and back; persistence across a restart
  - Read-only proof: hash and modified time of every fixture before and after link, scan, thumbnails, add-as-waypoint, unlink, relink and rescan
  - Measurements against the §0 targets on a generated tree of 40,000 photos (3 MB files) across 100 folders, recorded in design §0; the all-trips map with 50,000 points; launch with every folder linked
  - iCloud Drive check on a real folder with an undownloaded file, with the owner's permission; design §15 updated with the result
  - `npm run verify` and `npm run package` (with `verify:package`) pass
  - *Requirements: 3.6, 9.1, 9.4, 9.5, 11.4, 11.5*

Notes

9.4 Phase 2: Routes and Waypoints — Requirements — SPEC-PHASE-2-ROUTES-WAYPOINTS

Version: — · **Status:** unspecified · **Slug:** phase-2-routes-waypoints

9.4.1 Requirements

Status: not started. Blocked on Phase 1 verification.

Design and tasks for this phase are written after Phase 1 is built and tested, per section 20 of the source specification. These requirements exist now so Phase 1 can be shaped correctly, not so Phase 2 can start early.

Introduction Phase 2 makes the application useful for planning. Users draw and edit routes on the map, place and edit waypoints, see route statistics, and get an undo history over those edits. All objects belong to a trip per ADR 0004.

Requirements

Requirement 1: Route creation **User story:** As a trip planner, I want to draw a route on the map, so that I can plan where I intend to travel.

Acceptance criteria

1. WHEN the user activates route drawing THEN the system SHALL enter a mode where map clicks append route points, with the mode visibly indicated.
2. WHEN the user appends a point THEN the system SHALL draw the polyline segment and SHALL update the running distance.
3. WHEN the user finishes drawing THEN the system SHALL persist the route to the selected trip with a default name the user can change.
4. WHEN the user presses Escape during drawing THEN the system SHALL cancel without persisting a partial route.
5. THE system SHALL connect route points with straight segments in Phase 2, and SHALL NOT imply road-snapping that no routing provider yet performs.

Requirement 2: Route editing **User story:** As a trip planner, I want to adjust a route after drawing it.

Acceptance criteria

1. WHEN a route is selected THEN the system SHALL display draggable handles at its points.
2. WHEN the user drags a point THEN the system SHALL persist the new position on release and SHALL recompute statistics.
3. THE system SHALL support inserting an intermediate point by clicking a segment, deleting a point, and reordering points.
4. THE system SHALL support reversing, duplicating, renaming and deleting a route.
5. WHEN a route has fewer than two points after a deletion THEN the system SHALL keep it valid as a degenerate route rather than silently deleting it.
6. WHEN the user edits an imported route THEN the system SHALL apply the change only on explicit action, per the data integrity rule that imports are never modified implicitly.

Requirement 3: Waypoint management **User story:** As a trip planner, I want to mark and annotate places.

Acceptance criteria

1. THE system SHALL support creating a waypoint by map click and by manual coordinate entry.

2. THE system SHALL validate entered coordinates and SHALL reject values outside valid latitude and longitude ranges with a clear message.
3. WHEN the user drags a waypoint marker THEN the system SHALL persist the new coordinates.
4. THE properties panel SHALL allow editing name, coordinates, elevation, description, symbol and notes.
5. THE system SHALL support a symbol set rendered as map markers, with symbol names chosen to map onto Garmin's symbol vocabulary for later export fidelity.
6. THE system SHALL support grouping waypoints, searching waypoints by name, and deleting waypoints with confirmation.
7. WHERE elevation is unknown THE system SHALL store and display it as absent, not as zero.

Requirement 4: Route and waypoint statistics Acceptance criteria

1. THE properties panel SHALL show route total distance, point count, start coordinates and end coordinates.
2. WHERE route points carry elevation THE system SHALL show elevation gain and loss; WHERE they do not, it SHALL show that elevation data is unavailable rather than showing zero.
3. WHERE the user has configured a planning speed THE system SHALL show an estimated duration labelled as an estimate with that speed visible; it SHALL NOT present a bare duration implying measurement.
4. Statistics SHALL be computed from the stored points via `shared/geo`, and SHALL be unit tested against known fixtures.

Requirement 5: Undo and redo **User story:** As a user, I want to undo a mistaken edit, because there is no save point to revert to.

Acceptance criteria

1. THE system SHALL maintain a bounded undo history of user mutations.
2. THE Edit menu Undo and Redo items SHALL become enabled and SHALL reflect the next action's description.
3. WHEN an operation is undone THEN the database, map and properties panel SHALL all return to the prior state.
4. THE history SHALL cover route and waypoint edits and library organisation; destructive trip deletion SHALL remain confirmation-gated rather than relying on undo.

Requirement 6: Map selection and interaction Acceptance criteria

1. THE system SHALL support selecting a route or waypoint on the map and in the library, with selection synchronised between them.
2. THE map SHALL provide context menus on objects and on empty map space with contextually valid actions only.
3. THE system SHALL support fit-to-screen for a selected route and return-to-selection, wired to the View menu items enabled in this phase.
4. WHEN a trip is selected THEN the map SHALL display all of that trip's objects.

Requirement 7: Trip organisation completion Acceptance criteria

1. THE library SHALL populate the cross-trip Routes and Waypoints views.
2. THE system SHALL support moving a route or waypoint between trips as a reparent, with the UI stating that it is a move and not a copy.

9.4.2 Design

Overview Phase 2 makes TrailRT useful for planning: draw and edit routes on the map, place and annotate waypoints, see statistics derived from real geometry, and undo any of it.

Phase 1 established the shapes this builds on. Every mutation already goes through a service method that returns its inverse (ADR 0001), the map already owns a single imperative Leaflet instance, and `shared/geo` already computes distance and elevation. Phase 2 adds entities and interaction; it does not restructure anything.

Two decisions carry the weight of this phase: how undo works, and how route geometry edits are represented. Both are settled below before any code.

Undo and redo: a command log in the main process Requirement 5 needs a bounded undo history. ADR 0001 ruled out "revert to last saved file" because there is no save point, so this has to be a command history.

It lives in the main process, not the renderer. The database is the single source of truth, and an undo stack in the renderer would desynchronise the moment a mutation arrived from anywhere else — a menu action, a second window, or a backup restore. Keeping it beside the data means the stack can never describe a state the database was not in.

The symmetry that makes it work Every service mutation already returns `Mutation<T>`:

```
interface Mutation<T> {
  result: T;
  inverse: { kind: string; payload: Record<string, unknown> };
}
```

A command registry maps `kind` to the service call that applies it. Because applying a command itself returns a `Mutation`, the inverse of an inverse comes back for free:

```
do(cmd)   -> { result, inverse: undoCmd }   push undoCmd onto the undo stack
undo()    -> apply(undoCmd) -> { inverse: redoCmd }   push redoCmd onto redo
redo()    -> apply(redoCmd) -> { inverse: undoCmd }   push undoCmd onto undo
```

No separate "reverse operation" code, and no risk of a hand-written inverse drifting from its forward operation. A new undoable action is a service method plus one registry entry.

Rules that keep it honest:

- A new user action clears the redo stack. Branching history is not worth the confusion.
- The stack is bounded (50 entries) and holds only descriptors, not snapshots of whole objects, so it cannot grow without limit.
- **Trip and folder deletion stay out of the stack**, remaining confirmation-gated as in Phase 1. Their cascade spans several tables and a reliable inverse would mean snapshotting all of it. ADR 0001 already said this; it is restated because it is the obvious thing to get wrong.
- Route and waypoint deletion *is* undoable, via a restore command that recreates the object with its original id. Reusing the id matters: selection, and later attachments, refer to it.
- Each entry carries a human label ("Move route point", "Delete waypoint") so the Edit menu can say what it will undo rather than just "Undo".

Route geometry: whole-array replacement Requirement 2 lists point move, insert, delete and reorder, plus route reverse. Representing each as its own command, each with its own inverse, is four opportunities to get an off-by-one wrong.

Instead there is **one geometry command**: `route:setGeometry`, carrying the complete point array. Its inverse is the same command with the previous array. Move, insert, delete, reorder and reverse are all callers that compute a new array.

This is viable because planned routes are small. A route is tens to low hundreds of points — a human places them. That is a few kilobytes per undo entry, which is nothing against a 50-entry bound.

It would *not* be viable for tracks, which run to hundreds of thousands of points. Phase 3 track editing therefore needs a different approach, and ADR 0003 already anticipates that with its separate source-of-truth and render-cache split. The two are deliberately not unified.

Statistics are a cache, recomputed on write `routes.distance_m`, `elev_gain_m`, `elev_loss_m` and `point_count` are caches (Phase 1 schema). Every `setGeometry` recomputes them from the new points inside the same transaction, so they cannot drift from the geometry.

Elevation gain stays `null` when no point carries elevation — never `0`. A flat route and a route with unknown elevation are different facts and the properties panel must be able to tell them apart. `shared/geo/elevationStats` already returns nulls for this; Phase 2 just must not paper over it.

Duration is shown only as an estimate at the user's configured planning speed, with the speed visible. Requirement 4.3 forbids presenting a bare number that implies measurement.

Module layout Additions only; nothing moves.

```
src/
  shared/
    domain/
      types.ts          + Route, RoutePoint, Waypoint, WaypointGroup, stats
      symbols.ts        waypoint symbol vocabulary (Garmin-aligned names)
    ipc/
      channels.ts       + routes:*, waypoints:*, undo:*
      schemas.ts        + geometry, waypoint and undo request schemas
      api.ts            + the three new namespaces
  main/
    db/repositories/
      routes.ts         routes + route_points
      waypoints.ts      waypoints + waypoint_groups
    services/
      routeService.ts
      waypointService.ts
      undoService.ts    the stack
      commands.ts       kind -> service call registry
    ipc/register.ts     + new channels, each recording an undo entry
  renderer/
    stores/
      editorStore.ts    drawing mode, hover, pending geometry
    features/map/
      useRouteLayers.ts      polylines + draggable vertices
      useWaypointLayers.ts   markers
      markers.ts             DivIcon factories
    features/properties/
      RouteProperties.tsx
      WaypointProperties.tsx
```

`features/map/` still must not import trip management. Route and waypoint data reach it as props from the app layer, not by importing the library store.

Map interaction The Leaflet instance stays single and imperative. Phase 2 adds layer effects that attach to it, in the same pattern as the tile layer.

Draw mode. A mode flag in `editorStore` plus a Leaflet `click` handler that appends a point. The cursor changes, a hint bar appears, Escape cancels without persisting, and Enter or a double-click finishes. Segments are straight lines: there is no routing provider until Phase 4, and drawing must not imply road-snapping that nothing performs (Requirement 1.5).

Vertex editing. A selected route renders draggable circle markers at its points. Drag commits on `dragend`, not on every move — one undo entry per gesture, not per pixel. Clicking a segment inserts a point at the projected position; Alt-clicking a vertex deletes it.

Performance. Vertices are only materialised for the *selected* route. Rendering handles for every route in a trip would be both visually unreadable and the obvious source of a slow map.

Selection synchronisation. The library tree and the map read the same `selectionStore`, so clicking either highlights both. This already works for trips; routes and waypoints join the same mechanism.

Waypoint symbols Requirement 3.5 asks for a symbol set whose names map onto Garmin's vocabulary, so Phase 3 export keeps fidelity. `shared/domain/symbols.ts` holds the mapping: a stable internal id, a display label, a glyph for the marker, and the Garmin `<sym>` name used on export.

The set is deliberately small — around twenty symbols covering the common outdoor cases — rather than an exhaustive clone. An unmapped imported symbol is preserved verbatim in the database and rendered with a default glyph, so Phase 3 can round-trip it even though this build has no icon for it. Discarding it would violate the import-fidelity rule.

Technical risks

Risk	Impact	Mitigation
Undo stack desynchronises from the database	Corrupt-looking state, lost edits	Stack lives in the main process beside the data; every entry derives from a real mutation's inverse rather than being hand-written
Vertex drag generating an undo entry per mouse-move	Unusable history	Commit on `dragend` only; one entry per gesture
Draw mode leaking: clicks still appending after finish	Phantom points	Mode is a single store flag; the Leaflet handler is attached and removed by effect, not left bound
Leaflet marker churn on every store update	Janky map	Layers keyed by object id and updated in place; full rebuild only when the selected trip changes
Elevation nulls rendered as 0	Misleading data, breaks a steering rule	`shared/geo` returns nulls; properties panel renders `–`; covered by unit tests
Route point `seq` gaps after edits	Broken ordering, unique-constraint failures	`setGeometry` deletes and reinserts the whole array in one transaction, so `seq` is always dense

Risk	Impact	Mitigation
Deleted-then-undone object losing its id	Stale selection, future attachment orphans	Restore commands recreate with the original id

Out of scope for Phase 2 Stated so their absence is not read as an oversight: road-snapped routing (Phase 4), elevation profile charts (Phase 4), external elevation lookup (Phase 4), track editing and import/export (Phase 3), and sort-by-distance in the library (Phase 3, when tracks also have distance). Waze hand-off is Phase 4 per ADR 0005.

9.4.3 Tasks

Execute in order. The data and command layers come first so the map and panels are built against a settled contract, and the undo stack exists before the editing gestures that need it — retrofitting history onto working gestures is how per-pixel undo entries happen.

Update `docs/progress-mindmap.svg` as each task changes status.

- ☒ 1. Shared contracts for routes and waypoints
  - `shared/domain/types.ts`: `Route`, `RoutePoint`, `Waypoint`, `WaypointGroup`, `RouteStats`, extend `ObjectRef` usage
  - `shared/domain/symbols.ts`: the waypoint symbol vocabulary with internal id, label, glyph and Garmin `<sym>` name
  - `shared/ipc/channels.ts`: `routes:*`, `waypoints:*`, `undo:*`, new menu actions for route and waypoint creation
  - `shared/ipc/schemas.ts`: coordinate-validated point arrays, waypoint fields, undo requests
  - `shared/ipc/api.ts`: the three new namespaces
  - Unit tests: schema rejection of out-of-range coordinates, empty point arrays, over-long names, and unknown symbol ids
  - *Requirements: 3.2, 3.5, 3.7*
- ☒ 2. Route and waypoint repositories
  - `main/db/repositories/routes.ts`: routes plus dense-`seq` point writes
  - `main/db/repositories/waypoints.ts`: waypoints and waypoint groups
  - Cross-trip listing queries for the library views
  - Unit tests: point ordering survives a round trip, `seq` stays dense after a replace, cascade on trip delete still holds
  - *Requirements: 7.1*
- ☒ 3. Route service with geometry and derived statistics
  - create, rename, set colour, duplicate, delete, restore-with-id
  - `setGeometry`: delete and reinsert the whole array in one transaction, then recompute `distance_m`, `point_count`, `elev_gain_m`, `elev_loss_m`
  - Point operations expressed as callers that build a new array: append, move, insert at segment, delete, reorder, reverse
  - Every mutation returns its inverse
  - Unit tests: statistics match `shared/geo` fixtures; elevation stays `null` when no point carries elevation; a degenerate one-point route is valid; imported geometry is never modified implicitly
  - *Requirements: 1.3, 1.5, 2.3, 2.4, 2.5, 2.6, 4.1, 4.2, 4.4*
- ☒ 4. Waypoint service
  - create (map click or manual coordinates), update, move, delete, restore-with-id, group assignment, search
  - Coordinate validation rejects out-of-range values with a clear message
  - Elevation absent means `null`, never `0`

  - Unit tests: validation, grouping, search, cascade, null elevation
  - *Requirements: 3.1, 3.2, 3.3, 3.4, 3.6, 3.7*
- ☒ 5. Command registry and undo service
  - `main/services/commands.ts`: `kind` → service call, covering `route:*` and `waypoint:*` plus the Phase 1 `trip:*` and `folder:*` mutations that are safely invertible
  - `main/services/undoService.ts`: bounded stack, redo cleared on new action, human labels, deletion of trips and folders deliberately excluded
  - Unit tests: undo restores prior state; redo reapplies; a new action clears redo; the bound is enforced; an undone delete restores the original id; undo of a geometry edit restores the exact point array
  - *Requirements: 5.1, 5.3, 5.4*
- ☒ 6. IPC and preload wiring
  - Handlers for all route, waypoint and undo channels, each recording an undo entry where appropriate
  - Preload bridge methods; no generic passthrough
  - Unit tests: invalid payloads rejected at the boundary
  - *Requirements: 3.2, 5.1*
- ☒ 7. Waypoint map layer
  - `features/map/markers.ts`: DivIcon factories driven by the symbol vocabulary
  - `features/map/useWaypointLayers.ts`: markers keyed by id, updated in place
  - Click to select, drag to reposition committing on `dragend`
  - *Requirements: 3.3, 6.1, 6.4*
- ☒ 8. Route map layer
  - Polylines for every route in the selected trip, coloured per route
  - Draggable vertices for the selected route only
  - Start and end emphasis
  - *Requirements: 6.1, 6.4*
- ☒ 9. Route drawing mode
  - `editorStore`: mode flag, pending points, hint bar
  - Map click appends; running distance shown live
  - Enter or double-click finishes and persists; Escape cancels without persisting a partial route
  - Straight segments only, with the UI making clear no road-snapping occurs
  - *Requirements: 1.1, 1.2, 1.3, 1.4, 1.5*
- ☒ 10. Route vertex editing
  - Drag a vertex to move it; click a segment to insert; Alt-click to delete
  - Reverse, duplicate, rename and delete a route
  - One undo entry per gesture
  - *Requirements: 2.1, 2.2, 2.3, 2.4, 2.5*
- ☒ 11. Route and waypoint properties panels
  - Route: name, colour, distance, point count, start and end coordinates, elevation gain and loss where available, estimated duration labelled as an estimate with the planning speed shown
  - Waypoint: name, coordinates (editable with validation), elevation, description, symbol, notes
  - Absent data renders as “not available”, never as zero
  - *Requirements: 3.4, 3.7, 4.1, 4.2, 4.3*
- ☒ 12. Library tree completion
  - Routes and waypoints nested under their trip
  - Cross-trip Routes and Waypoints views populated
  - Move a route or waypoint between trips as a reparent, with the UI stating it is a move and not a copy
  - *Requirements: 7.1, 7.2*

- ☒ 13. Undo and redo in the UI
  - Edit menu Undo and Redo enabled from the stack state, labelled with the action they will reverse
  - ⌘Z and ⇧⌘Z
  - Map, library and properties all reflect an undone change
  - *Requirements: 5.2, 5.3*
- ☒ 14. Context menus and map controls
  - Object and empty-space context menus offering only contextually valid actions
  - Fit route to screen, fit trip to screen, return to selection — now that there is geometry to fit, the View menu items become functional
  - *Requirements: 6.2, 6.3*
- ☒ 15. End-to-end tests
  - Draw a route on the map, save it, reopen and confirm the geometry
  - Drag a vertex and confirm the statistics update
  - Create a waypoint by coordinate entry and by map click
  - Undo and redo across a geometry edit and a deletion
  - Route geometry preserved exactly across a restart
  - *Requirements: 1.1, 2.2, 3.1, 5.3*
- ☒ 16. Phase 2 verification pass
  - `npm run verify` green, and the e2e suite green against the packaged build
  - Walk the source-spec Phase 2 acceptance criteria: a route can be created on the map, routes save and reopen, waypoints persist, route geometry is preserved
  - Update the mindmap to show Phase 2 complete
  - Record any discovered limitation in `docs/adr/` rather than leaving a placeholder in the UI
  - *Requirements: all*

Notes

9.5 Phase 3: Tracks and Import/Export — Requirements — SPEC-PHASE-3-TRACKS-IMPORT-EXPORT

Version: — · **Status:** unspecified · **Slug:** phase-3-tracks-import-export

9.5.1 Requirements

Status: not started. Blocked on Phase 2.

Introduction Phase 3 is the phase that makes TrailRT a viable BaseCamp replacement, because it is the phase that reads existing data. Format support, track handling and an honest import report are the core deliverables.

Format tiering and licensing constraints are set by ADR 0006. Track storage is set by ADR 0003.

Requirements

Requirement 1: GPX import **User story:** As a BaseCamp user, I want to import my existing GPX files.

Acceptance criteria

1. THE system SHALL parse GPX 1.0 and 1.1, extracting waypoints (`wpt`), routes (`rte`) and tracks (`trk`) with multi-segment support.
2. THE system SHALL preserve coordinates, elevations, timestamps, names, descriptions, symbols and comments exactly as present in the source.
3. THE system SHALL NOT re-sample, simplify or round geometry on import.
4. THE system SHALL retain the original file bytes per ADR 0003, so that "original data preserved" is literally true.
5. WHERE a GPX extension namespace is present but unsupported THE system SHALL record it in the import report as unmapped metadata rather than discarding it silently.
6. IF the file is malformed THEN the system SHALL fail with a message naming the problem and its location, and SHALL import nothing from that file.
7. THE system SHALL treat all text content as untrusted plain text and SHALL NOT render it as markup.

Requirement 2: Additional format support Acceptance criteria

1. THE system SHALL import and export KML, and SHALL import KMZ by extracting its contained KML.
2. THE system SHALL import TCX, mapping activity trackpoints to tracks, and SHALL retain heart rate and cadence as unmapped metadata where present.
3. THE system SHALL import FIT using the MIT-licensed `fit-file-parser`. Garmin's own `@garmin/fitsdk` SHALL NOT be used: its licence is a proprietary bilateral agreement incompatible with the project's GPL-3.0 licence. The gate that ADR 0006 opened is now closed, and the answer is recorded in ADR 0007.
4. THE system SHALL import and export CSV for waypoints, with a documented column format and a header-mapping step for unrecognised layouts.
5. THE system SHALL export and import a native JSON backup covering the whole collection, suitable for migration between installations.
6. THE system SHALL list GDB as not yet supported with a brief explanation, rather than accepting the file and failing opaquely. GDB is Phase 6 work; it became viable once the project licence was set to GPL-3.0, which permits GPSBabel.

Requirement 3: Import workflow Acceptance criteria

1. THE system SHALL support importing via file dialog, multi-file selection, and drag-and-drop onto the window.
2. WHEN files are chosen THEN the system SHALL show a preview listing the routes, tracks and waypoints found, with counts, before anything is written.
3. THE preview SHALL allow choosing the destination trip, or creating a new one; WHERE no destination is chosen THE system SHALL create a trip named after the source file, per ADR 0004.
4. THE preview SHALL allow deselecting individual objects.
5. WHEN importing THEN the system SHALL show progress, and SHALL remain responsive for a file containing at least 200,000 track points.
6. WHEN an import completes THEN the system SHALL show a report listing successfully imported objects, unsupported objects, invalid records, warnings and any metadata that could not be mapped.
7. THE system SHALL NOT silently discard any input data.
8. THE system SHALL record every import in `import_history` and SHALL surface it in the Imported Data library view.
9. IF an import fails partway THEN the system SHALL roll back that file's objects as a unit rather than leaving a partial trip.

Requirement 4: Export Acceptance criteria

1. THE system SHALL export an individual route, track or waypoint, a complete trip, and a multi-trip selection.
2. THE system SHALL preserve coordinates, timestamps, elevations and metadata wherever the target format supports them.
3. WHERE the target format cannot represent some data THE system SHALL warn before writing, naming what will be lost.
4. A GPX export followed by a re-import SHALL produce geometrically identical objects, verified by a round-trip test.
5. Exported files SHALL be valid against the format's schema where one exists.

Requirement 5: Track display and statistics Acceptance criteria

1. THE system SHALL display tracks on the map with configurable colour and thickness, persisted per track.
2. THE system SHALL optionally display individual track points.
3. THE system SHALL show start and finish markers.
4. THE system SHALL show distance, point count, duration where timestamps exist, and elevation gain and loss where elevation exists.
5. THE system SHALL show an elevation profile where elevation data exists, and SHALL state its absence otherwise rather than drawing a flat line.
6. THE map SHALL render large tracks via the simplified geometry cache below a zoom threshold and real points above it, per ADR 0003, at interactive frame rates for a 200,000-point track.

Requirement 6: Track editing, non-destructive Acceptance criteria

1. THE system SHALL support removing points, splitting, joining, reversing, simplifying and renaming tracks.
2. Simplification SHALL be explicit and user-initiated, SHALL show the resulting point count before applying, and SHALL be undoable.
3. THE original imported data SHALL remain recoverable after any edit, via the retained source bytes.

4. WHEN a track is edited THEN the system SHALL invalidate and rebuild the render geometry cache.

Requirement 7: Library completion Acceptance criteria

1. THE library SHALL populate the cross-trip Tracks and Imported Data views.
2. THE library SHALL support sort by distance, deferred from Phase 1 because no distance-bearing objects existed then.

Requirement 8: Test fixtures Acceptance criteria

1. `test-data/` SHALL contain synthetic GPX, KML, KMZ, TCX and FIT fixtures, including edge cases: no elevation, no timestamps, multi-segment tracks, empty files, malformed XML, and a large track for performance testing.
2. Fixtures SHALL be synthetic. Real user GPS data SHALL NOT be used.
3. Round-trip tests SHALL cover every implemented export format.

9.5.2 Design

Overview Phase 3 is the phase that makes TrailRT a viable BaseCamp replacement, because it is the first phase that reads data the user already has. Two subsystems: recorded tracks, and a format conversion pipeline.

The hard constraints come from the steering rules and ADR 0003, not from the feature list:

- **Imported data is never silently modified or discarded.** No re-sampling, no rounding, no quiet dropping of fields we have no column for.
- **Absent data stays absent.** No elevation means `null`, not `0`.
- **Tracks are large.** A multi-day recording at 1 Hz is 100k–500k points. Every design choice below that looks over-engineered is there because of that number.

Track storage: three layers, one source of truth Straight from ADR 0003, now implemented:

Layer	Table	Role
Source of truth	`track_points`	Normalised rows, ordered by `(segment_index, seq)`
Render cache	`tracks.render_geometry`	Simplified polyline, rebuildable, invalidated on edit
Original bytes	`track_sources`	The imported file verbatim, with its hash and format

`render_geometry` is a packed binary blob, not JSON: a 200k-point track is ~3.2 MB as `Float64Array` pairs versus ~9 MB of JSON text, and it skips a parse on every map render. The stored tolerance travels with it so a cache built at the wrong tolerance can be detected and rebuilt rather than silently reused.

The cache must never be the only copy. Any path that writes `render_geometry` without `track_points` already being correct is a bug, and the rebuild is a pure function of the points.

Why tracks do not reuse the route geometry command Phase 2 made every route edit a whole-array replacement, because a human-placed route is tens of points. Doing that for a 200k-point track would put megabytes on the undo stack per edit.

Track edits therefore use **targeted commands** — delete a point range, split, join, simplify — and their inverses store only what changed, or in the case of simplify, the fact that the original is recoverable from `track_sources`. The two models are deliberately not unified; ADR 0003 anticipated this.

Import pipeline Three stages, deliberately separated so nothing is written before the user has seen what will be written.

```
          parse                    preview                  commit
file          ParsedFile        user choices       database
     (pure, no I/O               (rendered in              (one transaction
      beyond reading)             a dialog)                 per file)
```

Parse is pure: bytes in, a `ParsedFile` out, containing candidate routes, tracks and waypoints plus a list of warnings and unmapped metadata. No database access, so every parser is unit-testable against fixtures with no Electron and no SQLite.

Preview shows counts, per-object detail, the destination trip, and anything the parser could not map. The user can deselect individual objects.

Commit writes each file's objects in a single transaction. A failure mid-file rolls that file back as a unit rather than leaving a half-imported trip (Requirement 3.9). Multiple files are independent: one bad file does not lose the others.

The report is a first-class output, not logging Requirement 3.6 and 3.7 are the ones most easily fudged. The parser returns:

```ts
interface ImportReport {
  imported: { routes: number; tracks: number; waypoints: number; points: number };
  skipped: SkippedItem[];      // present in the file, not imported, with why
  warnings: string[];          // imported, but something was odd
  unmappedMetadata: string[];  // fields with no column, preserved as JSON
}
```

`unmappedMetadata` is the honest half. GPX extensions, TCX heart rate and cadence, KML `ExtendedData` — none of these have columns. They are preserved verbatim in the `metadata` JSON column and listed in the report, so the user knows they survived and Phase 6 can round-trip them. Dropping them silently would breach the steering rule; pretending we understand them would be worse.

Every import, successful or not, is recorded in `import_history`, which is what finally populates the Imported Data library view.

Format support and what each one costs One parsing approach for all three XML formats — `fast-xml-parser` — rather than a GeoJSON intermediate. `@tmcw/togeojson` was considered and rejected: GeoJSON flattens exactly the names, descriptions and extension data we are obliged to preserve, and it needs an XML DOM implementation under Node.

Format	Library	Reads	Writes	Notes
GPX 1.0/1.1	fast-xml-parser	☒	☒	`wpt`, `rte`, `trk` with multi-segment `trkseg`
KML	fast-xml-parser	☒	☒	`Placemark` Point/LineString, `gx:Track`, `MultiGeometry`

Format	Library	Reads	Writes	Notes
KMZ	jszip	✓	✓	A zip containing KML; the first `.kml` entry is the document
TCX	fast-xml-parser	✓	—	Activity trackpoints → tracks; HR/cadence become unmapped metadata
FIT	fit-file-parser	✓	—	MIT licensed. Garmin's own SDK is rejected — see ADR 0006/0007
CSV	in-house	✓	✓	Waypoints only, with a header-mapping step
JSON	in-house	✓	✓	Native full-collection backup

TCX and FIT are read-only by design. Writing them would mean inventing activity metadata — lap structure, sport type, device ids — that a planning app does not have. A route exported as TCX would be a fabrication, so the export dialog simply does not offer them, rather than offering them and producing something misleading.

GDB stays unsupported and is listed as such in the import dialog with a short explanation (ADR 0006, as amended by 0007).

Streaming and responsiveness A 200k-point FIT or GPX file must not freeze the UI (Requirement 3.5).

- Parsing happens in the **main process**, never the renderer, so a slow parse cannot block painting.
- Point insertion uses one prepared statement inside one transaction. The measured difference against per-row autocommit is roughly two orders of magnitude, and it is the single thing that makes a large import feasible.
- Progress is reported back over IPC at coarse intervals — per file and per 10,000 points — rather than per point, because the messaging would otherwise cost more than the work.

Map rendering for large tracks Below a zoom threshold the map draws `render_geometry`; above it, real points within the current viewport. Requirement 5.6 asks for interactive frame rates on a 200k-point track, and drawing 200k SVG vertices is not that.

Simplification is Douglas-Peucker in `shared/geo/simplify.ts` — pure, testable, and shared between the render cache and the user-initiated simplify action.

Module layout Additions only.

```
src/
  shared/
    io/
      types.ts        ParsedFile, ImportReport, SkippedItem
    geo/
      simplify.ts     Douglas-Peucker, pure
      packed.ts       Float64 polyline encode/decode
  main/
    db/repositories/
      tracks.ts       tracks, track_points, track_sources
      importHistory.ts
    io/
      detect.ts       format from extension plus content sniffing
      gpx/{read,write}.ts
      kml/{read,write}.ts
      kmz/{read,write}.ts
      tcx/read.ts
      fit/read.ts
      csv/{read,write}.ts
      json/{read,write}.ts
      importService.ts   parse → preview → commit
      exportService.ts
      garmin/            GarminCompatibility: symbol and extension mapping
    services/
      trackService.ts
  renderer/
    features/
      import/         ImportDialog: preview, selection, report
      export/         ExportDialog
    features/map/
      useTrackLayers.ts
    features/properties/
      TrackProperties.tsx
```

`main/io/` is a leaf: it depends on `shared/` and the repositories, and nothing depends on it except the IPC layer. That is what keeps Phase 6's Garmin work from reaching into trip management.

Technical risks

Risk	Impact	Mitigation
A 200k-point import freezes the app	Unusable on real data	Parse in main, single prepared-statement transaction, coarse progress reporting
Drawing 200k vertices on the map	Unusable map	Packed simplified render cache below a zoom threshold; viewport-limited real points above
Silent data loss on import	Breaks the core promise and a steering rule	`unmappedMetadata` and `skipped` are returned, surfaced in the report, and asserted in tests

Risk	Impact	Mitigation
Elevation or time rendered as 0 when absent	Misleading data	Parsers emit `null`; round-trip tests assert nulls survive
Coordinate precision lost on export	Round-trip corruption	No rounding anywhere in the write path; a round-trip test compares exact values
KML coordinate order	Silently swapped lat/lon	KML is `lon,lat,ele`, GPX is attributes — covered by a dedicated test per format
Partial import on malformed input	Half-imported trip	One transaction per file, rolled back as a unit
Simplify destroying the original	Irreversible data loss	Original bytes in `track_sources`; simplify is explicit, previewed, and undoable

Out of scope for Phase 3 Elevation profile charts and routing remain Phase 4. GDB, IMG and device transfer remain Phase 6 investigations. TCX and FIT writing is not planned at all, for the fabrication reason above.

9.5.3 Tasks

Execute in order. The parsers come before any UI so they can be developed against fixtures as pure functions, and the fixtures come early because every later task is verified against them.

GPX is built first and fully — read, write and round-trip — because it is the format that matters most and the one whose shape the import pipeline is designed around. The others follow the pattern it establishes.

Update `docs/progress-mindmap.svg` as each task changes status.

- ☒ 1. Shared IO contracts and geometry helpers
  - `shared/io/types.ts`: `ParsedFile`, `ParsedTrack`, `ImportReport`, `SkippedItem`, `ImportSelection`
  - `shared/geo/simplify.ts`: Douglas-Peucker, pure
  - `shared/geo/packed.ts`: `Float64Array` polyline encode/decode for the render cache
  - Unit tests: simplification preserves endpoints, respects tolerance, is idempotent at the same tolerance; packed round-trips exactly
  - *Requirements: 5.6, 6.2*
- ☒ 2. Synthetic test fixtures
  - `test-data/` with GPX 1.0 and 1.1, KML, KMZ, TCX, FIT and CSV
  - Edge cases, each as its own file: no elevation, no timestamps, multi-segment track, empty file, malformed XML, unknown symbol, unmapped extensions, and a large track for the performance assertion
  - A README stating that every fixture is synthetic
  - *Requirements: 8.1, 8.2*
- ☒ 3. Track repository and service
  - `main/db/repositories/tracks.ts`: tracks, `track_points` with dense `(segment_index, seq)`, `track_sources`
  - Bulk point insert in one prepared-statement transaction
  - `main/services/trackService.ts`: create from parsed data, rename, recolour, set width, delete, restore, plus statistics from points
  - Render cache build and invalidation

- Unit tests: 200,000 points insert within a time budget; cache rebuilds from points; cache invalidated on edit; original bytes retained
- *Requirements: 1.4, 5.4, 6.3, 6.4*

☒ 4. Format detection
- `main/io/detect.ts`: extension plus content sniffing, so a mislabelled file is still identified
- GDB recognised and reported as unsupported with an explanation, never accepted and failed opaquely
- Unit tests per format, including a renamed file and an unrecognised one
- *Requirements: 2.6*

☒ 5. GPX read
- `wpt`, `rte`, `trk` with multi-segment `trkseg`
- Preserve coordinates, elevation, time, name, description, symbol, comment exactly; absent values become `null`
- Unknown extension namespaces collected into `unmappedMetadata`, never dropped
- Malformed XML fails with a message naming the problem, importing nothing
- All text treated as untrusted plain text
- Unit tests against every GPX fixture
- *Requirements: 1.1, 1.2, 1.3, 1.5, 1.6, 1.7*

☒ 6. GPX write and round-trip
- Write waypoints, routes and tracks with full precision and no rounding
- Garmin-compatible `<sym>` names via the symbol vocabulary
- Preserved metadata written back where it belongs
- Unit tests: export then re-import yields geometrically identical objects, including nulls; output validates against the GPX schema
- *Requirements: 4.2, 4.4, 4.5*

☒ 7. KML and KMZ
- Read `Placemark` Point and LineString, `gx:Track` with timestamps, and `MultiGeometry`; `ExtendedData` into `unmappedMetadata`
- KMZ via jszip, taking the contained KML document
- Write both
- Unit tests, including the `lon,lat,ele` coordinate order, which is the opposite convention to the rest of the app
- *Requirements: 2.1, 4.1*

☒ 8. TCX read
- Activity trackpoints become tracks
- Heart rate and cadence retained as unmapped metadata rather than discarded
- Read-only: writing TCX would mean inventing lap and device data
- *Requirements: 2.2*

☒ 9. FIT read
- `fit-file-parser` (MIT). Garmin's own SDK is not used — ADR 0006 and 0007
- Semicircle coordinate conversion, which is FIT's native representation
- Read-only, for the same reason as TCX
- *Requirements: 2.3*

☒ 10. CSV waypoints
- Read and write with a documented column format
- Header mapping for unrecognised layouts, rather than refusing the file
- Unit tests: quoted fields, embedded commas and newlines, missing optional columns, absent elevation staying null
- *Requirements: 2.4*

☒ 11. JSON native backup
- Full-collection export and import, suitable for moving between installs
- Versioned envelope so a future format change is detectable

  - Unit tests: a whole collection round-trips, including trips, folders, routes, tracks and waypoints
  - *Requirements: 2.5*
- ☒ 12. Import service
  - parse preview commit, with nothing written before the user confirms
  - One transaction per file; a failure rolls that file back as a unit
  - Destination trip chosen, or auto-created named after the source file
  - Progress reported per file and per 10,000 points
  - Every import recorded in `import_history`
  - Unit tests: partial failure leaves no partial trip; report counts match what was written; nothing is silently discarded
  - *Requirements: 3.2, 3.3, 3.5, 3.6, 3.7, 3.8, 3.9*
- ☒ 13. Export service
  - Individual route, track or waypoint; a whole trip; a multi-trip selection
  - Warn before writing when the target format cannot represent something, naming what will be lost
  - TCX and FIT deliberately absent from the writable formats
  - *Requirements: 4.1, 4.2, 4.3*
- ☒ 14. IPC and preload wiring
  - Channels for detect, parse, commit, export, track mutations and import history
  - File dialogs in the main process; the renderer never receives a path to read
  - Imported file content treated as untrusted throughout
  - *Requirements: 3.1*
- ☒ 15. Track map layer
  - Packed render cache below a zoom threshold, real points above
  - Configurable colour and width, persisted per track
  - Start and finish markers; optional point display
  - Interactive frame rates on a 200,000-point track
  - *Requirements: 5.1, 5.2, 5.3, 5.6*
- ☒ 16. Track properties and editing
  - Distance, point count, duration where timestamps exist, elevation gain and loss where elevation exists, and an explicit statement when it does not
  - Remove points, split, join, reverse, simplify, rename
  - Simplify shows the resulting point count before applying and is undoable
  - The original import stays recoverable from the retained source bytes
  - *Requirements: 5.4, 5.5, 6.1, 6.2, 6.3*
- ☒ 17. Import dialog
  - File dialog, multi-file selection, and drag-and-drop onto the window
  - Preview with counts and per-object detail; individual objects deselectable
  - Destination trip chooser
  - Progress indicator for large files
  - Report listing imported, unsupported, invalid, warnings and unmapped metadata
  - *Requirements: 3.1, 3.2, 3.4, 3.5, 3.6*
- ☒ 18. Export dialog
  - Scope and format selection, with unsupported combinations explained rather than hidden
  - Loss warnings surfaced before writing
  - *Requirements: 4.1, 4.3*
- ☒ 19. Library completion
  - Tracks and Imported Data views populated
  - Sort by distance, now that tracks and routes both have one
  - *Requirements: 7.1, 7.2*
- ☒ 20. End-to-end tests

- Import a GPX fixture and see its routes, tracks and waypoints on the map
- Export GPX and re-import it, confirming identical geometry
- Import warnings visible in the report
- A malformed file is refused without partial import
- Tracks persist across a restart
- *Requirements: 1.6, 3.6, 4.4*

☒ 21. Phase 3 verification pass

- `npm run verify` green, and the e2e suite green against the packaged build
- Walk the source-spec Phase 3 acceptance criteria: existing GPX files import, routes/tracks/waypoints display correctly, exported GPX reopens, import warnings are visible
- Update the supported format list only for formats actually tested
- Update the mindmap to show Phase 3 complete
- *Requirements: all*

Notes

9.6 Phase 4: Advanced Mapping — Requirements — SPEC-PHASE-4-ADVANCED-MAPPING

Version: — · **Status:** unspecified · **Slug:** phase-4-advanced-mapping

9.6.1 Requirements

Status: not started. Phase 3 is complete, so this phase is unblocked.

Introduction Phase 4 widens the mapping and routing architecture that Phase 1 established as an interface, and adds profiling as a first-class feature. The `MapProvider` interface already exists; this phase proves it by adding a second real provider and a routing abstraction.

Three capabilities were added to this phase after Phase 3:

- **Snap to road.** Promoted from a sub-criterion of the routing abstraction to a requirement of its own (Requirement 3), because it is the feature most of the routing work exists to deliver.
- **Route profile.** One profile view over distance carrying elevation, speed and road class, rather than an elevation chart alone (Requirement 5).
- **Speed identification and distribution.** Per-segment speed with a percentage-of-route breakdown (Requirement 6).

Two distinctions run through all three and are not negotiable, because conflating them would make the app state things that are not true:

1. **A posted speed limit is not a recorded speed.** A planned route can carry the legal limit of the roads it follows, read from road data. A recorded track can carry the speed actually travelled, derived from its timestamps. These are different facts with different sources and different reliability, and the UI must never present one as the other. See ADR 0011.
2. **Unknown is a value.** Road data coverage for speed limits is partial and varies by road class and country, so "no limit recorded in the data" is a normal outcome and gets its own band in every breakdown. It is never defaulted to a national limit, inferred from road class, or rolled into 0.

Requirements

Requirement 1: Multiple map providers Acceptance criteria

1. THE system SHALL add at least one appropriately licensed terrain or topographic provider alongside OpenStreetMap.
2. Adding a provider SHALL require only a registry entry and a CSP allowlist entry, with no change to map components or trip management. This is the test of whether the Phase 1 abstraction held.
3. THE layer selector SHALL list available providers and SHALL persist the choice.
4. WHERE a provider requires an API key THE system SHALL store it in settings, SHALL NOT embed it in the repository, and SHALL disable the provider with an explanatory message when no key is present.
5. THE system SHALL display each provider's required attribution and SHALL respect each provider's usage policy.
6. THE system SHALL support overlay layers independently of the base layer.
7. Google Maps SHALL be added only if its licensing terms for this use are confirmed to permit it; absent that confirmation it SHALL NOT be shipped.

Requirement 2: Routing provider abstraction Acceptance criteria

1. THE system SHALL define a `RoutingProvider` interface in `shared/`, separate from map rendering.

2. THE interface SHALL be designed so that a provider failure degrades to straight-line routing rather than blocking route editing.
3. THE system SHALL NOT assume tile providers offer route calculation.
4. WHEN a route is created or edited with routing assistance THEN the system SHALL record which provider produced the geometry, so its origin is auditable.
5. THE default configuration SHALL remain straight-line, requiring no external service for the application to function.
6. THE interface SHALL accept a user-supplied endpoint, so a self-hosted routing service can be used instead of a public one.
7. THE system SHALL declare a routing provider's hosts in the same way map providers declare tile hosts, so a provider cannot reach a host it has not declared.

Requirement 3: Snap to road A route drawn by clicking the map is a straight-line polyline, which is honest but not what a user wants when the route follows a road. Snapping asks a routing provider for the road geometry between the points the user placed.

Acceptance criteria

1. THE system SHALL offer snap-to-road as an explicit mode when drawing a route, visually and textually distinguished from straight-line drawing.
2. THE system SHALL offer snapping an existing route as a separate, user-initiated action, and SHALL NOT snap any route implicitly.
3. WHEN snapping is applied THEN the points the user placed SHALL be retained as the route's control points, and the road geometry between them SHALL be stored as the route's shape, so a later edit of a control point can re-snap only the affected legs.
4. THE system SHALL show the resulting point count before applying a snap to an existing route, because snapping replaces geometry.
5. Snapping an existing route SHALL be undoable.
6. WHERE a routing provider cannot match a leg to a road THE system SHALL keep that leg straight, SHALL mark it as unmatched, and SHALL say so, rather than silently inventing a detour or failing the whole operation.
7. THE system SHALL NOT snap tracks. A recorded track is measured data; moving its points onto a road would be fabrication. Map matching a track for the purpose of *reading* road attributes is permitted under Requirement 6 and SHALL NOT modify the track's stored geometry.
8. THE selected transport mode (such as driving, cycling or walking) SHALL be explicit and recorded with the route, because the same points snap to different roads under different modes.
9. THE system SHALL require explicit opt-in before any route geometry is sent to a routing service, SHALL state plainly that this transmits the user's planned locations to that service, and SHALL name the service.

Requirement 4: Improved map controls Acceptance criteria

1. THE system SHALL provide fit route to screen, fit track to screen, fit trip to screen and return to selection.
2. THE system SHALL provide toggles for waypoint, route and track visibility.
3. THE system SHALL display a scale bar and the cursor's coordinates, in the configured units.
4. THE system SHALL support keyboard navigation of the map.
5. THE system SHALL support measuring distance between arbitrary map points without creating a route.

Requirement 5: Route and track profile Replaces the elevation-only chart. One profile component over distance, with elevation, speed and road class as separate bands, so the same hover position

reads across all of them.

Acceptance criteria

1. THE system SHALL display a profile for a selected route or track plotted against distance along it.
2. THE profile SHALL support an elevation band showing minimum elevation, maximum elevation, total ascent and total descent.
3. THE profile SHALL support a speed band, sourced per Requirement 6.
4. THE profile SHALL support a road-class band where road attribution is available.
5. WHEN the user hovers the profile THEN the corresponding map position SHALL be indicated, and hovering the map SHALL indicate the corresponding profile position.
6. WHERE a band has no data for the selected object THE system SHALL omit that band and SHALL state why, and SHALL NOT render an empty or flat band that implies a measurement.
7. THE profile SHALL be rendered from the simplified render geometry for a large track, so a 200,000-point track does not block the UI, and SHALL state that the view is simplified when it is.
8. THE system SHALL NOT synthesise elevation values.
9. WHERE an external elevation service is configured THE system SHALL be able to request elevation for a geometry as an explicit user action, SHALL mark the result as externally sourced and distinguishable from recorded data, and SHALL NOT overwrite recorded elevation.
10. THE profile SHALL be exportable as an image for a route or track.

Requirement 6: Speed identification and distribution The feature the user asked for: see what proportion of a route is at each speed. Its correctness depends entirely on being honest about where each number came from.

Acceptance criteria

1. THE system SHALL distinguish two speed sources, SHALL label which is in use, and SHALL NOT mix them in one series:
   - **Posted limit** — the legal maximum for the road, read from road data via a provider. Available for routes and for tracks.
   - **Recorded** — speed derived from a track's own timestamps and distances. Available only where the track carries timestamps.
2. THE system SHALL obtain posted limits by map matching the geometry to the road network through a provider interface, and SHALL store the result as a **cache** attributed to the provider and the date it was fetched.
3. THE speed cache SHALL be invalidated when the geometry changes, and SHALL be recomputable from the geometry, per the derived-data rule in `.kiro/steering/architecture.md`.
4. THE system SHALL present the distribution as a percentage of **route distance** in each speed band, not as a percentage of points, because sample spacing is not uniform and a point-based percentage would be misleading.
5. THE distribution SHALL include an explicit **Unknown** band for distance whose speed could not be determined, SHALL size it honestly, and SHALL NOT omit, hide or redistribute it.
6. THE system SHALL NOT infer a speed limit from road class, country defaults or any other proxy. Where the data has no limit, the answer is Unknown.
7. THE bands SHALL be presented in the user's configured units, and the underlying values SHALL be stored in SI.
8. WHERE a track has timestamps THE system SHALL additionally be able to show recorded speed, with minimum, maximum, mean and moving mean, and SHALL state which it is showing.
9. WHEN recorded speed is shown THEN the system SHALL note that GPS-derived speed is noisy, and any smoothing applied SHALL be stated and SHALL NOT replace the stored values.

10. THE system SHALL NOT present a posted limit as a prediction of travel time. Route duration remains the explicit planning-speed estimate from Phase 2, and the two SHALL NOT be silently combined.
11. WHERE a comparison of recorded speed against posted limit is offered THE system SHALL present it as a comparison of two independently sourced measurements, SHALL NOT characterise it as an infringement, and SHALL NOT report it anywhere outside the user's own machine.
12. THE system SHALL function fully with no speed provider configured; the speed band and distribution are then absent with an explanation, not empty.

Requirement 7: Waze hand-off Acceptance criteria

1. THE system SHALL offer a single-destination Waze hand-off for a waypoint, and for a route or track resolving to its end point, per ADR 0005.
2. THE menu label SHALL state that it navigates to the end point, so it does not imply multi-stop routing that the platform does not support.
3. THE system SHALL NOT present an "open route in Waze" action, because Waze rejects multi-waypoint URLs.
4. THE system SHALL open the link via `shell.openExternal` after coordinate validation, never in an in-app window.

Requirement 8: Offline map groundwork Acceptance criteria

1. THE system SHALL support a user-supplied local tile source as a provider implementation, requiring no change to the trip database.
2. THE system SHALL NOT implement tile prefetching, area download or any bulk retrieval from public tile servers.
3. WHEN no network is available THEN locally stored trip data SHALL remain fully usable and the map SHALL degrade gracefully.
4. WHERE a routing or speed provider is unreachable THE system SHALL continue to display any cached profile and speed data it already holds, marked with the date it was fetched.

Open questions for the design stage

1. **Which provider.** Valhalla is the only open-source engine known to return posted speed limits and road class from map matching, and it is MIT licensed, which is compatible. Whether to default to a public instance, require a self-hosted endpoint, or ship with no default at all is a decision with privacy and usage-policy consequences, recorded in ADR 0011.
2. **Speed band boundaries.** Fixed bands, data-derived bands, or units-dependent bands (50/80/100/120 km/h against 30/50/60/70 mph). A mapping between the two that silently rounds would misreport the distribution.
3. **Control points versus shape.** Requirement 3.3 implies routes gain a second geometry layer — placed control points plus snapped shape — which is a schema change. Whether that reuses the track `render_geometry` pattern or adds a column needs designing against the existing route storage.

9.6.2 Design

Scope Eight requirements, delivered as sixteen independently testable tasks. The phase proves three abstractions that Phase 1 declared but never exercised: a second map provider, a routing provider, and derived road attribution.

The shape of the problem Phases 1–3 built a local-first application with **no network calls except tile images**. `src/main/security/csp.ts` sets `connect-src 'self'`, so the routing work is the first

time the app talks to a service at all. That is the organising constraint of this design: every network capability is opt-in, declares its host, and the application remains completely functional without it.

```
                        no provider configured (default)   straight lines, as today
RoutingProvider
                        user-supplied endpoint   snap-to-road, speed limits, road class
```

Decisions

1. Map providers: OpenTopoMap, plus a user-supplied source `MAP_PROVIDERS` gains two entries and an overlay concept.

Provider	Kind	Key	Hosts
`osm`	raster	no	`tile.openstreetmap.org`
`opentopomap`	raster	no	`a/b/c.tile.opentopomap.org`
`custom`	offline / user-supplied	no	declared by the user

OpenTopoMap is the terrain provider required by Requirement 1.1. Its tiles are CC-BY-SA 2.0 over OpenStreetMap data under ODbL, so the attribution string names both the data and the rendering, and it is rendered verbatim by the existing attribution control.

Overlays are a separate registry with the same host-declaration contract. Requirement 1.6 asks for overlays independent of the base layer; the mechanism is implemented and proven by the user-supplied overlay, which is also what satisfies Requirement 8.1's local tile source. **No third-party overlay server is adopted**, because doing so means accepting another operator's usage policy, and the licensing steering requires that check before adoption rather than after. A documented gap beats an unvetted dependency.

The `custom` provider doubles as the offline groundwork: a `file://` or `http://localhost` template with a declared host needs no new code path.

2. `RoutingProvider` **lives in** `shared/`**, its wire format does not**

```
shared/routing/RoutingProvider.ts   interface + domain types, no HTTP
main/routing/valhalla.ts            the only file that knows Valhalla's JSON
main/routing/client.ts              fetch with timeout, abort, error mapping
main/routing/polyline.ts            polyline6 decode
```

The interface speaks domain terms — `RouteLeg`, `MatchedEdge`, `SnapResult` — and never Valhalla's. That is what makes a second provider a new adapter rather than a refactor, and it means a wire-format mistake is contained in one file.

Valhalla is the first adapter because it is MIT licensed and, per ADR 0011, the only open-source engine that returns posted speed limits and road class from map matching. Two wire details that are easy to get wrong and are therefore isolated and tested:

- **Shape is polyline6**, not polyline5. Decoding at precision 5 yields coordinates in the wrong place entirely rather than failing loudly.
- `length` **and** `speed_limit` **default to kilometres and km/h.** The adapter requests `directions_options: { units: 'kilometers' }` explicitly and converts to SI metres and m/s at the boundary, because the rest of the app stores SI.

`trace_attributes` is requested with an explicit `filters.attributes` include list rather than the default "everything", because the default response for a long trace is very large and most of it is unused.

No default endpoint ships. `routingEndpoint` defaults to `''`. There is no public instance baked in, so the app cannot send a user's route anywhere until they configure a host and accept the consent notice.

3. Snapping: control points are the user's intent, the shape is derived Requirement 3.3 asks that placed points be retained so an edit re-snaps only the affected legs. That needs a second geometry layer.

```
route_control_points   what the user placed       (new table, Phase 4)
route_points           the shape that is drawn    (existing, source of truth
                                                   for display and export)
```

A route with no control points is a hand-drawn route, exactly as today — which is how Requirement 2.5's "normal behaviour is unchanged" stays true without a feature flag.

`routes` gains `routing_provider`, `transport_mode` and `snapped_at` so the geometry's origin is auditable (Requirement 2.4).

The point-count problem, and what this phase does about it

`routePointsSchema` caps geometry at 10,000 points because `route:setGeometry`'s undo inverse carries the entire previous array. Snapped road geometry is denser than hand-placed geometry by one to two orders of magnitude.

The resolution here is deliberately narrow, because the general answer belongs to Phase 5:

- The IPC cap stays at 10,000. It bounds a **renderer-supplied** payload, and snapped geometry is generated in the main process and never arrives that way.
- Snapped geometry is capped at `MAX_SNAPPED_POINTS = 25_000`, enforced in the service. Beyond that the snap is refused with the count, rather than stored and silently unexportable.
- **Undo carries the previous array only when it is under 10,000 points.** Above that the snap reports `inverse: { kind: 'none' }` and the UI says the operation cannot be undone, following the precedent set by `trackService.restoreOriginal` — an honest refusal beats an inverse that would be a multi-megabyte snapshot.

Unmatched legs

A leg the provider cannot match stays as the straight line the user drew and is recorded in `routes.metadata.unmatchedLegs` as control-point index pairs. The map draws those legs dashed, the same visual language already used for the unsaved drawing preview. Partial success is normal and is reported, not failed.

4. Road attributes are a cache keyed by a geometry hash Map-matched speed limits and road classes go in one new table:

```
road_attributes(owner_type, owner_id, provider, geometry_hash,
                fetched_at, payload JSON)
```

`geometry_hash` is a SHA-256 over the rounded coordinate sequence. On read, the hash is recomputed and a mismatch means the cache is stale — which is how Requirement 6.3's invalidation-on-edit is enforced without a trigger. Same contract as every other derived value: rebuildable, never the only copy.

`payload` holds per-edge records (`startIndex`, `endIndex`, `speedLimitMps`, `roadClass`, `surface`) plus the provider and fetch date, so Requirement 8.4 can display stale cached data with its age when the provider is unreachable.

5. Profiles are computed in the main process, drawn as hand-written SVG `shared/geo/profile.ts` turns geometry into bands over cumulative distance. `shared/geo/speedBands.ts` turns per-edge attributes into a distance-weighted distribution.

Both are pure, in `shared/`, and tested without Electron. Two properties that tests pin down because they are the easy things to get wrong:

- **Distribution is weighted by distance, not point count.** Sample spacing is not uniform, so counting points over-weights slow sections — ADR 0011 §3.
- **Unknown is a band.** Distance with no determinable speed is reported, never redistributed. Given that OSM `maxspeed` coverage is partial, this band is often the largest one, and that is the correct answer.

The chart is hand-written SVG. No charting library, which keeps the licence register unchanged and avoids a dependency for something that is a `<path>` and a `<rect>` per band. A 200,000-point track is profiled from the existing render cache, and the panel says when the view is simplified.

6. Measuring, scale and coordinates `L.control.scale` is built into Leaflet and honours the configured unit system. Cursor coordinates reuse `formatCoordinates` from `shared/geo/units.ts`.

The measure tool is a distinct editor mode alongside `drawRoute` and `placeWaypoint`, reusing `useDrawMode`'s pending-points pattern and `pathDistance`. It creates nothing — Requirement 4.5 is explicit that measuring must not create a route — so it has no service, no IPC and no undo entry.

7. Waze stays as ADR 0005 specified it Single destination, resolving a route or track to its **end point**, with the menu label saying so. `shell.openExternal` after coordinate validation through the existing `security/navigation.ts` allowlist. No "open route in Waze", because Waze rejects multi-waypoint URLs.

Data model changes Migration `0002_routing_and_attributes`:

```
routes          + routing_provider TEXT      (null for hand-drawn)
                + transport_mode    TEXT
                + snapped_at        INTEGER

route_control_points  (new)  id, route_id, seq, lat, lon
                             unique(route_id, seq)

road_attributes       (new)  id, owner_type, owner_id, provider,
                             geometry_hash, fetched_at, payload
                             unique(owner_type, owner_id)
```

`AppSettings` gains:

Key	Default	Purpose
`routingEndpoint`	`''`	Base URL. Empty means no provider.
`routingConsentAccepted`	`false`	Gates every outbound routing call.
`transportMode`	`'auto'`	`auto` / `motorcycle` / `bicycle` / `pedestrian`
`mapOverlayId`	`''`	Overlay independent of the base layer
`customTileUrl`	`''`	The `custom` provider's template

Key	Default	Purpose
`showScaleBar`	`true`	
`layerVisibility`	all true	Routes / tracks / waypoints toggles

CSP becomes a function of settings `buildCsp()` currently takes tile hosts. It gains the configured routing origin in `connect-src`. The origin is parsed and validated — scheme must be `https:`, or `http:` only for a loopback host, so a self-hosted instance works without opening the door to plaintext anywhere else.

`connect-src` is never widened to a wildcard. A provider reaches exactly the origin the user named.

Error handling and fallback

Failure	Behaviour
No endpoint configured	Snap and speed features unavailable, with an explanation. Drawing works.
Consent not accepted	Same, plus the consent prompt naming the host.
Timeout / network error	`ServiceError('PROVIDER_UNAVAILABLE')`, geometry untouched, cached attributes still shown with their age.
Provider returns no match	Legs kept straight and marked unmatched; the snap still succeeds partially.
Snap exceeds the point cap	Refused with the count, before anything is written.

A failed snap never leaves a route half-modified: the write happens in one transaction after the provider has responded.

Testing strategy The provider-dependent code is tested without a live Valhalla instance, because a test that needs a service is a test that does not run.

1. **Adapter unit tests against canned responses.** Synthetic Valhalla JSON fixtures in `test-data/routing/`, authored by hand from the published API reference. These catch the polyline-precision and unit-conversion mistakes.
2. **A loopback HTTP server in the test** for the client: timeout, abort, non-200, malformed JSON. This exercises the real `fetch` path.
3. **Pure maths tested directly** — bearing, turn angle, profile resampling, distance-weighted banding, the Unknown band.
4. **E2E with the dialog stubbed in the main process**, as Phase 3 does for file dialogs, plus a loopback routing endpoint for the snap path.

The honest limitation, recorded here and in the README: **the adapter is verified against the published API reference and synthetic fixtures, not against a live Valhalla deployment.** Until it is exercised against a real instance, the wire-format mapping is evidence-based but unproven. That is stated in the UI as well as here, rather than implied to be tested.

Out of scope, deliberately

- Curvature-aware routing — Phase 5, which this phase unblocks.

- Tile prefetching or area download — forbidden by Requirement 8.2 and the OSM tile usage policy.
- Bundling an OSM extract or a routing engine binary.
- A second routing adapter. One proves the interface; a second without a user asking for it is speculation.

9.6.3 Tasks

Execute in order. The pure, offline-testable work comes first so that the provider integration lands on top of maths and storage that are already proven.

Update `docs/progress-mindmap.svg` as each task changes status.

☒ 1. Shared contracts
- `shared/routing/RoutingProvider.ts`: `RoutingProvider`, `TransportMode`, `RouteLeg`, `SnapResult`, `MatchedEdge`, `RoadClass`, `SurfaceKind`
- `shared/map/MapProvider.ts`: overlay registry, `custom` provider, OpenTopoMap entry with its CC-BY-SA attribution
- Domain types for profiles and speed distribution
- Unit tests: registry lookup, host collection, overlay independence
- *Requirements: 1.1, 1.2, 1.5, 1.6, 2.1, 2.7*

☒ 2. Pure geometry: bearing, profile, speed bands
- `shared/geo/bearing.ts`: initial bearing, turn angle, normalisation. The first angle maths in the project
- `shared/geo/profile.ts`: cumulative distance, elevation band, band resampling for a large track
- `shared/geo/speedBands.ts`: distance-weighted distribution with an explicit Unknown band, metric and imperial band sets
- Unit tests: bearing against known pairs, antimeridian, distribution sums to 100%, Unknown never redistributed, point-count vs distance weighting produce different answers on uneven spacing
- *Requirements: 5.1, 5.2, 6.4, 6.5, 6.7*

☒ 3. Map provider registry and dynamic CSP
- OpenTopoMap and `custom` wired into the registry
- `buildCsp()` takes the configured routing origin; origin validated as https, or http only for loopback
- Layer selector lists providers and overlays, persists the choice
- Unit tests: CSP contains the configured origin and nothing else; a non-loopback http origin is refused; wildcard never emitted
- *Requirements: 1.2, 1.3, 1.4, 1.5, 1.6, 8.1, 12.2*

☒ 4. Migration 0002 and repositories
- `routes` + `routing_provider`, `transport_mode`, `snapped_at`
- `route_control_points` table, dense `(route_id, seq)`
- `road_attributes` table with `geometry_hash` and JSON payload
- Repository functions, and `geometryHash()` over rounded coordinates
- Unit tests: migration applies to a Phase 3 database; control points round-trip; a geometry change changes the hash
- *Requirements: 2.4, 6.2, 6.3*

☒ 5. Routing client and Valhalla adapter
- `main/routing/polyline.ts`: polyline6 decode
- `main/routing/client.ts`: POST with timeout and abort, error mapping to `ServiceError`
- `main/routing/valhalla.ts`: `/route` and `/trace_attributes`, explicit `units: kilometers`, SI conversion at the boundary, explicit attribute filter list
- Fixtures in `test-data/routing/`, hand-authored from the API reference

  - Unit tests: polyline6 decodes to the documented coordinates and precision 5 is visibly wrong; km⁄m and km/h⁄m/s conversion; absent `speed_limit` stays null; a loopback server covers timeout, 500 and malformed JSON
  - *Requirements: 2.2, 2.6, 2.7, 6.1*
- ☒ 6. Routing service: snap a route
  - Snap on create and as a separate action on an existing route
  - Control points retained; snapped shape written to `route_points`
  - Unmatched legs kept straight and recorded
  - `MAX_SNAPPED_POINTS` enforced before any write; refusal names the count
  - Undo carries the previous array under 10,000 points, otherwise reports `kind: 'none'` and says why
  - Unit tests: control points preserved, unmatched legs marked, statistics recomputed, point cap refused cleanly, undo both ways
  - *Requirements: 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.8*
- ☒ 7. Road attributes service
  - Map match a route or track, store the cache keyed by geometry hash
  - **Tracks are never modified** — matching reads attributes only
  - Speed distribution assembled from the cache
  - Unit tests: cache hit avoids a second call, geometry edit invalidates, track geometry unchanged after matching, Unknown band sized honestly
  - *Requirements: 3.7, 6.1, 6.2, 6.3, 6.4, 6.5, 6.6, 6.12*
- ☒ 8. Settings and consent
  - `routingEndpoint`, `routingConsentAccepted`, `transportMode`, `mapOverlayId`, `customTileUrl`, `showScaleBar`, `layerVisibility`
  - Consent names the host and states what is transmitted; every outbound call is gated on it
  - Unit tests: a call without consent is refused; an invalid endpoint is rejected at the schema
  - *Requirements: 2.5, 3.9, 12.1, 12.5, 12.6*
- ☒ 9. IPC and preload wiring
  - Channels for snap, match, profile, distribution, provider status
  - Zod schemas for every payload; results never throw across the boundary
  - *Requirements: 2.2, 12.1*
- ☒ 10. Map controls
  - Scale bar, cursor coordinates in configured units
  - Visibility toggles for routes, tracks and waypoints
  - Keyboard navigation of the map
  - Unit and e2e tests for the toggles and the scale bar
  - *Requirements: 4.1, 4.2, 4.3, 4.4*
- ☒ 11. Measure tool
  - A distinct editor mode; creates nothing and has no undo entry
  - Running total and segment distances in configured units
  - *Requirements: 4.5*
- ☒ 12. Snap-to-road UI
  - Snap mode in the draw flow, visually distinct from straight-line drawing
  - Snap action on an existing route with a point-count preview
  - Unmatched legs drawn dashed and explained
  - Transport mode selector
  - *Requirements: 3.1, 3.2, 3.4, 3.6, 3.8, 9*
- ☒ 13. Profile panel
  - Hand-written SVG; elevation, speed and road-class bands
  - Hover syncs to the map and back
  - A band with no data is omitted with a stated reason
  - Simplified for a large track, and says so

  - Export the profile as an image
  - *Requirements: 5.1–5.10*
- ☒ 14. Speed distribution UI
  - Percentage of distance per band, Unknown included and never hidden
  - Source labelled: posted limit or recorded
  - Provider and fetch date shown; stale cache shown with its age
  - *Requirements: 6.1, 6.4, 6.5, 6.7, 6.8, 6.9, 6.11, 8.4*
- ☒ 15. Waze hand-off
  - Waypoint, and route or track resolving to its end point
  - Label states “end point”; no multi-waypoint action exists
  - `shell.openExternal` after validation
  - *Requirements: 7.1, 7.2, 7.3, 7.4*
- ☒ 16. Phase 4 verification pass
  - `npm run verify` green; e2e green against the packaged build
  - Walk the acceptance criteria: a second provider needs only a registry and CSP entry; straight-line drawing is unchanged with no provider configured; the Unknown band is visible; no wildcard in the CSP
  - Record in the README that the Valhalla adapter is verified against the published reference and synthetic fixtures, **not** against a live instance
  - Update the mindmap to show Phase 4 complete
  - *Requirements: all*

Added after the verification pass

- ☒ 17. Interactive snap routing tool
  - A distinct editor mode: each click is a **control point**, and the leg from the previous one is routed immediately so the road-following line appears as the user builds it
  - One provider call per click (`routing:routeLeg`), not a re-snap of the whole line, so the tenth click costs what the first did
  - Legs are routed strictly in order with a generation guard, because replies can arrive late or out of order and `legs` must stay index-aligned with the gaps between control points
  - Saves the **previewed** geometry (`routing:createSnapped`), not a re-route: the user accepted a specific line and must get it
  - Bounded by `MAX_UNDOABLE_POINTS` rather than `MAX_SNAPPED_POINTS`, so a route built with the tool stays editable through `route:setGeometry`
  - Three visually distinct leg states: routed, unmatched (dashed red, matching a whole-route snap), and pending
  - Toolbar control is **disabled with the reason** when no provider is available, never silently inert
  - Unit tests: one call per leg, unmatched reported not thrown, previewed geometry saved verbatim, leg/point mismatch refused with nothing written, point cap refused with the count, undo removes the control points too
  - E2E: the tool is present but disabled with no provider; `routeLeg` refuses with `PROVIDER_UNAVAILABLE`; a leg/point mismatch is refused at the schema
  - *Requirements: 3.1, 3.2, 3.3, 3.6, 3.8, 9*
- ☒ 18. Embed the routing engine
  - Build Valhalla 3.9.1 from source with `ENABLE_NODE_BINDINGS=ON` and `ENABLE_SERVICES=OFF`: the N-API bindings wrap `tyr::actor_t`, which takes the same JSON as the HTTP API, so there is no daemon, port or supervisor
  - `scripts/build-valhalla.sh`, `scripts/bundle-valhalla.sh`, `scripts/verify-valhalla.mjs`, `npm run build:valhalla`, `npm run verify:valhalla`
  - Relocatable bundle: 113 dylibs rewritten to `@rpath` with `@loader_path`-relative rpaths, ad-hoc re-signed

  - Adapter refactored so the embedded and HTTP providers share every line of request construction and response translation — only the transport differs
  - arm64 only; `minimumSystemVersion` raised to 13.3 for `std::to_chars`
  - Licence audit of the native tree, including the libspatialite / readosm tri-licence election to LGPL-2.1-or-later
  - Verification: the bundler relocates a copy and loads it in Electron, because a path audit passed on a bundle that could not load
  - *Requirements: 2.1, 2.2, 2.6, 3.1, 6.1; ADR 0013*
- ☒ 19. Wire the UI to the embedded engine
  - `LocalValhallaProvider` (`main/routing/embedded.ts`), preferred when no endpoint is configured — needs no endpoint and **no consent**, because nothing leaves the machine, and contributes no CSP origin
  - An explicitly configured remote endpoint still wins: a deliberate choice is respected rather than overruled
  - Map region import: `routing:importRegion` spawns `valhalla_build_tiles` into a staging directory and swaps it in only on success, so a failed or cancelled build cannot leave a half-written graph that reports as usable
  - Region metadata stored beside the graph, not in settings, so deleting the graph takes its label with it
  - Three distinguishable unavailable states — `no-engine` (broken build), `no-map-data` (user action), `consent-required` (remote only) — each with the action that actually resolves it
  - Unit tests: presence checks, config generation, region round-trip, embedded-vs-remote preference, consent still enforced for remote
  - E2E against the real engine and a real graph: status available with no endpoint or consent, a leg routed in-process, a drawn route snapped with control points kept, posted speed limits off the local graph, the toolbar tool enabled, the properties panel naming the engine
  - *Requirements: 3.1, 3.2, 3.3, 6.1, 6.4, 12.1; ADR 0013*
- ☒ 20. Region download from the visible map area
  - `shared/geo/polygon.ts`: pure point-in-polygon, bounding-box area and containment — planar on purpose, since these are coarse administrative outlines and the question is "which country am I looking at"
  - `main/routing/geofabrik.ts`: parses Geofabrik's `index-v1.json` (554 regions, nested three deep, polygon boundaries plus a direct `.pbf` URL) and ranks candidates for a viewport
  - **Ranked by the viewport centre, not by full coverage.** The obvious rule is wrong and measurably so: zoomed to Vaduz the viewport corners cross Liechtenstein's border, so the smallest fully-containing region becomes the 2.3 GB Alps bundle. `coversViewport` is reported per candidate instead, so the UI can offer the choice for a view spanning a border
  - Confirmed again on the Buxton test geography once it replaced Vaduz (ADR 0015), where the case is not a near-miss but the normal state: the test loop spans Derbyshire, Cheshire and Staffordshire, and Geofabrik publishes England by county, so **no county extract covers it.** A view of the whole loop ranks `staffordshire` first with `coversViewport: false`, and offers `england` as the one that does
  - Real download sizes via HEAD through the `-latest.osm.pbf` → dated-file redirect, followed only within the same origin
  - Oversized candidates marked unselectable up front rather than refused at the end of a multi-gigabyte download
  - Index cached on disk with a week's TTL, and used even when stale if the network fails — a week-old index beats no picker
  - Extract deleted after the graph is built; it is the raw input and several times the size of what it produces

- Geofabrik chosen over an Overpass bounding box because a highway-only Overpass extract loses the relations that encode turn restrictions, so the graph would route the wrong way down no-entry streets with nothing to indicate why
- Unit tests on synthetic geometry reproducing the border-spanning and spill-over cases; `npm run verify:geofabrik` checks the real index still parses and still ranks as assumed
- *Requirements: 3.1, 6.1, 8.1; ADR 0013*

Notes

9.7 Phase 5: Object and point management — Requirements — SPEC-PHASE-5-OBJECT-POINT-MANAG

Version: — · **Status:** unspecified · **Slug:** `phase-5-object-point-management`

9.7.1 Requirements

Managing what is already in the collection, rather than creating more of it. Three capabilities, in increasing order of risk:

1. Select several routes, tracks or waypoints at once and move or delete them in one action.
2. Edit the points of a single route or track in a table.
3. Selecting rows in that table highlights those points on the map, so a point in a list of thousands can be identified without counting.

Inserted ahead of windy-roads routing, which moves to Phase 6 and Garmin compatibility to Phase 7. The ordering is deliberate: Phase 6 generates routes with thousands of points, and a user with no way to inspect or trim them would be worse off than before.

Context the investigation established A codebase assessment was run before these requirements were written, because all three capabilities extend existing single-object machinery rather than sitting beside it. Five findings shape what follows.

There is no multi-selection anywhere. `selectionStore` holds one `ObjectRef` plus five mutually exclusive detail slots, and `ensureGeometry` enforces that exactly one trip's geometry is loaded at a time. Seven components read `selected` directly.

There is no composite undo. One `commit()` is one undo entry, one library refresh and one native-menu rebuild. Looping an existing channel N times gives N of each, and ⌘Z would reverse one item per press.

No track mutation is currently undoable, and the code claims otherwise. `commands.ts` registers no `track:*` handlers, so `undoService.record()` silently discards every track inverse — including `track:deleteRange`, whose service comment states that "the removed run is small by construction, so it is cheap to carry as a real inverse". That is a latent defect, not a design choice, and this phase owns fixing it.

Route points have no stable identity. `replacePoints` deletes every row and re-inserts with a fresh `randomUUID()`, so a table cannot key rows on a point id — only on index, which shifts under every insert and delete.

Track points are deliberately not editable per point. `deletePointRange` is contiguous-only and collapses all segments into one. ADR 0003 and `join()` both treat a segment break as real information about the recording.

Requirement 1: Multi-selection in the library **User story.** As someone tidying up an imported collection, I want to select several objects at once so that moving or deleting them is one action rather than twenty.

Acceptance criteria 1.1. ⌘-click toggles an object's membership in the selection. Shift-click selects the inclusive range between the anchor and the clicked row, computed over the **currently visible rows** — a collapsed trip's hidden children are not silently included.

1.2. A plain click replaces the selection with the clicked object, and sets it as the anchor.

1.3. The selection may contain routes, tracks and waypoints, including a mixture of them. Trips and folders are **excluded** from multi-selection: they are containers whose deletion cascades, and batching a cascade behind one keystroke is not a risk worth offering.

1.4. The selection is visible: every selected row is marked, and the panel states how many objects are selected and of what kinds.

1.5. The tree exposes the selection to assistive technology — `aria-multiselectable`, `aria-selected` per row, and a roving `tabIndex` so the tree is keyboard-navigable at all. It is not today.

1.6. Keyboard: arrow keys move focus, Space toggles, Shift-arrow extends, ⌘A selects every selectable object in the visible tree, Escape collapses to the anchor alone.

1.7. Right-clicking a row **inside** an existing multi-selection keeps that selection and offers the batch actions. Right-clicking outside it replaces the selection with that row, matching the current single-select behaviour.

1.8. The same object appearing in two tree positions — under its trip and in a cross-trip view, per ADR 0004 — is one member of the selection, not two. Selecting it in either position marks both.

1.9. Selection survives a library refresh. An object that has disappeared is dropped from the selection silently; it is not an error.

1.10. The properties panel continues to describe the **anchor** object. It does not attempt to merge or multi-edit, and when more than one object is selected it says so and names what the batch actions will apply to.

Requirement 2: Batch move and delete **User story.** As someone reorganising trips, I want to move or delete a selection in one step, and undo it in one step.

Acceptance criteria 2.1. A multi-selection can be moved to a trip, by drag-and-drop onto the trip or by a menu action. Tracks are included — they are currently absent from the draggable set, which is an omission rather than a decision.

2.2. A multi-selection can be deleted.

2.3. **One batch is one undo entry.** Undoing a batch of twelve deletions restores all twelve in one ⌘Z, and the Edit menu names the count and kind: “Undo delete 12 routes”.

2.4. A batch applies in a single transaction. If any member fails, nothing is applied and the message names which object failed and why.

2.5. **Partial undoability is stated before the action, not discovered after it.** Route and waypoint deletions are undoable; track deletion is not, by ADR 0003, because the point array is too large to carry. A mixed batch must say plainly which part cannot be reversed, and require confirmation.

2.6. A batch delete containing a track is confirmation-gated, naming the number of tracks and total points that will be irreversibly removed.

2.7. Batch operations fire **one** library-changed notification and **one** undo-state publication, not one per member.

2.8. A batch move across trips updates the map’s loaded geometry once, afterwards — not once per member.

2.9. The count is bounded, with the bound stated. A selection larger than the bound is refused with the number rather than attempted.

Requirement 3: Point table for a route **User story.** As someone who drew a route slightly wrong, I want to see its points in a table so I can correct a coordinate or remove a stray point without hunting for a 4-pixel handle on the map.

Acceptance criteria 3.1. The table lists the selected route's points in order with their index, latitude, longitude, elevation, name, symbol and notes, and the cumulative distance from the start.

3.2. Latitude, longitude and elevation are editable in place, validated against the same bounds the IPC schema enforces. Elevation accepts empty, meaning unknown — it does not coerce to 0.

3.3. Rows can be deleted, including a non-contiguous selection of rows. A route must keep at least one point; emptying it is refused with the suggestion to delete the route instead.

3.4. Rows can be reordered, and a point can be inserted before or after a selected row.

3.5. Every edit goes through the existing `routes.setGeometry` channel, so it is one undo entry and inherits the existing 10,000-point cap. **No new per-point channel is introduced.**

3.6. The table reflects external changes. Dragging a vertex on the map updates the table, because both read the same canonical geometry.

3.7. Indices are presented as 1-based for display and documented as such, since the user-facing count is "point 1 of 240".

Requirement 4: Point table for a track **User story.** As someone whose recording has a GPS spike through a building, I want to find and remove those samples without discarding the whole recording.

Acceptance criteria 4.1. The table lists a track's points with index, latitude, longitude, elevation, timestamp, and the distance and derived speed from the previous point. Speed is the existing recorded-speed calculation, null where untimed — never 0.

4.2. **The table is virtualised.** A 200,000-point track must scroll without materialising 200,000 rows.

4.3. The table always reads exact points, never the simplified render cache. Showing an index from a simplified payload would mean the row the user edits is not the sample they are looking at. Where the map is drawing simplified geometry, the table says so.

4.4. Rows can be deleted, including non-contiguous selections. **Segment structure survives.** Deleting across a segment boundary does not merge the segments, and deleting the points either side of a break does not remove the break. The existing `deletePointRange` collapses everything into one segment, which is acceptable for a trim and not acceptable here.

4.5. Coordinates, elevation and timestamp are editable, so a single bad sample can be corrected rather than only removed.

4.6. **Editing a recording is labelled as such.** The panel states that the track is measured data and that the imported original remains recoverable where it was retained, with the restore action to hand.

4.7. Track point edits are undoable where the carried inverse is bounded, and honestly non-undoable where it is not, with the threshold stated. This requires registering the missing `track:*` command handlers — without them the existing `deleteRange` inverse is discarded and the feature would silently not undo.

4.8. A track must keep at least one point.

4.9. Deleting a large number of points reports what it will do before doing it, including whether it will be undoable.

Requirement 5: Map highlighting of selected points **User story.** As someone looking at a table row, I want to see which point on the map it is, so I can tell whether I have found the right one.

Acceptance criteria 5.1. Selecting one or more table rows highlights exactly those points on the map, visibly distinct from ordinary vertices and from the selection colour used for whole objects.

5.2. The highlight is bidirectional. Clicking a highlighted point on the map selects its table row and scrolls it into view.

5.3. Highlighting a point outside the current view offers to bring it into view; it does not pan the map on every row change, which would fight the pointer during a drag-select.

5.4. Highlighting is bounded and does not degrade the map. Leaflet is configured with `preferCanvas`, so a highlighted subset is drawn in its own layer rather than by restyling markers in the shared canvas renderer. Above a stated number of highlighted points the map shows the extent of the selection instead of every individual point, and says that is what it is doing.

5.5. The **table is the accessible surface**. Canvas-rendered points cannot take keyboard focus, so the map mirrors the table's selection and is not an independent input for assistive technology.

5.6. Highlighting is read-only. It changes no data and creates no undo entry.

Requirement 6: Consistency with what exists 6.1. Existing single-selection behaviour is unchanged when exactly one object is selected. The seven current consumers of `selected` continue to work without modification.

6.2. The point table is an additional view of geometry, not a second source of truth. It reads the same canonical data the map draws.

6.3. Absent data stays absent throughout. A missing elevation or timestamp displays as unavailable and round-trips as null.

6.4. No operation added here writes `render_geometry` without `track_points` already being correct, per ADR 0003.

6.5. Every new IPC channel validates its payload with a zod schema and returns a discriminated result, per the architecture rules.

Out of scope, stated

- **Multi-object property editing.** Setting the colour of twelve routes at once is a reasonable want and is not in this phase; the properties panel describes the anchor only.
- **Multi-select in search results.** Search renders its own flat list for trips and waypoints only; extending selection into it is deferred rather than half-implemented.
- **Trips and folders in a multi-selection**, per 1.3.
- **Editing a track's segment structure directly.** Splitting and joining remain the operations for that.
- **Cross-trip multi-selection driving the map.** The map shows one trip's geometry; a selection spanning trips is permitted for move and delete, but the map follows the anchor.

Open questions for the owner

1. **Should editing a track's coordinates be possible at all**, or should the table be delete-and-inspect only? Requirement 4.5 assumes correcting a single bad sample is worth having. The opposing view is that a recording is measurement and should only ever be trimmed, never rewritten. The retained original makes either position defensible.
2. **What bound on a batch?** Requirement 2.9 asks for one without naming it.
3. **What bound on highlighted points** before switching to an extent, per 5.4?

9.7.2 Design

Three features that each extend single-object machinery: multi-selection, point tables, and point highlighting on the map. The interesting content of this design is not the features — it is the five existing invariants they collide with, and what each collision resolves to.

1. Multi-selection: a set beside the anchor, not instead of it `selectionStore` is single-select by shape: one `ObjectRef` plus five mutually exclusive detail slots, where `reload()` spreads `EMPTY` on every branch so exactly one slot is ever populated. Seven components read `selected`.

Rewriting that into a collection would touch all seven and change the meaning of `route`, `track` and `waypoint` for every reader. Instead:

```
interface SelectionState {
  /** The anchor: last clicked, and the object the properties panel describes. */
  selected: ObjectRef | null;
  /**
   * Every selected object including the anchor, keyed `kind:id`.
   *
   * A Set of keys rather than an array of refs so membership is O(1) per tree
   * row - the tree re-renders every row on selection change, and a linear
   * scan per row is quadratic in the selection size.
   */
  selectedKeys: ReadonlySet<string>;
  /** Where a Shift-click range starts. Null until something is clicked. */
  anchorKey: string | null;
  // ... the five detail slots, unchanged, describing the anchor only
}
```

Three consequences fall out, and each is a deliberate answer to a constraint the investigation surfaced:

Detail loading stays keyed on the anchor. `tracks.get` returns every point — up to 200,000 — which the store's own comment notes is paid "once per selection rather than per render". Loading detail for a multi-selection of tracks would multiply that payload by the selection size for no benefit, because the properties panel shows one object.

`ensureGeometry` keeps its one-trip rule. The map shows a single trip's geometry. A selection spanning trips follows the anchor, which is Requirement 6.1's "unchanged when one object is selected" extended honestly: the map was never going to show two trips, and pretending otherwise would be a bigger change than this phase is buying.

Pruning is explicit. The existing `reload()` branches already null `selected` when an object has vanished. `selectedKeys` needs the same treatment, and a batch delete needs it for N objects at once — so pruning becomes a single operation taking the keys that survived rather than a per-branch conditional.

Two keyspaces that must not be conflated `TreeNode.id` is a **path** (`view:routes/route:abc`), deliberately distinct from the object ref, because ADR 0004 puts the same route under its trip and in the cross-trip Routes view. Expansion state and React keys use the path; selection uses `kind:id`.

A Shift-click range is computed over the **flattened visible row list keyed by path**, and produces members keyed by `kind:id`. Requirement 1.8 — one object selected in two places is one member — is a direct consequence of that asymmetry, and it falls out for free rather than needing special handling.

`TreeRow.onClick` currently discards its event, so the modifier keys are not available. It must forward `{ metaKey, shiftKey }` and the row's path.

2. Batch operations: a composite command `commit()` in `register.ts` is the single funnel for every mutation: it records one inverse, notifies the library once, and republishes the undo state — which also rebuilds the native menu, because Electron menus are immutable once built.

Looping an existing channel for a batch therefore gives N undo entries, N library refreshes and N menu rebuilds, and ⌘Z would reverse one item per press. Requirements 2.3 and 2.7 both rule that out.

The composite fits the existing contract without changing it:

```
// commands.ts
| 'batch'

'batch': (db, p) => {
  const commands = commandArray(p);
  const applied = db.transaction(() =>
    commands.map((command) => applyCommand(db, command)),
  );
  return {
    result: applied.map((m) => m.result),
    // Reversed: undoing a batch must unwind in the opposite order it applied,
    // or a restore can run before the delete that made room for it.
    inverse: {
      kind: 'batch',
      payload: { commands: applied.map((m) => m.inverse).toReversed() },
    },
  };
}
```

Because `applyCommand` returns a `Mutation`, the inverse of a batch is a batch — the same trick that makes single commands self-inverting, applied one level up.

Three details that are easy to get wrong:

`isUndoable` must look inside. A batch whose children are all `none` — a batch of track deletions — would otherwise report as undoable and put a step on the stack that does nothing. The batch handler filters `none` children, and a batch that filters down to nothing records nothing.

Labels need the count. `LABELS` is a flat `Record<CommandKind, string>`, so "delete 12 routes" cannot come from it. `record(inverse, label?)` already accepts an optional label that no caller currently uses; batch callers pass one.

The stack bound still applies. `MAX_DEPTH = 50`. A batch is one entry, so batching makes the stack hold *more* user intent, not less.

Service layer Batch move and delete are composed in a service from the existing single-object operations, not reimplemented:

```
// batchService.ts
export function remove(db, refs): Mutation<BatchOutcome>
export function moveToTrip(db, refs, tripId): Mutation<BatchOutcome>
```

Each collects the per-object `Mutation`s inside one transaction and returns a batch inverse. `BatchOutcome` reports counts per kind and, critically, which members were **not** undoable — Requirement 2.5 needs that at the UI before the action, so the same calculation is exposed as a dry-run query.

The track undo gap `commands.ts` registers no `track:*` handlers. `trackService` emits `track:update`, `track:move`, `track:reverse`, `track:restorePoints`, `track:deleteRange` and `track:delete` as inverses; `commit()` passes them all to `record()`; and `record()` drops every one because `isUndoable` tests `kind in HANDLERS`.

So today a track move or rename cannot be undone, and `deleteRange` claims an inverse it does not have. This phase registers the handlers that have bounded inverses — `track:update`, `track:move`, `track:reverse`, `track:restorePoints`, `track:deleteRange` — and leaves `track:delete` as `none` deliberately, per ADR 0003.

That is a bug fix rather than a feature, and it is a prerequisite: Requirement 4.7 cannot be satisfied without it.

3. Point tables: one shape, two very different data paths A single `PointTable` component over a normalised row model, with a virtualised body. The data paths behind it diverge completely, because routes and tracks are stored differently on purpose (ADR 0003).

	Route	Track
Source	`routes.get` — whole array	New paged channel
Cap	10,000, already enforced	~200,000, unbounded today
Row identity	**Index only**	**Index only**
Edit channel	Existing `routes.setGeometry`	New targeted channels
Undo	Whole previous array, already bounded	Targeted inverse, bounded by count

Routes: reuse `setGeometry`, add nothing Route edits build a new array and send it through the existing channel. One undo entry, the existing 10,000 cap, the existing validation. Requirement 3.5 forbids a per-point channel, and the reason is concrete: `setGeometry`'s inverse is already the previous array, so a per-point channel would produce a worse undo story for no gain.

Tracks: paged reads, targeted writes A 200,000-point track cannot be sent to the renderer as one payload for a table that shows forty rows. A paged read channel returns a window plus the total:

```
tracks.points({ id, offset, limit }) -> { total, offset, points, segmentBreaks }
```

`segmentBreaks` carries the indices where a segment ends, because Requirement 4.4 demands segment structure survive a delete and the renderer cannot infer breaks from a window that does not contain one.

Writes are targeted. The existing `deletePointRange` is contiguous-only **and collapses every segment into one** — fine for trimming a tail, wrong here. A new operation takes a set of indices and rebuilds segments preserving breaks:

```
trackService.deletePointsAt(db, id, indices): Mutation<TrackWithPoints>
```

Its inverse carries the removed points with their indices and segment positions, so it is bounded by how many were deleted rather than by track size. Above a threshold it reports `none` and says so, following the precedent set by `simplifyTrack` and `routingService.snapRoute`.

Row identity is the index, and that is a real constraint `replacePoints` re-UUIDs every row, so there is no stable point id to key on. The table keys on index, which means:

- An edit invalidates every index after the edit point.
- The table reloads after any mutation rather than patching locally.
- A pending selection of rows is remapped or cleared on reload — not silently reinterpreted against shifted indices, which would delete the wrong points.

This is worth stating because "just key on the id" is the obvious instinct and there is no id to use.

4. Point highlighting: a separate layer, because of the canvas Leaflet runs with `preferCanvas: true`. Consequences:

- Track points are `L.circleMarker`s drawn into a **shared canvas renderer**. There is no DOM node per point, so there is nothing to restyle with CSS, and `setStyle()` on one marker redraws the whole renderer pane.
- Canvas-drawn points cannot take keyboard focus, which is why Requirement 5.5 makes the **table** the accessible surface and the map a mirror.

So highlighting is a **third layer group** holding only the highlighted subset, rather than restyling members of the existing layer:

```
useRouteLayers   lines | handles | highlight   (new)
useTrackLayers   lines | markers | highlight   (new)
```

The existing handles effect does a full `clearLayers()` and depends on `[map, routes, selectedId, enabled]`. Adding `highlightedIndices` to it would rebuild every vertex handle on every table-row click. A separate layer means a row click touches only the highlight group.

Above the Requirement 5.4 bound the highlight layer draws the **extent** of the selection — a bounding rectangle — instead of individual points, and the table says that is what it is showing. That keeps a select-all on a 200,000-point track from attempting 200,000 markers, which is precisely what the existing `showPoints` guards exist to prevent.

`showPoints` itself stays as it is: opt-in, selected-track-only, exact-geometry- only. Highlighting is independent of it, because a highlighted point must be visible whether or not every point is.

5. Where things live

```
src/
  shared/
    domain/types.ts           SelectionSet helpers, PointRow
    ipc/schemas.ts            batch, paged points, point-edit payloads
  main/
    services/
      batchService.ts         composes existing mutations into one
      trackService.ts         + deletePointsAt, editPointAt
      commands.ts             + 'batch', + the missing track:* handlers
    db/repositories/
      tracks.ts               + paged point read
  renderer/
```

```
stores/selectionStore.ts  + selectedKeys, anchorKey, pruning
features/
  library/                multi-select, keyboard model, batch menu
  points/                 PointTable, virtualised body, row model
  map/                    + useHighlightLayer
```

6. Risks, and what each resolves to

Risk	Resolution
A mixed batch delete is partly irreversible	Stated before the action and confirmation-gated (2.5, 2.6), not discovered afterwards
Index-keyed rows go stale under concurrent edits	Reload after every mutation; remap or clear a pending row selection rather than reinterpret it
Non-contiguous track delete destroys segment breaks	New `deletePointsAt` rebuilds segments; `deletePointRange` keeps its trimming behaviour unchanged
Highlighting many canvas points stalls the map	Separate layer, bounded count, extent rectangle above it
A paged table disagrees with the map	Table always reads exact points; where the map is simplified the table says so (4.3)
The tree gains a keyboard model it never had	Roving `tabIndex` and `aria-multiselectable` are explicit requirements (1.5, 1.6), not incidental

7. Testing Unit, against the services:

- Batch undo is one entry, unwinds in reverse order, and a batch of non-undoable members records nothing.
- A batch that fails mid-way applies nothing.
- `deletePointsAt` with non-contiguous indices preserves segment breaks, and preserves a break whose adjacent points were both deleted.
- The missing `track:*` handlers now round-trip: a track move undoes.
- Route point edits remain within the 10,000 cap and produce one undo entry.
- Paged point reads return consistent totals and correct segment breaks at a window boundary.

End-to-end:

- ⌘-click and Shift-click build the expected selection over visible rows only.
- One ⌘Z reverses a batch delete of several routes.
- A mixed batch naming a track is confirmation-gated and says what is irreversible.
- Selecting table rows highlights the matching map points; clicking a highlighted point selects its row.
- A large track's table scrolls without materialising every row.

9.7.3 Tasks

Execute in order. The undo-stack repair comes first because two later tasks depend on it, and the pure work comes before the UI so the services are proven before anything draws them.

Update `docs/progress-mindmap.svg` as each task changes status.

- ☒ 1. Repair the track undo gap
  - Register the missing `track:*` handlers in `commands.ts`: `track:update`, `track:move`, `track:reverse`, `track:restorePoints`, `track:deleteRange`

  - `track:delete` stays `none` deliberately, per ADR 0003 — a point array that large is not carried on the stack
  - Add labels for each, so the Edit menu names what it will undo
  - A bug fix, not a feature: every track inverse is currently discarded by `record()` because `isUndoable` tests `kind in HANDLERS`, so a track rename or move cannot be undone today and `deleteRange` claims an inverse it does not have
  - Unit tests: a track move, rename, reverse and range delete each round-trip through undo and redo; a track deletion still reports non-undoable
  - *Requirements: 4.7*
- ☒ 2. Composite batch command
  - `'batch'` kind in `commands.ts`, applying children in one transaction and returning a batch inverse with the children **reversed**
  - `isUndoable` looks inside a batch: one whose children are all `none` records nothing rather than a step that does nothing
  - Batch-aware labels through the `label` argument `record()` already accepts
  - Unit tests: one entry per batch, reverse unwind order, all-`none` batch records nothing, a mid-batch failure applies nothing
  - *Requirements: 2.3, 2.4, 2.7*
- ☒ 3. Batch service
  - `batchService.remove` and `batchService.moveToTrip`, composing the existing per-object mutations rather than reimplementing them
  - `BatchOutcome` reports counts per kind and which members are not undoable
  - A dry-run query returning the same assessment, so the UI can warn before the action rather than after
  - Selection bound enforced, refusing with the count
  - Unit tests: mixed-kind batches, the undoability assessment matches what the batch actually records, the bound is refused cleanly
  - *Requirements: 2.1, 2.2, 2.5, 2.9*
- ☒ 4. Non-contiguous track point deletion
  - `trackService.deletePointsAt(db, id, indices)` rebuilding segments with breaks preserved, including a break whose adjacent points were both deleted
  - `deletePointRange` keeps its current trimming behaviour unchanged — it collapses segments, which is right for a trim and wrong here
  - Inverse carries the removed points with their indices and segment positions, so it is bounded by the number deleted rather than track size; above a threshold it reports `none` and says so
  - Refuses to empty a track
  - Unit tests: non-contiguous deletion across a boundary preserves breaks, inverse restores exactly, the threshold is honoured
  - *Requirements: 4.4, 4.7, 4.8*
- ☒ 5. Track point editing and paged reads
  - `trackService.editPointAt` for coordinates, elevation and timestamp
  - Paged read: `tracks.points({ id, offset, limit })` returning the window, the total, and `segmentBreaks` — the renderer cannot infer a break from a window that does not contain one
  - Reads exact points, never the render cache
  - Unit tests: window boundaries, breaks at a boundary, totals consistent under concurrent edits, absent elevation and timestamp round-trip as null
  - *Requirements: 4.1, 4.3, 4.5, 6.3, 6.4*
- ☒ 6. IPC and preload wiring
  - Channels for batch move, batch delete, the dry-run assessment, paged track points, and the track point edit and delete operations
  - Zod schemas for every payload; results never throw across the boundary

  - Route point editing adds **no** channel — it reuses `routes.setGeometry`
  - *Requirements: 3.5, 6.5*
- ☒ 7. Multi-selection in the selection store
  - `selectedKeys` and `anchorKey` beside `selected`, which stays the anchor
  - Detail loading stays keyed on the anchor; `ensureGeometry` keeps its one-trip rule
  - Pruning takes the surviving keys in one operation, for a batch delete
  - Unit tests: toggle, range, replace, pruning after a batch delete, the five detail slots still describe the anchor only
  - *Requirements: 1.1, 1.2, 1.9, 6.1*
- ☒ 8. Library tree: selection and keyboard model
  - `TreeRow` forwards `{ metaKey, shiftKey }` and its path — it discards the event today
  - Range selection computed over the flattened **visible** rows, keyed by path, producing members keyed by `kind:id`
  - Roving `tabIndex`, `aria-multiselectable`, `aria-selected` per row, arrow and Shift-arrow navigation, Space, ⌘A, Escape
  - Trips and folders excluded from multi-selection
  - Selection count and composition shown in the panel
  - E2E: modifier-click selection over visible rows only; keyboard navigation
  - *Requirements: 1.3, 1.4, 1.5, 1.6, 1.8, 1.10*
- ☒ 9. Batch actions in the UI
  - Context menu keeps an existing multi-selection when right-clicked inside it, and replaces it when right-clicked outside
  - Batch drag-and-drop onto a trip, with `track` added to the draggable set
  - Confirmation naming what is irreversible, for any batch containing a track
  - Native menu and Edit-menu labels carry the count
  - E2E: one ⌘Z reverses a batch delete; a mixed batch warns before acting
  - *Requirements: 1.7, 2.1, 2.2, 2.5, 2.6, 2.8*
- ☒ 10. Point table
  - One virtualised component over a normalised row model, for both kinds
  - Route columns: index, lat, lon, elevation, name, symbol, notes, cumulative distance. Track columns: index, lat, lon, elevation, time, distance and derived speed from the previous point
  - In-place editing with the same bounds the schemas enforce; empty elevation means unknown, not 0
  - Row selection, insert, reorder, and non-contiguous delete
  - Reloads after every mutation; a pending row selection is remapped or cleared rather than reinterpreted against shifted indices
  - Says when the map is drawing simplified geometry
  - States that a track is measured data, with the restore action to hand
  - E2E: a 200,000-point table scrolls without materialising every row
  - *Requirements: 3.1–3.7, 4.1, 4.2, 4.3, 4.6, 4.9, 6.2*
- ☒ 11. Map highlighting
  - `useHighlightLayer`: a third layer group, so a row click does not rebuild every vertex handle
  - Bidirectional — clicking a highlighted point selects and scrolls to its row
  - Offers to bring an off-screen point into view rather than panning on every row change
  - Above the bound, draws the extent of the selection and says so
  - Read-only: no data change, no undo entry
  - E2E: selecting rows highlights the matching points; clicking a point selects its row
  - *Requirements: 5.1–5.6*
- ☒ 12. Phase 5 verification pass
  - `npm run verify` green; e2e green against the packaged build
  - Walk the acceptance criteria: single selection behaves exactly as before; one batch is

one undo entry; a mixed batch states what it cannot reverse; segment breaks survive a non-contiguous delete; the table never shows simplified indices
- Confirm the track undo repair from task 1 holds end to end
- Update the mindmap to show Phase 5 complete
- *Requirements: all*

Notes

9.8 Phase 6: Route alternatives and shaping — Requirements — SPEC-PHASE-6-ROUTE-ALTERNATIVES

Version: — · **Status:** unspecified · **Slug:** phase-6-route-alternatives-shaping

9.8.1 Requirements

Status: **complete.** All 15 tasks done and verified, including against the packaged app.

When a route is made between two places, show the routes the engine can offer between them and let the user pick one. Then let them shape it directly on the map: drag the line to pull it onto a different road, drag the points they placed, and click to add or remove points.

Inserted ahead of windy-roads routing, which moves to Phase 7, and Garmin compatibility, which moves to Phase 8. See ADR 0017. The ordering is deliberate. Windy-roads routing produces routes that a user will want to compare with the conventional one and then adjust. Building the comparison and the shaping first means Phase 7 adds a cost model to an editing surface that already exists, instead of building both at once.

Terms

Term	Meaning
Control point	A point the user placed. A snapped route's geometry is computed from its control points. Stored in `route_control_points`.
Via point	A control point the route is meant to *visit*: the start, the end, and every point clicked in the snap routing tool. Shown as a numbered handle.
Shaping point	A control point that only *bends* the route onto a road. Created by dragging the line or by choosing an alternative. Shown as a small unnumbered handle.
Leg	The geometry between two adjacent control points.
Candidate	One of the routes the engine offers for a single leg. Candidate 1 is the engine's preferred route. The others are its alternatives.

Context the investigation established A codebase assessment was run before writing these requirements, against Phase 4's routing as it actually shipped. Eight findings shape what follows.

There are no alternatives anywhere. `RoutingProvider` has two methods, `snap` and `match`. Each leg asks for exactly one route and gets one.

Valhalla can offer alternatives, but only between two locations. The `/route` request takes an `alternates` count, and the bundled `config.template.json` sets `service_limits.max_alternates` to 2. That gives at most three candidates. Valhalla does not return alternatives for a request with more than two locations. That limit happens to fit Phase 4's design: every leg is already its own two-location `/route` call (see the comment in `valhalla.ts`), so alternatives can be offered **per leg** without changing how routing is done.

Clicking the line of a selected snapped route corrupts it. The click handler in `useRouteLayers` calls `insertRouteVertex`, which inserts a raw vertex into the *derived* road geometry through `route:setGeometry`. The control points are left unchanged, so the stored shape no longer

matches the points it is supposed to be derived from. The next re-snap silently throws the inserted vertex away. This is a live defect and this phase fixes it.

Dragging one control point re-snaps the whole route. `moveControlPoint` calls `snapRoute` with the full control-point list. On a twenty-point route, that is nineteen provider calls for one moved point. The undo inverse is then the whole previous geometry, so the drag cannot be undone on any route above `MAX_UNDOABLE_POINTS`. Phase 4 Requirement 3.3 retained control points "so a later edit of a control point can re-snap only the affected legs". That storage was built, but re-snapping only the affected legs was not.

Leg boundaries are not stored. `route_points` is one flat sequence, and `route_control_points` holds only `seq`, `lat` and `lon`. Nothing records where leg *i* starts in the shape, so there is no way to replace one leg's geometry without re-routing everything. `flattenLegs` also de-duplicates the shared vertex at each join, so a boundary cannot be recovered after the fact by counting.

Every point is a full stop. Each leg is routed independently, so the engine treats each control point as a destination. A point placed beside the road, or on the far side of a dual carriageway, can produce a route that arrives, turns round and leaves. That makes a short out-and-back spur, which is the most common way a drag-to-shape tool goes wrong. Valhalla's `break_through` location type keeps the leg split but forbids a U-turn at that location.

The snap routing tool's points are not editable. `useSnapRouting` draws placed points with `interactive: false`. The tool's only edit is Backspace, which removes the last point.

The TrailRT JSON format drops control points. `io/json/write.ts` writes a route's shape and nothing from `route_control_points`. A snapped route that is exported and re-imported comes back as a hand-drawn route with thousands of vertex handles. Backup is not affected, because it copies the whole database through SQLite's backup API. Once routes carry shaping, losing them on the one path meant for moving between installs is a larger loss, so this phase closes the gap.

User stories

- As a rider planning a day out, I want to see the routes available between two towns and choose one, so I am not limited to the engine's first answer.
- As a user, I want to drag the line onto the road I actually want, so I don't have to work out where to click to force that road.
- As a user, I want to move, add and remove the points of a route I have already saved, without the rest of the route changing under me.
- As a user, I want to undo any of those edits with ⌧Z, exactly as for any other edit.

Requirements

Requirement 1: Candidates while drawing Acceptance criteria

1. WHEN the snap routing tool routes the leg to the most recently placed point THE system SHALL request up to three candidates for that leg and SHALL draw every candidate returned at the same time.
2. THE first candidate SHALL be the engine's preferred route and SHALL be selected by default.
3. THE system SHALL show for each candidate:
   - its distance
   - its duration, labelled as an estimate per ADR 0011 §8
   - its difference in distance and estimated duration from candidate 1
4. THE user SHALL be able to select a candidate in three ways: by clicking its line, by choosing it in a candidate list, or by pressing `1`, `2` or `3`. Hovering over a candidate in either place SHALL highlight it in the other.

5. THE selected candidate SHALL be drawn as the route, and the others SHALL be drawn in a style that cannot be mistaken for it.
6. Placing the next point, or finishing the route, SHALL accept the selected candidate for that leg. The candidates for a leg SHALL then be dismissed.
7. WHEN the engine returns fewer than three candidates THE system SHALL show those it returned. WHEN it returns only one, THE system SHALL say that no alternative was found. THE system SHALL NOT fabricate an alternative.
8. Candidates SHALL be requested only for a leg that matched a road. An unmatched leg SHALL stay the straight line Phase 4 Requirement 3.6 specifies, with no candidates.
9. THE displacement check of ADR 0016 SHALL apply to every candidate, not only the first.
10. THE system SHALL provide a setting that turns automatic candidates off. It SHALL be on by default. With it off, the tool behaves exactly as in Phase 4.

Requirement 2: Candidates for a leg of a saved route Acceptance criteria

1. WHEN a snapped route is selected THE system SHALL offer, from the context menu of any matched leg, a command to show alternatives for that section of the route.
2. THE candidates SHALL be presented as in Requirement 1.3–1.5. The leg's current geometry SHALL be shown alongside them so the user can see what would change.
3. Choosing a candidate SHALL replace that leg's geometry only. The geometry of every other leg SHALL remain identical, point for point.
4. Choosing a candidate SHALL be one undo entry. Dismissing the candidates SHALL change nothing.
5. THE command SHALL NOT be offered for a hand-drawn route or for an unmatched leg.

Requirement 3: A chosen alternative survives re-routing The engine does not remember which alternative was chosen. If a leg is re-routed later, for example because the user dragged one of its ends, the engine would return its preferred route again and silently undo the choice.

Acceptance criteria

1. WHEN the user chooses a candidate other than candidate 1 THE system SHALL insert shaping points on that candidate so that re-routing the leg reproduces the chosen roads.
2. THE system SHALL verify this by re-routing through the inserted shaping points. WHEN the result diverges from the chosen candidate by more than a stated tolerance THE system SHALL insert further shaping points, up to a stated bound.
3. WHEN the bound is reached without a match THE system SHALL still keep the chosen geometry, and SHALL tell the user that a later re-route of that section may not reproduce it.
4. THE saved geometry SHALL be the candidate exactly as the user saw it, not the result of the verification re-route. This follows Phase 4's rule that what the user accepted is what is stored (see `PreviewedLeg`).

Requirement 4: Drag the line to shape it Acceptance criteria

1. WHEN the user presses on a leg of the selected snapped route, or of the snap routing tool's preview, and drags, THE system SHALL create a shaping point at the pointer and SHALL route that leg as two legs through it.
2. WHILE dragging, THE system SHALL show a preview that follows the pointer:
   - With the embedded engine, the preview SHALL be road-following. At most one request SHALL be in flight at a time, and only the reply for the latest pointer position SHALL be drawn.
   - With a remote provider, the preview MAY be straight lines from the neighbouring control points, with road routing on release.

3. Releasing SHALL commit the shaping point and the re-routed legs as one edit, which is one undo entry on a saved route.
4. Pressing Escape during a drag SHALL cancel it and leave the route exactly as it was.
5. A click without a drag on the selected snapped route's line SHALL insert a shaping point at the nearest position on the line. It SHALL NOT change the route's geometry. This replaces the current behaviour described in the investigation, which inserts a raw vertex.
6. A click and a drag SHALL be told apart by pointer movement, so a click is never mistaken for a tiny drag and a drag never starts by accident.
7. THE route SHALL NOT turn round at a shaping point wherever a through path exists. A shaping point SHALL bend the route, not create an out-and-back spur.

Requirement 5: Drag control points Acceptance criteria

1. Dragging a via or shaping point SHALL re-route only the legs that touch it: one leg for the start or end, two otherwise. Every other leg SHALL remain identical, point for point.
2. WHILE dragging, THE preview SHALL behave as in Requirement 4.2.
3. Releasing SHALL be one undo entry on a saved route.
4. Points placed in the snap routing tool SHALL be draggable before the route is saved, with the same behaviour.

Requirement 6: Add, remove and convert points Acceptance criteria

1. THE user SHALL be able to remove a control point by Alt-clicking it or from its context menu. The two legs it joined SHALL be merged and re-routed as one leg.
2. Removing the start or end point SHALL be allowed while at least three control points remain. The route SHALL then start or end at the neighbouring point, and the leg that led to the removed point SHALL be dropped without any re-routing.
3. THE system SHALL refuse to remove a point that would leave fewer than two, and SHALL say why.
4. THE user SHALL be able to convert a shaping point to a via point, and back, from its context menu. Conversion SHALL NOT change the geometry. The start and end SHALL always be via points.
5. THE context menu of a saved snapped route's end point SHALL offer to extend the route. While extending, each map click SHALL append a via point and route a new leg, with candidates as in Requirement 1. Each click SHALL be one undo entry. Enter or Escape SHALL stop extending.

Requirement 7: Via and shaping points are distinct Acceptance criteria

1. Via points SHALL be drawn as numbered handles. Start and end SHALL keep the endpoint badges they have today.
2. Shaping points SHALL be drawn as smaller, unnumbered handles that cannot be mistaken for via points.
3. A control point's role SHALL be stored with the route. It SHALL survive restart, undo and backup.
4. THE TrailRT JSON collection format SHALL carry a snapped route's control points, roles and leg boundaries, so a route moved between installs stays editable. Files written before this phase, which have none, SHALL still import, as hand-drawn routes as today. A file whose control points do not fit its shape SHALL import the route as hand-drawn and say so. It SHALL NOT be rejected.
5. GPX, KML and other exports SHALL be unchanged in this phase: a route exports its full shape as today. Storing the role now lets the Garmin phase export shaping points without a migration.

Requirement 8: Undo, integrity and limits Acceptance criteria

1. Every committed edit to a saved snapped route SHALL be exactly one undo entry. Undo SHALL restore the control points, their roles and the geometry exactly.
2. THE undo inverse SHALL carry only the legs the edit replaced, not the whole route. An edit to a route above `MAX_UNDOABLE_POINTS` SHALL therefore still be undoable when the replaced section is within that bound. WHEN it is not, the edit SHALL declare itself non-undoable and say so, following `trackService.restoreOriginal`.
3. An edit that would take a route past `MAX_SNAPPED_POINTS` SHALL be refused with the count, and nothing SHALL be written.
4. Route statistics SHALL be recomputed from the stored geometry after every edit and SHALL NOT be taken from a provider response.
5. Cached road attributes SHALL be invalidated by any geometry change, through the existing geometry-hash mechanism.
6. The unmatched-leg and displaced-point records (`unmatchedLegs`, `displacedControlPoints`) SHALL stay correct after points are inserted or removed. Their indices refer to control points and have to move with them.
7. Within the snap routing tool, before the route is saved, ⌧Z SHALL undo the last edit of any kind: placing, dragging, inserting or removing a point, or choosing a candidate. Backspace SHALL keep its Phase 4 meaning of removing the last point.
8. Routes snapped before this phase have no stored leg boundaries and SHALL still be editable. The first edit MAY re-route the whole route once to establish the boundaries, and SHALL say that it is doing so.

Requirement 9: Responsiveness and failure Acceptance criteria

1. A reply that belongs to a superseded gesture or preview SHALL never be applied. This extends the `generation` rule in `snapRoutingStore`.
2. While an edit to a saved route is being routed, THE system SHALL show that it is routing and SHALL NOT accept a second edit to the same route until the first is committed or has failed.
3. A provider failure during an edit SHALL leave the route exactly as it was and SHALL be reported. A transport failure (unreachable, timed out, cancelled) SHALL be reported as a failure of the service, never as an unmatched leg. This is the same distinction `valhalla.ts` already draws.
4. With the embedded engine, a drag preview on a leg of up to 50 km SHALL update with a median latency of 300 ms or less on the Buxton-square fixture geography (ADR 0015). This target SHALL be measured and the result recorded.

Requirement 10: Provider contract Acceptance criteria

1. Alternatives SHALL be an optional capability declared by the provider. A provider that does not offer them SHALL still satisfy the interface. THE UI SHALL then hide the candidate features and say why.
2. THE Valhalla adapter SHALL implement the capability with the `alternates` request parameter, SHALL request no more than the engine's configured limit, and SHALL keep the wire format inside `main/routing/valhalla.ts`.
3. THE adapter SHALL support a per-location "pass through, do not turn" hint, which serves Requirement 4.7. A provider that cannot honour it SHALL still produce a route, and that route SHALL be checked for a reversal as in the design.
4. No new network origin SHALL be introduced, and the CSP SHALL be unchanged. Alternatives and shaping send the same data as routing already does, under the existing consent.

Requirement 11: Testing Acceptance criteria

1. Unit tests SHALL cover:
   - parsing a Valhalla response that contains alternates, from a fixture
   - zero, one and two alternates
   - displacement on an alternate
   - splicing legs leaves untouched legs identical
   - unmatched and displaced indices move correctly with insertion and removal
   - the shaping-point verifier against a fake provider
   - every new undo inverse round-trips exactly
   - editing a route that has no stored leg boundaries
2. An opt-in live test against the embedded engine SHALL show that **Buxton → Macclesfield** returns at least two candidates. Two real routes exist there: the A537 over the Cat and Fiddle, and the A5004 Long Hill. The test SHALL also show that choosing the non-preferred one and then re-routing reproduces it.
3. An end-to-end test SHALL cover the whole flow: place two points, see more than one candidate, choose the second, drag the line, save, undo.

Non-functional requirements

- Module boundaries per `.kiro/steering/architecture.md`. Candidate parsing lives in the adapter, leg splicing in `main/services/`, and the pure geometry (divergence, nearest position on the line, reversal detection) in `shared/geo/`.
- No new dependencies are expected. Any that turn out to be needed are subject to the licence check in `.kiro/steering/licensing.md`.
- The map SHALL stay at interactive frame rates while candidates are drawn. That is at most three polylines for one leg at a time.

Out of scope

- **Place search and geocoding.** "Two places" means two points placed on the map, as today. A geocoder would be a network dependency for a core feature, which ADR 0007 rules out without an offline source. An offline one is now specified as Phase 10 Requirement 11.
- **Alternatives across more than two points at once.** The engine does not offer them (see the investigation). Alternatives are per leg.
- **Ranking candidates by twistiness or scenery.** That is Phase 7. Its Requirement 5.3 rejects re-ranking alternatives as a way of finding winding roads, so this phase's candidates must not be presented as that feature.
- **Exporting shaping points** as GPX route points or Garmin shaping points. That belongs to the Garmin phase. The role is stored now so that phase needs no migration.
- **Changes to hand-drawn route editing.** A route with no control points keeps its Phase 2 per-vertex handles.
- **Time-dependent routing and live traffic.** Valhalla does not combine them with alternates, and the app has no traffic source.

Open questions for the owner

1. Should candidates be offered automatically for every leg while drawing (as written in Requirement 1), or only on request? Automatic is closer to what was asked for, and the requirement includes a setting to turn it off.
2. In the snap routing tool, should Escape dismiss the candidates before it cancels the tool? As written, Escape still cancels the tool, as in Phase 4.

9.8.2 Design

Two features, one of each kind. Candidates are a read: ask the engine for more than one answer and show them. Shaping is a write: change a route by editing the points it is computed from. Candidates are the easy part. Most of this design is about shaping, because Phase 4 stored a snapped route in a way that only supports replacing all of it.

Decisions that are hard to reverse are recorded in ADR 0017.

1. The provider gains two optional capabilities

```
// shared/routing/RoutingProvider.ts

export type ControlRole = 'via' | 'shaping';

/** One location in a section routed as a single request. */
export interface SectionLocation {
  point: LatLon;
  /**
   * Arrive and leave without turning round. Set for shaping points.
   * Valhalla: `type: 'break_through'`. The leg is still split here.
   */
  passThrough: boolean;
}

export interface RouteCandidate {
  shape: LatLon[];
  distanceM: number | null;
  /** An engine estimate, never a measurement. ADR 0011 §8. */
  estimatedDurationS: number | null;
  /** Measured, as for snap. ADR 0016. */
  startDisplacementM: number;
  endDisplacementM: number;
}

export interface RoutingProvider {
  // ...snap and match unchanged...

  /**
   * Up to `count` candidates between two points. The engine's preferred route
   * comes first. Absent on a provider that cannot offer alternatives.
   */
  alternatives?(
    from: LatLon,
    to: LatLon,
    mode: TransportMode,
    count: number,
  ): Promise<RouteCandidate[]>;

  /**
   * Routes a short section as **one** request, splitting legs at every
   * location and honouring `passThrough`. Absent when unsupported; callers
   * fall back to `snap` and check for reversals (§5).
   */
```

```
  snapSection?(
    locations: readonly SectionLocation[],
    mode: TransportMode,
  ): Promise<SnapResult>;
}
```

Both methods are optional. This keeps Phase 4's promise that a second provider is an adapter, not a refactor: a provider with only `snap` and `match` still works, and the UI loses the features it cannot back (Requirement 10.1). `RoutingStatus` gains `capabilities: { alternatives: boolean; passThrough: boolean }`, so the renderer knows what to offer without probing.

Valhalla adapter

- `alternatives` sends `/route` with two locations and `alternates: min(count - 1, 2)`. The 2 is the bundled `service_limits.max_alternates`. A remote host may configure a lower limit, so an error that mentions alternates is retried once with `alternates: 0`. The primary route is in `trip` and the others are in `alternates[].trip`. `ValhallaRouteResponse` gains the `alternates` field. Nothing outside `valhalla.ts` sees it.
- Every candidate goes through the same displacement measurement as `snap`. A candidate with either end beyond `MAX_SNAP_DISPLACEMENT_M` is dropped from the list rather than shown (Requirement 1.9). If that leaves none, the leg is unmatched, exactly as today.
- `snapSection` sends one `/route` with N locations. Shaping points are typed `break_through` and via points `break`. A `break_through` location keeps the leg split, which §2 depends on, but forbids a U-turn there (Requirement 4.7). A multi-location request fails as a unit. On a `PROVIDER_ERROR`, `snapSection` falls back to per-leg `snap`, so one bad point degrades to Phase 4 behaviour rather than failing. Transport errors propagate, for the reason `snap` already documents.

2. Leg boundaries are stored To replace one leg, the system has to know where that leg is in `route_points`. Migration `0003` adds two columns:

```
ALTER TABLE route_control_points ADD `role` text DEFAULT 'via' NOT NULL;
-- Index into route_points of the first vertex of the leg that starts here.
-- Null for routes snapped before Phase 6: boundaries unknown.
ALTER TABLE route_control_points ADD `shape_index` integer;
```

Invariant. For a route with control points `c[0..n-1]` and shape `p`:

- `b[k] = c[k].shape_index` is non-decreasing, `b[0] = 0` and `b[n-1] = p.length - 1`.
- For `k  1`, `b[k]` is the index of the **last vertex of leg** `k-1`, and leg `k` is the *inclusive* slice `p[b[k] ..= b[k+1]]`. Adjacent legs always share the boundary vertex. Where two engine legs did not meet exactly, both vertices are kept and the short step belongs to the later leg.
- Inclusive rather than half-open, which is a correction made during implementation. With half-open legs, the boundary vertex is either shared (the join was de-duplicated) or the later leg's own first vertex (it was not), and nothing records which. A splice would then drop or duplicate a vertex depending on a fact it cannot see.
- Every write that touches a snapped route's points also rewrites `shape_index`. That covers `snapRoute`, `createSnappedRoute`, `restoreSnap` and the new splice. A write that cannot maintain the invariant clears the control points instead. `route:setGeometry` on a snapped route is that case, and it is now unreachable from the map (§4).

`flattenLegs` becomes `flattenLegsWithBoundaries`. It returns the points and `b[]` together, so the boundaries come from the same pass that removes duplicate join vertices and cannot drift from it. Splicing is a pure function in `shared/routing/splice.ts`:

```
spliceLegs(
  points: readonly LatLon[],
  boundaries: readonly number[],
  fromLeg: number,           // first leg replaced
  toLeg: number,             // one past the last leg replaced
  replacement: readonly { shape: readonly LatLon[] }[],
): { points: LatLon[]; boundaries: number[] }
```

It keeps `p[0 ..= b[fromLeg]]` and `p[b[toLeg] ..]` unchanged as values. The middle is re-flattened, and the two seams are de-duplicated with the same 1e-6 rule `flattenLegs` uses. This is the function behind Requirements 2.3 and 5.1 ("every other leg identical, point for point"). Its tests include a property test showing that a splice is *exactly* equal, in both points and boundaries, to flattening the kept legs around the replacement.

Legacy routes (`shape_index` null). Their first edit runs the existing whole-route `snapRoute` on the current control points to establish the boundaries, then applies the edit. The toast says "Re-routing the whole route once to enable section editing" (Requirement 8.8). No migration back-fill is attempted. Recovering boundaries by nearest-point search is unreliable on loops and out-and-back routes, which are exactly where a wrong boundary would do damage.

3. One edit model for the tool and for saved routes The snap routing tool (a draft in the renderer) and a saved snapped route (rows in SQLite) need the same edits. They share one pure planner, so the two cannot behave differently:

```
// shared/routing/edit.ts - no I/O
type RouteEdit =
  | { kind: 'append'; at: LatLon }
  | { kind: 'move'; index: number; to: LatLon }
  | { kind: 'insert'; leg: number; at: LatLon }                  // drag the line
  | { kind: 'insertOnLine'; leg: number; shapeIndex: number }    // click the line
  | { kind: 'remove'; index: number }
  | { kind: 'setRole'; index: number; role: ControlRole }
  | { kind: 'choose'; leg: number; candidate: RouteCandidate; shaping: LatLon[] };

interface EditPlan {
  control: { point: LatLon; role: ControlRole }[];   // after the edit
  replace: [fromLeg: number, toLeg: number];          // in the old leg numbering
  /** What to route for the replaced span. Empty when no routing is needed. */
  section: SectionLocation[];
  /** Geometry known without routing (insertOnLine, choose, setRole). */
  knownLegs: { shape: LatLon[] }[] | null;
  /** Remaps unmatchedLegs / displacedControlPoints indices. */
  remap: (controlIndex: number) => number | null;
}

planEdit(control, boundaries, edit): EditPlan
```

Edit	Legs replaced	Routed	Notes
`append`	none (one new leg)	`[last, at]`	Candidates per §6
`move` start / end	1	2 locations	
`move` middle	2	3 locations, middle `passThrough` if shaping	

Edit	Legs replaced	Routed	Notes
`insert` (drag line)	1 → 2	`[c[k], at(pass), c[k+1]]`	
`insertOnLine`	1 → 2	nothing	Splits the shape at `shapeIndex`; geometry unchanged (4.5)
`remove` middle	2 → 1	`[c[k-1], c[k+1]]`	
`remove` start / end	1 → 0	nothing	Drops the end leg (6.2)
`setRole`	0	nothing	Geometry unchanged (6.4)
`choose`	1 → 1 + shaping	nothing	Splits the candidate at each shaping point (§7)

Section ends are never pass-through. This is a correction made during implementation. A section starts and ends at control points whose *other* leg is not being re-routed. Making those ends pass-through would only mean something if the neighbouring legs were routed in the same request, and that would change legs Requirement 5.1 says stay identical. So only a section's interior locations carry `passThrough`, and a reversal at a section end is caught afterwards by the check in §5. The planner also enforces the refusals: fewer than two points (6.3), and start or end never being shaping (6.4). Removing the start or end point turns its neighbour into a via point.

Saved routes: preview, then apply Phase 4 established that what the user watched appear is what gets stored (`PreviewedLeg`). The same holds here, so routing and writing are separate calls:

```
renderer                                  main
planEdit(...)            routing:routeSection    provider.snapSection | snap
                            PreviewedLeg[]     (no database access)
(user sees the preview; drag may repeat this many times)
on release  routing:applyEdit { routeId, expected, edit, legs }
                                          re-plan from the DB, check `expected`,
                                          splice, write, record undo
```

`expected` is the control points and boundaries the renderer planned against. Main re-plans the edit from the database. If the result differs from `expected`, nothing is written and `INVALID_REQUEST` is returned, for the same reason `createSnappedRoute` refuses mismatched legs: a preview computed against a route that has since changed describes neither version.

New IPC, all behind the existing consent gate and validated by zod in `shared/ipc/schemas.ts`:

Channel	Writes	Purpose
`routing:routeSection`	no	Route 2–N locations; returns `PreviewedLeg[]`
`routing:alternatives`	no	Candidates between two points
`routing:applyEdit`	yes	Commit a planned edit with its previewed legs

`routing:routeLeg` stays. It is `routeSection` with two locations, and the drain loop keeps using it.

4. Map interaction `useRouteLayers` keeps everything for hand-drawn routes. For a selected **snapped** route, the line's click handler no longer calls `insertRouteVertex` (the defect in the investigation). Instead, a new hook, `useRouteShaping`, owns the interaction:

- **Hit area.** A transparent 16 px polyline under the visible line, for the selected route only. A 4 px line is too thin to grab.
- **Press on the line.** The hook records the press, disables `map.dragging` and starts watching movement. Movement under 4 px followed by release counts as a click. The hook then finds the nearest position on the line (`closestSegment`), takes the leg from the boundaries, and applies `insertOnLine`. Movement beyond 4 px starts an `insert` drag with a temporary shaping handle. Leaflet's own `click` after `mouseup` is suppressed once a drag has started (Requirement 4.6).
- **Press on a handle.** This uses Leaflet marker dragging, as today, but runs through the same controller, so a handle drag is a `move`.
- **During a drag.** A `DragPreview` controller keeps the latest pointer position in a ref. When no request is in flight, it sends `routeSection` for that position. When the reply lands, it draws the reply only if the position has not moved on. Otherwise it sends the new position and discards the reply. That gives at most one request in flight, with the latest position winning (Requirement 4.2). On a remote provider, it draws straight lines and routes once on release.
- **Release** calls `applyEdit` with the preview legs if they match the final position. Otherwise it routes the final position first.
- **Escape** during a drag restores the handles and clears the preview, and nothing is sent (4.4).
- **Alt-click** on a handle is `remove`. The **context menu** on a handle offers *Remove point*, *Make shaping point* or *Make via point*, and on the end handle *Extend route*. The context menu on the line gains *Show alternatives for this section*.
- **While an edit is in flight** (9.2), the route's handles are non-interactive, the cursor shows progress, and a second press is ignored.

Panes are SVG and click-through. This is a correction made during implementation, found by the end-to-end test. The map renders paths on canvas, and a canvas renderer puts a full-map canvas in each pane it serves. Once it has held a path, that canvas takes every pointer event over the whole map, so a custom pane above the others blocks clicks on every route and marker beneath it. The shaping and candidate panes therefore use an SVG renderer with `pointer-events: none` on the pane (`interactivePane.ts`). Leaflet's interactive paths and markers carry an explicit `pointer-events: auto`, so they still receive events, and empty areas fall through.

Handles (Requirement 7): via points use the new numbered `viaIcon`, and the start and end keep `endpointIcon`. Shaping points get a new `shapingIcon()` in `markers.ts`: a 9 px hollow circle in the route colour.

In the snap routing tool, `useSnapRouting` makes its placed points draggable and uses the same `useRouteShaping` controller against the draft. The edit is applied to `snapRoutingStore` instead of through IPC (Requirement 5.4).

5. Reversal check, for providers without pass-through `shared/geo/reversal.ts`: `reversalAt(shape, index, windowM = 150): boolean`. It compares the bearing into and out of `shape[index]` over `windowM` metres each side. A difference above 150° is a reversal. It runs on every shaping point after `snap`-based routing. That covers a provider without `snapSection` and the per-leg fallback in §1. A reversal does not block the edit. The shaping handle gets a warning ring, and the toast says the route turns back at that point and suggests moving it. A spur the user can see and fix is better than refusing an edit the engine was happy with.

6. Candidates

In the tool `snapRoutingStore` gains:

```
roles: ControlRole[];                          // parallel to controlPoints
legs: (PreviewedLeg | null)[];                 // was a prefix array; null = needs routing
candidates: { leg: number; items: Candidate[]; selected: number } | null;
history: Snapshot[];                           // for Z, bounded at 50
```

`legs` changes from a prefix to a sparse array, because a `move` in the middle of the draft invalidates two legs that are not at the end. `nextGap()` returns the first `null`. `drainSnapLegs` is unchanged apart from that, and so is its one-at-a-time ordering guarantee. When the gap is the last leg, the setting is on and the provider has `alternatives`, the drain calls `routing:alternatives` instead of `routeLeg`. It stores candidate 1 as the leg and the full list in `candidates`.

Selecting a candidate swaps the leg and does not route. Placing the next point or finishing applies `choose` for the selected candidate (§7) and clears `candidates` (Requirement 1.6).

`Candidate` is a `PreviewedLeg` plus `estimatedDurationS`. The field is named for what it is: `PreviewedLeg` dropped duration on the grounds that nothing displayed it, and something now does, labelled as an estimate.

Rendering `useCandidateLayers(map, candidates, selected, onSelect, onHover)`:

- **Selected:** route accent, weight 5.
- **Others:** slate `#64748b`, weight 4, opacity 0.7, drawn underneath, and clickable with a 16 px hit area.
- **Labels:** each candidate gets a small label at its point of greatest divergence from candidate 1: `B · +3.2 km · +6 min (est.)`.

Next to the existing snap-route readout in `MapView`, a candidate list shows the same figures with a radio per row. Pressing `1`, `2` or `3` selects. Hovering over a row or a line highlights the other.

For a saved route's leg *Show alternatives for this section* calls `routing:alternatives` with the leg's two control points and shows the same layers. The current leg is drawn dashed in the route colour so it can be compared. Choosing a candidate is `applyEdit` with `choose`, which is one undo entry. Escape or clicking away dismisses the candidates and writes nothing (Requirement 2.4).

7. Making a chosen alternative stick `shared/geo/divergence.ts`, all pure:

- `farthestFrom(candidate, reference)` returns the segment of `candidate` whose midpoint is farthest from the `reference` polyline. It uses the midpoint and not a vertex, because a vertex is often a junction, and a shaping point on a junction can snap onto the crossing road.
- `overlap(a, b, toleranceM = 25)` returns the fraction of `a`'s length that lies within `toleranceM` of `b`, taking the lower of `a→b` and `b→a`.

Algorithm for `choose` with candidate `C` over leg `A→B`, where `P` is candidate 1:

```
shaping = [ midpoint of farthestFrom(C, P) ]
repeat up to 3 times:
    R = snapSection([A, ...shaping(pass), B])
    if overlap(C, R)  0.98: stop - verified
    shaping += midpoint of farthestFrom(C, R), ordered along C
unverified → keep, warn (Requirement 3.3)
```

The stored geometry is `C` split at each shaping point. Each midpoint lies on `C` by construction, so the split inserts one vertex into a straight segment and moves nothing (Requirement 3.4). The verification routes are only used for the comparison and are then discarded.

Verification is asynchronous. In the tool, it runs after the user moves on. On a saved route, it runs before `applyEdit` while the candidates are still shown, with a short "Checking this route can be reproduced" state. The 0.98 threshold, 25 m tolerance and bound of 3 are constants in `divergence.ts`. Their values are confirmed by the live test in task 11, not assumed.

8. Undo A new command kind, `route:spliceLegs`:

```
payload: {
  id: string;
  fromShape: number;            // first index replaced in the current points
  toShape: number;              // one past the last index replaced
  points: RoutePoint[];         // what to put back
  control: { lat; lon; role; shapeIndex }[];   // whole list: small
  metadata: Record<string, unknown>;           // unmatchedLegs, displaced…
}
```

The handler splices `points` into `[fromShape, toShape)`, replaces the control points and metadata, and recomputes stats. It returns its own `route:spliceLegs` inverse, so redo comes for free and is exactly symmetric. The payload carries the **replaced slice**, not the route. If that slice is over `MAX_UNDOABLE_POINTS`, the edit's inverse is `{ kind: 'none' }` and the toast says so (Requirement 8.2). Control lists are carried whole: they are tens of points, and diffing them would be complexity for nothing.

Undo-stack memory is the reason for the small inverse. The database write is a different matter: the write itself uses `replacePoints` with the whole new array, inside one transaction. Rewriting 25,000 rows in SQLite takes milliseconds. Shifting `seq` in place under the `(route_id, seq)` unique index would need a two-pass renumber, and nothing gained would justify the risk.

`undoService.labelFor` gains `'route:spliceLegs': 'route shaping'`. The draft in the tool keeps its own `history` of snapshots, because nothing is in the database yet. ⌘Z pops a snapshot and Backspace keeps its Phase 4 behaviour (Requirement 8.7).

9. Extend from the end *Extend route* puts `editorStore` into an `extendRoute` mode for that route. Each map click is an `append` edit applied through `applyEdit` straight away with candidate 1, so each click is one undo entry (Requirement 6.5). The new leg's candidates are then shown. Choosing another is a second `choose` edit and a second undo entry. Enter or Escape leaves the mode. It reuses the tool's keyboard handling, so the keys mean the same thing in both places.

10. Whole-route re-snap with shaping points `snapRoute` (the Phase 4 *Snap to roads* button and a transport-mode change) must not undo the shaping a user has done. It groups the control points into runs from one via point to the next. A run with shaping points is routed with one `snapSection` call, and a run without them uses the per-leg `snap` as today. The boundaries come out of the same `flattenLegsWithBoundaries` pass.

11. Settings and status

- `settings.routeAlternatives: boolean`, default `true`, in `shared/domain/defaults.ts`. It appears in the Routing section of `SettingsDialog` as "Show alternative routes while drawing".
- `RoutingStatus.capabilities` comes from the presence of `provider.alternatives` and `provider.snapSection`. When `alternatives` is false, the candidate UI is absent, and the Settings toggle is disabled with the reason.

12. The TrailRT JSON format carries control points Each route in the collection gains an optional field:

```
control?: { lat: number; lon: number; role: ControlRole; shapeIndex: number | null }[];
```

It is optional, so `COLLECTION_VERSION` stays at 1: the format's own rule is that additive fields need no bump, and older readers ignore the field. Older files are read as they are today (Requirement 7.4). A sibling `routing` field carries the provider and transport mode, so the imported route says where its road geometry came from. On import, the invariant in §2 is checked against the imported shape. A file that fails the check keeps its geometry and drops the control points, so the route becomes hand-drawn, and that fact goes into the existing import report. Rejecting the whole file would lose a route over a fact about how it was edited.

Requirement traceability

Requirement	Section
1 Candidates while drawing	§6
2 Candidates for a saved leg	§6, §8
3 Chosen alternative survives	§7
4 Drag the line	§3, §4
5 Drag control points	§3, §4
6 Add, remove, convert	§3, §4, §9
7 Via vs shaping	§2, §4, §12
8 Undo and integrity	§2, §8
9 Responsiveness and failure	§1, §4
10 Provider contract	§1, §11
11 Testing	tasks.md

9.8.3 Tasks

Execute in order. The pure, offline-testable work comes first: contracts, geometry, splicing and the edit planner are all proven before anything talks to an engine or the map.

Update `docs/progress-mindmap.svg` as each task changes status.

- ☒ 1. Shared contracts
  - `shared/routing/RoutingProvider.ts`: `ControlRole`, `SectionLocation`, `RouteCandidate`, optional `alternatives` and `snapSection`
  - `shared/routing/results.ts`: `Candidate` (`PreviewedLeg + estimatedDurationS`), `RoutingStatus.capabilities`
  - `providerCapabilities(provider)` derives capabilities from method presence
  - Unit tests: a provider with only `snap`/`match` reports no capabilities and still type-checks as a `RoutingProvider`
  - *Requirements: 10.1*
- ☒ 2. Pure geometry: divergence and reversal
  - `shared/geo/divergence.ts`: `farthestFrom` (segment midpoint, not vertex), `overlap` with tolerance, named constants for 0.98 / 25 m / 3
  - `shared/geo/reversal.ts`: `reversalAt(shape, index, windowM)`
  - Unit tests: identical lines overlap 1.0; disjoint 0; a parallel line at 20 m is inside a 25 m tolerance and at 30 m outside; a midpoint is returned, not a junction vertex; an out-and-back spur is a reversal and a 90° turn is not
  - *Requirements: 3.2, 4.7, 10.3*
- ☒ 3. Leg boundaries and splicing
  - `flattenLegsWithBoundaries` replaces `flattenLegs`; existing callers migrated
  - `shared/routing/splice.ts`: `spliceLegs`, plus `checkBoundaries` for the §2 invariant
  - Unit tests: untouched slices are exactly equal (not approximately) after a splice at the start, middle and end; seams de-duplicate at 1e-6 and keep a genuine step; a splice to

zero legs at either end; property test over random leg sets that flatten▯splice▯flatten agrees with flattening the replaced set directly
  - *Requirements: 2.3, 5.1, 8.6*

☒ 4. Edit planner
  - `shared/routing/edit.ts`: `RouteEdit`, `EditPlan`, `planEdit`, index `remap` for `unmatchedLegs` and `displacedControlPoints`
  - Refusals: fewer than two points; start/end as shaping
  - Unit tests: one case per row of the design §3 table; `insertOnLine` and `setRole` route nothing and change no geometry; remove next to shaping points keeps their `passThrough`; remap after insert and after remove
  - *Requirements: 4.1, 4.5, 5.1, 6.1, 6.2, 6.3, 6.4, 8.6*

☒ 5. Migration 0003 and repositories
  - `route_control_points` + `role` (default `via`) + nullable `shape_index`, via `npm run db:generate`
  - Repository read/write of role and boundaries; `replaceControlPoints` takes them
  - `snapRoute`, `createSnappedRoute`, `restoreSnap` write boundaries
  - Unit tests: migration applies to a Phase 5 database and existing rows read as `via` / null; boundaries written by every path satisfy `checkBoundaries`; undoing a snap restores the boundaries too
  - *Requirements: 7.3, 8.8*

☒ 6. Valhalla adapter: alternatives and sections
  - `alternatives`: `alternates: min(count-1, 2)`, parse `alternates[].trip`, displacement per candidate, retry once with `alternates: 0` on a host-limit error
  - `snapSection`: one N-location `/route`, `break_through` for pass-through, per-leg `snap` fallback on `PROVIDER_ERROR`, transport errors propagate
  - Synthetic responses hand-authored from the API reference, served by the adapter tests' existing loopback server: 0, 1 and 2 alternates; a 3-location `break_through` response
  - Unit tests: candidate order and units; a displaced alternate is dropped; no alternates means one candidate, never a fabricated one; the `break_through` type is sent for pass-through only; fallback runs on `PROVIDER_ERROR` and not on timeout
  - *Requirements: 1.7, 1.9, 4.7, 10.2, 10.3, 11.1*

☒ 7. Routing service, IPC and undo
  - `routingService`: `routeSection`, `alternatives`, `applyEdit` (re-plan from the database, refuse on `expected` mismatch, splice, `replacePoints`, stats, metadata remap, one transaction)
  - Legacy route (null boundaries): whole-route snap first, reported
  - `route:spliceLegs` command, its symmetric inverse, `'route shaping'` label; `none` inverse when the replaced slice exceeds `MAX_UNDOABLE_POINTS`
  - `snapRoute` groups via-to-via runs and uses `snapSection` for runs with shaping points (design §10)
  - IPC channels `routing:routeSection`, `routing:alternatives`, `routing:applyEdit` behind the consent gate; zod schemas; preload
  - Unit tests: undo then redo of every edit kind round-trips exactly; an edit to a 20,000-point route whose replaced slice is 300 points is undoable; a result over `MAX_SNAPPED_POINTS` writes nothing; stale `expected` writes nothing; a provider failure leaves the route byte-identical; the road-attribute cache reads as stale after an edit; whole-route re-snap keeps shaping points
  - *Requirements: 2.3, 2.4, 8.1, 8.2, 8.3, 8.4, 8.5, 8.8, 9.3, 10.4, 11.1*

☒ 8. Making a chosen alternative stick
  - The design §7 verifier: initial shaping point, up to three refinements, unverified outcome reported, stored geometry = candidate split at the shaping points
  - Unit tests against a fake provider: a provider that honours shaping verifies in one pass; one that needs two points gets two; one that never converges stops at the bound and

reports; the stored geometry equals the candidate with only on-line vertices added
  - *Requirements:* 3.1, 3.2, 3.3, 3.4, 11.1

☒ 9. Shaping a saved route on the map
  - `useRouteShaping`: 16 px hit area, click vs drag at 4 px, Leaflet click suppressed after a drag, Escape cancels, in-flight lock with a progress cursor
  - `DragPreview`: one in flight, latest position wins; straight-line preview on a remote provider
  - Snapped-route line click no longer reaches `insertRouteVertex` (fixes the investigation defect); hand-drawn routes unchanged
  - `shapingIcon()`, numbered via handles, reversal warning ring; Alt-click remove; handle and line context menus
  - Unit tests: the click/drag state machine and the `DragPreview` latest-wins logic, both extracted as pure functions
  - *Requirements:* 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 5.1, 5.2, 5.3, 6.1, 6.2, 6.3, 6.4, 7.1, 7.2, 9.1, 9.2

☒ 10. Candidates on the map
  - `useCandidateLayers`: selected/other styles, 16 px hit areas, divergence labels with "(est.)"
  - Candidate list beside the snap-route readout; `1/2/3`; hover linking
  - *Show alternatives for this section* on a saved route's leg, current leg drawn dashed, choose applies `choose`, Escape dismisses without writing
  - Unit tests: label figures and deltas from fixture candidates; the command is absent for hand-drawn routes and unmatched legs
  - *Requirements:* 1.3, 1.4, 1.5, 2.1, 2.2, 2.4, 2.5

☒ 11. The snap routing tool
  - `snapRoutingStore`: `roles`, sparse `legs`, `candidates`, bounded `history`; `nextGap` returns the first null
  - `drainSnapLegs` requests candidates for the last leg when the setting and capability allow
  - Placed points draggable through `useRouteShaping`; ⌘Z pops history; Backspace unchanged
  - `settings.routeAlternatives` (default on) and its Settings toggle, disabled with the reason when unsupported
  - Unit tests: a middle move nulls exactly two legs and the drain re-routes them in order; a stale generation is still dropped; ⌘Z restores each edit kind; with the setting off the store behaves as Phase 4
  - *Requirements:* 1.1, 1.2, 1.6, 1.7, 1.8, 1.10, 5.4, 8.7, 9.1

☒ 12. Extend a saved route
  - `extendRoute` editor mode from the end handle's context menu; each click is an `append` applied at once; candidates shown for the new leg; Enter/Escape leave
  - Unit tests: three clicks are three undo entries; choosing a candidate after a click is a fourth
  - *Requirements:* 6.5

☒ 13. TrailRT JSON carries control points
  - Optional `control` and `routing` per route; no `COLLECTION_VERSION` bump, per the format's rule that additive fields do not need one; import checks the boundary invariant and falls back to hand-drawn with a report entry
  - Unit tests: a shaped route round-trips with roles and boundaries; a pre-Phase 6 file imports unchanged; a file with mismatched boundaries imports as hand-drawn and is reported, not rejected; GPX export of a shaped route is byte-identical to the same shape exported in Phase 5
  - *Requirements:* 7.4, 7.5

☒ 14. Live tests and the latency measurement
  - Opt-in live test on the embedded engine, `npm run verify:shaping` (`src/main/routing/shaping.liv`
  - Measure median drag-preview latency for legs up to 50 km; record the result and the

machine in this file; revisit the §7 constants if the live data disagrees with them
- **Geography changed from the plan.** The spec named Buxton → Macclesfield, but the graph loaded here is the Geofabrik `derbyshire` extract and Macclesfield is in Cheshire, where the engine drags the end 3 km to the county boundary (ADR 0016). The test uses Buxton → Ashbourne, the square's eastern leg, and takes `TRAILRT_LIVE_PAIR=macclesfield` for a graph that covers Cheshire.
- **Recorded 2026-11-08, Apple M4, Valhalla 3.9.1 in-process, Derbyshire graph:**
  * Buxton → Ashbourne offered 3 candidates: 34.6 km / 30 min, 49.0 km / 41 min, 46.4 km / 40 min (estimates).
  * Choosing candidate B needed **2** shaping points; re-routing through them reproduced it with overlap **1.000**. The §7 constants (0.98, 25 m, at most 3 pins) held without change.
  * A shaping point on the A515 routed with no reversal.
  * Drag preview, 20 pointer positions over a 32 km leg: **median 4 ms**, max 4 ms, against the 300 ms target. Live road-following while dragging is comfortably affordable on the embedded engine.
- *Requirements: 9.4, 11.2*

☒ 15. End-to-end and verification pass
- Playwright (`tests/e2e/shaping.spec.ts`): place two points, see more than one candidate, choose the second, drag the line, save, undo; restart and confirm shaping points persist
- `npm run verify` against the packaged build; update `docs/progress-mindmap.svg`, `README.md` and `.kiro/specs/README.md`
- **Recorded 2026-11-08:** `npm run verify` passed (1,123 unit tests, 119 end-to-end); the full end-to-end suite also passed against the packaged `TrailRT.app` (`TRAILRT_E2E_APP`), 119 of 119.
- The end-to-end test found a defect the unit tests could not: the map renders paths on canvas, and a canvas renderer's full-map canvas in a custom pane swallows every click beneath it once it has held a path. The candidate pane, empty after a choice, blocked the next drag. Fixed by giving the shaping and candidate panes SVG renderers with pointer events off on the pane (`features/map/interactivePane.ts`); see design §4.
- *Requirements: 11.3*

Notes

9.9 Phase 7: UK regions, road-following by default, and a fuller profile — Requirements — SPEC-PHASE-7-REGIONS-PROFILE

Version: — · **Status:** unspecified · **Slug:** `phase-7-regions-profile`

9.9.1 Requirements

Status: **complete.** All 16 tasks done and verified. Tasks 1–13 against the packaged app; tasks 14–16, fixes and owner feedback reported afterwards, against the development build.

Five changes the owner asked for after Phase 6 shipped:

1. Map regions for snap-to-road data are chosen from a **map of the UK with its counties**, and several can be selected at once.
2. Following roads becomes **the default way to make a route**, so the separate "snap to road" controls go.
3. The profile shows **altitude** for routes, which today have none.
4. Every profile gets a **twisty score**.
5. A UK road with **no posted limit** in the map data is counted at the **UK national speed limit** instead of as Unknown.

Inserted ahead of windy-roads routing, which moves to Phase 8, and Garmin compatibility, which moves to Phase 9. See ADR 0018. The order suits the work: the twisty score is the scoring function that windy-roads routing needs (its Requirement 3), so that phase now starts with scoring already built and calibrated.

What the investigation found The investigation was done against the code as it stands after Phase 6, and against the real engine, with live measurements.

The region picker offers one region at a time. `RegionPickerDialog` ranks Geofabrik extracts by the centre of the visible map, and `regionCatalogue.downloadAndBuild` takes a single `regionId`. The graph is replaced each time. The test loop round Buxton crosses three counties (ADR 0015), so routing the whole loop has needed the 1.4 GB England extract.

Valhalla cannot build from overlapping extracts. `valhalla_build_tiles` accepts several `.osm.pbf` files. Given Derbyshire, Cheshire and Staffordshire, it warns ("Tile building using more than one osm.pbf extract is discouraged", issue 3925), logs `Failed tile 2/824388/0: vector`, and aborts with an uncaught exception (exit 134). Geofabrik's county polygons are buffered, so neighbouring counties always overlap. Merged first with `osmium merge`, the same three counties take 1.5 s to merge and 11 s to build: 40 tiles, 141 MB. The whole Buxton square then routes: 98.8 km, four legs.

The UK in Geofabrik's index. `united-kingdom` has England, Scotland and Wales as children. England has 47 county-level children. Northern Ireland is not published on its own. The only extract containing it is `ireland-and-northern-ireland`. The `united-kingdom` boundary polygon, at 150 points, correctly puts Belfast, Enniskillen and Shetland inside, and Dublin, Dundalk, Letterkenny, the Isle of Man and Jersey outside.

There are three snap controls.

- A "Snap routing" button in the map toolbar starts the road-following tool.
- "New Route" starts straight-line drawing, with a "Straight lines — snap to roads" toggle in the hint bar.
- The properties panel has a "Snap to roads" button, which converts an existing route.

Routes have no elevation. Snapped geometry comes from the engine with `ele = null`, so the profile says "No point in this geometry carries an elevation". The engine supports elevation, but

only from DEM tiles that are not installed. The open Terrain Tiles dataset on AWS publishes 1-arc-second (about 30 m) HGT tiles; `N53W002.hgt.gz` is 7.2 MB. Sampled at the test towns, it gives Buxton 297 m against a known 300 m, and Ashbourne 131 m against 130 m.

Points per kilometre does not measure twistiness. Routed on the three-county graph:

Road	Points/km	Heading change of counted bends
A537 Cat and Fiddle	45.7	201°/km
A57 Snake Pass	28.1	79°/km
A523 Macclesfield–Leek	24.9	70°/km
A515 Buxton–Ashbourne	24.2	53°/km
A53 Leek–Buxton	29.4	45°/km
M6 Stoke–Crewe	16.9	31°/km

By point density, Snake Pass and the A53 look about the same, and the A53 looks twistier than the A523. A GPS recording logs at a fixed rate, so its point density measures how slowly it was ridden. Phase 8 Requirement 3.5 already records that point density follows mapping effort, not road shape.

Speed limits. `trace_attributes` returns `speed_limit` only where OSM has a `maxspeed` tag. Along the Buxton loop, every unclassified, residential and service road came back with none. The same call can also return `edge.traversability` (one-way or both ways) and `edge.use`. With those and the road class, the UK default limit can be derived.

ADR 0011 §2 currently forbids that ("never … filled from a country default"). The owner has now asked for it. ADR 0018 amends §2 so that a default may be applied only when it is labelled as assumed.

Terms

Term	Meaning
Region	A downloadable Geofabrik extract. For the UK: an English county, Scotland, Wales, or Ireland and Northern Ireland.
Loaded regions	The regions the current routing graph was built from.
Terrain elevation	Height sampled from a digital elevation model at a point. A model value, not a measurement.
Bend	A run of same-direction turning that turns far enough, and tightly enough, to count. Defined exactly in the design.
Twisty score	0–10, from degrees of bend per kilometre.
Assumed limit	A speed limit the system inferred from UK law because the map data has no posted one. Never presented as posted.

Requirements

Requirement 1: Choosing regions on a UK county map

1. THE region picker SHALL open on a map of the United Kingdom showing every downloadable region for it: England's counties, Scotland, Wales, and Ireland and Northern Ireland.

2. Clicking a region SHALL toggle its selection, and any number SHALL be selectable. Hovering SHALL name the region. A list beside the map SHALL name the selected regions, and each SHALL be removable from there.
3. THE picker SHALL state that Northern Ireland is published only together with the Republic of Ireland, rather than leave the user to discover it.
4. Regions in the loaded graph SHALL be marked as loaded, and SHALL be selected when the picker opens. Adding a county is then one click and a rebuild.
5. THE total download size of the selection SHALL be shown before anything is downloaded. A selection above the build limit (2 GB) SHALL be refused before the download starts, with the reason.
6. Download and build SHALL produce **one** routing graph covering every selected region. Where more than one region is selected, the extracts SHALL be merged before building. Valhalla aborts on overlapping extracts, and neighbouring regions always overlap.
7. Progress SHALL name the stage: downloading (region *n* of *m*, with bytes), merging, building, and fetching terrain.
8. THE loaded-regions record SHALL list every region. Settings and the success message SHALL name them.
9. WHERE the bundled merge tool is missing THE picker SHALL still build a single region, and SHALL refuse more than one with that reason.
10. THE previous view-based list SHALL remain available as “Elsewhere”, for places outside the UK.
11. THE picker SHALL say that a new selection replaces the loaded graph, and that every selected region is downloaded again, because extracts are not kept after a build.
12. Offline, or without the index, the existing fallback (import a `.osm.pbf`) SHALL remain.

Requirement 2: Following roads is the default

1. “New Route”, wherever it is offered (menu, library context menu, properties panel, keyboard), SHALL start the road-following route tool when the engine is available.
2. THE “Snap routing” map-toolbar button and the snap toggle in the drawing hint bar SHALL be removed.
3. WHERE routing is unavailable, “New Route” SHALL draw straight lines. The hint bar SHALL give the reason and offer to add map regions. A route SHALL NOT be silently straight.
4. A straight-line route SHALL remain possible on purpose, for paths the engine does not know: off-road, ferries, private land. It SHALL be offered as a menu command, “New Straight-Line Route”, not as a toggle.
5. THE properties panel SHALL offer “Snap to roads” only for a route that does not already follow roads: an import, or one drawn straight. A road-routed route re-routes itself when edited (Phase 6) and needs no button.

Requirement 3: Altitude in the profile

1. WHERE a geometry carries no recorded elevation, THE profile SHALL plot terrain elevation sampled from a digital elevation model, with its gain and loss.
2. Terrain elevation SHALL be labelled as such wherever it appears: on the chart, in the gain and loss figures, and in the chart’s accessible name. Recorded and modelled heights SHALL never be presented as the same thing.
3. Recorded elevation SHALL take precedence. Terrain elevation SHALL NOT be written into route or track points, and SHALL NOT be exported. It is a derived value, computed when the profile is drawn.
4. Terrain tiles SHALL be downloaded with a region, covering the selected regions. WHERE a profile needs tiles that are missing, it SHALL offer to download them, stating how many and roughly how large. After download, terrain elevation SHALL work offline.

5. A void in the model, or a point with no tile (open sea), SHALL be a gap in the line, never 0.
6. Gain and loss from terrain SHALL suppress model noise with a stated threshold, documented with its reasoning.
7. A downloaded tile SHALL be validated before use. A file of the wrong size SHALL be rejected, not sampled.
8. THE data source SHALL be attributed in the About window and the licence register.

Requirement 4: A twisty score on every profile

1. Every profile of a route or track SHALL show a twisty score from 0 to 10, with:
   - a label (Straight, Gentle, Winding, Twisty, Very twisty)
   - degrees of bend per kilometre
   - bends per kilometre
   - the number of tight bends and hairpins
2. THE score SHALL be computed from heading change along the line after resampling it at a fixed spacing, so that point density has no effect. Why points per kilometre was rejected SHALL be documented with the measured figures.
3. THE score SHALL be **unknown, with a reason**, where it cannot be measured: a line shorter than half a kilometre, or one too sparse to show a bend, such as straight lines between clicked points. Unknown SHALL never be shown as 0.
4. THE profile SHALL show twistiness along the route, kilometre by kilometre.
5. THE scoring function SHALL be pure and SHALL live in `src/shared/geo/`. It delivers Phase 8 Requirement 3's scoring, which that phase SHALL reuse rather than rebuild.
6. THE thresholds SHALL be documented with their reasoning, and calibrated against the real roads above. A live test SHALL show that the score ranks them: the Cat and Fiddle above Snake Pass, Snake Pass above the A53, and the M6 lowest.
7. THE stated limitations SHALL include that a turn at a junction counts as a bend.

Requirement 5: UK default limits where none is posted

1. WHERE a road inside the UK has no posted limit in the map data, the speed distribution and the profile SHALL count it at the UK default for its kind:

Road	Assumed limit	Basis
Motorway (including its slip roads)	70 mph	Motorway limit
One-way trunk or primary road (a dual carriageway)	70 mph	National speed limit, dual carriageway
Other trunk, primary, secondary, tertiary and unclassified roads	60 mph	National speed limit, single carriageway
Residential and service roads	30 mph	Restricted road (street lighting)

2. THE assumed share SHALL be distinguishable from the posted share wherever either is shown:
   - the distribution bar SHALL mark the assumed portion of each band
   - the table SHALL give each band's assumed distance
   - the profile band SHALL draw assumed spans differently, with a key.
3. THE rule SHALL apply only inside the UK boundary. The Republic of Ireland, the Isle of Man and the Channel Islands have their own limits, and SHALL stay Unknown.

4. A road with no road class, and distance not matched to any road, SHALL stay Unknown.
5. A setting SHALL turn assumed limits off, returning to posted limits only. The default SHALL be on.
6. Recorded speeds SHALL be unaffected. This is about posted limits only.
7. THE rules SHALL live in one module, with their legal basis cited, so that correcting one rule is a one-line change.
8. ADR 0018 SHALL amend ADR 0011 §2 to allow this, and SHALL record the conditions under which it is allowed.

Requirement 6: A plainer route panel (owner feedback) Added after the phase shipped. The owner found the route panel "too techie for the standard person", and reported that changing the transport mode did not change the estimated duration.

1. THE route panel SHALL lead with name, colour, distance, an estimated time, the transport mode, and the speed-limit summary. Start and end coordinates, point count, elevation detail, source, last-modified time, the routing engine and the snap provenance SHALL move to a Details popup opened from an ⓘ button. This amends Phase 2 Requirement 4.1: those values are still shown, one click away.
2. THE estimated time SHALL depend on the transport mode. Walking SHALL use the planning speed from Settings. Cycling, car and motorcycle SHALL use a stated typical average. The caption SHALL give the speed and where it came from, so the figure is still labelled as an estimate (Phase 2 Requirement 4.3).
3. WHEN a route or track is selected AND the built-in engine is in use, road data SHALL be looked up without a button press. The lookup SHALL run at most once per selection, SHALL NOT toast, and SHALL show a failure inline with a retry. A remote service SHALL still be contacted only when the user asks, because consent covers actions the user takes.
4. Problems with the road data (points outside the loaded map, sections with no road) SHALL stay in the panel, not behind Details.
5. Duplicate and Delete SHALL be removed from the route panel, and Delete from the track panel. Both stay in the library's context menu.
6. Point editing SHALL open inside the profile drawer, under the chart. Points selected in the table SHALL be shaded on the chart. Clicking the chart SHALL select the nearest point in the table, and open the table if it is closed. Shift-click SHALL extend the selection and Cmd-click SHALL toggle a point, as they do in the table. THE drawer SHALL follow the selection: selecting another route or track SHALL show that one, and selecting anything without a profile SHALL close it.
7. THE Waze hand-off SHALL be a small icon. For a route or track it SHALL offer the start or the end, each as its own single-point hand-off, and each label SHALL say which point it sends. ADR 0005 is amended to allow the start, and there is still no whole-route action.

Non-functional requirements

1. **No regression.** `npm run verify` stays green; existing tests change only where the behaviour they assert was deliberately changed (the snap controls).
2. **Licences.** `osmium-tool` is GPL-3.0-or-later, which is compatible. It is bundled as a native tool and recorded in the register, with its dependencies. Terrain data is attributed. No new npm dependency.
3. **Module boundaries hold.** Scoring, HGT sampling, the UK boundary and the limit rules are pure modules in `shared/`. Downloading, the tile cache and the merge live in `main/`.
4. **Real geography in tests**, per ADR 0015: the Buxton square for the live multi-county build, and the test towns for terrain values.

Out of scope

- Keeping downloaded extracts so a county can be added without re-downloading the others. Possible later. Disk cost versus download time is the trade-off.
- Region maps for other countries. “Elsewhere” keeps the view-based list.
- Ascent for routes in the properties panel. The profile shows terrain gain; route statistics stay computed from stored points.
- Telling junction turns from bends. That needs graph topology, and belongs to Phase 8.

9.9.2 Design

Written against the code after Phase 6, and against the live engine. The figures quoted here were measured on an Apple M4. The engine was the bundled Valhalla 3.9.1 addon, on a graph built from Derbyshire, Cheshire and Staffordshire.

1. Regions: a UK map, several at once, merged before building

1.1 Which regions the map shows The region list is derived from the Geofabrik index the app already caches (`regionCatalogue.loadCatalogue`), so nothing new is fetched to draw the map.

`ukRegions(regions)` in `main/routing/geofabrik.ts` is a pure function. It returns:

- every region whose parent is `england` (47 counties)
- `scotland` and `wales`
- `ireland-and-northern-ireland`, carrying a note that Northern Ireland is published only with the Republic.

Rings are rounded to 4 decimal places (about 10 m) for transfer. They are used only to draw the picker, never to decide coverage, so the rounding cannot reach routing. About 16,000 points in total go over IPC once per open.

```
// shared/routing/results.ts
interface RegionMapEntry {
  id: string;
  name: string;
  /** Outer rings, [lon, lat], rounded for drawing only. */
  rings: [number, number][][];
  loaded: boolean;
  note: string | null;
}
```

1.2 IPC

Channel	Request	Response
`routing:regionMap`	none	`RegionMapEntry[]`
`routing:regionSizes`	`{ regionIds: string[] }` (1–64)	`{ id, sizeBytes | null }[]`
`routing:downloadRegion`	`{ regionIds: string[] }` (1–64), **changed** from `{ regionId }`	`LoadedRegion`

Sizes are fetched for the selection, not for all 50 regions when the map opens. The cost is shown when the user has chosen something, and HEAD requests to Geofabrik stay proportional to what is actually being considered. Sizes are cached for the session in `regionCatalogue`.

1.3 Download, merge, build `downloadAndBuild(paths, regionIds, options)` replaces the single-region version:

1. Resolve every id in the index (`NOT_FOUND` names the missing one).
2. With more than one id and no merge tool, refuse with `INVALID_REQUEST`: "This build cannot combine regions. Choose one."
3. Fetch the sizes. If the known total exceeds `MAX_REGION_BYTES` (2 GB), refuse before downloading anything.
4. Download each extract to `downloads/<id>.osm.pbf`, reporting `{ phase: 'downloading', regionIndex, regionCount, regionName, receivedBytes, totalBytes }`.
5. With more than one extract, run `osmium merge … -o downloads/merged.osm.pbf --overwrite --no-progress`, reporting `{ phase: 'merging' }`.
6. `importRegion` builds from the single file (unchanged: staging directory, swap on success).
7. Fetch terrain tiles covering the regions' bounding boxes (`{ phase: 'terrain', tilesDone, tileCount }`). A terrain failure does **not** fail the build. The graph is the thing the user asked for. The outcome carries `terrainWarning`, and the profile can fetch tiles later (Requirement 3.4).
8. `finally`: delete every extract and the merged file.

`LoadedRegion` gains `regionIds: string[]`. `regionName` becomes the joined display names ("Derbyshire, Cheshire and Staffordshire"; past three names, "Derbyshire, Cheshire and 4 more"). `readLoadedRegion` accepts an old record without `regionIds` and reports it as `[]`, so a graph built before this phase shows nothing pre-selected rather than failing to load.

1.4 The merge tool `osmium-tool` 1.19.1 (GPL-3.0-or-later), from Homebrew. It is bundled at `resources/valhalla/bin/osmium` by `scripts/bundle-valhalla.sh`, beside the Valhalla tools. That script already walks a binary's dylib closure and rewrites it to `@loader_path`, so `osmium` picks up `boost_program_options`, `lz4` and `expat` the same way. `scripts/build-valhalla.sh` adds `osmium-tool` to its required Homebrew formulae.

`EmbeddedPaths` gains `mergeTool`. `mergeToolPresent(paths)` is a plain `existsSync`, so a build without it degrades to single-region, which Requirement 1.9 asks for.

Why not merge in TypeScript: deduplicating overlapping PBF extracts means decoding and re-encoding every block with fresh string tables, which is a PBF writer, written and maintained here. `osmium merge` is the standard tool and takes 1.5 s on the three-county case. Rejected in ADR 0018.

1.5 The picker `RegionPickerDialog` gets two tabs. **United Kingdom** is the default. **Elsewhere** is the existing view-based list, which now calls `downloadRegion({ regionIds: [id] })`.

The UK tab is an inline SVG drawn by `RegionMap.tsx`:

- **Projection.** Equirectangular, with longitude scaled by `cos(54.5°)`, the UK's mid-latitude. It is computed by the pure function `projectRegions(entries, width, height)` in `shared/geo/regionMap.ts`, which returns SVG path strings and a label point per region.
- **One `<path>` per region**, with `role="checkbox"`, `aria-checked`, an `aria-label` of the region's name, and Space and Enter to toggle. A map of fifty clickable shapes must still be usable from the keyboard.
- **Fill.** Unselected regions are neutral. Selected ones use the accent colour. Loaded ones get a heavier outline and are listed as "loaded".
- **Overlap.** Neighbouring polygons overlap slightly, because Geofabrik buffers them. The click goes to the topmost path, and the hover name says which region that is.
- **Beside the map,** a list of the selection with sizes, a total, and a remove (×) per row, plus the two statements Requirement 1.11 asks for.

2. Following roads is the default `actions.startDrawingRoute()` becomes the single entry point for "New Route":

```
status = await routing.loadStatus()
status.available → startSnapRouting()            (road-following tool)
otherwise        → editor.startDrawRoute(tripId, { straightBecause: 'unavailable' })
```

`startStraightLineRoute()` is the deliberate path: `straightBecause: 'chosen'`. It is reached from the application menu ("New Straight-Line Route", Route menu, dispatch `route:newStraight`) and from the library context menu.

Removed:

- the toolbar "Snap routing" button
- the hint-bar toggle
- `editorStore.snapWhileDrawing` and `setSnapWhileDrawing`
- the `snap` parameter of `createRouteFromDrawnPoints`, which no longer snaps.

Draw mode in the hint bar:

`straightBecause`	Hint
`unavailable`, no map data	"Straight lines — no map region loaded yet" plus an **Add map regions…** button
`unavailable`, other	"Straight lines — road routing unavailable" plus the engine's message
`chosen`	"Straight lines — this route will not follow roads"

The **Add map regions…** button sits in a hint bar that is otherwise `pointer-events: none`, so the button itself takes pointer events. It opens the region picker, and drawing continues underneath.

`RoadDataSection` shows "Snap to roads" only when `provenance.routingProvider === null`, meaning a route that does not already follow roads.

3. Terrain elevation

3.1 Data Terrain Tiles on the AWS Registry of Open Data, `skadi` layout: `https://elevation-tiles-prod.s3.am` Each tile is 1° × 1°. Big-endian `int16` metres, row-major from the north-west corner. Either 3601 × 3601 (1", about 30 m; what the UK gets) or 1201 × 1201 (3"). −32768 is a void.

3.2 `shared/geo/hgt.ts` (pure)

- `tileNameFor(lat, lon)` gives `N53W002`, from the floor of each coordinate. The south-west corner names the tile.
- `tilesCovering(box)` lists the names a bounding box touches.
- `hgtSize(byteLength)` returns 3601, 1201, or `null` (invalid; Requirement 3.7).
- `sampleHgt(view: DataView, size, tileLat, tileLon, lat, lon)` interpolates bilinearly. If any of the four corners is a void it returns `null`, not a blend with −32768.
- `gainLoss(values, thresholdM)` accumulates a climb only once it has gone `thresholdM` beyond the last turning point (hysteresis). `TERRAIN_GAIN_THRESHOLD_M = 3`. The reasoning: SRTM's vertical noise is a few metres. Sampled every 20–30 m along a road, cell-to-cell noise would otherwise be counted as climbing, inflating gain on flat ground. Recorded elevation keeps the existing, unthresholded `elevationProfile`, so no existing figure changes.

3.3 `main/terrain/terrainStore.ts`

- Tiles are kept gzipped under `userData/terrain/<name>.hgt.gz`, about 7 MB each against 26 MB unpacked. They are decompressed into memory on first use, with an LRU of 4 tiles. A tile that S3 answers 403 or 404 for is open sea, and is recorded as `<name>.none`, so it is not asked for again.
- `downloadTiles(names, { onProgress, signal })` downloads with `redirect: 'error'` and validates by `gunzipSync` plus `hgtSize`. The file is written to a temp name and renamed on success, so a partial download never sits in the cache.
- `elevations(points)` returns `{ values: (number | null)[], missingTiles: string[] }`.
- `TERRAIN_HOST` is declared in the module. It is main-process only, so the renderer CSP is unaffected.

3.4 Profile `profileService.profileFor` gains an injected terrain source, so tests do not touch disk. When no source point carries elevation:

- every tile present: the samples get terrain heights, `elevationSource = 'terrain'`, and the summary comes from `gainLoss(…, 3)`.
- tiles missing: elevation is absent with the reason “Terrain heights for this area are not downloaded yet.”, plus `terrainMissing: { tileCount, approxBytes }`, where `approxBytes` is 7.5 MB per tile.

New IPC `routing:downloadTerrain { ownerType, ownerId }` downloads the tiles covering that geometry’s bounding box. The panel then reloads the profile.

The chart titles the line “Terrain elevation (model, about 30 m)”. Gain and loss read “Gain ≈ …”.

4. Twisty score `shared/geo/twistiness.ts`, already written and calibrated during the investigation:

1. **Unknown** when the length is under 500 m, or the median source spacing is over 150 m. Hand-drawn lines have spacings in kilometres. Engine geometry and 1 Hz recordings are well under 50 m.
2. **Resample** every 20 m, which discards point density.
3. **Smooth** with a 3-point moving average, so alternating jitter cancels.
4. **Turning.** The signed heading change per step.
5. **Bends.** A same-sign run of steps with |turn| ≥ 2° (a radius under about 570 m), tolerating one quiet step. The run counts if its total turn is at least 20° and its radius (arc ÷ angle) is at most 400 m. It is classed as hairpin (≥ 135° and ≤ 45 m), tight (≤ 60 m), bend (≤ 150 m) or sweeping. These radii are **measured** at 20 m resolution, which has a floor of roughly 25–30 m: a true 12 m hairpin measures 32 m, and a true 25 m one measures 45 m.
6. **Score** = min(10, degrees per km ÷ 30). Levels: Straight below 1.5, Gentle below 2.5, Winding below 5, Twisty below 7.5, Very twisty above.
7. **Windows** of 1 km for the band along the profile. A tail under 500 m is folded into the last window.

Calibration, measured:

Road	°/km	Score	Level
A537 Cat and Fiddle	201	6.7	Twisty
A57 Snake Pass	79	2.6	Winding
A523	70	2.3	Gentle
A515	53	1.8	Gentle
A53	45	1.5	Gentle

Road	°/km	Score	Level
M6	31	1.0	Straight

Snake Pass is mostly long sweeping curves (30 of its 39 counted bends), which is why it lands below the Cat and Fiddle's tight sequence.

The live test re-measures with the fixture's via points and the Nominatim-resolved endpoints (task 12): Cat and Fiddle 7.3, Snake Pass 2.7, A515 1.6, M6 0.9. The Cat and Fiddle scores higher than in the table because its via points keep it on the A537 through every bend.

Known limitation. A kilometre containing junction turns in a town scores high, because a 90° turn at a T-junction is a bend by geometry. The overall score is distance-weighted, so it is robust on a real ride. The kilometre band will show a town centre as twisty, and the panel says so.

`Profile` gains `twistiness: Twistiness | null` and `twistinessUnknownBecause: string | null`. It is computed on the **full** source points, not the thinned samples. That is O(n), and the full points are already in memory.

The chart adds a "Twistiness" band. Each window is coloured by level on a sequential ramp, with a five-level key. The summary line reads: "Twisty score 6.7 / 10 · Twisty · 201°/km · 3.4 bends/km · 2 tight".

5. UK default limits

5.1 Data `MatchedEdge` gains `oneway: boolean | null`, from `edge.traversability` (`forward` or `backward` is one-way, `both` is not). `edge.traversability` is added to `MATCH_ATTRIBUTES`. Cached payloads from before this phase have no `oneway`. It reads as `null`, which the rule treats as single carriageway. A "Look up road data" refresh brings it in.

5.2 `shared/geo/ukBoundary.ts` `UK_BOUNDARY: Ring`, 150 points, generated by `scripts/generate-uk-bounda` from the `united-kingdom` feature of Geofabrik's index. The polygon is derived from OpenStreetMap: © OpenStreetMap contributors, ODbL, and the header says so. `inUk(point)` uses a bounding-box prefilter, then `pointInRing`.

5.3 `shared/routing/ukSpeedLimits.ts`

```
ukDefaultLimit(roadClass, oneway): { mps: number; basis: UkLimitBasis } | null
```

Class	One-way	Limit	Basis
motorway	any	70 mph	`motorway`
trunk, primary	true	70 mph	`nsl-dual`
trunk, primary	false or null	60 mph	`nsl-single`
secondary, tertiary, unclassified	any	60 mph	`nsl-single`
residential, service_other	any	30 mph	`restricted-road`
null	—	none	—

Legal basis, cited in the module:

- the Motorways Traffic (Speed Limit) Regulations 1974 (motorways)
- the 70 mph, 60 mph and 50 mph (Temporary Speed Limit) (Continuation) Order 1978 (national limits)
- the Road Traffic Regulation Act 1984, s.81–82 (restricted roads: 30 mph where street lighting is no more than 183 m apart).

Two judgement calls, stated here and in the UI:

1. Residential and service roads take 30 mph, not 60. The owner asked for the national speed limit, but in UK law a street-lit road is a restricted road at 30 mph, and counting a housing estate at 60 would be the larger error.
2. One-way trunk or primary is read as dual carriageway. A one-way street in a town on an A-road is the case this gets wrong. It is rare on trunk and primary roads, and the assumed label covers it.

5.4 Distribution and profile

- `SpeedSample` gains `assumed?: boolean`.
- `BandShare` gains `assumedDistanceM` and `assumedPercent`.
- `SpeedDistribution` gains `assumed: { distanceM, percent }`.
- `speedDistribution` adds an assumed sample to its band's distance and to the band's assumed distance. Unknown keeps only what no rule could place.

`roadAttributeService.postedSpeedSamples(shape, edges, options)` takes `{ ukDefaults: boolean }`. For an edge with a null limit, it takes the midpoint of the edge's own shape, and if that is `inUk` and the rule returns a limit, emits an assumed sample. Otherwise the edge stays Unknown, as before.

`Profile` gains `assumedSpeedSpans: ProfileSpan<number>[] | null`. `postedSpeedSpans` keeps its meaning (posted only), so nothing that reads it changes. The band is drawn when either is non-empty.

The setting is `ukDefaultSpeedLimits: boolean`, default `true`, read by the IPC handlers and passed in. The Settings dialog labels it: "Count UK roads with no posted limit at the national default (shown as assumed)".

5.5 Display

- `SpeedDistributionBar`**.** Each band's segment draws its assumed part with a diagonal hatch over the band colour. The table adds an "of which assumed" column, shown only when anything is assumed. Under the bar: "Includes 41% assumed from UK default limits — no limit is posted in the map data for those roads."
- **Profile.** Assumed spans use the same colour at 45% opacity with the hatch, plus a key line: "Hatched: assumed UK default limit".

6. Attribution and licences

- **About window, data sources:** "Elevation: Terrain Tiles (Mapzen / AWS Open Data), derived from SRTM and other sources", and "UK boundary: © OpenStreetMap contributors, ODbL".
- **Register, native tools:** osmium-tool 1.19.1 GPL-3.0-or-later, boost (program_options) BSL-1.0, lz4 BSD-2-Clause, expat MIT, bzip2 and zlib from the system. A new "Data" section covers the terrain tiles.

7. Testing

Layer	What
Unit	Twistiness: straight is 0; a uniform 100 m radius scores; alternating jitter does not; hairpins are classed; sparse and short input is unknown; windows.

Layer	What
Unit	HGT: tile naming in all four hemispheres, covering tiles, size validation, bilinear values on a synthetic grid, voids are null, hysteresis gain.
Unit	UK rules table; `inUk` at Belfast, Shetland, Dublin, Douglas, Jersey; assumed share in the distribution; unknown stays unknown outside the UK and with no class; the setting off.
Unit	`ukRegions` from a fixture index; `projectRegions`; multi-region `downloadAndBuild` with fakes (refusals, merge invoked only for more than one, extracts deleted on failure); `readLoadedRegion` back-compat.
Unit	Profile service: terrain fallback, recorded precedence, missing tiles, twistiness, assumed spans.
Live (`npm run verify:regions`)	Download Derbyshire, Cheshire and Staffordshire, merge, build, and route the whole Buxton square; score the six reference roads and assert the ranking; terrain at Buxton and Ashbourne within 15 m of the known heights.
E2E	New Route with no region gives the straight-line hint and an Add regions button; no “Snap routing” toolbar button; the region picker map with a seeded index (select two counties, see the total); the profile shows the twisty score; the settings toggle exists.

The region-picker end-to-end test seeds `geofabrik-index.json` from `test-data/geofabrik-uk.json`. That is the UK features of the real index, simplified to at most 40 points a ring: ODbL map data, not user GPS data. Because `loadCatalogue` reads a fresh cached index before going to the network, the test needs no network and no special hook.

9.9.3 Tasks

Execute in order. Pure modules come first, each proven offline, then the main process, then the UI, then the live and end-to-end passes.

Update `docs/progress-mindmap.svg` as each task changes status.

- ☒ 1. Spec, ADR 0018 and renumbering
  - Requirements, design and tasks for this phase; ADR 0018
  - Windy roads moves to `phase-8-…` and Garmin to `phase-9-…`; live documents and links are updated; historical records are left as written
  - *Requirements: 4.5, 5.8*
- ☒ 2. Twistiness scoring
  - `shared/geo/twistiness.ts`: resample, smooth, bends, score, windows, unknown
  - Unit tests: a straight line scores 0; a constant 100 m radius curve is counted and classed; a 15 m radius 180° turn is a hairpin; ±3 m zigzag jitter on a straight line scores Straight; the same road at 5 m and at 40 m source spacing scores the same within 0.3; a 2 km line of 4 points and a 300 m line are unknown; windows cover the length

  - Found while testing: the measured radius has a floor of about 25–30 m at 20 m resampling, so a true 12 m hairpin measures 32 m. Hairpin and tight thresholds moved to the measured scale (45 m, 60 m); design §4 corrected
  - *Requirements:* 4.1, 4.2, 4.3, 4.5, 4.7
- ☒ 3. UK boundary and default limits
  - `scripts/generate-uk-boundary.mjs`, `shared/geo/ukBoundary.ts` (`inUk`)
  - `shared/routing/ukSpeedLimits.ts` (`ukDefaultLimit`, cited)
  - `shared/geo/speedBands.ts`: assumed samples, per-band assumed share
  - Unit tests: inside/outside for Belfast, Shetland, Buxton, Dublin, Douglas, Jersey; the rule table row by row; a distribution with assumed distance sums to 100% with Unknown kept separate
  - The boundary is 150 points from Geofabrik's `united-kingdom` feature, so the generator is checked in and the output is reproducible
  - *Requirements:* 5.1, 5.3, 5.4, 5.7
- ☒ 4. Road data carries one-way, and assumed limits reach the services
  - `valhalla.ts`: `edge.traversability` ▯ `MatchedEdge.oneway`
  - `roadAttributeService.postedSpeedSamples` with `{ ukDefaults }`; `profileService` `assumedSpeedSpans`; setting `ukDefaultSpeedLimits` (default on) in types, defaults, schema and the IPC handlers
  - Unit tests: traversability parsing; an unposted UK tertiary edge is 60 mph assumed; the same edge outside the UK, or with the setting off, is Unknown; old payloads without `oneway` read as single carriageway
  - *Requirements:* 5.1, 5.2, 5.4, 5.5, 5.6
- ☒ 5. Terrain: HGT sampling and the tile store
  - `shared/geo/hgt.ts`: names, covering tiles, size validation, bilinear sampling, voids, `gainLoss` with hysteresis
  - `main/terrain/terrainStore.ts`: gzipped cache, LRU, sea markers, validated atomic download
  - Unit tests: as design §7, plus the store against a synthetic gzipped tile and a fake fetch (403 is sea, a wrong size is rejected and not cached)
  - Tiles are resolved once per lookup rather than once per point, so a 200,000-point recording does not cost 200,000 filesystem checks, and the memory cap still holds across many tiles
  - *Requirements:* 3.1, 3.5, 3.6, 3.7
- ☒ 6. Profile: terrain elevation and twistiness
  - `profileService`: injected terrain source; recorded precedence; `elevationSource`, `terrainMissing`, `twistiness`, `twistinessUnknownBecause`
  - IPC `routing:downloadTerrain`; preload; api
  - Unit tests: a snapped route with tiles gets terrain heights and gain; with a tile missing it reports the count; a GPX route with recorded heights keeps them; twistiness present for road geometry and unknown for a hand-drawn route
  - *Requirements:* 3.1, 3.2, 3.3, 3.4, 4.1, 4.3
- ☒ 7. Region map data and multi-region build
  - `geofabrik.ukRegions`; `regionCatalogue.regionMap` and `regionSizes`; `downloadAndBuild(paths, regionIds)` with merge and terrain stages; `EmbeddedPaths.mergeTool`; `LoadedRegion.regionIds`
  - IPC `routing:regionMap`, `routing:regionSizes`; `downloadRegion` takes `regionIds`
  - `scripts/build-valhalla.sh` requires `osmium-tool`; `scripts/bundle-valhalla.sh` bundles `osmium` and its dylibs; re-bundled
  - Unit tests: `ukRegions` from a fixture; refusals (over 2 GB, several without a merge tool); merge invoked only for several; extracts removed on failure; old `region.json` still reads
  - Re-bundled: the relocated `osmium` loads and merges, and the bundle's own load check passes. The check failed at first only because this editor exports `ELECTRON_RUN_AS_NODE`; `bundle-valhalla.sh` now unsets it

  - *Requirements: 1.1, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 1.12*
- ☒ 8. Region picker: UK map and Elsewhere
  - `shared/geo/regionMap.ts` `projectRegions` (unit-tested)
  - `RegionMap.tsx`; `RegionPickerDialog` with United Kingdom and Elsewhere tabs, selection list, sizes, total, the replace and re-download notes; progress lines for every stage; Settings names loaded regions
  - *Requirements: 1.1, 1.2, 1.3, 1.4, 1.5, 1.7, 1.10, 1.11*
- ☒ 9. Following roads by default
  - `startDrawingRoute` chooses the road tool when available; `startStraightLineRoute`; `straightBecause` in the editor store
  - Remove the toolbar button, the hint toggle and `snapWhileDrawing`; hint bar with “Add map regions…”; “New Straight-Line Route” in the app menu and library context menu; “Snap to roads” only for routes not following roads
  - *Requirements: 2.1, 2.2, 2.3, 2.4, 2.5*
- ☒ 10. Profile and distribution display
  - Terrain-labelled elevation line and gain, and the download offer; twisty score line and twistiness band with key; hatched assumed speed spans with key; `SpeedDistributionBar` assumed hatch, column and note; Settings toggle
  - *Requirements: 3.2, 3.4, 4.1, 4.4, 5.2, 5.5*
- ☒ 11. Attribution and licence register
  - About window data sources; register: osmium-tool and dependencies, terrain data; `npm run licences` still passes
  - Found while doing it: `npm run licences` rewrote the whole register, so the hand-maintained Valhalla section had been silently deleted by every regeneration. It now lives in `docs/licences/native-and-data.md`, which the generator appends, and the lost section is restored
  - *Requirements: 3.8, NFR 2*
- ☒ 12. Live tests
  - `src/main/routing/regions.live.test.ts`, `npm run verify:regions`: three-county merge and build, the Buxton square routed, twistiness ranking of the six reference roads, terrain at Buxton and Ashbourne
  - Snake Pass and M6 endpoints resolved through Nominatim and added to `places.json` with corridor checks (passes the Snake Pass Inn; ≥ 80% of the M6 route is motorway), rather than coordinates from memory
  - Result: three counties downloaded, merged and built in 75 s (40 tiles, 148 MB); the Buxton square routes, 98.8 km in four legs, nothing displaced; Cat and Fiddle 7.3 Twisty > Snake Pass 2.7 Winding > A515 1.6 Gentle > M6 0.9 Straight; terrain Buxton 299 m (known 300), Ashbourne 129 m (known 130)
  - Noted: the Macclesfield–Leek leg scored 5.3 with three “hairpins”, because its fixture via points sit off the A523 and the route turns into villages and back. Real geometry, and the junction limitation in design §4 made visible
  - *Requirements: 1.6, 3.1, 4.6*
- ☒ 13. End-to-end and verification pass
  - E2E as design §7; existing specs updated for the removed controls
  - `npm run verify`; `npm run package`; e2e against the packaged app; README; mindmap
  - Result: `npm run verify` green (lint, typecheck, 1,214 unit tests, build, 122 end-to-end); packaged `TrailRT.app` carries `bin/osmium` and passes the same 122 end-to-end tests
  - Visual check in the running app, with the three-county graph: the picker map, the twisty band and score, hatched assumed limits, and terrain heights rising from Buxton to 517 m at the Cat and Fiddle (known: 515 m). The map’s height was cut to fit the dialog after the first look
  - *Requirements: all*
- ☒ 14. Fix: Enter and double-click did not finish a new route

  - Reported by the owner after task 13. Reproduced in the app with the Derbyshire graph: every way of starting New Route, finished with Enter, showed “That request was not valid” and saved nothing
  - Cause 1, from Phase 6 and exposed by Requirement 2.1: keeping route A in the alternatives stored the `Candidate` itself as the draft leg, and its `estimatedDurationS` failed the strict leg schema. Phase 6’s end-to-end test only chose route B, whose leg is rebuilt. `asPreviewedLeg` in `shared/routing/results.ts` now projects every stored and sent leg
  - Cause 2: the first click of a double-click puts that point’s handle under the pointer, so the `dblclick` landed on the handle and never reached the map. `useSnapRouting` now listens on the map container in the capture phase
  - Also: finishing waits up to 5 s for a leg still in flight rather than refusing, and straight-line drawing ignores the duplicate click of a double-click, which had left a zero-length last segment
  - Unit test against the real request schema (fails before the fix); `tests/e2e/finishing.spec.ts`: Enter keeping route A, Enter straight after a click, double-click on a handle, straight-line double-click ☒ 2 points
  - `npm run verify` green: 1,215 unit, 126 end-to-end
  - *Requirements: 2.1, 2.4*

☒ 15. Fix: the map went blank after switching base map while zoomed in

  - Reported by the owner with the profile open; the profile was not the cause. Reproduced: zoomed to 19 on OpenStreetMap, switching to OpenTopoMap (which serves to 17) left the map at 19 with a tile layer that drew nothing past its own `maxZoom` — no tiles at all, and zooming did not bring them back. The attribution read “© OpenStreetMap contributors, Map data © OpenStreetMap contributors, SRTM · Rendering © OpenTopoMap”, exactly as in the owner’s screenshot
  - Cause 1: a provider’s serving limit was used as the map’s zoom limit. Now `MAP_MAX_ZOOM` (19) is the map’s and every tile layer’s `maxZoom`, and the provider’s limit is `maxNativeZoom`, so past it Leaflet enlarges the deepest tiles rather than showing nothing
  - Cause 2: layer cleanup called `off()` before `remove()`, deleting Leaflet’s one-off listener that takes the layer’s attribution back off the map, so the old provider stayed credited for the rest of the session
  - `tests/e2e/basemap.spec.ts`: fails without the fix (no tiles), passes with it; checks the tiles requested are zoom 17 and the credit appears once
  - `npm run verify` green: 1,215 unit, 128 end-to-end
  - *Requirements: NFR 1*

☒ 16. Owner feedback: a plainer route panel

  - Route panel (`features/properties/RouteProperties.tsx`) leads with distance, time, transport mode and the speed-limit summary. Everything technical is in a Details popup behind ☒. Duplicate and Delete removed; the track panel loses Delete too (both stay in the library menu)
  - Time follows the transport mode: `shared/routing/travelTime.ts`. Walking uses the Settings planning speed; the others use a stated average
  - `features/properties/roadData.tsx` replaces `RoadDataSection.tsx`: `RoadSummary` for the panel, `RoadDetails` for the popup, and `useRoadData`, which looks roads up on selection with the built-in engine only, once per selection, quietly, with an inline retry on failure
  - Points open inside the profile drawer. `ProfileSample.index` carries each sample’s source point; `features/profile/profilePoints.ts` maps indices to distances and back, so the table’s selection is shaded on the chart and a chart click selects the nearest row
  - Waze is an icon with a start / end menu (`wazeTargetsFor`); ADR 0005 amended
  - Unit tests: `travelTime.test.ts` (the 34-mile route is ~12 h walking, ~1 h driving), `profilePoints.test.ts` (exact, thinned, round trip, runs), `waze.test.ts` (start and end,

empty geometry)
- End-to-end: chart click selects and shades a point; Waze menu offers start and end only; with the built-in engine, speed limits appear without a button and the engine details are in the popup
- Found by the first verify run: a road lookup dropped the route's snap provenance, so after an automatic lookup the map lost unmatched legs and dragging the line added no shaping point. `matchRoads` now keeps it
- Follow-up reported by the owner: selecting another route left the drawer on the first. `features/profile/drawerTarget.ts` now makes it follow the selection: another route or track retargets it (the point list stays open), anything without a profile closes it. The drawer header names the geometry. Unit-tested, and an end-to-end test that fails without the fix
- `npm run verify` green: 1,244 unit, 130 end-to-end
- *Requirements:* *6.1*, *6.2*, *6.3*, *6.4*, *6.5*, *6.6*, *6.7*

Notes

9.10 Phase 8: Intelligent Windy Roads Routing — Requirements — SPEC-PHASE-8-WINDY-ROADS-ROUT

Version: — · **Status:** unspecified · **Slug:** `phase-8-windy-roads-routing`

9.10.1 Requirements

Status: requirements written. Phase 4 has shipped, and this phase follows Phase 6 (route alternatives and shaping), whose shaping-point model and alternatives picker it builds on, and Phase 7, which built and calibrated the twistiness score this phase's Requirement 3 asks for (`shared/geo/twistiness.ts`, ADR 0018). See `investigation.md` for why that dependency is hard rather than a sequencing preference.

Introduction Generate motorcycle-friendly routes between two points that actively favour winding, interesting roads rather than the shortest or fastest line.

What the investigation changed about this phase The feature brief describes an enhancement to "the existing routing, OpenStreetMap data, snap-to-road functionality and GPX export capabilities". The mandatory codebase investigation found that **of those four, only GPX export exists**:

- there is no routing engine, no routing API client, and no HTTP client at all
- OpenStreetMap is consumed as raster tile imagery only; `connect-src 'self'` in `src/main/security/csp.ts` blocks every outbound request today
- snap-to-road is Phase 4 requirement text, unimplemented
- there is no bearing, turn-angle or curvature maths anywhere in the project

So this phase does not enhance a routing architecture. It is the phase that makes Phase 4's routing architecture *curvature-aware*, and it cannot begin before Phase 4 exists. The brief's instruction — "do not introduce a second routing engine without establishing why the existing implementation cannot support the feature" — resolves as: there is no first engine, Phase 4 chooses one, and this phase's job is to establish whether that choice can carry custom curvature weighting rather than to displace an incumbent.

The constraint that will shape the design most `src/shared/ipc/schemas.ts` caps route geometry at **10,000 points**, because `routeService.setGeometry`'s undo inverse carries the entire previous point array. A deliberately twisty 200 km route at OSM digitisation density is plausibly 20,000–50,000 points. Requirement 8 exists to resolve that; it is an architecture question, not a tuning one.

Two distinctions carried in from earlier phases

1. **Calculating a route is not map matching a track.** Map matching takes an existing recorded trace and finds the roads it followed. Routing takes two points and finds a path. They share a provider and nothing else, and ADR 0011 §4 already forbids snapping tracks because a recording is measured data.
2. **Unknown is a value.** A road whose curvature cannot be determined is not a straight road, exactly as absent elevation is `null` and never `0`.

User stories

1. As a motorcyclist planning a weekend ride, I want to enter a start and destination and get a route that favours bends, so that I do not have to hand-draw every interesting road I already know.
2. As the same rider, I want to control *how* twisty, because "as twisty as possible" and "a pleasant route that still gets me there" are different requests on different days.

3. As someone with a destination to reach, I want to cap how much extra distance the route may add, so that a scenic route does not become an impractical one.
4. As a rider on a road bike, I want unsuitable surfaces and inaccessible roads excluded, so that the route does not send me down a rutted farm track.
5. As an existing user who does not want any of this, I want normal routing to behave exactly as it did, so that a new feature does not change my workflow.
6. As a privacy-conscious user, I want to know before my planned route leaves my machine, and to be able to run the whole thing locally.
7. As a rider exporting to a device, I want the generated route to export as GPX that my device accepts.

Requirements

Requirement 1: Routing engine capability assessment The deliverable is a documented finding, not a feature. It gates Requirement 5.

Acceptance criteria

1. THE phase SHALL begin by assessing whether the routing provider selected in Phase 4 can express a caller-supplied, per-edge curvature cost, and SHALL record the finding with evidence.
2. THE assessment SHALL re-verify, against the then-current release, whether GraphHopper's custom models can use a curvature value as a term in an expression rather than only as a constant multiplier, because a project forum answer from 2022 states the latter and that limitation would materially change the design.
3. THE assessment SHALL state explicitly that a hosted routing API cannot accept caller-supplied per-edge weights, and SHALL therefore NOT propose one as a way to satisfy Requirement 5.3.
4. WHERE a second routing engine is proposed THE assessment SHALL compare integration effort, hosting requirements, data requirements, licence, performance and maintenance against extending the Phase 4 provider, and SHALL justify the addition rather than assert it.
5. THE assessment SHALL prefer the option that adds the least new infrastructure and operational complexity, and SHALL say plainly what each option costs the user in download size, runtime dependencies and setup.
6. Any engine adopted SHALL have a licence compatible with GPL-3.0, verified per `.kiro/steering/licensin` before adoption, and recorded in the third-party register.

Requirement 2: Routing modes and preferences Acceptance criteria

1. THE system SHALL provide a routing mode selector offering, at minimum: **Normal**, **Slightly Winding**, **Winding** and **Very Winding**.
2. **Normal SHALL preserve existing behaviour exactly.** No change to geometry, statistics, storage, undo behaviour or export for a user who does not opt in. This is a regression requirement and SHALL be covered by tests that would fail if normal routing changed.
3. THE winding modes SHALL differ in the strength with which curvature is favoured, not in which roads are permitted, so that the suitability rules in Requirement 4 apply identically across all of them.
4. THE selected mode SHALL be recorded with the route, so the geometry's origin is auditable, consistent with Phase 4 Requirement 2.4.
5. THE system SHALL function fully with no routing provider configured; the winding modes are then unavailable with an explanation, not silently degraded to straight lines under a label that promises bends.
6. THE mode SHALL be re-applicable to an existing A-to-B route without redrawing it, and SHALL be undoable.

Requirement 3: Road curvature and twistiness scoring A pure, dependency-free scoring function in `src/shared/geo/`, built from scratch — the investigation confirmed there is no bearing or angle maths in the project to build on.

> **Delivered early, in Phase 7.** `shared/geo/twistiness.ts` scores a line from heading change after resampling, with unknown for sparse or short input, and is calibrated against six real roads around Buxton. This phase reuses it for per-edge costs and verifies the remaining criteria (curve-radius terms in the cost, edge-level caching) rather than writing a second scorer.

Acceptance criteria

1. THE system SHALL compute a twistiness score for a road segment from its geometry, normalised so that segments of different lengths are comparable.
2. THE score SHALL be derived from, at minimum: heading change between consecutive geometry points, meaningful bends per kilometre, bend angle and an estimate of curve radius.
3. THE algorithm SHALL distinguish, and SHALL be tested to distinguish: straight roads, gentle sweeping curves, regular winding roads, tight bends and hairpins.
4. THE algorithm SHALL apply filtering and smoothing so that digitisation irregularity is not counted as a genuine bend, and the thresholds SHALL be documented with the reasoning for their values.
5. THE algorithm SHALL account for the fact that **OSM geometry point density varies with mapping quality, not only with road shape** — a densely digitised motorway curve and a sparsely digitised lane are not comparable without normalisation. This is a documented limitation of the approach, not a defect to be hidden.
6. WHERE a segment has too few geometry points to score meaningfully THE system SHALL return **unknown** rather than a low score, because sparse geometry is not evidence of a straight road.
7. THE scoring function SHALL be pure, SHALL live in `src/shared/geo/`, and SHALL NOT import `electron`, `node:*` or any driver, per `.kiro/steering/architecture.md`.
8. THE algorithm and its limitations SHALL be documented, including what it cannot know: camber, sight lines, surface quality in practice, traffic, and whether a road is actually enjoyable to ride.

Requirement 4: Road suitability and restrictions Acceptance criteria

1. WHERE the underlying data supports it THE system SHALL offer preferences to: avoid motorways, avoid major dual carriageways, prefer secondary and tertiary roads, prefer suitable unclassified roads, and avoid unsuitable surfaces.
2. THE system SHALL respect access restrictions, one-way restrictions, and roads inaccessible to the selected vehicle type, and SHALL NOT produce a route that violates them.
3. THE system SHALL NOT assume that an unclassified road is a suitable motorcycle road.
4. THE system SHALL NOT include private roads, access-restricted roads or unsuitable tracks in a generated route unless the user has explicitly enabled them, and SHALL state what enabling them means.
5. WHERE suitability cannot be determined from the data THE system SHALL treat the road as **unknown suitability** and SHALL NOT silently treat unknown as suitable.
6. Any hardcoded road classification or suitability assumption SHALL be documented in one place with its reasoning, rather than scattered through the costing code.

Requirement 5: Custom routing cost model Acceptance criteria

1. THE cost for a road segment SHALL combine distance, expected travel time, twistiness score, road classification, road suitability and user preferences.

2. THE weighting SHALL be expressed as a named model rather than scattered constants, so that a mode is a set of parameters and not a code path.
3. **Twistiness SHALL influence route selection during the search, and SHALL NOT be computed after a conventional route has been produced.** A design that generates alternatives and re-ranks them does not satisfy this requirement and SHALL be rejected.
4. THE routing SHALL return a viable route when no sufficiently winding roads are available, rather than failing, and SHALL state that it could not find a winding route.
5. THE cost model SHALL be independently testable without a live routing service.
6. THE system SHALL NOT present a curvature-weighted route's duration as a measurement; duration remains the labelled estimate described in ADR 0011 §8.

Requirement 6: Detour management Acceptance criteria

1. THE system SHALL offer a configurable maximum detour relative to the normal route, with at least the options **10%**, **20%**, **30%**, **50%** and **no strict limit**.
2. **THE detour limit SHALL be enforced during route selection, not reported after the fact.** A route that exceeds the budget SHALL NOT be returned as the chosen route.
3. THE system SHALL avoid routing loops, unnecessary backtracking and repeated traversal of the same segment in the same direction.
4. WHERE the budget cannot be met THE system SHALL return the best route within it, SHALL say that the twistiness preference was limited by the detour budget, and SHALL NOT silently exceed the budget.
5. THE system SHALL report the chosen route's distance and estimated time against the normal route's, so the trade-off accepted is visible.

Requirement 7: Snap-to-road integration and geometry integrity Builds directly on Phase 4 Requirement 3 and must not duplicate it.

Acceptance criteria

1. Generated route geometry SHALL follow actual connected roads, SHALL preserve valid connectivity, and SHALL respect direction restrictions.
2. THE system SHALL NOT produce shortcuts across fields, water or inaccessible areas.
3. THE system SHALL reuse the Phase 4 snap-to-road implementation and SHALL NOT introduce a parallel one.
4. THE system SHALL NOT map-match an existing track as a way of generating a route, and SHALL NOT modify any track's stored geometry.
5. THE generated geometry SHALL avoid redundant points. Because `src/main/io/gpx/write.ts` performs **no rounding, re-sampling or decimation on export** by deliberate policy, density SHALL be controlled at generation time or by an explicit, previewed simplification using the existing `shared/geo/simplify.ts`, following the Phase 3 precedent of showing the resulting point count before applying.
6. THE resulting route SHALL remain usable for display at interactive frame rates and for export.

Requirement 8: Geometry volume and undo Exists because of a specific, measured constraint rather than a general concern.

Acceptance criteria

1. THE design SHALL resolve the interaction between snapped geometry volume, the **10,000-point cap** in `routePointsSchema`, and `route:setGeometry` undo entries that carry the entire previous point array.
2. THE resolution SHALL NOT be to remove the cap without replacing the bound it provides on a single undo entry.

3. WHERE routes gain a second geometry layer — placed control points plus snapped shape, per Phase 4 Requirement 3.3 — the schema change SHALL be designed against the existing `routes` / `route_points` tables, and SHALL state whether it follows the tracks `render_geometry` cache pattern or adds new storage.
4. Undoing a snap SHALL restore the previous geometry exactly, or SHALL declare itself non-undoable and say why, following the honesty precedent in `trackService.restoreOriginal`.
5. Route statistics SHALL remain correct and recomputed from the stored geometry, never carried over from a provider response.

Requirement 9: User interface Acceptance criteria

1. THE feature SHALL integrate into the existing routing interface and SHALL NOT redesign unrelated functionality.
2. THE system SHALL provide controls for routing mode, twistiness preference, maximum detour and road avoidance options.
3. THE system SHALL indicate clearly when winding-road routing is active, including on the map, so a generated route is never mistaken for a hand-drawn one.
4. THE controls SHALL use existing components, styling and patterns from `src/renderer/components/` and SHALL be usable at both desktop and narrow window widths.
5. THE system SHALL show progress for a long calculation and SHALL allow it to be cancelled.
6. THE system SHALL report the achieved twistiness of the generated route, so the user can see whether the preference had any effect.

Requirement 10: GPX compatibility Acceptance criteria

1. Generated routes SHALL export through the existing GPX writer with no new format and no new metadata conventions.
2. Export SHALL preserve coordinate ordering, route continuity and full coordinate precision, per the existing no-rounding policy.
3. A generated route SHALL survive an export-then-reimport round trip with identical geometry, tested the same way Phase 3 tests GPX round trips.
4. Curvature scores, mode and provider SHALL be recorded as route metadata where they are preserved, SHALL be reported as unmapped on export to formats that cannot carry them, and SHALL NOT be written into geometry fields.

Requirement 11: Performance, caching and reliability Acceptance criteria

1. Curvature scores SHALL be cached, keyed to the geometry they were computed from, SHALL be invalidated when that geometry changes, and SHALL be recomputable — the same contract as every other derived value in the project (`.kiro/steering/architecture.md`).
2. THE system SHALL NOT recompute curvature for unchanged road geometry.
3. THE system SHALL handle a routing failure gracefully and SHALL fall back to normal routing with an explanation, never silently.
4. WHERE a routing provider is unreachable THE system SHALL say so and SHALL leave existing routes untouched.
5. THE system SHALL respect any applicable API rate limits and SHALL NOT retry in a way that would breach a provider's usage policy.
6. Calculation SHALL NOT block the user interface. The investigation found no worker or background process in the application today, so introducing one is an explicit design decision to be recorded, not an implementation detail.
7. Memory use SHALL be bounded for a continental-scale road network, and the design SHALL state what bound it relies on.

Requirement 12: Security, privacy and licensing Acceptance criteria

1. THE system SHALL require explicit opt-in before route geometry is sent to any external service, SHALL name the service, and SHALL state what is transmitted — inheriting ADR 0011 §6.
2. Any new network host SHALL be declared by its provider and added to the CSP allowlist deliberately. `connect-src 'self'` SHALL NOT be widened to a wildcard.
3. THE system SHALL NOT bulk-download or area-download road data from OpenStreetMap's own infrastructure, per the tile usage policy recorded in `src/shared/map/MapProvider.ts`.
4. WHERE OSM-derived road data is bundled or cached locally THE system SHALL honour ODbL attribution and share-alike obligations, and the licence implications SHALL be assessed before adoption.
5. THE application SHALL remain fully functional for all existing features with no network connection.
6. Route and location data SHALL NOT be transmitted anywhere other than the routing provider the user explicitly configured.

Requirement 13: Testing Acceptance criteria

1. Unit tests SHALL cover: straight roads scoring low, winding roads scoring higher, hairpins being identified, sparse geometry returning unknown rather than low, and digitisation noise not producing artificial bends.
2. Tests SHALL cover the cost model and the detour budget independently of any live routing service.
3. Tests SHALL assert that **normal routing is unchanged**, in a form that would fail if it regressed.
4. Tests SHALL cover motorway avoidance, one-way restrictions, access restrictions and the no-route case.
5. Tests SHALL cover snap-to-road compatibility and GPX export round-trip fidelity.
6. End-to-end tests SHALL cover the UI at desktop and narrow widths.
7. Fixtures SHALL follow the existing rule in `test-data/README.md`: no real user GPS data. Real OSM road extracts are ODbL-licensed map data rather than user traces and are therefore permitted, but their size and attribution consequences SHALL be settled at design time rather than by committing a large extract.

Non-functional requirements

1. **No regression.** Every existing test continues to pass; `npm run verify` stays green.
2. **Module boundaries hold.** Scoring is pure and lives in `shared/geo/`; routing integration lives in `main/`; the renderer reaches it only through the preload bridge.
3. **No new dependency without a licence check** recorded in the register.
4. **Errors cross IPC as results, never exceptions**, per `src/main/ipc/handler.ts`.
5. **Documented assumptions.** Every road-classification assumption and every scoring threshold is written down with its reasoning.

Assumptions

1. The brief names the application "Trail GPX"; this repository is **TrailRT**. Assumed to be the same product under an earlier working name. Worth a one-line confirmation.
2. "Motorcycle-friendly" is assumed to mean a road-legal motorcycle on sealed roads by default, not adventure or off-road riding. Surface preferences are offered, but the default excludes unsealed tracks.
3. Phase 4 is assumed to deliver a routing provider whose transport modes include a motorcycle-equivalent, or a car profile close enough to serve.
4. The user is assumed to accept either a third-party routing service or the setup cost of running one locally. Requirement 1.5 exists to make that cost explicit rather than discovered.

Unresolved questions for the design stage

1. **Which engine, and where does it run?** The three shapes are: extend the Phase 4 provider if it can express custom per-edge costs; adopt a second self-hosted engine; or implement an in-house graph search over a bounded, user-supplied OSM extract. Each has a different answer to "what does the user have to install", and that is the deciding question for a desktop app that currently ships as a single DMG with no server.
2. **Where does road geometry come from?** Scoring curvature requires road geometry, not just a routing result. A bounded local extract, a provider that returns geometry with attributes, or a per-request fetch are different designs with different offline behaviour.
3. **Can twistiness genuinely influence the search** with the chosen engine, or only bias it coarsely? If only coarsely, Requirement 5.3 forces the in-house option, and that needs to be known before committing.
4. **The 10,000-point question.** Raise the cap with a different bound on undo entries, store the snapped shape as a cache keyed to control points, or simplify at generation time. Requirement 8 must be answered before any storage is written.
5. **Default planning speed.** `planningSpeedKmh` defaults to 4.5 km/h — walking pace. A motorcycle feature needs mode-aware defaults, which is a settings change beyond this feature's surface.
6. **Does the user want the twistiness breakdown** to reuse the Phase 4 speed distribution component? "% of route at each twistiness band" is the same shape of answer as "% of route at each speed", and sharing it would be cheaper than building a second one.

Notes

9.11 Phase 9: Garmin Compatibility — Requirements — SPEC-PHASE-9-GARMIN-COMPATIBILITY

Version: — · **Status:** unspecified · **Slug:** phase-9-garmin-compatibility

9.11.1 Requirements

Status: not started. Blocked on Phase 4.

Introduction Phase 9 is predominantly an **investigation** phase. Its primary deliverables are documented findings, not features. That framing is deliberate: GDB and IMG are proprietary formats with no viable JavaScript tooling and real licensing constraints, as established in ADR 0006.

Nothing in this phase may be advertised as supported until it has been tested against real sample files.

Requirements

Requirement 1: GDB support **Deliverable:** a findings document, and an implementation if the findings support one. The project's GPL-3.0 licence permits GPSBabel (GPL-2.0-or-later), so unlike the other items in this phase this one has a realistic path to being a feature.

Acceptance criteria

1. THE investigation SHALL document the GDB container structure, versions in circulation, and which record types carry routes, tracks and waypoints.
2. THE investigation SHALL compare two routes to support: invoking GPSBabel, or writing an independent parser. It SHALL cover, for the GPSBabel route, the obligation to publish its corresponding source, the practical cost of signing and notarising a nested Qt-linked binary inside the app bundle, and the size impact.
3. THE investigation SHALL recommend one route with reasoning.
4. WHERE GPSBabel is bundled THE release process SHALL publish its complete corresponding source alongside the binary, as GPL-3.0 requires.
5. IF GDB reading is implemented THEN it SHALL be validated against real sample files before appearing in the supported format list.
6. UNTIL then THE import dialog SHALL list GDB as not yet supported with a short explanation.

Requirement 2: Garmin IMG investigation **Deliverable:** a findings document and an interface design.

Acceptance criteria

1. THE investigation SHALL document IMG file structure, subfile organisation, map indexing, and metadata.
2. THE investigation SHALL document the licensing position for Garmin map products, distinguishing freely redistributable community maps from locked commercial products.
3. THE system SHALL NOT attempt to bypass encryption, DRM or map licensing under any circumstances.
4. THE phase SHALL produce a `GarminMapProvider` interface design that fits the existing `MapProvider` abstraction, with Garmin map parsing separated from the mapping interface.
5. A complete IMG renderer is explicitly NOT required. The architecture SHALL be shown not to prevent one.
6. IF a reader is prototyped THEN it SHALL be scoped to metadata and index reading before any rendering work.

Requirement 3: Device transfer investigation Acceptance criteria

1. THE investigation SHALL document how modern Garmin devices present themselves on macOS, distinguishing USB mass storage from MTP.
2. THE investigation SHALL document the on-device directory conventions for GPX and FIT files.
3. WHERE a device mounts as mass storage THE system SHALL be able to detect it and offer file-level transfer using already-implemented formats, which is feature work that follows from the investigation.
4. THE system SHALL NOT write to a device without explicit confirmation naming the device and the files.
5. MTP-only devices SHALL be reported as unsupported rather than partially handled.

Requirement 4: Expanded import and export Acceptance criteria

1. THE system SHALL improve Garmin symbol vocabulary fidelity on GPX export, validated against real Garmin-authored files.
2. THE system SHALL support Garmin GPX extension namespaces for round-tripping where documented.
3. A `GarminCompatibility` module SHALL encapsulate all Garmin-specific quirks, separate from the general format readers.

Requirement 5: Compatibility testing Acceptance criteria

1. THE system SHALL be tested against files exported by real Garmin devices and by BaseCamp itself.
2. Test files SHALL be synthetic or explicitly licensed for this use. Real user GPS data SHALL NOT be used.
3. THE supported format list SHALL be updated only on the basis of passing tests against real samples.
4. Findings SHALL be recorded as ADRs where they constitute decisions.

Notes

10 Architecture Decisions

No architecture decision records found in the source tree.

11 Traceability Matrix

Requirement → Design → Task → Code/Test, derived from spec cross references.

Requirement ID	Requirement	Design	Tasks	Code/Test
SPEC-PHASE-1-FOUNDATION:R1	Req 1	design	3, 15	see tasks
SPEC-PHASE-1-FOUNDATION:R2	Req 2	design	3, 4, 7	see tasks
SPEC-PHASE-1-FOUNDATION:R3	Req 3	design	8, 10, 11	see tasks
SPEC-PHASE-1-FOUNDATION:R4	Req 4	design	9, 11	see tasks
SPEC-PHASE-1-FOUNDATION:R5	Req 5	design	6, 9, 11	see tasks
SPEC-PHASE-1-FOUNDATION:R6	Req 6	design	2, 4, 5, 6	see tasks
SPEC-PHASE-1-FOUNDATION:R7	Req 7	design	6, 8, 9, 10, 14, 15	see tasks
SPEC-PHASE-1-FOUNDATION:R8	Req 8	design	10	see tasks
SPEC-PHASE-1-FOUNDATION:R9	Req 9	design	14	see tasks
SPEC-PHASE-1-FOUNDATION:R10	Req 10	design	12	see tasks
SPEC-PHASE-1-FOUNDATION:R11	Req 11	design	4, 6, 8, 13	see tasks
SPEC-PHASE-1-FOUNDATION:R12	Req 12	design	1, 2, 17	see tasks
SPEC-PHASE-1-FOUNDATION:R13	Req 13	design	1, 15	see tasks
SPEC-PHASE-1-FOUNDATION:R14	Req 14	design	16, 17	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R1	Req 1	design	3, 8, 9, 10, 17	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R2	Req 2	design	3, 8, 9, 17	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R3	Req 3	design	2, 3, 9, 17	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R4	Req 4	design	18	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R5	Req 5	design	5, 11, 14, 21	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R6	Req 6	design	4, 5, 14, 19	see tasks

Requirement ID	Requirement	Design	Tasks	Code/Test
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R7	Req 7	design	5, 19	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R8	Req 8	design	15, 19	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R9	Req 9	design	4, 14, 17, 19	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R10	Req 10	design	7, 16	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R11	Req 11	design	6, 11, 12, 13, 20, 21	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R12	Req 12	design	1, 10, 14, 21, 22	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R13	Req 13	design	2, 3, 4, 6, 7, 11, 12, 16, 22	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R14	Req 14	design	23, 27, 30	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R15	Req 15	design	23, 24, 25, 26, 30	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R16	Req 16	design	23, 28, 30	see tasks
SPEC-PHASE-10-ROAD-TRIP-PLANNING:R17	Req 17	design	23, 29, 30	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R1	Req 1	design	6, 7, 8, 9	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R2	Req 2	design	1, 3, 4	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R3	Req 3	design	5, 6, 7, 9, 13	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R4	Req 4	design	9	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R5	Req 5	design	5, 10	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R6	Req 6	design	8, 11	see tasks

Requirement ID	Requirement	Design	Tasks	Code/Test
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R7	Req 7	design	5, 12	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R8	Req 8	design	6, 7, 8, 12	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R9	Req 9	design	1, 4, 7, 8, 13	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R10	Req 10	design	5, 12	see tasks
SPEC-PHASE-11-GEOTAGGED-PHOTOS:R11	Req 11	design	2, 3, 4, 13	see tasks
SPEC-PHASE-2-ROUTES-WAYPOINTS:R1	Req 1	design	3, 9, 15	see tasks
SPEC-PHASE-2-ROUTES-WAYPOINTS:R2	Req 2	design	3, 10, 15	see tasks
SPEC-PHASE-2-ROUTES-WAYPOINTS:R3	Req 3	design	1, 4, 6, 7, 11, 15	see tasks
SPEC-PHASE-2-ROUTES-WAYPOINTS:R4	Req 4	design	3, 11	see tasks
SPEC-PHASE-2-ROUTES-WAYPOINTS:R5	Req 5	design	5, 6, 13, 15	see tasks
SPEC-PHASE-2-ROUTES-WAYPOINTS:R6	Req 6	design	7, 8, 14	see tasks
SPEC-PHASE-2-ROUTES-WAYPOINTS:R7	Req 7	design	2, 12	see tasks
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R1	Req 1	design	3, 5, 20	see tasks
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R2	Req 2	design	4, 7, 8, 9, 10, 11	see tasks
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R3	Req 3	design	12, 14, 17, 20	see tasks

Requirement ID	Requirement	Design	Tasks	Code/Test
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R4	Req 4	design	6, 7, 13, 18, 20	see tasks
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R5	Req 5	design	1, 3, 15, 16	see tasks
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R6	Req 6	design	1, 3, 16	see tasks
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R7	Req 7	design	19	see tasks
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT:R8	Req 8	design	2	see tasks
SPEC-PHASE-4-ADVANCED-MAPPING:R1	Req 1	design	1, 3	see tasks
SPEC-PHASE-4-ADVANCED-MAPPING:R2	Req 2	design	1, 4, 5, 8, 9, 18	see tasks
SPEC-PHASE-4-ADVANCED-MAPPING:R3	Req 3	design	6, 7, 8, 12, 17, 18, 19, 20	see tasks
SPEC-PHASE-4-ADVANCED-MAPPING:R4	Req 4	design	10, 11	see tasks
SPEC-PHASE-4-ADVANCED-MAPPING:R5	Req 5	design	2, 13	see tasks
SPEC-PHASE-4-ADVANCED-MAPPING:R6	Req 6	design	2, 4, 5, 7, 14, 18, 19, 20	see tasks
SPEC-PHASE-4-ADVANCED-MAPPING:R7	Req 7	design	15	see tasks
SPEC-PHASE-4-ADVANCED-MAPPING:R8	Req 8	design	3, 14, 20	see tasks
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R1	Req 1	design	7, 8, 9	see tasks
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R2	Req 2	design	2, 3, 9	see tasks

Requirement ID	Requirement	Design	Tasks	Code/Test
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R3	Req 3	design	6, 10	see tasks
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R4	Req 4	design	1, 4, 5, 10	see tasks
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R5	Req 5	design	11	see tasks
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT:R6	Req 6	design	5, 6, 7, 10	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R1	Req 1	design	6, 10, 11	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R2	Req 2	design	3, 7, 10	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R3	Req 3	design	2, 8	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R4	Req 4	design	2, 4, 6, 9	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R5	Req 5	design	3, 4, 9, 11	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R6	Req 6	design	4, 9, 12	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R7	Req 7	design	5, 9, 13	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R8	Req 8	design	3, 4, 5, 7, 11	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R9	Req 9	design	7, 9, 11, 14	see tasks

Requirement ID	Requirement	Design	Tasks	Code/Test
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R10	Req 10	design	1, 2, 6, 7	see tasks
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING:R11	Req 11	design	6, 7, 8, 14, 15	see tasks
SPEC-PHASE-7-REGIONS-PROFILE:R1	Req 1	design	7, 8, 12	see tasks
SPEC-PHASE-7-REGIONS-PROFILE:R2	Req 2	design	9, 14	see tasks
SPEC-PHASE-7-REGIONS-PROFILE:R3	Req 3	design	5, 6, 10, 11, 12	see tasks
SPEC-PHASE-7-REGIONS-PROFILE:R4	Req 4	design	1, 2, 6, 10, 12	see tasks
SPEC-PHASE-7-REGIONS-PROFILE:R5	Req 5	design	1, 3, 4, 10	see tasks
SPEC-PHASE-7-REGIONS-PROFILE:R6	Req 6	design	16	see tasks
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R1	Req 1	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R2	Req 2	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R3	Req 3	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R4	Req 4	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R5	Req 5	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R6	Req 6	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R7	Req 7	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R8	Req 8	—	—	—

Requirement ID	Requirement	Design	Tasks	Code/Test
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R9	Req 9	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R10	Req 10	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R11	Req 11	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R12	Req 12	—	—	—
SPEC-PHASE-8-WINDY-ROADS-ROUTING:R13	Req 13	—	—	—
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R1	Req 1	—	—	—
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R2	Req 2	—	—	—
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R3	Req 3	—	—	—
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R4	Req 4	—	—	—
SPEC-PHASE-9-GARMIN-COMPATIBILITY:R5	Req 5	—	—	—

12 Git Change History

Commits touching `.kiro/specs` (most recent first).

Commit	Date	Author	Subject
3b5e32d	2026-11-11	Andrew Kentigern-Fox	feat: Phase 10 road trip planning, owner feedback, and Phase 11 spec
e96452c	2026-11-09	Andrew Kentigern-Fox	feat: Phases 5–7 — objects and points, route shaping, UK regions and profile
41fabd6	2026-11-07	Andrew Kentigern-Fox	feat: Phase4 Road Snap and Road Speed Analysis
cefb2a1	2026-11-07	Andrew Kentigern-Fox	feat: Phase4 p2
f97b51d	2026-11-06	Andrew Kentigern-Fox	feat: Phase4 p1
212fcc1	2026-11-06	Andrew Kentigern-Fox	feat: Phase3 Completed
767301b	2026-11-06	Andrew Kentigern-Fox	feat: Phase3 Completed
a4ac6e4	2026-11-06	Andrew Kentigern-Fox	feat: First Fix, UI created
b9c92b3	2026-11-05	Andrew Kentigern-Fox	Specs

13 Appendices

13.1 Appendix A — Reference Files

No supplementary reference files found.

13.2 Appendix B — Tokenizer Summary

Estimated token counts for each specification and steering document, to help identify bloat. Counts are deterministic approximations based on characters per token (Claude Opus 4.8 ≈ 3.5 chars/token; Claude Haiku ≈ 3.5 chars/token; GPT-4o (o200k) ≈ 4 chars/token). They are estimates, not exact tokenizer output.

13.2.1 Specifications

Spec ID	Title	Chars	Words	Claude Opus 4.8	Claude Haiku	GPT-4o (o200k)
SPEC-PHASE-10-ROAD-TRIP-PLANNING	Phase 10: Road Trip Planning — Requirements	96,202	15,116	26,703	26,703	23,365
SPEC-PHASE-11-GEOTAGGED-PHOTOS	Phase 11: Geotagged Photos from a Linked Folder — Requirements	92,449	14,983	25,742	25,742	22,525
SPEC-PHASE-6-ROUTE-ALTERNATIVES-SHAPING	Phase 6: Route alternatives and shaping — Requirements	53,387	8,371	14,774	14,774	12,927
SPEC-PHASE-7-REGIONS-PROFILE	Phase 7: UK regions, road-following by default, and a fuller profile — Requirements	47,519	7,451	13,227	13,227	11,573
SPEC-PHASE-8-WINDY-ROADS-ROUTING	Phase 8: Intelligent Windy Roads Routing — Requirements	43,305	6,157	11,851	11,851	10,369
SPEC-PHASE-1-FOUNDATION	Phase 1: Application Foundation — Requirements	45,674	6,273	11,822	11,822	10,344
SPEC-PHASE-4-ADVANCED-MAPPING	Phase 4: Advanced Mapping — Requirements	37,940	5,727	10,455	10,455	9,149
SPEC-PHASE-5-OBJECT-POINT-MANAGEMENT	Phase 5: Object and point management — Requirements	33,529	5,194	9,357	9,357	8,188
SPEC-PHASE-3-TRACKS-IMPORT-EXPORT	Phase 3: Tracks and Import/Export — Requirements	23,840	3,623	6,550	6,550	5,732
SPEC-PHASE-2-ROUTES-WAYPOINTS	Phase 2: Routes and Waypoints — Requirements	21,436	3,320	5,922	5,922	5,182
SPEC-PHASE-9-GARMIN-COMPATIBILITY	Phase 9: Garmin Compatibility — Requirements	4,262	621	1,187	1,187	1,039
Total		**499,543**	**76,836**	**137,590**	**137,590**	**120,393**

13.2.2 Steering Documents

Document	Chars	Words	Claude Opus 4.8	Claude Haiku	GPT-4o (o200k)
toolchain.md	6,758	915	1,641	1,641	1,436
architecture.md	5,779	835	1,552	1,552	1,358
licensing.md	3,976	509	975	975	853
agent-workflow.md	2,211	369	611	611	535
progress-tracking.md	1,767	314	501	501	438
Total	**20,491**	**2,942**	**5,280**	**5,280**	**4,620**